MULESOFT · PRACTICE PROBLEMS · WITH FULL SOLUTIONS

1000 DataWeave Problems With Solutions

Volume 1 — Selectors & Basic Data Access. 100 hands-on problems, each with input, solution code, output, and a one-line explanation.

Problems 1–100

Volume 1 of 10

2026 EDITION

How to Use This Volume

This is a companion practice book to *The Complete DataWeave Handbook* — 100 short, focused problems on selectors, the very first skill every DataWeave developer needs. Each problem follows the same format:

- **A short problem statement** — what you need to extract or produce.
- **Input** — the sample payload you're working with.
- **Solution** — a complete, correct DataWeave expression, shown in the same black code window used throughout the Handbook.
- **Output** — exactly what the solution produces.
- **A one-line explanation** of the technique used.

Try to write the solution yourself from the problem statement and input before looking at the answer — that's what actually builds the skill. This volume is organized into ten sections of ten problems each, moving from simple field access to mixed real-world challenges.

VOLUME 1

Selectors & Basic Data Access

100 problems on reaching into data and pulling out exactly what you need — dot selectors, index selectors, descendants, safe navigation, and more, across ten focused sections.

SECTION 1 — Basic Dot Selectors (Problems 1–10)

#1 Get the Customer's Email

Extract the customer's email address from the order payload.

INPUT

```
{ "order": { "customer": { "email": "maria@example.com" } } }
```

SOLUTION

```
payload.order.customer.email
```

OUTPUT

```
"maria@example.com"
```

A simple dot selector walks straight down three levels to the target field.

#2 Get the Employee's Department

Extract the department name from the employee record.

INPUT

```
{ "employee": { "name": "Raj Patel", "department": "Logistics" } }
```

SOLUTION

```
payload.employee.department
```

OUTPUT

```
"Logistics"
```

Sibling fields like `name` are simply ignored by a selector targeting only `department`.

#3 Get the Product's Category

Extract the category from a product record.

INPUT

```
{ "product": { "sku": "PALLET-01", "category": "Freight Equipment" } }
```

SOLUTION

```
payload.product.category
```

OUTPUT

```
"Freight Equipment"
```

One dot selector per level of nesting — here, two levels deep.

#4 Get the City From a Weather Payload

Extract the city name from a weather API response.

INPUT

```
{ "location": { "city": "Austin", "country": "US" } }
```

SOLUTION

```
payload.location.city
```

OUTPUT

```
"Austin"
```

The field name in the selector must match the key in the payload exactly, including case.

#5 Get the Book's Author

Extract the author's name from a book record.

INPUT

```
{ "book": { "title": "Data Pipelines", "author": "L. Owens" } }
```

SOLUTION

```
payload.book.author
```

OUTPUT

```
"L. Owens"
```

A straightforward two-level dot selector.

#6 Get the Movie's Director

Extract the director's name from a movie record.

INPUT

```
{ "movie": { "title": "Night Freight", "director": "S. Alvarez" } }
```

SOLUTION

```
payload.movie.director
```

OUTPUT

```
"S. Alvarez"
```

Selector paths mirror the payload's structure one to one.

#7 Get the Invoice Currency

Extract the currency code from an invoice payload.

INPUT

```
{ "invoice": { "number": "INV-88", "currency": "USD" } }
```

SOLUTION

```
payload.invoice.currency
```

OUTPUT

```
"USD"
```

A String value is returned exactly as stored — no coercion needed here.

#8 Get the Flight's Destination

Extract the destination airport code from a flight booking.

INPUT

```
{ "flight": { "origin": "JFK", "destination": "LAX" } }
```

SOLUTION

```
payload.flight.destination
```

OUTPUT

```
"LAX"
```

Even with a similarly-named sibling (`origin`), the selector only returns the exact field requested.

#9 Get the Recipe's Cuisine

Extract the cuisine type from a recipe record.

INPUT

```
{ "recipe": { "name": "Paella", "cuisine": "Spanish" } }
```

SOLUTION

```
payload.recipe.cuisine
```

OUTPUT

```
"Spanish"
```

Basic field access remains the same regardless of the business domain.

#10 Get the Account Holder's Name

Extract the account holder's name from a bank account payload.

INPUT

```
{ "account": { "id": "ACC-501", "holder": "Wei Zhang" } }
```

SOLUTION

```
payload.account.holder
```

OUTPUT

```
"Wei Zhang"
```

This closes out Section 1 — every problem used the same core pattern: one dot per level.

SECTION 2 — Nested Object Selectors (Problems 11–20)

#11 Get the Customer's City (3 Levels Deep)

Extract the city from an order's customer address.

INPUT

```
{ "order": { "customer": { "address": { "city": "Denver" } } } }
```

SOLUTION

```
payload.order.customer.address.city
```

OUTPUT

```
"Denver"
```

Chained dot selectors work at any depth — four in this case.

#12 Get the Employee's Manager Name

Extract the manager's name from a nested employee record.

INPUT

```
{ "employee": { "name": "Raj Patel", "manager": { "name": "Dana Kim" } } }
```

SOLUTION

```
payload.employee.manager.name
```

OUTPUT

```
"Dana Kim"
```

Two different `name` fields exist at different levels — the selector's full path disambiguates them.

#13 Get the Manufacturer's Country

Extract the manufacturer's country from a product record.

INPUT

```
{ "product": { "manufacturer": { "name": "Acme Co", "country": "DE" } } }
```

SOLUTION

```
payload.product.manufacturer.country
```

OUTPUT

```
"DE"
```

Three-level nested access, same pattern as before.

#14 Get the Warehouse Code

Extract the warehouse code from a shipment's origin details.

INPUT

```
{ "shipment": { "origin": { "warehouse": { "code": "WH-EAST-4" } } } }
```

SOLUTION

```
payload.shipment.origin.warehouse.code
```

OUTPUT

```
"WH-EAST-4"
```

Four-level nesting is handled identically to two-level nesting — just a longer chain.

#15 Get the School Principal's Name

Extract the principal's name from a student's school record.

INPUT

```
{ "student": { "school": { "principal": { "name": "Mr. Osei" } } } }
```

SOLUTION

```
payload.student.school.principal.name
```

OUTPUT

```
"Mr. Osei"
```

Reading the selector aloud ("student's school's principal's name") often matches the payload structure directly.

#16 Check If a Hotel Room Has WiFi

Extract the boolean WiFi amenity flag from a hotel room record.

INPUT

```
{ "hotel": { "room": { "amenities": { "wifi": true } } } }
```

SOLUTION

```
payload.hotel.room.amenities.wifi
```

OUTPUT

```
true
```

Selectors work the same way regardless of the target field's type — here it returns a Boolean, not a String.

#17 Get the Engine Horsepower

Extract the horsepower spec from a vehicle's engine details.

INPUT

```
{ "vehicle": { "engine": { "specs": { "horsepower": 310 } } } }
```

SOLUTION

```
payload.vehicle.engine.specs.horsepower
```

OUTPUT

```
310
```

The result here is a Number, returned exactly as it appears in the source.

#18 Get the Vendor's Tax ID

Extract the vendor's tax ID from a contract's parties section.

INPUT

```
{ "contract": { "parties": { "vendor": { "taxId": "TX-99871" } } } }
```

SOLUTION

```
payload.contract.parties.vendor.taxId
```

OUTPUT

```
"TX-99871"
```

camelCase field names like `taxId` are matched exactly, case-sensitively.

#19 Get the Venue Address

Extract the venue address from a ticket's event details.

INPUT

```
{ "ticket": { "event": { "venue": { "address": "500 Main St" } } } }
```

SOLUTION

```
payload.ticket.event.venue.address
```

OUTPUT

```
"500 Main St"
```

Another four-level dot chain — depth alone never changes the technique.

#20 Get the Billing Cycle

Extract the billing cycle from a subscription's plan details.

INPUT

```
{ "subscription": { "plan": { "billing": { "cycle": "monthly" } } } }
```

SOLUTION

```
payload.subscription.plan.billing.cycle
```

OUTPUT

```
"monthly"
```

Section 2 complete — nested access is just the dot selector, repeated for every level.

SECTION 3 — Array Index Selectors (Problems 21–30)

#21 Get the First Item's SKU

Extract the SKU of the first item in an order.

INPUT

```
{ "items": [ { "sku": "PALLET-01" }, { "sku": "CRATE-07" } ] }
```

SOLUTION

```
payload.items[0].sku
```

OUTPUT

```
"PALLET-01"
```

Array indexes are zero-based — `[0]` is always the first element.

#22 Get the Third Passenger's Name

Extract the name of the third passenger on a flight booking.

INPUT

```
{ "passengers": [ {"name":"A. Lee"}, {"name":"B. Cruz"}, {"name":"C. Diaz"} ]
}
```

SOLUTION

```
payload.passengers[2].name
```

OUTPUT

```
"C. Diaz"
```

The third element sits at index 2, since counting starts from 0.

#23 Get the First Comment's Text

Extract the text of the first comment on a support ticket.

INPUT

```
{ "comments": [ {"text":"Please expedite"}, {"text":"Confirmed"} ] }
```

SOLUTION

```
payload.comments[0].text
```

OUTPUT

```
"Please expedite"
```

Index selectors combine naturally with a following dot selector to reach a field inside that element.

#24 Get the Second Employee's Name

Extract the name of the second employee in a department roster.

INPUT

```
{ "employees": [ {"name":"Raj Patel"}, {"name":"Dana Kim"} ] }
```

SOLUTION

```
payload.employees[1].name
```

OUTPUT

```
"Dana Kim"
```

The second element is at index 1.

#25 Get the First Image URL

Extract the URL of the first image in a product's image list.

INPUT

```
{ "images": [ "https://cdn.site/a.jpg", "https://cdn.site/b.jpg" ] }
```

SOLUTION

```
payload.images[0]
```

OUTPUT

```
"https://cdn.site/a.jpg"
```

When an array holds plain Strings rather than Objects, no trailing dot selector is needed.

#26 Get the Third Quiz Question

Extract the text of the third question in a quiz.

INPUT

```
{ "quiz": { "questions": [ {"text":"Q1"}, {"text":"Q2"}, {"text":"Q3"} ] } }
```

SOLUTION

```
payload.quiz.questions[2].text
```

OUTPUT

```
"Q3"
```

Dot and index selectors freely combine, regardless of how deep the array is nested.

#27 Get the First Ingredient

Extract the name of the first ingredient in a recipe.

INPUT

```
{ "recipe": { "ingredients": [ {"name":"Rice"}, {"name":"Saffron"} ] } }
```

SOLUTION

```
payload.recipe.ingredients[0].name
```

OUTPUT

```
"Rice"
```

The array sits one level deep inside `recipe`, so the dot selector comes first.

#28 Get the Second Stop's City

Extract the city of the second stop on a delivery route.

INPUT

```
{ "route": { "stops": [ {"city":"Reno"}, {"city":"Salt Lake City"} ] } }
```

SOLUTION

```
payload.route.stops[1].city
```

OUTPUT

```
"Salt Lake City"
```

Consistent pattern: locate the array, then index into it, then select the target field.

#29 Get the First Reviewer's Name

Extract the reviewer's name from the first review on a product.

INPUT

```
{ "reviews": [ {"reviewer":"J. Ortiz","rating":5} ] }
```

SOLUTION

```
payload.reviews[0].reviewer
```

OUTPUT

```
"J. Ortiz"
```

Even a single-element array still requires an explicit index — DataWeave never auto-unwraps arrays.

#30 Get the Third Player's Score

Extract the score of the third player on a leaderboard.

INPUT

```
{ "players": [ {"score":900}, {"score":870}, {"score":845} ] }
```

SOLUTION

```
payload.players[2].score
```

OUTPUT

```
845
```

Section 3 complete — index selectors are always zero-based, whatever the array contains.

SECTION 4 — Negative Index & Range Selectors (Problems 31–40)

#31 Get the Last Item's SKU

Extract the SKU of the last item in an order, without knowing the array's length.

INPUT

```
{ "items": [ {"sku":"PALLET-01"}, {"sku":"CRATE-07"}, {"sku":"DRUM-14"} ] }
```

SOLUTION

```
payload.items[-1].sku
```

OUTPUT

```
"DRUM-14"
```

A negative index of `-1` always means "the last element," regardless of array length.

#32 Get the Last Comment

Extract the text of the most recent (last) comment on a ticket.

INPUT

```
{ "comments": [ {"text":"Opened"}, {"text":"In progress"}, {"text":"Resolved"}
] }
```

SOLUTION

```
payload.comments[-1].text
```

OUTPUT

```
"Resolved"
```

Negative indexing is especially useful for logs and timelines where you want the latest entry.

#33 Get the First Two Items

Extract the first two items from an order as a sub-array.

INPUT

```
{ "items": [ {"sku":"A"}, {"sku":"B"}, {"sku":"C"} ] }
```

SOLUTION

```
payload.items[0 to 1]
```

OUTPUT

```
[ {"sku":"A"}, {"sku":"B"} ]
```

A range selector `[a to b]` is inclusive on both ends and returns a sub-array.

#34 Get the Last Two Passengers

Extract the last two passengers on a flight, whatever the total count.

INPUT

```
{ "passengers": [ {"n":"A"}, {"n":"B"}, {"n":"C"}, {"n":"D"} ] }
```

SOLUTION

```
payload.passengers[-2 to -1]
```

OUTPUT

```
[ {"n":"C"}, {"n":"D"} ]
```

Negative indexes can be used inside a range too, counting from the end in both directions.

#35 Get the Middle Route Stops

Extract stops 2 and 3 (skipping the first and last) from a 4-stop route.

INPUT

```
{ "stops": [ {"c":"Reno"}, {"c":"Elko"}, {"c":"Ogden"}, {"c":"Cheyenne"} ] }
```

SOLUTION

```
payload.stops[1 to 2]
```

OUTPUT

```
[ {"c":"Elko"}, {"c":"Ogden"} ]
```

Combine indexes deliberately to slice out exactly the middle section you need.

#36 Get the Most Recent Review

Extract the most recently added review, assuming reviews are appended in order.

INPUT

```
{ "reviews": [ {"r":5}, {"r":3}, {"r":4} ] }
```

SOLUTION

```
payload.reviews[-1]
```

OUTPUT

```
{ "r": 4 }
```

Without a trailing field selector, the entire object at that index is returned.

#37 Get the First Three Scores

Extract just the first three scores from a longer leaderboard array.

INPUT

```
{ "scores": [900, 870, 845, 810, 790] }
```

SOLUTION

```
payload.scores[0 to 2]
```

OUTPUT

```
[900, 870, 845]
```

Range selectors work on arrays of plain values just as well as arrays of objects.

#38 Get the Latest Chat Message

Extract the text of the newest message in a chat thread.

INPUT

```
{ "messages": [ {"text":"Hi"}, {"text":"Order shipped"} ] }
```

SOLUTION

```
payload.messages[-1].text
```

OUTPUT

```
"Order shipped"
```

Negative indexing plus a field selector reaches the newest entry's specific value in one line.

#39 Get All Items Except the First

Extract every item in an order except the very first one.

INPUT

```
{ "items": [ {"sku":"A"}, {"sku":"B"}, {"sku":"C"} ] }
```

SOLUTION

```
payload.items[1 to -1]
```

OUTPUT

```
[ {"sku":"B"}, {"sku":"C"} ]
```

Mixing a positive start and a negative end is a concise way to say "from here to the very end."

#40 Get the Most Recent Transaction

Extract the amount of the most recent transaction on an account.

INPUT

```
{ "transactions": [ {"amount":50}, {"amount":-20}, {"amount":100} ] }
```

SOLUTION

```
payload.transactions[-1].amount
```

OUTPUT

```
100
```

Section 4 complete — negative indexes and ranges remove the need to know an array's exact length in advance.

SECTION 5 — Descendants Selector (Problems 41–50)

#41 Get Every SKU in the Order

Extract every `sku` value anywhere in the payload, at any depth.

INPUT

```
{ "items": [ {"sku":"A"}, {"sku":"B"} ], "bundle": {"sku":"C"} }
```

SOLUTION

```
payload..sku
```

OUTPUT

```
["A", "B", "C"]
```

The descendants selector `..` searches the entire structure, not just one branch.

#42 Get Every Email Address

Extract every `email` field from a payload with several contact objects.

INPUT

```
{ "customer": {"email":"a@x.com"}, "support": {"agent": {"email":"b@x.com"}} }
```

SOLUTION

```
payload..email
```

OUTPUT

```
["a@x.com", "b@x.com"]
```

Even fields buried at very different depths are found by the same descendants selector.

#43 Get Every Price in a Multi-Order Batch

Extract every `price` value across a batch of orders, each with its own items.

INPUT

```
{ "orders": [ {"items":[{"price":10},{"price":20}]}, {"items":[{"price":5}]} ]
}
```

SOLUTION

```
payload..price
```

OUTPUT

```
[10, 20, 5]
```

Descendants selectors traverse both arrays and objects freely while collecting matches.

#44 Get Every ID in a Nested Structure

Extract every `id` field from an order, its customer, and its items.

INPUT

```
{ "id":"ORD-1", "customer":{"id":"CUST-9"}, "items":[{"id":"ITM-1"}] }
```

SOLUTION

```
payload..id
```

OUTPUT

```
["ORD-1", "CUST-9", "ITM-1"]
```

Note that the top-level `id` is included too — `..` searches from the payload root downward, inclusively.

#45 Get Every Status Field

Extract every `status` value from a shipment and its sub-legs.

INPUT

```
{ "status":"In Transit", "legs":[{"status":"Complete"},{"status":"Pending"}] }
```

SOLUTION

```
payload..status
```

OUTPUT

```
["In Transit", "Complete", "Pending"]
```

Descendants collect same-named fields wherever they occur, even mixed with different types of values.

#46 Get Every Amount in a Statement

Extract every `amount` field from an account statement's transactions and fees.

INPUT

```
{ "transactions":[{"amount":50}], "fees":[{"amount":2},{"amount":1}] }
```

SOLUTION

```
payload..amount
```

OUTPUT

```
[50, 2, 1]
```

This is exactly the kind of pattern useful for quickly summing a value scattered across a payload — often paired with `sum()`.

#47 Get Every Date Field

Extract every `date` field from an itinerary with multiple flight legs.

INPUT

```
{ "legs": [ {"date":"2026-03-01"}, {"date":"2026-03-04"} ] }
```

SOLUTION

```
payload..date
```

OUTPUT

```
["2026-03-01", "2026-03-04"]
```

The values here are still Strings unless separately coerced — `..` only selects, it never converts types.

#48 Get Every Discount Code

Extract every `code` field applied anywhere in a shopping cart.

INPUT

```
{ "cart": {"code":"WELCOME10"}, "items":[{"code":"BULK5"}] }
```

SOLUTION

```
payload..code
```

OUTPUT

```
["WELCOME10", "BULK5"]
```

Useful whenever a field name is reused at multiple, unpredictable levels of a payload.

#49 Get Every Quantity Across Line Items

Extract every `qty` field from an order's items array.

INPUT

```
{ "items": [ {"qty":4}, {"qty":2}, {"qty":1} ] }
```

SOLUTION

```
payload..qty
```

OUTPUT

```
[4, 2, 1]
```

Here `..qty` reaches the same result a `map` could — but is more concise for a simple flat extraction.

#50 Sum Every Amount With Descendants + sum()

Compute the total of every `amount` field scattered across a payload's transactions and fees.

INPUT

```
{ "transactions":[{"amount":50}], "fees":[{"amount":2},{"amount":1}] }
```

SOLUTION

```
sum(payload..amount)
```

OUTPUT

```
53
```

Section 5 complete — combining `..` with a function like `sum()` is one of the most practical real-world uses of descendants.

SECTION 6 — Safe Navigation & the default Operator (Problems 51–60)

#51 Handle a Missing Phone Number

Extract the customer's phone number, falling back to "N/A" if it's missing.

INPUT

```
{ "customer": { "email": "a@x.com" } }
```

(NO PHONE FIELD)

SOLUTION

```
payload.customer.phone default "N/A"
```

OUTPUT

```
"N/A"
```

Selecting a field that doesn't exist returns `null`, which `default` then replaces.

#52 Default a Missing Discount to Zero

Extract an order's discount amount, defaulting to 0 if not present.

INPUT

```
{ "orderId": "ORD-9" }
```

(NO DISCOUNT FIELD)

SOLUTION

```
payload.discount default 0
```

OUTPUT

```
0
```

Defaulting numeric fields avoids downstream arithmetic errors caused by an unexpected `null`.

#53 Default a Missing Middle Name

Extract a customer's middle name, defaulting to an empty string if absent.

INPUT

```
{ "customer": { "firstName": "Maria" } }
```

SOLUTION

```
payload.customer.middleName default ""
```

OUTPUT

```
""
```

An empty string default keeps later string concatenation safe without adding the literal text "null."

#54 Default a Missing Nested Object Entirely

Extract a shipment's tracking info object, defaulting to an empty object if the whole branch is missing.

INPUT

```
{ "shipmentId": "SHP-4" }
```

(NO TRACKING FIELD)

SOLUTION

```
payload.tracking default {}
```

OUTPUT

```
{}
```

`default` *works with any type of fallback value — an object, array, string, or number.*

#55 Default an Empty Items Array

Extract an order's items array, defaulting to an empty array if missing, before calling `sizeOf`.

INPUT

```
{ "orderId": "ORD-9" }
```

(NO ITEMS FIELD)

SOLUTION

```
sizeOf(payload.items default [])
```

OUTPUT

```
0
```

Without the default, `sizeOf(null)` would throw an error instead of safely returning 0.

#56 Chain default After a Deep Selector

Extract a customer's loyalty tier, several levels deep, defaulting to "Standard".

INPUT

```
{ "customer": { "profile": {} } }
```

(NO TIER FIELD)

SOLUTION

```
payload.customer.profile.tier default "Standard"
```

OUTPUT

```
"Standard"
```

`default` is placed once, at the very end of the full selector chain.

#57 Default Only Triggers on null, Not on Empty String

Show that an existing but empty string field is *not* replaced by `default`.

INPUT

```
{ "notes": "" }
```

SOLUTION

```
payload.notes default "No notes provided"
```

OUTPUT

```
""
```

Since `""` is not `null`, `default` leaves it untouched — a common source of confusion for newcomers.

#58 Combine default With a Computed Fallback

Extract a shipping label, defaulting to a value built from the order ID if missing.

INPUT

```
{ "orderId": "ORD-77" }
```

(NO LABEL FIELD)

SOLUTION

```
payload.label default ("LBL-" ++ payload.orderId)
```

OUTPUT

```
"LBL-ORD-77"
```

The fallback side of `default` can be any expression, not just a literal.

#59 Default an Array Element Field

Extract the first item's discount, defaulting to 0 if that specific item has none.

INPUT

```
{ "items": [ { "sku": "A" } ] }
```

(NO DISCOUNT ON ITEM)

SOLUTION

```
payload.items[0].discount default 0
```

OUTPUT

```
0
```

`default` applies the same way whether the missing field is at the top level or inside an array element.

#60 Default Every Item's Missing Notes Field via map

Return every item's `notes` field, defaulting each missing one to an empty string.

INPUT

```
{ "items": [ {"notes":"Fragile"}, {"sku":"B"} ] }
```

SOLUTION

```
payload.items map (item) -> item.notes default ""
```

OUTPUT

```
["Fragile", ""]
```

Section 6 complete — combining `default` inside `map` guards an entire array of records at once.

SECTION 7 — Multi-Value & Wildcard Selectors (Problems 61–70)

#61 Get Every "note" Field at One Level

An object has several repeated `note` keys at the same level (unusual but valid in XML-derived data) — select all of them.

INPUT

```
{ "note": ["Handle with care", "Keep dry"] }
```

SOLUTION

```
payload.*note
```

OUTPUT

```
["Handle with care", "Keep dry"]
```

The multi-value selector `*fieldName` gathers every value under that key at the current level into an array.

#62 Get Every Tag on a Single Item

An item has several repeated `tag` entries — select them all as an array.

INPUT

```
{ "item": { "tag": ["fragile", "priority", "export"] } }
```

SOLUTION

```
payload.item.*tag
```

OUTPUT

```
["fragile", "priority", "export"]
```

This pattern is especially common when reading XML, where a repeated element naturally has multiple values under one key.

#63 Count Every Matching Field With sizeOf

Count how many `warning` entries exist at the top level of a payload.

INPUT

```
{ "warning": ["Low stock", "Delayed", "Backordered"] }
```

SOLUTION

```
sizeOf(payload.*warning)
```

OUTPUT

```
3
```

*Combining * with `sizeOf` is a quick way to count repeated same-named entries.*

#64 Distinguish * From .. on a Simple Case

Given a flat object with two `error` entries, show that * and `..` return the same result at this single level.

INPUT

```
{ "error": ["Timeout", "Retry limit reached"] }
```

SOLUTION

```
{
  multiValue: payload.*error,
  descendants: payload..error
}
```

OUTPUT

```
{ "multiValue": ["Timeout","Retry limit reached"], "descendants":
["Timeout","Retry limit reached"] }
```

*They diverge once nesting is involved: * only checks one level, while `..` searches every level.*

#65 Select Every Attribute-Like Key at the Top Level

Select every `flag` value directly under an item's metadata object.

INPUT

```
{ "metadata": { "flag": ["urgent", "reviewed"] } }
```

SOLUTION

```
payload.metadata.*flag
```

OUTPUT

```
["urgent", "reviewed"]
```

The multi-value selector can be applied after any dot selector path, not just at the payload root.

#66 Map Over Multi-Value Results

Uppercase every tag collected from a multi-value selector.

INPUT

```
{ "tag": ["fragile", "export"] }
```

SOLUTION

```
payload.*tag map (t) -> upper(t)
```

OUTPUT

```
["FRAGILE", "EXPORT"]
```

Because `*fieldName` always returns an Array, it can be piped directly into `map`, `filter`, or any other array function.

#67 Filter Multi-Value Results

From a multi-value selection of scores, keep only those above 80.

INPUT

```
{ "score": [95, 60, 88, 72] }
```

SOLUTION

```
payload.*score filter (s) -> s > 80
```

OUTPUT

```
[95, 88]
```

Multi-value results are ordinary arrays, so every array function from Chapter 9 works on them directly.

#68 Handle a Single Occurrence With *

Show what a multi-value selector returns when there's only one matching field, not several.

INPUT

```
{ "tag": "priority" }
```

SOLUTION

```
payload.*tag
```

OUTPUT

```
["priority"]
```

*Even a single match is still wrapped in an array — * always returns an Array, for consistency.*

#69 Handle No Occurrences With *

Show what a multi-value selector returns when the field doesn't exist at all.

INPUT

```
{ "sku": "A" }
```

(NO TAG FIELD)

SOLUTION

```
payload.*tag
```

OUTPUT

```
[]
```

*Unlike a plain dot selector on a missing field (which returns `null`), * safely returns an empty array.*

#70 Sum Multi-Value Numeric Fields

Sum every `fee` value found at one level of a payload.

INPUT

```
{ "fee": [2.50, 1.00, 0.75] }
```

SOLUTION

```
sum(payload.*fee)
```

OUTPUT

```
4.25
```

*Section 7 complete — * always yields a clean Array, ready for any downstream array function.*

SECTION 8 — XML Attribute & Element Selectors (Problems 71–80)

#71 Select an XML Attribute

Extract the `id` attribute from an XML order element.

INPUT (XML)

```
<order id="ORD-1042"><total>482.0</total></order>
```

SOLUTION

```
payload.order.@id
```

OUTPUT

```
"ORD-1042"
```

The @ prefix selects an XML attribute rather than a child element.

#72 Select an XML Child Element's Text

Extract the total amount from the same XML order element's child.

INPUT (XML)

```
<order id="ORD-1042"><total>482.0</total></order>
```

SOLUTION

```
payload.order.total
```

OUTPUT

```
"482.0"
```

Child elements use the same dot selector as JSON objects; only attributes need the @ prefix.

#73 Combine Attribute and Element Access

Build an object combining an XML attribute and a child element's value.

INPUT (XML)

```
<shipment code="SHP-9"><status>Delivered</status></shipment>
```

SOLUTION

```
{
  code: payload.shipment.@code,
  status: payload.shipment.status
}
```

OUTPUT

```
{ "code": "SHP-9", "status": "Delivered" }
```

A single expression can freely mix attribute (@) and element selectors in the same object.

#74 Select Repeated XML Elements as an Array

Extract every `item` element under `items` as an array.

INPUT (XML)

```
<items><item>A</item><item>B</item></items>
```

SOLUTION

```
payload.items.*item
```

OUTPUT

```
["A", "B"]
```

*Repeated XML sibling elements are exactly the case the multi-value selector * from Section 7 was built for.*

#75 Select an Attribute Inside a Repeated Element

Extract the `sku` attribute from every repeated `item` element.

INPUT (XML)

```
<items><item sku="A"/><item sku="B"/></items>
```

SOLUTION

```
payload.items.*item.@sku
```

OUTPUT

```
["A", "B"]
```

Selectors chain naturally: multi-value first to fan out over each element, then `@sku` on each one.

#76 Coerce an XML Numeric Value

Extract and coerce the XML order total to a real Number for arithmetic.

INPUT (XML)

```
<order><total>482.0</total></order>
```

SOLUTION

```
payload.order.total as Number
```

OUTPUT

```
482.0
```

Since XML text is always text underneath, coerce with `as` before treating a value as a Number.

#77 Default a Missing XML Attribute

Extract an XML element's optional `priority` attribute, defaulting to "normal".

INPUT (XML)

```
<order id="ORD-1"/>
```

(NO PRIORITY ATTRIBUTE)

SOLUTION

```
payload.order.@priority default "normal"
```

OUTPUT

```
"normal"
```

The `default` operator works identically on attribute selectors as it does on element selectors.

#78 Reshape XML Attributes Into a JSON Object

Convert an XML element's attributes into a clean JSON object.

INPUT (XML)

```
<customer id="CUST-9" email="a@x.com"/>
```

SOLUTION

```
{
  id: payload.customer.@id,
  email: payload.customer.@email
}
```

OUTPUT

```
{ "id": "CUST-9", "email": "a@x.com" }
```

This is the core building block of every JSON-to-XML or XML-to-JSON transformation from Chapter 18.

#79 Select a Deeply Nested XML Element

Extract the tracking number from a deeply nested XML shipment structure.

INPUT (XML)

```
<shipment><tracking><number>1Z999</number></tracking></shipment>
```

SOLUTION

```
payload.shipment.tracking.number
```

OUTPUT

```
"1Z999"
```

Deep XML nesting is navigated with exactly the same dot-chain technique as deep JSON nesting.

#80 Filter Repeated XML Elements by Attribute

From repeated XML `item` elements, keep only those marked `fragile="true"`.

INPUT (XML)

```
<items><item sku="A" fragile="true"/><item sku="B" fragile="false"/></items>
```

SOLUTION

```
payload.items.*item filter (i) -> i.@fragile == "true"
```

OUTPUT

```
[{"sku@":"A","fragile@":"true"}]
```

Section 8 complete — `filter` combines with attribute selectors exactly as it does with any other condition.

SECTION 9 — Combining Selectors With sizeOf, isEmpty & typeOf (Problems 81–90)

#81 Count an Order's Items

Count how many items are in an order.

INPUT

```
{ "items": [ {"sku":"A"}, {"sku":"B"}, {"sku":"C"} ] }
```

SOLUTION

```
sizeOf(payload.items)
```

OUTPUT

```
3
```

`sizeOf` combines directly with any selector that resolves to an array.

#82 Check If an Order Has No Items

Determine whether an order's items array is empty.

INPUT

```
{ "items": [] }
```

SOLUTION

```
isEmpty(payload.items)
```

OUTPUT

```
true
```

`isEmpty` works on arrays, objects, and strings alike, always returning a Boolean.

#83 Get the Data Type of a Field

Determine the type of the `total` field before deciding how to process it.

INPUT

```
{ "total": 482.5 }
```

SOLUTION

```
typeOf(payload.total)
```

OUTPUT

```
Number
```

`typeOf` is useful when a script needs to branch its logic based on a field's actual type.

#84 Count Characters in a String Field

Count how many characters are in a customer's notes field.

INPUT

```
{ "notes": "Leave at back door" }
```

SOLUTION

```
sizeOf(payload.notes)
```

OUTPUT

```
19
```

`sizeOf` on a String returns its character count, not its byte length.

#85 Count an Object's Fields

Count how many fields exist on a customer object.

INPUT

```
{ "customer": { "firstName": "Maria", "lastName": "Chen", "email": "m@x.com" }
}
```

SOLUTION

```
sizeOf(payload.customer)
```

OUTPUT

```
3
```

On an Object, `sizeOf` counts the number of key-value pairs.

#86 Guard Against an Empty Array Before Indexing

Safely get the first item's SKU, or "NONE" if the items array is empty.

INPUT

```
{ "items": [] }
```

SOLUTION

```
if (isEmpty(payload.items)) "NONE" else payload.items[0].sku
```

OUTPUT

```
"NONE"
```

Checking `isEmpty` before indexing avoids selecting from an array that has nothing at position 0.

#87 Check the Type Before Coercing

Only coerce a field to Number if it's currently a String.

INPUT

```
{ "qty": "4" }
```

SOLUTION

```
if (typeOf(payload.qty) == "String") payload.qty as Number else payload.qty
```

OUTPUT

```
4
```

This defensive pattern is common when a field's type is inconsistent across different source systems.

#88 Count Non-Empty Notes Across Items

Count how many items have a non-empty `notes` field.

INPUT

```
{ "items": [ {"notes":"Fragile"}, {"notes":""}, {"notes":"Rush"} ] }
```

SOLUTION

```
sizeOf(payload.items filter (i) -> not isEmpty(i.notes))
```

OUTPUT

```
2
```

Combining `filter`, `isEmpty`, and `sizeOf` answers a common "how many records have X" question in one line.

#89 Check if a Field Is Null vs. Missing vs. Empty

Build a diagnostic object showing three different "no value" states for the same field name.

INPUT

```
{ "a": null, "b": "" }
```

(FIELD "C" DOESN'T EXIST)

SOLUTION

```
{
  aIsNull: payload.a == null,
  bIsEmpty: isEmpty(payload.b),
  cIsMissing: payload.c == null
}
```

OUTPUT

```
{ "aIsNull": true, "bIsEmpty": true, "cIsMissing": true }
```

A genuinely `null` value and a missing field both compare equal to `null` — DataWeave doesn't distinguish "absent" from "explicitly null."

#90 Validate Every Item Has a Positive Quantity

Check whether every single item in an order has a quantity greater than zero.

INPUT

```
{ "items": [ {"qty":4}, {"qty":2}, {"qty":1} ] }
```

SOLUTION

```
payload.items every (item) -> item.qty > 0
```

OUTPUT

```
true
```

Section 9 complete — `every` (the counterpart to `some` from Chapter 18) checks that all elements satisfy a condition.

SECTION 10 — Mixed Real-World Selector Challenges (Problems 91–100)

#91 Build a Customer Summary From Scattered Fields

Combine the customer's name, the order ID, and the first item's SKU into one summary object.

INPUT

```
{ "orderId":"ORD-1042", "customer":{"firstName":"Maria","lastName":"Chen"},
"items":[{"sku":"PALLET-01"}] }
```

SOLUTION

```
{
  summary: payload.customer.firstName ++ " " ++ payload.customer.lastName
    ++ " ordered " ++ payload.items[0].sku ++ " (" ++ payload.orderId ++ ")"
}
```

OUTPUT

```
{ "summary": "Maria Chen ordered PALLET-01 (ORD-1042)" }
```

Real-world scripts routinely combine several unrelated selectors into a single human-readable output.

#92 Safely Report a Shipment's Latest Status

Return the most recent status update's text, or "No updates yet" if the array is empty.

INPUT

```
{ "updates": [] }
```

SOLUTION

```
if (isEmpty(payload.updates)) "No updates yet" else payload.updates[-1].text
```

OUTPUT

```
"No updates yet"
```

Combines an emptiness guard with negative indexing — a very common real-world defensive pattern.

#93 Extract and Total All Fees Across Nested Groups

Sum every fee amount scattered across a payload's shipping and handling groups.

INPUT

```
{ "shipping":{"fee":12.5}, "handling":{"fees":[{"fee":2},{"fee":1.5}]} }
```

SOLUTION

```
sum(payload..fee)
```

OUTPUT

```
16
```

Descendants plus `sum` handles inconsistent nesting shapes without needing to know the exact structure in advance.

#94 Pick Out Only Rush Items

Return the SKUs of every item with quantity greater than 3, defaulting missing quantities to 0.

INPUT

```
{ "items": [ {"sku":"A","qty":5}, {"sku":"B"}, {"sku":"C","qty":4} ] }
```

SOLUTION

```
(payload.items filter (i) -> (i.qty default 0) > 3) map (i) -> i.sku
```

OUTPUT

```
["A", "C"]
```

Chaining `filter` then `map`, with a `default` guarding the missing field, is a common three-part real-world pattern.

#95 Build a Compact Line-Item Report

Build an array of short strings like "PALLET-01 x4" for every item.

INPUT

```
{ "items": [ {"sku":"PALLET-01","qty":4}, {"sku":"CRATE-07","qty":2} ] }
```

SOLUTION

```
payload.items map (i) -> i.sku ++ " x" ++ (i.qty as String)
```

OUTPUT

```
["PALLET-01 x4", "CRATE-07 x2"]
```

Coercing a Number to String with `as String` is required before concatenating it with ++.

#96 Find the Highest-Priced Item

Return the SKU of the single most expensive item in an order.

INPUT

```
{ "items": [ {"sku":"A","price":50}, {"sku":"B","price":120},
{"sku":"C","price":90} ] }
```

SOLUTION

```
(payload.items orderBy (i) -> -i.price)[0].sku
```

OUTPUT

```
"B"
```

Sorting by the negated price puts the highest value first, then `[0]` grabs it — a compact way to avoid a separate max function.

#97 Check Whether Any Item Is Out of Stock

Determine whether any item in the order has a quantity of 0, defaulting missing quantities to 0 as well.

INPUT

```
{ "items": [ {"sku":"A","qty":4}, {"sku":"B","qty":0} ] }
```

SOLUTION

```
payload.items some (i) -> (i.qty default 0) == 0
```

OUTPUT

```
true
```

A defensive default plus `some` answers a real business question in a single, readable line.

#98 Extract a Nested Value From an Array of Arrays

Get the SKU of the second item in the first sub-order of a multi-order batch.

INPUT

```
{ "orders": [ { "items": [ {"sku":"A"}, {"sku":"B"} ] } ] }
```

SOLUTION

```
payload.orders[0].items[1].sku
```

OUTPUT

```
"B"
```

Chained index selectors reach into arbitrarily nested arrays, one index at a time.

#99 Build a Null-Safe Full Address String

Combine street, city, and an optional apartment number into one address string, safely handling the missing apartment.

INPUT

```
{ "address": { "street": "500 Main St", "city": "Denver" } }
```

SOLUTION

```
payload.address.street
  ++ (if (payload.address.apt != null) (" Apt " ++ payload.address.apt) else
"")
  ++ ", " ++ payload.address.city
```

OUTPUT

```
"500 Main St, Denver"
```

Real address-formatting logic almost always needs this kind of conditional piece to avoid printing "Apt null."

#100 Full CargoLink Order Summary

Bringing it all together: build a complete summary combining selectors, defaults, map, and sum from the full CargoLink order payload used throughout the DataWeave Handbook.

INPUT

```
{ "orderId":"ORD-1042", "customer":{"firstName":"Maria","lastName":"Chen"},
"items":[{"sku":"PALLET-01","qty":4,"price":120.50},{"sku":"CRATE-
07","qty":2,"price":45.00}], "shippingCountry":"US" }
```

SOLUTION

```
{
  orderId: payload.orderId,
  customerName: payload.customer.firstName ++ " " ++
payload.customer.lastName,
  itemCount: sizeOf(payload.items),
  skus: payload.items map (i) -> i.sku,
  orderTotal: sum(payload.items map (i) -> i.qty * i.price),
  isDomestic: (payload.shippingCountry default "") == "US"
}
```

OUTPUT

```
{ "orderId":"ORD-1042", "customerName":"Maria Chen", "itemCount":2, "skus":
["PALLET-01","CRATE-07"], "orderTotal":572.0, "isDomestic":true }
```

Volume 1 complete — every technique from all ten sections comes together in this one closing, production-shaped script.

END OF VOLUME 1

100 Down, 900 to Go

You've worked through 100 problems on selectors and basic data access — dot selectors, indexes, ranges, descendants, safe navigation, multi-value selectors, and XML attributes — the foundation every later volume builds on.

Volume 2 moves to Data Types & Type Coercion: another 100 problems on precisely converting and validating values as they move between systems.

MULESOFT · PRACTICE PROBLEMS · WITH FULL SOLUTIONS

1000 DataWeave Problems With Solutions

Volume 2 — Data Types & Type Coercion. 100 hands-on problems on precisely converting and validating values as they move between systems.

Problems 101–200

Volume 2 of 10

2026 EDITION

How to Use This Volume

Volume 2 focuses entirely on **type coercion** — the `as` operator, format options, and the discipline of knowing exactly what type a value is before working with it. Same format as Volume 1:

- **A short problem statement**, the **input** payload, a complete **solution** in a black code window, the exact **output**, and a one-line explanation.
- Ten sections of ten problems each, moving from basic String/Number coercion through Dates, DateTimes, type checks, and mixed real-world validation challenges.

Try each problem yourself before checking the solution — coercion bugs are some of the most common real-world DataWeave errors, and this volume is built to make them second nature to catch.

VOLUME 2

Data Types & Type Coercion

100 problems on the `as` operator, number and date formats, type checks, and the null-safety patterns that keep coercion from becoming a source of production bugs.

SECTION 1 — String ↔ Number Coercion (Problems 101–110)

#101 Coerce a String Quantity to Number

Convert a quantity received as a String into a real Number.

INPUT

```
{ "qty": "4" }
```

SOLUTION

```
payload.qty as Number
```

OUTPUT

```
4
```

The `as Number` coercion parses a numeric String into a real Number, ready for arithmetic.

#102 Coerce a Number to String for Concatenation

Build an order label by combining a String prefix with a numeric order ID.

INPUT

```
{ "orderId": 1042 }
```

SOLUTION

```
"ORD-" ++ (payload.orderId as String)
```

OUTPUT

```
"ORD-1042"
```

The ++ operator requires both sides to be the same kind of value, so the Number must be coerced first.

#103 Coerce a Decimal String to Number

Convert a price received as a String into a Number for calculation.

INPUT

```
{ "price": "120.50" }
```

SOLUTION

```
payload.price as Number
```

OUTPUT

```
120.5
```

DataWeave has a single Number type, so a decimal String coerces the same way an integer String does.

#104 Coerce a Negative Number String

Convert a negative adjustment amount, received as a String, into a Number.

INPUT

```
{ "adjustment": "-15.75" }
```

SOLUTION

```
payload.adjustment as Number
```

OUTPUT

```
-15.75
```

A leading minus sign is parsed correctly as part of the numeric value.

#105 Sum Two String Quantities After Coercion

Add two quantities that both arrived as Strings.

INPUT

```
{ "a": "10", "b": "5" }
```

SOLUTION

```
(payload.a as Number) + (payload.b as Number)
```

OUTPUT

```
15
```

Without coercion, + on two Strings would fail — both operands must be Numbers first.

#106 Coerce a String With Leading Zeros

Convert a zero-padded code to a Number and observe what happens to the padding.

INPUT

```
{ "code": "007" }
```

SOLUTION

```
payload.code as Number
```

OUTPUT

```
7
```

Coercing to Number drops insignificant leading zeros — if the padding matters, keep the field as a String instead.

#107 Coerce Every Element of an Array of Number Strings

Convert an array of numeric Strings into an array of real Numbers.

INPUT

```
{ "scores": ["10", "20", "30"] }
```

SOLUTION

```
payload.scores map (s) -> s as Number
```

OUTPUT

```
[10, 20, 30]
```

`as Number` applies to one value at a time, so it's combined with `map` to coerce every element.

#108 Round-Trip a Number Through String

Convert a Number to String and back to Number, confirming the value survives the round trip.

INPUT

```
{ "value": 42 }
```

SOLUTION

```
(payload.value as String) as Number
```

OUTPUT

```
42
```

Useful to confirm when a value must pass through a text-only system field without losing its numeric meaning.

#109 Coerce a Currency-Formatted String to Number

Convert a currency-formatted amount String, including a dollar sign and thousands separator, into a plain Number.

INPUT

```
{ "amount": "$1,234.50" }
```

SOLUTION

```
payload.amount as Number {format: "$#,##0.00"}
```

OUTPUT

```
1234.5
```

The `format` option tells DataWeave exactly how to interpret the currency symbol and separators while parsing.

#110 Multiply Two Coerced String Fields

Compute a line total from a quantity and a price, both received as Strings.

INPUT

```
{ "qtyStr": "3", "priceStr": "45.00" }
```

SOLUTION

```
(payload.qtyStr as Number) * (payload.priceStr as Number)
```

OUTPUT

```
135
```

Section 1 complete — coercing each operand individually before an arithmetic operation is the standard, safe pattern.

SECTION 2 — String ↔ Boolean Coercion (Problems 111–120)

#111 Coerce a String Flag to Boolean

Convert a "true"/"false" String flag into a real Boolean.

INPUT

```
{ "isActive": "true" }
```

SOLUTION

```
payload.isActive as Boolean
```

OUTPUT

```
true
```

A String reading exactly "true" or "false" coerces directly to the matching Boolean.

#112 Coerce a Boolean to String

Convert a Boolean field into its text representation for a log message.

INPUT

```
{ "isRush": true }
```

SOLUTION

```
"Rush: " ++ (payload.isRush as String)
```

OUTPUT

```
"Rush: true"
```

Just like Numbers, Booleans must be coerced to String before use with the ++ concatenation operator.

#113 Compare a Coerced Boolean in a Condition

Use a String flag directly inside an if/else after coercing it to Boolean.

INPUT

```
{ "isVerified": "false" }
```

SOLUTION

```
if (payload.isVerified as Boolean) "Approved" else "Pending Review"
```

OUTPUT

```
"Pending Review"
```

Coercing first keeps the `if` condition a real Boolean rather than a truthy/falsy String comparison.

#114 Coerce a Yes/No Field With a Manual Check

Convert a "Y"/"N" style field to Boolean, since `as Boolean` only understands "true"/"false".

INPUT

```
{ "consent": "Y" }
```

SOLUTION

```
payload.consent == "Y"
```

OUTPUT

```
true
```

Non-standard Boolean text like "Y"/"N" needs an explicit comparison instead of relying on `as Boolean`.

#115 Coerce an Array of Flag Strings to Booleans

Convert every flag in an array of "true"/"false" Strings into real Booleans.

INPUT

```
{ "flags": ["true", "false", "true"] }
```

SOLUTION

```
payload.flags map (f) -> f as Boolean
```

OUTPUT

```
[true, false, true]
```

The same coercion-inside-`map` pattern from Section 1 applies to Booleans just as it does to Numbers.

#116 Count How Many Flags Are True

Count how many "true" String flags exist in an array, after coercion.

INPUT

```
{ "flags": ["true", "false", "true", "true"] }
```

SOLUTION

```
sizeOf(payload.flags filter (f) -> f as Boolean)
```

OUTPUT

```
3
```

Coercion, `filter`, and `sizeOf` combine into a compact "count how many are true" pattern.

#117 Coerce a Number to a Boolean-Style Check

Determine whether a stock count field indicates "in stock" using a comparison rather than direct coercion.

INPUT

```
{ "stockCount": 0 }
```

SOLUTION

```
payload.stockCount > 0
```

OUTPUT

```
false
```

DataWeave does not coerce Numbers directly to Boolean — express the intent with an explicit comparison instead.

#118 Build a Human-Readable Flag Summary

Convert a Boolean into a friendlier "Yes"/"No" String than a raw coercion would give.

INPUT

```
{ "isPriority": true }
```

SOLUTION

```
if (payload.isPriority) "Yes" else "No"
```

OUTPUT

```
"Yes"
```

For display purposes, an explicit `if` often reads better to end users than the literal "true"/"false" from `as String`.

#119 Validate an Uppercase/Lowercase Boolean String

Safely coerce a Boolean String regardless of its case, since `as Boolean` expects lowercase.

INPUT

```
{ "flag": "TRUE" }
```

SOLUTION

```
lower(payload.flag) as Boolean
```

OUTPUT

```
true
```

Normalizing the String's case with `lower` before coercing avoids a subtle bug with inconsistently-cased source data.

#120 Build a Status Object Combining Both Coercions

Build a small status object combining a coerced Boolean and its String representation.

INPUT

```
{ "isDelivered": "true" }
```

SOLUTION

```
{
  delivered: payload.isDelivered as Boolean,
  label: if (payload.isDelivered as Boolean) "Delivered" else "In Transit"
}
```

OUTPUT

```
{ "delivered": true, "label": "Delivered" }
```

Section 2 complete — Boolean coercion is most useful right before a conditional, exactly as shown here.

SECTION 3 — Number Formatting With as String (Problems 121–130)

#121 Format a Number as Currency

Format an order total as a US-style currency String.

INPUT

```
{ "total": 482.5 }
```

SOLUTION

```
payload.total as String {format: "$#,##0.00"}
```

OUTPUT

```
"$482.50"
```

The format pattern's `0.00` forces exactly two decimal places, padding with a trailing zero if needed.

#122 Format a Large Number With Thousands Separators

Format a large inventory count with comma thousands separators.

INPUT

```
{ "unitsInStock": 1284392 }
```

SOLUTION

```
payload.unitsInStock as String {format: "#,##0"}
```

OUTPUT

```
"1,284,392"
```

The `#,##0` pattern inserts a comma every three digits with no decimal places.

#123 Zero-Pad a Numeric Code

Format a bin number as a zero-padded 3-digit code.

INPUT

```
{ "binNumber": 7 }
```

SOLUTION

```
payload.binNumber as String {format: "000"}
```

OUTPUT

```
"007"
```

Each `0` in the pattern forces a digit position, padding with leading zeros as needed.

#124 Format a Ratio as a Percentage

Format a decimal fill-rate value as a percentage String.

INPUT

```
{ "fillRate": 0.853 }
```

SOLUTION

```
payload.fillRate as String {format: "#0.0%"}
```

OUTPUT

```
"85.3%"
```

A `%` in the format pattern automatically multiplies the value by 100 and appends the percent sign.

#125 Round a Number to Two Decimal Places as a String

Format a computed average to exactly two decimal places.

INPUT

```
{ "average": 12.34567 }
```

SOLUTION

```
payload.average as String {format: "#0.00"}
```

OUTPUT

```
"12.35"
```

Formatting to a String not only displays but also rounds the value to the pattern's decimal precision.

#126 Format Without Trailing Decimal Noise

Format a whole number total so it doesn't show unnecessary decimal places.

INPUT

```
{ "total": 200.0 }
```

SOLUTION

```
payload.total as String {format: "#,##0.##"}
```

OUTPUT

```
"200"
```

The `#` symbol (versus `0`) means "show this digit only if present," dropping insignificant trailing zeros.

#127 Round Numbers Using round() Before Formatting

Round a price to the nearest whole number, then format it as a plain integer String.

INPUT

```
{ "price": 19.6 }
```

SOLUTION

```
round(payload.price) as String
```

OUTPUT

```
"20"
```

`round()` is a Number function, applied before coercion, rather than something a format pattern controls.

#128 Format a Negative Number as Currency

Format a negative refund amount using the same currency pattern as a positive value.

INPUT

```
{ "refund": -45.0 }
```

SOLUTION

```
payload.refund as String {format: "$#,##0.00"}
```

OUTPUT

```
"-$45.00"
```

The sign is preserved automatically, placed before the currency symbol.

#129 Format an Order Total Inside an Object

Build a receipt object where the total is a nicely formatted currency String, not a raw Number.

INPUT

```
{ "orderId": "ORD-9", "total": 1042.4 }
```

SOLUTION

```
{
  orderId: payload.orderId,
  totalFormatted: payload.total as String {format: "$#,##0.00"}
}
```

OUTPUT

```
{ "orderId": "ORD-9", "totalFormatted": "$1,042.40" }
```

Formatted display values are typically computed right where they're used, keeping the raw Number available elsewhere if still needed.

#130 Format Every Item's Price in an Array

Format every item's price as currency across an entire array.

INPUT

```
{ "items": [ {"price":120.5}, {"price":45} ] }
```

SOLUTION

```
payload.items map (i) -> i.price as String {format: "$#,##0.00"}
```

OUTPUT

```
["$120.50", "$45.00"]
```

Section 3 complete — formatting inside `map` applies the same pattern consistently across every element.

SECTION 4 — String → Date Coercion (Custom Formats) (Problems 131–140)

#131 Coerce an ISO Date String

Convert a standard ISO-format date String into a real Date value.

INPUT

```
{ "orderDate": "2026-03-15" }
```

SOLUTION

```
payload.orderDate as Date
```

OUTPUT

```
|2026-03-15|
```

ISO format (`yyyy-MM-dd`) is DataWeave's default, so no explicit `format` option is needed.

#132 Coerce a US-Style Date String

Convert a US-style MM/dd/yyyy date String into a Date, with an explicit format.

INPUT

```
{ "orderDate": "03/15/2026" }
```

SOLUTION

```
payload.orderDate as Date {format: "MM/dd/yyyy"}
```

OUTPUT

```
|2026-03-15|
```

A non-ISO String always needs an explicit `format` describing exactly how it's written.

#133 Coerce a European-Style Date String

Convert a European-style dd-MM-yyyy date String into a Date.

INPUT

```
{ "deliveryDate": "15-03-2026" }
```

SOLUTION

```
payload.deliveryDate as Date {format: "dd-MM-yyyy"}
```

OUTPUT

```
|2026-03-15|
```

Without the correct format, "15-03-2026" could easily be misread as day 15 of a nonexistent month 03 in the wrong order.

#134 Coerce a Compact Date String

Convert a compact 8-digit date String (no separators) into a Date.

INPUT

```
{ "batchDate": "20260315" }
```

SOLUTION

```
payload.batchDate as Date {format: "yyyyMMdd"}
```

OUTPUT

```
|2026-03-15|
```

Legacy batch systems often use compact, separator-free date formats exactly like this one.

#135 Coerce a Full Month-Name Date String

Convert a written-out date String, like "March 15, 2026", into a Date.

INPUT

```
{ "eventDate": "March 15, 2026" }
```

SOLUTION

```
payload.eventDate as Date {format: "MMMM d, yyyy"}
```

OUTPUT

```
|2026-03-15|
```

Format patterns can spell out full month names with `MMMM`, matching however the source system writes dates.

#136 Try Coercing an Invalid Date Safely

Safely attempt to coerce a malformed date String without crashing the whole script.

INPUT

```
{ "orderDate": "not-a-date" }
```

SOLUTION

```
(try(() -> payload.orderDate as Date)).success
```

OUTPUT

```
false
```

Wrapping a risky coercion in `try` (from Chapter 17) lets a script detect and handle bad data gracefully.

#137 Coerce a Date From Only Month and Year

Convert a "MM-yyyy" String, representing a billing period, into a Date defaulting to the first of the month.

INPUT

```
{ "billingPeriod": "03-2026" }
```

SOLUTION

```
("01-" ++ payload.billingPeriod) as Date {format: "dd-MM-yyyy"}
```

OUTPUT

```
|2026-03-01|
```

Prepending a fixed day value is a common trick when a source field only carries month and year.

#138 Coerce Every Date String in an Array

Convert every date String in an array of shipment legs into real Date values.

INPUT

```
{ "legDates": ["03/01/2026", "03/04/2026"] }
```

SOLUTION

```
payload.legDates map (d) -> d as Date {format: "MM/dd/yyyy"}
```

OUTPUT

```
[|2026-03-01|, |2026-03-04|]
```

Date coercion combines with `map` exactly the same way Number and Boolean coercion did in earlier sections.

#139 Compare Two Coerced Dates

Determine whether a delivery date is after an order date, both received as differently-formatted Strings.

INPUT

```
{ "orderDate": "2026-03-01", "deliveryDate": "05-03-2026" }
```

SOLUTION

```
(payload.deliveryDate as Date {format: "dd-MM-yyyy"}) > (payload.orderDate as
Date)
```

OUTPUT

```
true
```

Once coerced to real Date values, comparison operators work correctly regardless of the original text formats.

#140 Guard a Date Coercion With a Format Check

Safely coerce a date field that might be in either of two known formats, trying the primary format first.

INPUT

```
{ "date": "2026-03-15" }
```

SOLUTION

```
do {
  var attempt = try(() -> payload.date as Date)
  ---
  if (attempt.success) attempt.result else payload.date as Date {format:
"dd/MM/yyyy"}
}
```

OUTPUT

```
|2026-03-15|
```

Section 4 complete — combining `try` with a fallback format handles source systems that aren't perfectly consistent.

SECTION 5 — Date → String Coercion (Reformatting) (Problems 141–150)

#141 Reformat a Date to US Style

Convert a parsed Date value into a US-style MM/dd/yyyy String.

INPUT

```
{ "orderDate": "2026-03-15" }
```

SOLUTION

```
(payload.orderDate as Date) as String {format: "MM/dd/yyyy"}
```

OUTPUT

```
"03/15/2026"
```

Parse first with `as Date`, then reformat with a second `as String` — the standard two-step reformatting pattern.

#142 Reformat a Date to Show the Day Name

Convert a Date into a String showing the full day-of-week name.

INPUT

```
{ "eventDate": "2026-03-15" }
```

SOLUTION

```
(payload.eventDate as Date) as String {format: "EEEE"}
```

OUTPUT

```
"Sunday"
```

The `EEEE` pattern token produces the full weekday name derived from the actual calendar date.

#143 Reformat a Date to a Written-Out Style

Convert a Date into a friendly, written-out String for a customer-facing message.

INPUT

```
{ "deliveryDate": "2026-03-15" }
```

SOLUTION

```
(payload.deliveryDate as Date) as String {format: "MMMM d, yyyy"}
```

OUTPUT

```
"March 15, 2026"
```

Customer-facing messages often reformat an internal ISO date into a more natural, readable style like this one.

#144 Convert Between Two Non-ISO Formats Directly

Convert a date String from European style directly to US style.

INPUT

```
{ "date": "15-03-2026" }
```

SOLUTION

```
(payload.date as Date {format: "dd-MM-yyyy"}) as String {format:
"MM/dd/yyyy"}
```

OUTPUT

```
"03/15/2026"
```

Parse with the source format, then output with the target format — DataWeave always routes through a real Date in between.

#145 Build a Compact Filename-Safe Date String

Format a Date as a compact, filename-safe String with no separators.

INPUT

```
{ "batchDate": "2026-03-15" }
```

SOLUTION

```
"batch_" ++ ((payload.batchDate as Date) as String {format: "yyyyMMdd"}) ++
".csv"
```

OUTPUT

```
"batch_20260315.csv"
```

Reformatted dates are frequently embedded directly into generated filenames, exactly as shown here.

#146 Reformat Every Date in an Array

Reformat every leg date in a shipment's route from ISO to US style.

INPUT

```
{ "legDates": ["2026-03-01", "2026-03-04"] }
```

SOLUTION

```
payload.legDates map (d) -> (d as Date) as String {format: "MM/dd/yyyy"}
```

OUTPUT

```
["03/01/2026", "03/04/2026"]
```

The two-step reformatting pattern combines with `map` exactly like every other coercion in this book.

#147 Extract Just the Year From a Date

Get just the four-digit year portion of a Date as a String.

INPUT

```
{ "orderDate": "2026-03-15" }
```

SOLUTION

```
(payload.orderDate as Date) as String {format: "yyyy"}
```

OUTPUT

```
"2026"
```

A format pattern can request just one component of a date, ignoring the rest.

#148 Build a Report Header With a Reformatted Date

Build a report title string embedding a friendly-formatted report date.

INPUT

```
{ "reportDate": "2026-03-15" }
```

SOLUTION

```
"Monthly Report — " ++ ((payload.reportDate as Date) as String {format: "MMMM yyyy"})
```

OUTPUT

```
"Monthly Report — March 2026"
```

Combining coercion, formatting, and concatenation in one expression is a very common real-world pattern.

#149 Reformat Only If the Field Is Present

Reformat an optional cancellation date, defaulting to "N/A" if the field is missing.

INPUT

```
{ "orderId": "ORD-1" }
```

(NO CANCELLATIONDATE FIELD)

SOLUTION

```
if (payload.cancellationDate == null) "N/A"
else (payload.cancellationDate as Date) as String {format: "MMMM d, yyyy"}
```

OUTPUT

```
"N/A"
```

Always guard an optional date field before coercing it — coercing `null` directly would throw an error.

#150 Full Date Reformat Report

Build a small object showing the same date in three different output formats at once.

INPUT

```
{ "orderDate": "2026-03-15" }
```

SOLUTION

```
do {
  var parsed = payload.orderDate as Date
  ---
  {
    iso: parsed as String {format: "yyyy-MM-dd"},
    us: parsed as String {format: "MM/dd/yyyy"},
    written: parsed as String {format: "MMMM d, yyyy"}
  }
}
```

OUTPUT

```
{ "iso": "2026-03-15", "us": "03/15/2026", "written": "March 15, 2026" }
```

Section 5 complete — parsing once into a local variable, then formatting it multiple ways, avoids repeating the coercion.

SECTION 6 — DateTime & Time Coercion (Problems 151–160)

#151 Coerce an ISO DateTime String

Convert a timestamped String into a real DateTime value.

INPUT

```
{ "createdAt": "2026-03-15T14:30:00Z" }
```

SOLUTION

```
payload.createdAt as DateTime
```

OUTPUT

```
|2026-03-15T14:30:00Z|
```

DateTime includes both the date and time, plus a timezone offset — `Z` denotes UTC.

#152 Extract Just the Date Portion of a DateTime

Get just the calendar Date from a full DateTime value.

INPUT

```
{ "createdAt": "2026-03-15T14:30:00Z" }
```

SOLUTION

```
(payload.createdAt as DateTime) as Date
```

OUTPUT

```
|2026-03-15|
```

Coercing a DateTime down to Date simply drops the time-of-day and timezone components.

#153 Extract Just the Time Portion of a DateTime

Get just the time-of-day from a full DateTime value.

INPUT

```
{ "createdAt": "2026-03-15T14:30:00Z" }
```

SOLUTION

```
(payload.createdAt as DateTime) as String {format: "HH:mm:ss"}
```

OUTPUT

```
"14:30:00"
```

`HH:mm:ss` is a 24-hour time pattern, extracting only the time-of-day portion as a String.

#154 Format a DateTime for Display

Format a DateTime as a friendly, combined date-and-time String.

INPUT

```
{ "createdAt": "2026-03-15T14:30:00Z" }
```

SOLUTION

```
(payload.createdAt as DateTime) as String {format: "MMMM d, yyyy 'at' HH:mm"}
```

OUTPUT

```
"March 15, 2026 at 14:30"
```

Literal text inside a format pattern (like `'at'`) is wrapped in single quotes so it isn't mistaken for a pattern token.

#155 Coerce a DateTime With a Non-UTC Offset

Convert a timestamp String with a specific timezone offset into a DateTime.

INPUT

```
{ "eventTime": "2026-03-15T09:00:00-05:00" }
```

SOLUTION

```
payload.eventTime as DateTime
```

OUTPUT

```
|2026-03-15T09:00:00-05:00|
```

A DateTime preserves its original offset — it does not automatically convert to UTC unless you explicitly ask it to.

#156 Compare Two DateTime Values

Determine whether an order was placed before a promotion's end time.

INPUT

```
{ "orderTime": "2026-03-15T10:00:00Z", "promoEnds": "2026-03-15T12:00:00Z" }
```

SOLUTION

```
(payload.orderTime as DateTime) < (payload.promoEnds as DateTime)
```

OUTPUT

```
true
```

DateTime values compare chronologically once coerced, exactly like Numbers compare numerically.

#157 Coerce a Standalone Time String

Convert a String representing only a time-of-day into a Time value.

INPUT

```
{ "pickupTime": "09:15:00" }
```

SOLUTION

```
payload.pickupTime as Time
```

OUTPUT

```
|09:15:00|
```

DataWeave's `Time` type holds a time-of-day with no associated calendar date.

#158 Combine a Date and a Time Into a DateTime

Combine a separate Date field and Time field into one combined DateTime String, then coerce it.

INPUT

```
{ "date": "2026-03-15", "time": "09:15:00" }
```

SOLUTION

```
(payload.date ++ "T" ++ payload.time ++ "Z") as DateTime
```

OUTPUT

```
|2026-03-15T09:15:00Z|
```

Building the ISO-shaped String first, then coercing once, is simpler than trying to coerce the two fields separately.

#159 Format a DateTime as a Log Timestamp

Format a DateTime into a precise, sortable timestamp String suitable for log entries.

INPUT

```
{ "eventTime": "2026-03-15T14:30:05Z" }
```

SOLUTION

```
(payload.eventTime as DateTime) as String {format: "yyyy-MM-dd'T'HH:mm:ss"}
```

OUTPUT

```
"2026-03-15T14:30:05"
```

A fixed, sortable timestamp format like this is standard for structured application logs.

#160 Build a Full Timestamp Report Object

Build an object showing an event's date, time, and full combined timestamp from one DateTime field.

INPUT

```
{ "eventTime": "2026-03-15T14:30:00Z" }
```

SOLUTION

```
do {
  var dt = payload.eventTime as DateTime
  ---
  {
    date: dt as String {format: "yyyy-MM-dd"},
    time: dt as String {format: "HH:mm:ss"},
    full: dt as String {format: "MMMM d, yyyy 'at' HH:mm"}
  }
}
```

OUTPUT

```
{ "date": "2026-03-15", "time": "14:30:00", "full": "March 15, 2026 at 14:30"
}
```

Section 6 complete — parsing a DateTime once and reformatting it multiple ways avoids repeated, redundant coercion.

SECTION 7 — typeOf & Type Checks (Problems 161–170)

#161 Get the Type of a String Field

Check the type of a customer's name field.

INPUT

```
{ "name": "Maria Chen" }
```

SOLUTION

```
typeOf(payload.name)
```

OUTPUT

```
String
```

`typeOf` returns one of DataWeave's core type names, useful for branching logic on unpredictable input.

#162 Get the Type of an Array Field

Check the type of an order's items field.

INPUT

```
{ "items": [ {"sku":"A"} ] }
```

SOLUTION

```
typeOf(payload.items)
```

OUTPUT

```
Array
```

Confirming a field is genuinely an Array before calling array functions on it prevents a whole class of runtime errors.

#163 Check if a Field Is Null Using typeOf

Determine whether a discount field is Null.

INPUT

```
{ "discount": null }
```

SOLUTION

```
typeOf(payload.discount) == "Null"
```

OUTPUT

```
true
```

Comparing against the exact type name "Null" is one way to explicitly check for null, alongside `== null`.

#164 Branch Logic Based on a Field's Type

Handle a quantity field that might arrive as either a Number or a String, normalizing both to Number.

INPUT

```
{ "qty": "4" }
```

SOLUTION

```
if (typeOf(payload.qty) == "String") payload.qty as Number else payload.qty
```

OUTPUT

```
4
```

This type-check-then-coerce pattern is common when integrating with source systems that aren't type-consistent.

#165 Match on Type Using match

Use a `match` expression to describe a value's type in one word.

INPUT

```
{ "value": 42 }
```

SOLUTION

```
payload.value match {
  case is Number -> "numeric"
  case is String -> "text"
  case is Boolean -> "flag"
  else -> "other"
}
```

OUTPUT

```
"numeric"
```

The `match` expression's `is Type` pattern (Chapter 13) is often clearer than several chained `typeOf` comparisons.

#166 Validate That a Field Is an Object Before mapObject

Guard a `mapObject` call so it only runs if the field is genuinely an Object.

INPUT

```
{ "customer": { "firstName": "Maria" } }
```

SOLUTION

```
if (typeOf(payload.customer) == "Object")
  payload.customer mapObject (v, k) -> { (upper(k)): v }
else {}
```

OUTPUT

```
{ "FIRSTNAME": "Maria" }
```

Guarding before calling a type-specific function avoids a runtime error if the field ever arrives in an unexpected shape.

#167 Check Every Item's Type Consistency

Confirm every element in an array is a Number before summing them.

INPUT

```
{ "scores": [10, 20, 30] }
```

SOLUTION

```
payload.scores every (s) -> typeOf(s) == "Number"
```

OUTPUT

```
true
```

Validating type consistency across an entire array before an aggregate operation is a solid defensive habit.

#168 Find Elements With an Unexpected Type

Find every element in a supposedly-numeric array that is actually a String.

INPUT

```
{ "scores": [10, "20", 30] }
```

SOLUTION

```
payload.scores filter (s) -> typeOf(s) == "String"
```

OUTPUT

```
["20"]
```

Filtering by type is a quick way to isolate exactly the "bad" records in an otherwise-consistent dataset.

#169 Report a Field's Type in a Diagnostic Object

Build a small diagnostic object reporting the type of every top-level field, for debugging an unfamiliar payload.

INPUT

```
{ "id": "ORD-1", "total": 200, "active": true }
```

SOLUTION

```
payload mapObject (value, key) -> { (key): typeOf(value) as String }
```

OUTPUT

```
{ "id": "String", "total": "Number", "active": "Boolean" }
```

This exact pattern is a handy quick script for exploring an unfamiliar payload's shape during debugging (Chapter 22).

#170 Safely Coerce Only When Needed

Write a reusable pattern that coerces a field to Number only if it isn't already one.

INPUT

```
{ "qty": 4 }
```

(ALREADY A NUMBER)

SOLUTION

```
if (typeOf(payload.qty) == "Number") payload.qty else payload.qty as Number
```

OUTPUT

```
4
```

Section 7 complete — this guard avoids an unnecessary (and occasionally invalid) coercion attempt on data that's already correct.

SECTION 8 — Coercion Inside map/filter Over Arrays (Problems 171–180)

#171 Coerce and Sum Mixed-Type Quantities

Sum a list of quantities where some arrived as Numbers and some as Strings.

INPUT

```
{ "quantities": [4, "2", 5, "1"] }
```

SOLUTION

```
sum(payload.quantities map (q) -> q as Number)
```

OUTPUT

```
12
```

Coercing every element to a consistent type before `sum` avoids a type-mismatch failure partway through.

#172 Filter an Array by a Coerced Numeric Threshold

From an array of price Strings, keep only those over $100 after coercion.

INPUT

```
{ "prices": ["45.00", "120.50", "89.99", "200.00"] }
```

SOLUTION

```
payload.prices filter (p) -> (p as Number) > 100
```

OUTPUT

```
["120.50", "200.00"]
```

The coercion happens inside the filter condition; the original String values are still what's returned.

#173 Coerce and Format in One Pass

Coerce an array of price Strings to Numbers, then reformat each as clean currency.

INPUT

```
{ "prices": ["45", "120.5"] }
```

SOLUTION

```
payload.prices map (p) -> (p as Number) as String {format: "$#,##0.00"}
```

OUTPUT

```
["$45.00", "$120.50"]
```

Coercing to Number first, then immediately back to a formatted String, normalizes inconsistent input into a clean display format.

#174 Sort an Array of Numeric Strings Correctly

Sort an array of numeric Strings in true numeric order, not alphabetic order.

INPUT

```
{ "scores": ["9", "10", "2"] }
```

SOLUTION

```
payload.scores orderBy (s) -> s as Number
```

OUTPUT

```
["2", "9", "10"]
```

Without coercion, alphabetic sorting would incorrectly place "10" before "2" — `orderBy` uses the coerced key, but returns the original values.

#175 Coerce Nested Object Fields Inside map

Coerce each item's `qty` field from String to Number, keeping the rest of the object unchanged.

INPUT

```
{ "items": [ {"sku":"A","qty":"4"}, {"sku":"B","qty":"2"} ] }
```

SOLUTION

```
payload.items map (item) -> item ++ { qty: item.qty as Number }
```

OUTPUT

```
[ {"sku":"A","qty":4}, {"sku":"B","qty":2} ]
```

The `++` merge overrides just the `qty` field with its coerced value, leaving `sku` untouched.

#176 Coerce Multiple Fields Inside One map

Coerce both `qty` and `price` from String to Number in every item.

INPUT

```
{ "items": [ {"sku":"A","qty":"4","price":"120.50"} ] }
```

SOLUTION

```
payload.items map (item) -> item ++ {
  qty: item.qty as Number,
  price: item.price as Number
}
```

OUTPUT

```
[ {"sku":"A","qty":4,"price":120.5} ]
```

Multiple fields can be coerced together in a single merge object, keeping the transformation concise.

#177 Compute a Total From Coerced Nested Fields

Compute an order's grand total from items whose `qty` and `price` both arrived as Strings.

INPUT

```
{ "items": [ {"qty":"4","price":"120.50"}, {"qty":"2","price":"45.00"} ] }
```

SOLUTION

```
sum(payload.items map (i) -> (i.qty as Number) * (i.price as Number))
```

OUTPUT

```
572.0
```

This is the exact real-world situation the CargoLink order total calculation assumes clean Number types are already in place.

#178 Coerce Dates Inside a map for a Timeline

Coerce and reformat every leg's date in a shipment timeline in one pass.

INPUT

```
{ "legs": [ {"city":"Reno","date":"2026-03-01"}, {"city":"Ogden","date":"2026-03-03"} ] }
```

SOLUTION

```
payload.legs map (leg) -> leg ++ {
  date: (leg.date as Date) as String {format: "MMM d"}
}
```

OUTPUT

```
[ {"city":"Reno","date":"Mar 1"}, {"city":"Ogden","date":"Mar 3"} ]
```

Reformatting a date field in place, inside a merge, keeps every other field on the object unaffected.

#179 Filter and Coerce Together in a Real Report

From a list of transactions with String amounts, keep only negative ones (refunds) as coerced Numbers.

INPUT

```
{ "transactions": [ {"amount":"50"}, {"amount":"-20"}, {"amount":"-5.5"} ] }
```

SOLUTION

```
(payload.transactions map (t) -> t.amount as Number) filter (a) -> a < 0
```

OUTPUT

```
[-20, -5.5]
```

Mapping to coerce, then filtering the coerced result, keeps each step of the transformation simple and readable.

#180 Build a Fully Type-Normalized Item List

Normalize an entire items array so every `qty`, `price`, and `inStock` field has a consistent, correct type.

INPUT

```
{ "items": [ {"sku":"A","qty":"4","price":"120.50","inStock":"true"} ] }
```

SOLUTION

```
payload.items map (i) -> i ++ {
  qty: i.qty as Number,
  price: i.price as Number,
  inStock: i.inStock as Boolean
}
```

OUTPUT

```
[ {"sku":"A","qty":4,"price":120.5,"inStock":true} ]
```

Section 8 complete — this "normalize on the way in" pattern is one of the most common real jobs a DataWeave script performs.

SECTION 9 — Coercion With default & Error-Safe Patterns (Problems 181–190)

#181 Default Before Coercing a Missing Number

Coerce a possibly-missing discount field to Number, defaulting first to avoid coercing null.

INPUT

```
{ "orderId": "ORD-1" }
```

(NO DISCOUNT FIELD)

SOLUTION

```
(payload.discount default "0") as Number
```

OUTPUT

```
0
```

Applying `default` before `as Number` ensures there's always a valid String to coerce, avoiding a coercion error on `null`.

#182 Safely Coerce a Field That Might Be Malformed

Attempt to coerce a price field that might contain garbage data, falling back to 0 on failure.

INPUT

```
{ "price": "N/A" }
```

SOLUTION

```
do {
  var attempt = try(() -> payload.price as Number)
  ---
  if (attempt.success) attempt.result else 0
}
```

OUTPUT

```
0
```

Unlike `default` (which only handles `null`), `try` catches a coercion that fails because the data is simply invalid.

#183 Write a Reusable Safe-Coercion Function

Write a custom function that safely coerces any String to Number, with a caller-provided fallback.

INPUT

```
{ "value": "abc" }
```

SOLUTION

```
fun safeNumber(value: String, fallback: Number): Number =
  do {
    var attempt = try(() -> value as Number)
    ---
    if (attempt.success) attempt.result else fallback
  }
---
safeNumber(payload.value, -1)
```

OUTPUT

```
-1
```

Wrapping this pattern in a reusable function (Chapter 12) avoids repeating the same `try`/`if` logic everywhere it's needed.

#184 Validate Every Item Before Coercing the Whole Array

Check whether every quantity String in an array is safely coercible before processing the array further.

INPUT

```
{ "quantities": ["4", "2", "bad"] }
```

SOLUTION

```
payload.quantities every (q) -> (try(() -> q as Number)).success
```

OUTPUT

```
false
```

A validation pass like this before processing catches bad records early, rather than failing partway through a larger transformation.

#185 Separate Valid and Invalid Records

Split an array of items into those whose price coerces safely, and those that don't.

INPUT

```
{ "items": [ {"sku":"A","price":"120.50"}, {"sku":"B","price":"bad"} ] }
```

SOLUTION

```
{
  valid: payload.items filter (i) -> (try(() -> i.price as Number)).success,
  invalid: payload.items filter (i) -> not (try(() -> i.price as
Number)).success
}
```

OUTPUT

```
{ "valid": [{"sku":"A","price":"120.50"}], "invalid":
[{"sku":"B","price":"bad"}] }
```

Separating bad records instead of letting them silently fail the whole batch is a strong real-world integration habit.

#186 Default a Boolean Coercion Safely

Safely coerce a flag field to Boolean, defaulting to `false` if the field is missing or unparseable.

INPUT

```
{ "orderId": "ORD-1" }
```

(NO ISRUSH FIELD)

SOLUTION

```
(payload.isRush default "false") as Boolean
```

OUTPUT

```
false
```

Defaulting to the String "`false`" (not the Boolean `false`) keeps the value valid for the subsequent `as Boolean` coercion.

#187 Safely Coerce a Date With a Clear Fallback

Safely coerce an optional promised-delivery date, falling back to `null` rather than crashing the script.

INPUT

```
{ "promisedDate": "not-available" }
```

SOLUTION

```
(try(() -> payload.promisedDate as Date)).result default null
```

OUTPUT

```
null
```

Even the fallback of "just return null safely" is meaningfully better than an unhandled coercion error stopping the whole flow.

#188 Report Why a Coercion Failed

Return the error message from a failed coercion attempt, for logging purposes.

INPUT

```
{ "price": "twelve dollars" }
```

SOLUTION

```
do {
  var attempt = try(() -> payload.price as Number)
  ---
  if (attempt.success) "OK" else attempt.error.message
}
```

OUTPUT

```
"Cannot coerce a String to a Number..." (exact wording varies by runtime)
```

`attempt.error.message` *gives a human-readable reason, useful when logging exactly why a record failed validation.*

#189 Count How Many Records Failed Coercion

Count how many items in a batch have a price field that fails to coerce.

INPUT

```
{ "items": [ {"price":"10"}, {"price":"bad"}, {"price":"also-bad"} ] }
```

SOLUTION

```
sizeOf(payload.items filter (i) -> not (try(() -> i.price as
Number)).success)
```

OUTPUT

```
2
```

A quick count like this is often logged or reported as a data-quality metric for an incoming batch.

#190 Build a Full Validation Report

Build a summary object reporting total records, valid records, and invalid records for a batch of quantities.

INPUT

```
{ "quantities": ["4", "2", "bad", "5"] }
```

SOLUTION

```
do {
  var results = payload.quantities map (q) -> (try(() -> q as
Number)).success
  ---
  {
    total: sizeOf(payload.quantities),
    valid: sizeOf(results filter (r) -> r),
    invalid: sizeOf(results filter (r) -> not r)
  }
}
```

OUTPUT

```
{ "total": 4, "valid": 3, "invalid": 1 }
```

Section 9 complete — this validation-report pattern is a realistic, production-shaped use of everything `try` and coercion together enable.

SECTION 10 — Mixed Real-World Type Coercion Challenges (Problems 191–200)

#191 Normalize a Whole Order From Mixed Types

Normalize an order payload where quantities, prices, and a rush flag all arrived as Strings.

INPUT

```
{ "orderId":"ORD-9", "isRush":"true", "items":
[{"sku":"A","qty":"4","price":"120.50"}] }
```

SOLUTION

```
{
  orderId: payload.orderId,
  isRush: payload.isRush as Boolean,
  items: payload.items map (i) -> i ++ {
    qty: i.qty as Number,
    price: i.price as Number
  }
}
```

OUTPUT

```
{ "orderId":"ORD-9", "isRush":true, "items":
[{"sku":"A","qty":4,"price":120.5}] }
```

Real integrations frequently need exactly this kind of full-payload type normalization pass at the very start of a script.

#192 Build an Invoice Line From Mixed Sources

Build a formatted invoice line combining a coerced date, a formatted currency amount, and a Boolean-driven label.

INPUT

```
{ "date": "2026-03-15", "amount": "482.5", "isPaid": "true" }
```

SOLUTION

```
((payload.date as Date) as String {format: "MMM d, yyyy"}) ++ " — "
  ++ ((payload.amount as Number) as String {format: "$#,##0.00"}) ++ " — "
  ++ (if (payload.isPaid as Boolean) "PAID" else "DUE")
```

OUTPUT

```
"Mar 15, 2026 — $482.50 — PAID"
```

Every type covered in this volume — Date, Number, and Boolean — comes together in a single realistic report line.

#193 Validate and Total a Batch, Skipping Bad Records

Compute a total from a batch of transactions, safely skipping any whose amount fails to coerce.

INPUT

```
{ "transactions": [ {"amount":"50"}, {"amount":"bad"}, {"amount":"25.5"} ] }
```

SOLUTION

```
sum(
  (payload.transactions map (t) -> try(() -> t.amount as Number))
    filter (r) -> r.success
    map (r) -> r.result
)
```

OUTPUT

```
75.5
```

Mapping to attempts, filtering to successes, then mapping to results is a clean three-stage pipeline for a "sum the valid ones" requirement.

#194 Reformat an Entire Timeline for a Customer Email

Reformat a shipment's tracking events into friendly, human-readable timeline entries.

INPUT

```
{ "events": [ {"status":"Picked Up","time":"2026-03-15T08:00:00Z"},
{"status":"Delivered","time":"2026-03-17T14:00:00Z"} ] }
```

SOLUTION

```
payload.events map (e) -> e.status ++ " on "
  ++ ((e.time as DateTime) as String {format: "MMMM d 'at' HH:mm"})
```

OUTPUT

```
["Picked Up on March 15 at 08:00", "Delivered on March 17 at 14:00"]
```

This is the exact kind of transformation that turns a raw system timestamp into copy a customer would actually want to read.

#195 Coerce a Payload With Deeply Mixed Types

Normalize a customer profile where nested loyalty points and a verified flag both need coercion.

INPUT

```
{ "customer": { "name": "Maria Chen", "loyalty": { "points": "1250",
"verified": "true" } } }
```

SOLUTION

```
payload.customer ++ {
  loyalty: payload.customer.loyalty ++ {
    points: payload.customer.loyalty.points as Number,
    verified: payload.customer.loyalty.verified as Boolean
  }
}
```

OUTPUT

```
{ "name":"Maria Chen", "loyalty": { "points":1250, "verified":true } }
```

Nested merges, one level at a time, keep a multi-level coercion readable instead of one deeply tangled expression.

#196 Build a CSV-Ready Row From Mixed Types

Prepare an order for CSV output, where every field must already be a flat String or Number.

INPUT

```
{ "orderId": 1042, "total": 482.5, "isRush": true, "date": "2026-03-15" }
```

SOLUTION

```
{
  orderId: payload.orderId as String,
  total: payload.total as String {format: "0.00"},
  isRush: payload.isRush as String,
  date: (payload.date as Date) as String {format: "MM/dd/yyyy"}
}
```

OUTPUT

```
{ "orderId":"1042", "total":"482.50", "isRush":"true", "date":"03/15/2026" }
```

Converting everything to a clean, flat String representation is exactly what's needed right before CSV output (Chapter 5).

#197 Validate an Entire Payload's Types Before Processing

Check whether an order's quantity, price, and rush flag are all coercible before running the rest of the transformation.

INPUT

```
{ "qty":"4", "price":"bad-price", "isRush":"true" }
```

SOLUTION

```
[
  (try(() -> payload.qty as Number)).success,
  (try(() -> payload.price as Number)).success,
  (try(() -> payload.isRush as Boolean)).success
] every (r) -> r
```

OUTPUT

```
false
```

A pre-flight validation like this lets a script reject a malformed record cleanly, before any real processing begins.

#198 Build a Coercion-Safe Custom Function Library Call

Use a reusable `safeNumber` function (from Problem 183) across several fields in one order.

INPUT

```
{ "qty": "bad", "price": "120.50" }
```

SOLUTION

```
fun safeNumber(value: String, fallback: Number): Number =
  do { var a = try(() -> value as Number) --- if (a.success) a.result else
fallback }
---
{
  qty: safeNumber(payload.qty, 0),
  price: safeNumber(payload.price, 0)
}
```

OUTPUT

```
{ "qty": 0, "price": 120.5 }
```

A well-designed helper function pays for itself the moment it's reused across more than one field, exactly as shown here.

#199 Compare Records Across Two Different Type Shapes

Determine whether two representations of the same amount — one a Number, one a formatted String — actually match.

INPUT

```
{ "systemA": 482.5, "systemB": "$482.50" }
```

SOLUTION

```
payload.systemA == (payload.systemB as Number {format: "$#,##0.00"})
```

OUTPUT

```
true
```

Reconciling values from two systems almost always requires coercing both sides to the same type before comparing.

#200 Full CargoLink Order Type-Safety Pass

Bringing it all together: fully normalize and validate a CargoLink order, reporting both the clean data and any coercion failures.

INPUT

```
{ "orderId":"ORD-1042", "isRush":"true", "orderDate":"03/15/2026", "items":
[{"sku":"PALLET-01","qty":"4","price":"120.50"},{"sku":"CRATE-
07","qty":"bad","price":"45.00"}] }
```

SOLUTION

```
do {
  var coerced = payload.items map (i) -> {
    sku: i.sku,
    qty: try(() -> i.qty as Number),
    price: try(() -> i.price as Number)
  }
  ---
  {
    orderId: payload.orderId,
    isRush: payload.isRush as Boolean,
    orderDate: (payload.orderDate as Date {format: "MM/dd/yyyy"}) as String
{format: "yyyy-MM-dd"},
    validItems: sizeOf(coerced filter (i) -> i.qty.success and
i.price.success),
    invalidItems: sizeOf(coerced filter (i) -> not (i.qty.success and
i.price.success))
  }
}
```

OUTPUT

```
{ "orderId":"ORD-1042", "isRush":true, "orderDate":"2026-03-15",
"validItems":1, "invalidItems":1 }
```

Volume 2 complete — this closing script combines every coercion technique from all ten sections into one production-shaped validation pass.

END OF VOLUME 2

200 Down, 800 to Go

You've worked through 200 problems now — selectors and basic data access in Volume 1, and a complete tour of type coercion across Strings, Numbers, Booleans, Dates, DateTimes, and error-safe patterns in Volume 2.

Volume 3 moves to Operators & Conditional Logic: another 100 problems on arithmetic, comparisons, if/else, and the `match` expression.

MULESOFT · PRACTICE PROBLEMS · WITH FULL SOLUTIONS

1000 DataWeave Problems With Solutions

Volume 3 — Operators & Conditional Logic. 100 hands-on problems on arithmetic, comparisons, logical operators, if/else, match/case, and default.

Problems 201–300

Volume 3 of 10

2026 EDITION

How to Use This Volume

Volume 3 is about **decisions** — the operators and control structures DataWeave scripts use to compare values and branch logic. Same format as Volumes 1 and 2:

- **A short problem statement**, the **input** payload, a complete **solution** in a black code window, the exact **output**, and a one-line explanation.
- Ten sections of ten problems each, moving from arithmetic and comparison operators through logical operators, merge operators, `if/else`, `match/case`, `default`, and mixed real-world decision logic.

A quick correction before you start: an earlier volume in this series referenced an `unless` keyword as an inverted form of `if`. DataWeave 2.0 does not actually have an `unless` keyword — its only conditional expressions are `if/else` and `match/case`. This volume uses only real, verified DataWeave syntax throughout.

VOLUME 3

Operators & Conditional Logic

100 problems on arithmetic and comparison operators, `and`/`or`/`not`, the `++`/`--` merge operators, `if`/`else`, `match`/`case`, and `default` — the building blocks every DataWeave decision is made of.

SECTION 1 — Arithmetic Operators (Problems 201–210)

#201 Add an Order Subtotal and Tax

Calculate the total for an order by adding the subtotal and tax amount.

INPUT

```
{ "subtotal": 240, "tax": 19.2 }
```

SOLUTION

```
payload.subtotal + payload.tax
```

OUTPUT

```
259.2
```

The + operator adds two Numbers directly — no coercion is needed once both sides are already numeric.

#202 Calculate a Remaining Balance

Subtract the amount already paid from the invoice total.

INPUT

```
{ "invoiceTotal": 500, "amountPaid": 350 }
```

SOLUTION

```
payload.invoiceTotal - payload.amountPaid
```

OUTPUT

```
150
```

The - operator works exactly as expected between two Numbers, producing the outstanding balance.

#203 Compute a Line Total

Multiply a unit price by a quantity to get the line total for a pallet.

INPUT

```
{ "unitPrice": 45.5, "qty": 4 }
```

SOLUTION

```
payload.unitPrice * payload.qty
```

OUTPUT

```
182
```

*The * operator multiplies two Numbers — the standard way to price a line item.*

#204 Compute an Average Weight

Divide a total shipment weight by the number of pallets to get the average weight per pallet.

INPUT

```
{ "totalWeightKg": 620, "palletCount": 4 }
```

SOLUTION

```
payload.totalWeightKg / payload.palletCount
```

OUTPUT

```
155
```

The / operator performs real division, returning a decimal Number whenever the values do not divide evenly.

#205 Check Whether a Pallet Count Is Even

Use the remainder of a division to determine whether a pallet count is an even number.

INPUT

```
{ "palletCount": 8 }
```

SOLUTION

```
(payload.palletCount mod 2) == 0
```

OUTPUT

```
true
```

DataWeave's `mod` operator returns the remainder of a division — a remainder of `0` means the number is even.

#206 Calculate a Taxed Line Total

Combine multiplication and addition to compute a line total that includes an 8% tax rate.

INPUT

```
{ "unitPrice": 100, "qty": 2 }
```

SOLUTION

```
(payload.unitPrice * payload.qty) * 1.08
```

OUTPUT

```
216
```

Parentheses make the intent explicit: compute the pre-tax total first, then apply the tax multiplier.

#207 Apply a Negative Refund Adjustment

Apply a refund adjustment (stored as a negative Number) to an order total.

INPUT

```
{ "orderTotal": 300, "adjustment": -45 }
```

SOLUTION

```
payload.orderTotal + payload.adjustment
```

OUTPUT

```
255
```

Storing an adjustment as a negative Number lets a single + handle both charges and refunds.

#208 Round a Division Result to Two Decimals

Divide a total cost across three shipments and format the result to exactly two decimal places.

INPUT

```
{ "totalCost": 100, "shipmentCount": 3 }
```

SOLUTION

```
(payload.totalCost / payload.shipmentCount) as String {format: "0.00"}
```

OUTPUT

```
"33.33"
```

Division alone can produce long decimals — coercing to String with a format pattern rounds and displays it cleanly.

#209 Combine Operators With Explicit Precedence

Calculate a discounted total: subtract a flat discount, then apply a tax rate to what remains.

INPUT

```
{ "subtotal": 200, "discount": 20 }
```

SOLUTION

```
(payload.subtotal - payload.discount) * 1.08
```

OUTPUT

```
194.4
```

Parentheses control evaluation order explicitly, rather than relying on remembering DataWeave's default operator precedence.

#210 Sum an Array of Line Totals

Add up an array of shipment costs into a single grand total.

INPUT

```
{ "costs": [120, 85.5, 60] }
```

SOLUTION

```
payload.costs reduce ((item, acc = 0) -> acc + item)
```

OUTPUT

```
265.5
```

`reduce` applies `+` across every element, building a running total that starts from the default accumulator `0`.

SECTION 2 — Comparison Operators (Problems 211–220)

#211 Check for an Exact Status Match

Determine whether an order's status is exactly `"DELIVERED"`.

INPUT

```
{ "status": "DELIVERED" }
```

SOLUTION

```
payload.status == "DELIVERED"
```

OUTPUT

```
true
```

The == operator compares both value and type, returning a Boolean.

#212 Exclude Cancelled Orders

Check whether an order's status is anything other than `"CANCELLED"`.

INPUT

```
{ "status": "PENDING" }
```

SOLUTION

```
payload.status != "CANCELLED"
```

OUTPUT

```
true
```

The != operator is the inverse of ==, useful for filtering out a single unwanted value.

#213 Flag a High-Value Order

Check whether an order total exceeds a $500 high-value threshold.

INPUT

```
{ "orderTotal": 640 }
```

SOLUTION

```
payload.orderTotal > 500
```

OUTPUT

```
true
```

The `>` operator works on Numbers exactly as it does in most languages.

#214 Flag an Underweight Shipment

Check whether a shipment's weight is below the 5kg minimum billable weight.

INPUT

```
{ "weightKg": 3.2 }
```

SOLUTION

```
payload.weightKg < 5
```

OUTPUT

```
true
```

The `<` operator compares two Numbers, returning `true` when the left side is strictly smaller.

#215 Check a Loyalty Tier Threshold

Determine whether a customer's lifetime spend qualifies for Gold tier at $1,000 or more.

INPUT

```
{ "lifetimeSpend": 1000 }
```

SOLUTION

```
payload.lifetimeSpend >= 1000
```

OUTPUT

```
true
```

`>=` *is inclusive of the boundary value itself, unlike* `>`.

#216 Check a Maximum Capacity Limit

Determine whether a truck's load is within its 2,000kg maximum capacity.

INPUT

```
{ "loadKg": 2000 }
```

SOLUTION

```
payload.loadKg <= 2000
```

OUTPUT

```
true
```

`<=` *is the inclusive counterpart to* `<`*, so a load exactly at capacity still passes.*

#217 Compare Two Strings Alphabetically

Determine whether one warehouse code sorts before another alphabetically.

INPUT

```
{ "a": "WH-AUSTIN", "b": "WH-DALLAS" }
```

SOLUTION

```
payload.a < payload.b
```

OUTPUT

```
true
```

DataWeave's comparison operators work on Strings too, using standard lexicographic ordering.

#218 Compare Two Dates

Determine whether a shipment's actual delivery Date is after its promised delivery Date.

INPUT

```
{ "promised": "2026-03-10", "actual": "2026-03-14" }
```

SOLUTION

```
(payload.actual as Date) > (payload.promised as Date)
```

OUTPUT

```
true
```

Once coerced to real Date values, comparison operators order them chronologically rather than alphabetically.

#219 Safely Compare a String Quantity to a Number

Compare a quantity that arrived as a String against a numeric reorder threshold.

INPUT

```
{ "qtyOnHand": "12" }
```

SOLUTION

```
(payload.qtyOnHand as Number) < 20
```

OUTPUT

```
true
```

Comparing a String directly to a Number would fail — coercing first keeps the comparison type-safe.

#220 Filter Orders Above a Value Threshold

Keep only the orders in an array whose total is greater than $200.

INPUT

```
{ "orders": [{"id": 1, "total": 150}, {"id": 2, "total": 320}] }
```

SOLUTION

```
payload.orders filter ($.total > 200)
```

OUTPUT

```
[{"id": 2, "total": 320}]
```

A comparison operator inside `filter`'s condition is the standard way to select array elements by value.

SECTION 3 — Logical Operators: and, or, not (Problems 221–230)

#221 Require Two Conditions at Once

Check whether an order is both high-value and marked as priority shipping.

INPUT

```
{ "total": 600, "priority": true }
```

SOLUTION

```
(payload.total > 500) and payload.priority
```

OUTPUT

```
true
```

`and` returns `true` only when both Boolean operands are `true`.

#222 Accept Either of Two Payment Methods

Check whether a payment method is either `"CARD"` or `"WIRE"`.

INPUT

```
{ "method": "WIRE" }
```

SOLUTION

```
(payload.method == "CARD") or (payload.method == "WIRE")
```

OUTPUT

```
true
```

`or` returns `true` when at least one of the two Boolean operands is `true`.

#223 Negate a Boolean Flag

Determine whether a shipment is not yet delivered, given a `delivered` flag.

INPUT

```
{ "delivered": false }
```

SOLUTION

```
not payload.delivered
```

OUTPUT

```
true
```

`not` flips a Boolean value — the cleanest way to express "the opposite of" a flag.

#224 Combine and/or With Parentheses

Check eligibility for free shipping: either the order is over $100, or the customer is a member and the order is over $50.

INPUT

```
{ "total": 60, "isMember": true }
```

SOLUTION

```
(payload.total > 100) or (payload.isMember and (payload.total > 50))
```

OUTPUT

```
true
```

Parentheses group the `and` clause explicitly, so it is evaluated as a single unit before the outer `or`.

#225 Negate a Comparison Directly

Check whether an order's total is not over the $1,000 fraud-review threshold.

INPUT

```
{ "total": 420 }
```

SOLUTION

```
not (payload.total > 1000)
```

OUTPUT

```
true
```

`not` can wrap an entire comparison expression, since the comparison itself already evaluates to a Boolean.

#226 Filter With an and Condition

Keep only orders that are both over $200 and marked as priority.

INPUT

```
{ "orders": [{"id": 1, "total": 250, "priority": true}, {"id": 2, "total": 300, "priority": false}] }
```

SOLUTION

```
payload.orders filter (($.total > 200) and $.priority)
```

OUTPUT

```
[{"id": 1, "total": 250, "priority": true}]
```

Combining a comparison and a Boolean field with `and` is a common real-world filter condition.

#227 Filter With an or Condition

Keep only orders whose status is either `"PENDING"` or `"PROCESSING"`.

INPUT

```
{ "orders": [{"id": 1, "status": "PENDING"}, {"id": 2, "status": "SHIPPED"},
{"id": 3, "status": "PROCESSING"}] }
```

SOLUTION

```
payload.orders filter (($.status == "PENDING") or ($.status == "PROCESSING"))
```

OUTPUT

```
[{"id": 1, "status": "PENDING"}, {"id": 3, "status": "PROCESSING"}]
```

Chaining `or` comparisons on the same field is a readable way to match against a small set of values.

#228 Chain Three Conditions With and

Check whether a shipment qualifies for express handling: in stock, under 10kg, and domestic.

INPUT

```
{ "inStock": true, "weightKg": 4, "domestic": true }
```

SOLUTION

```
payload.inStock and (payload.weightKg < 10) and payload.domestic
```

OUTPUT

```
true
```

`and` can be chained across more than two operands; the whole expression is `true` only if every part is.

#229 Rewrite a not (and) as an or of nots

Show that "not (in stock and domestic)" gives the same result as "not in stock or not domestic".

INPUT

```
{ "inStock": true, "domestic": false }
```

SOLUTION

```
{
  direct: not (payload.inStock and payload.domestic),
  expanded: (not payload.inStock) or (not payload.domestic)
}
```

OUTPUT

```
{ "direct": true, "expanded": true }
```

This is De Morgan's law in practice — both forms always agree, and picking whichever reads more clearly is a matter of style.

#230 Build a VIP Discount Eligibility Rule

A customer qualifies for a VIP discount if they are a Gold member and not flagged for fraud, or if their lifetime spend is over $5,000.

INPUT

```
{ "tier": "GOLD", "fraudFlag": false, "lifetimeSpend": 1200 }
```

SOLUTION

```
((payload.tier == "GOLD") and (not payload.fraudFlag)) or
(payload.lifetimeSpend > 5000)
```

OUTPUT

```
true
```

Combining `and`, `or`, and `not` with clear parentheses lets a single expression encode a full business rule.

SECTION 4 — Merge Operators: ++ and -- (Problems 231–240)

#231 Concatenate Two Strings

Build a full tracking code by joining a carrier prefix and a shipment number.

INPUT

```
{ "prefix": "CL", "number": "88213" }
```

SOLUTION

```
payload.prefix ++ "-" ++ payload.number
```

OUTPUT

```
"CL-88213"
```

++ concatenates two Strings, and can be chained across several values in one expression.

#232 Merge Two Arrays

Combine two arrays of item codes into a single array.

INPUT

```
{ "a": ["PALLET-01", "PALLET-02"], "b": ["CRATE-07"] }
```

SOLUTION

```
payload.a ++ payload.b
```

OUTPUT

```
["PALLET-01", "PALLET-02", "CRATE-07"]
```

On Arrays, ++ appends the second array's elements after the first array's elements, in order.

#233 Merge Two Objects

Merge a base order object with an overrides object, keeping every field from both.

INPUT

```
{ "base": {"status": "PENDING", "priority": false}, "overrides": {"priority": true} }
```

SOLUTION

```
payload.base ++ payload.overrides
```

OUTPUT

```
{ "status": "PENDING", "priority": true }
```

On Objects, ++ merges keys from both sides — when a key exists in both, the right-hand side's value wins.

#234 Remove a Key From an Object

Strip an internal `_debugNotes` field before returning an order to the customer-facing API.

INPUT

```
{ "orderId": "ORD-1042", "total": 240, "_debugNotes": "retry #2" }
```

SOLUTION

```
payload -- "_debugNotes"
```

OUTPUT

```
{ "orderId": "ORD-1042", "total": 240 }
```

The -- operator removes a key from an Object by name, leaving every other field untouched.

#235 Remove an Item From an Array

Remove a cancelled item code from an array of item codes.

INPUT

```
{ "items": ["PALLET-01", "CRATE-07", "PALLET-02"] }
```

SOLUTION

```
payload.items -- "CRATE-07"
```

OUTPUT

```
["PALLET-01", "PALLET-02"]
```

`--` also works on Arrays, removing every element that matches the given value.

#236 Remove Multiple Keys at Once

Strip both an internal `_debugNotes` field and a temporary `_cacheKey` field from a payload.

INPUT

```
{ "orderId": "ORD-1042", "_debugNotes": "n/a", "_cacheKey": "abc123" }
```

SOLUTION

```
(payload -- "_debugNotes") -- "_cacheKey"
```

OUTPUT

```
{ "orderId": "ORD-1042" }
```

Chaining `--` removes one key per call — each call returns a new Object with that key gone.

#237 Confirm the Right-Hand Side Wins in a Merge

Merge a default shipping config with a customer-specific config that overrides one field.

INPUT

```
{ "defaults": {"carrier": "STANDARD", "insured": false}, "custom": {"carrier": "EXPRESS"} }
```

SOLUTION

```
payload.defaults ++ payload.custom
```

OUTPUT

```
{ "carrier": "EXPRESS", "insured": false }
```

This "defaults then overrides" ordering is the standard pattern for layering configuration objects with ++.

#238 Combine Two Item Lists Into a Single Shipment

Merge two orders' item arrays into one combined shipment manifest.

INPUT

```
{ "orderA": {"items": [{"sku": "PALLET-01"}]}, "orderB": {"items": [{"sku": "CRATE-07"}]} }
```

SOLUTION

```
payload.orderA.items ++ payload.orderB.items
```

OUTPUT

```
[{"sku": "PALLET-01"}, {"sku": "CRATE-07"}]
```

++ works the same way on arrays of objects as it does on arrays of primitives — it simply concatenates them.

#239 Conditionally Remove a Key Based on Its Value

Remove the `discountCode` field entirely if it is an empty String.

INPUT

```
{ "orderId": "ORD-1042", "discountCode": "" }
```

SOLUTION

```
if (payload.discountCode == "") payload -- "discountCode" else payload
```

OUTPUT

```
{ "orderId": "ORD-1042" }
```

Combining `--` with `if/else` lets a key be dropped only under a specific condition, rather than unconditionally.

#240 Apply a Partial Update to a Full Record

Apply a partial update to a customer record: merge in new fields, and remove the temporary `_isDraft` flag.

INPUT

```
{ "record": {"name": "Maria Chen", "tier": "SILVER", "_isDraft": true},
"update": {"tier": "GOLD"} }
```

SOLUTION

```
(payload.record ++ payload.update) -- "_isDraft"
```

OUTPUT

```
{ "name": "Maria Chen", "tier": "GOLD" }
```

`++` and `--` combine naturally: merge the update in first, then strip any fields that should never reach the output.

SECTION 5 — if/else Basics (Problems 241–250)

#241 Return a Label Based on a Flag

Return `"Priority"` or `"Standard"` depending on a Boolean priority flag.

INPUT

```
{ "priority": true }
```

SOLUTION

```
if (payload.priority) "Priority" else "Standard"
```

OUTPUT

```
"Priority"
```

`if (condition) A else B` is an expression in DataWeave — it evaluates directly to a value, and `else` is required.

#242 Return Different Value Types Per Branch

Return the actual weight Number if known, or the String `"unknown"` if it is missing.

INPUT

```
{ "weightKg": null }
```

SOLUTION

```
if (payload.weightKg != null) payload.weightKg else "unknown"
```

OUTPUT

```
"unknown"
```

DataWeave does not require both branches of an `if/else` to share the same type, since the whole language is dynamically typed.

#243 Branch on a Comparison

Return `"Free"` if the order total is over $100, otherwise return the flat $9.99 shipping fee.

INPUT

```
{ "total": 140 }
```

SOLUTION

```
if (payload.total > 100) "Free" else 9.99
```

OUTPUT

```
"Free"
```

Any expression that evaluates to a Boolean can act as the `if` condition, including a direct comparison.

#244 Set an Object Field Conditionally

Build an order summary where the `rushFee` field is 15 if priority, or 0 otherwise.

INPUT

```
{ "orderId": "ORD-1042", "priority": true }
```

SOLUTION

```
{
  orderId: payload.orderId,
  rushFee: if (payload.priority) 15 else 0
}
```

OUTPUT

```
{ "orderId": "ORD-1042", "rushFee": 15 }
```

Because `if/else` is an expression, it can be dropped directly into a field's value inside an object literal.

#245 Calculate a Conditional Discount Amount

Apply a 10% discount if the order is over $200, otherwise apply no discount.

INPUT

```
{ "total": 250 }
```

SOLUTION

```
if (payload.total > 200) (payload.total * 0.9) else payload.total
```

OUTPUT

```
225
```

Each branch can hold a full calculation, not just a literal value.

#246 Branch on a Combined Condition

Approve an order automatically only if it is under $1,000 and the customer has no fraud flag.

INPUT

```
{ "total": 400, "fraudFlag": false }
```

SOLUTION

```
if ((payload.total < 1000) and (not payload.fraudFlag)) "AUTO_APPROVED" else
"MANUAL_REVIEW"
```

OUTPUT

```
"AUTO_APPROVED"
```

Logical operators combine naturally with `if`, letting one condition encode a full multi-part business rule.

#247 Use else as a Fallback Default

Use the customer's preferred carrier if set, otherwise fall back to `"STANDARD"`.

INPUT

```
{ "preferredCarrier": null }
```

SOLUTION

```
if (payload.preferredCarrier != null) payload.preferredCarrier else
"STANDARD"
```

OUTPUT

```
"STANDARD"
```

This pattern is a manual, explicit version of what the `default` operator does automatically for null values.

#248 Apply if/else to Every Item in an Array

Label every item in an array as `"fragile"` or `"standard"` based on its category.

INPUT

```
{ "items": [{"category": "GLASSWARE"}, {"category": "TOOLS"}] }
```

SOLUTION

```
payload.items map (item) -> if (item.category == "GLASSWARE") "fragile" else
"standard"
```

OUTPUT

```
["fragile", "standard"]
```

`if/else` works inside a `map` lambda just like any other expression, evaluating per element.

#249 Assign a Tier Label From a Numeric Threshold

Assign `"HEAVY"` if a shipment is 100kg or more, otherwise `"LIGHT"`.

INPUT

```
{ "weightKg": 140 }
```

SOLUTION

```
if (payload.weightKg >= 100) "HEAVY" else "LIGHT"
```

OUTPUT

```
"HEAVY"
```

Using `>=` instead of `>` makes the boundary value itself count as the heavier tier.

#250 Build a Shipping Cost Calculator

Calculate shipping cost: free if the order is over $150, otherwise a flat $12 base fee plus $2 per kg.

INPUT

```
{ "total": 90, "weightKg": 6 }
```

SOLUTION

```
if (payload.total > 150) 0 else (12 + (payload.weightKg * 2))
```

OUTPUT

```
24
```

This is the shape most real shipping calculators take: one condition guarding an otherwise ordinary arithmetic formula.

SECTION 6 — Nested & Chained if/else (Problems 251–260)

#251 Build a Three-Tier Shipping Speed Label

Label a shipment `"OVERNIGHT"`, `"EXPRESS"`, or `"STANDARD"` based on its speed code.

INPUT

```
{ "speedCode": 2 }
```

SOLUTION

```
if (payload.speedCode == 1) "OVERNIGHT"
else if (payload.speedCode == 2) "EXPRESS"
else "STANDARD"
```

OUTPUT

```
"EXPRESS"
```

Chaining `else if` is really just nesting one `if/else` inside the previous `else` branch — each check runs only if the ones before it failed.

#252 Build a Four-Tier Grading Scale

Convert a driver's on-time delivery percentage into a letter grade: A (95+), B (85+), C (70+), or D.

INPUT

```
{ "onTimePct": 88 }
```

SOLUTION

```
if (payload.onTimePct >= 95) "A"
else if (payload.onTimePct >= 85) "B"
else if (payload.onTimePct >= 70) "C"
else "D"
```

OUTPUT

```
"B"
```

Each condition only needs to check its own lower bound, because any higher value would already have matched an earlier branch.

#253 Mix Types Across Nested Branches

Return a Number surcharge for known zones, or a String warning for an unrecognized zone.

INPUT

```
{ "zone": "Z9" }
```

SOLUTION

```
if (payload.zone == "Z1") 5
else if (payload.zone == "Z2") 12
else "UNKNOWN_ZONE"
```

OUTPUT

```
"UNKNOWN_ZONE"
```

Since DataWeave is dynamically typed, nested branches are free to return completely different kinds of values.

#254 Classify Every Item in an Array With Nested Logic

Classify each item's weight as `"heavy"`, `"medium"`, or `"light"`.

INPUT

```
{ "items": [{"weightKg": 120}, {"weightKg": 15}, {"weightKg": 2}] }
```

SOLUTION

```
payload.items map (item) ->
  if (item.weightKg >= 100) "heavy"
  else if (item.weightKg >= 10) "medium"
  else "light"
```

OUTPUT

```
["heavy", "medium", "light"]
```

A nested `if/else` chain drops into a `map` lambda exactly like a simple one — it is evaluated fresh for each item.

#255 Combine Nested Branches With and/or

Determine a customer's support tier: `"PLATINUM"` if Gold member and lifetime spend over $10,000, `"GOLD"` if just a Gold member, otherwise `"STANDARD"`.

INPUT

```
{ "tier": "GOLD", "lifetimeSpend": 12000 }
```

SOLUTION

```
if ((payload.tier == "GOLD") and (payload.lifetimeSpend > 10000)) "PLATINUM"
else if (payload.tier == "GOLD") "GOLD"
else "STANDARD"
```

OUTPUT

```
"PLATINUM"
```

Placing the most specific, most restrictive condition first ensures it is checked before the broader fallback catches it instead.

#256 Build an Object Field With Nested Branches

Build a shipment summary where `handling` is `"FRAGILE"`, `"REFRIGERATED"`, or `"STANDARD"` based on category.

INPUT

```
{ "shipmentId": "SHP-501", "category": "REFRIGERATED" }
```

SOLUTION

```
{
  shipmentId: payload.shipmentId,
  handling:
    if (payload.category == "GLASSWARE") "FRAGILE"
    else if (payload.category == "REFRIGERATED") "REFRIGERATED"
    else "STANDARD"
}
```

OUTPUT

```
{ "shipmentId": "SHP-501", "handling": "REFRIGERATED" }
```

A multi-branch nested `if/else` reads cleanly as a field value once it is broken across indented lines.

#257 Always End a Nested Chain With a Default Branch

Assign a routing hub code, falling through to a general hub if none of the specific regions match.

INPUT

```
{ "region": "MIDWEST" }
```

SOLUTION

```
if (payload.region == "WEST") "HUB-LAX"
else if (payload.region == "EAST") "HUB-JFK"
else "HUB-GENERAL"
```

OUTPUT

```
"HUB-GENERAL"
```

Because `else` is required, a nested chain can never "fall through" with no result — the final `else` always catches every remaining case.

#258 Compare Two Fields Against Each Other in Nested Logic

Compare a shipment's actual weight to its declared weight and flag it as over, under, or matching.

INPUT

```
{ "declaredKg": 50, "actualKg": 55 }
```

SOLUTION

```
if (payload.actualKg > payload.declaredKg) "OVER_DECLARED"
else if (payload.actualKg < payload.declaredKg) "UNDER_DECLARED"
else "MATCHES"
```

OUTPUT

```
"OVER_DECLARED"
```

Nested conditions are not limited to comparing a field against a constant — they can compare two fields directly against each other.

#259 Guard a Nested Branch With a Null Check

Safely check a discount code: apply it only if present and non-empty, otherwise apply no discount.

INPUT

```
{ "discountCode": null }
```

SOLUTION

```
if (payload.discountCode == null) "NO_DISCOUNT"
else if (payload.discountCode == "") "NO_DISCOUNT"
else payload.discountCode
```

OUTPUT

```
"NO_DISCOUNT"
```

Checking for `null` before touching a field's value is the same null-safety discipline used throughout selectors, applied here to branching logic.

#260 Build a Shipping Tier and Surcharge Calculator

Calculate a total shipping cost: a base fee by weight tier, plus a $10 surcharge for international shipments.

INPUT

```
{ "weightKg": 25, "international": true }
```

SOLUTION

```
(
  if (payload.weightKg >= 50) 40
  else if (payload.weightKg >= 20) 25
  else 12
) + (if (payload.international) 10 else 0)
```

OUTPUT

```
35
```

Two independent `if/else` expressions can be combined with a plain arithmetic operator, keeping each piece of logic easy to read on its own.

SECTION 7 — match/case Basics (Problems 261–270)

#261 Match on a String Literal

Convert an order status code into a customer-facing label using `match`.

INPUT

```
{ "status": "OPEN" }
```

SOLUTION

```
payload.status match {
  case "OPEN" -> "Your order is open"
  case "CLOSED" -> "Your order is closed"
}
```

OUTPUT

```
"Your order is open"
```

`match` compares a value against a list of `case` patterns in order, returning the result of the first one that matches.

#262 Match on a Number Literal

Convert a numeric priority code into a text label using `match`.

INPUT

```
{ "priorityCode": 1 }
```

SOLUTION

```
payload.priorityCode match {
  case 1 -> "Urgent"
  case 2 -> "Normal"
}
```

OUTPUT

```
"Urgent"
```

`case` patterns can be Number literals just as easily as String literals.

#263 Add a Default Case With else

Match a shipment method, falling back to a generic message for any unrecognized value.

INPUT

```
{ "method": "DRONE" }
```

SOLUTION

```
payload.method match {
  case "TRUCK" -> "Ground shipment"
  case "AIR" -> "Air shipment"
  else -> "Unrecognized method"
}
```

OUTPUT

```
"Unrecognized method"
```

The `else` case acts as a catch-all, running whenever none of the earlier `case` patterns matched.

#264 Return Different Types Per Case

Match an inventory status, returning a Number restock quantity for one case and a String for another.

INPUT

```
{ "status": "OUT_OF_STOCK" }
```

SOLUTION

```
payload.status match {
  case "OUT_OF_STOCK" -> 100
  case "IN_STOCK" -> "no action needed"
  else -> "unknown status"
}
```

OUTPUT

```
100
```

Just like `if/else`, `match` does not require every branch to return the same type.

#265 Match a Value Stored in a Variable

Store a status in a `var`, then use `match` to decide the next action.

INPUT

```
{ "status": "DELAYED" }
```

SOLUTION

```
var currentStatus = payload.status
---
currentStatus match {
  case "DELAYED" -> "Notify customer"
  else -> "No action"
}
```

OUTPUT

```
"Notify customer"
```

`match` works on any expression, including a `var` reference, not just a direct selector.

#266 Use match Inside a map

Convert an array of status codes into an array of human-readable labels.

INPUT

```
{ "statuses": ["A", "B", "C"] }
```

SOLUTION

```
payload.statuses map (s) -> s match {
  case "A" -> "Active"
  case "B" -> "Backordered"
  else -> "Cancelled"
}
```

OUTPUT

```
["Active", "Backordered", "Cancelled"]
```

A `match` expression can be the body of a `map` lambda, evaluating once per array element.

#267 Match Several Related Codes to One Result

Route both `"NEW"` and `"REOPENED"` tickets to the same handling queue.

INPUT

```
{ "ticketStatus": "REOPENED" }
```

SOLUTION

```
payload.ticketStatus match {
  case "NEW" -> "TRIAGE_QUEUE"
  case "REOPENED" -> "TRIAGE_QUEUE"
  else -> "STANDARD_QUEUE"
}
```

OUTPUT

```
"TRIAGE_QUEUE"
```

DataWeave's `match` does not support combining literals in one `case`, so two related values are matched with two separate cases that share a result.

#268 Use match Inside an Object Field

Build an order summary where the `displayStatus` field comes from a `match` expression.

INPUT

```
{ "orderId": "ORD-1042", "status": "SHIPPED" }
```

SOLUTION

```
{
  orderId: payload.orderId,
  displayStatus: payload.status match {
    case "SHIPPED" -> "On the way"
    else -> "Processing"
  }
}
```

OUTPUT

```
{ "orderId": "ORD-1042", "displayStatus": "On the way" }
```

Just like `if/else`, a `match` expression can be nested directly inside an object literal as a field's value.

#269 Match on a Boolean Literal

Convert a Boolean `inStock` flag into a display label using `match`.

INPUT

```
{ "inStock": false }
```

SOLUTION

```
payload.inStock match {
  case true -> "In Stock"
  case false -> "Out of Stock"
}
```

OUTPUT

```
"Out of Stock"
```

`case` patterns can match Boolean literals directly, though for a plain two-way Boolean check, `if/else` is usually the more idiomatic choice.

#270 Build a Customer-Facing Status Message

Convert an internal order status into a friendly customer-facing message, with a safe fallback.

INPUT

```
{ "status": "OUT_FOR_DELIVERY" }
```

SOLUTION

```
payload.status match {
  case "PROCESSING" -> "We're preparing your order."
  case "OUT_FOR_DELIVERY" -> "Your order is out for delivery!"
  case "DELIVERED" -> "Your order has arrived."
  else -> "We're tracking your order's status."
}
```

OUTPUT

```
"Your order is out for delivery!"
```

This is the pattern most production status-message translators use: a short, explicit list of known cases with a safe generic fallback.

SECTION 8 — match With Type & Structural Patterns (Problems 271–280)

#271 Dispatch on Value Type With is

Return a different label depending on whether a field holds a String or a Number.

INPUT

```
{ "value": 42 }
```

SOLUTION

```
payload.value match {
  case n is Number -> "It's a Number: " ++ (n as String)
  case s is String -> "It's a String: " ++ s
  else -> "Some other type"
}
```

OUTPUT

```
"It's a Number: 42"
```

A type matcher (`case n is Number`) both checks the type and binds the matched value to the name `n` for use in the result.

#272 Handle a Null Field With a Type Matcher

Return a safe default message when an optional field is `Null`, and its value otherwise.

INPUT

```
{ "trackingNumber": null }
```

SOLUTION

```
payload.trackingNumber match {
  case n is Null -> "Not yet assigned"
  case t is String -> t
}
```

OUTPUT

```
"Not yet assigned"
```

`Null` is a real type in DataWeave's type system, so `case n is Null` is a clean way to branch specifically on a missing value.

#273 Distinguish Between an Array and an Object

Return a different message depending on whether a field holds an Array or an Object.

INPUT

```
{ "items": [1, 2, 3] }
```

SOLUTION

```
payload.items match {
  case a is Array -> "Array with " ++ (sizeOf(a) as String) ++ " elements"
  case o is Object -> "Single object"
}
```

OUTPUT

```
"Array with 3 elements"
```

Type matchers are especially useful for handling payloads whose shape can vary between an Array and a single Object.

#274 Dispatch on a Boolean Type

Confirm that a field genuinely holds a Boolean before treating it as a flag.

INPUT

```
{ "flag": true }
```

SOLUTION

```
payload.flag match {
  case b is Boolean -> "Valid flag: " ++ (b as String)
  else -> "Invalid flag value"
}
```

OUTPUT

```
"Valid flag: true"
```

`is Boolean` guards against a field that was supposed to be a flag arriving as some other type entirely, like a String.

#275 Combine a Type Check With Further Logic

If a discount value is a Number, apply it only when positive; otherwise treat it as no discount.

INPUT

```
{ "discount": -5 }
```

SOLUTION

```
payload.discount match {
  case n is Number -> if (n > 0) n else 0
  else -> 0
}
```

OUTPUT

```
0
```

A matched, bound value can be fed straight into further logic, such as an `if/else`, inside the same `case` branch.

#276 Match a Single-Element Array Pattern

Return a special message when a shipment has exactly one item, using an array selector pattern.

INPUT

```
{ "items": ["PALLET-01"] }
```

SOLUTION

```
payload.items match {
  case [single] -> "Single-item shipment: " ++ single
  else -> "Multi-item shipment"
}
```

OUTPUT

```
"Single-item shipment: PALLET-01"
```

An array pattern like `[single]` matches only arrays with exactly one element, binding it to the name `single`.

#277 Match a Two-Element Array Pattern

Bind the first and second elements of a two-item coordinate pair by pattern.

INPUT

```
{ "coords": [40.7, -74.0] }
```

SOLUTION

```
payload.coords match {
  case [lat, lon] -> "Lat: " ++ (lat as String) ++ ", Lon: " ++ (lon as
String)
  else -> "Invalid coordinates"
}
```

OUTPUT

```
"Lat: 40.7, Lon: -74.0"
```

Array patterns can bind more than one element at once, matching only arrays of exactly that length.

#278 Match an Object by a Field's Value

Route an order using an object selector pattern that checks the `status` field directly.

INPUT

```
{ "status": "OPEN", "orderId": "ORD-1042" }
```

SOLUTION

```
payload match {
  case {status: "OPEN"} -> "Route to fulfillment"
  case {status: "CLOSED"} -> "Route to archive"
  else -> "Route to review"
}
```

OUTPUT

```
"Route to fulfillment"
```

An object selector pattern matches the whole payload against a specific field's expected value, without needing a separate selector expression.

#279 Bind and Transform a Matched Number

Match a value as a Number and immediately double it inside the result expression.

INPUT

```
{ "quantity": 6 }
```

SOLUTION

```
payload.quantity match {
  case n is Number -> n * 2
  else -> 0
}
```

OUTPUT

```
12
```

Type matchers do double duty: they act as both a runtime type check and a variable binding, ready for use on the right of `->`.

#280 Route an Unpredictable Payload Shape

A webhook payload sometimes sends a single order Object, and sometimes an Array of orders. Handle both shapes safely.

INPUT

```
{ "orders": [{"id": 1}, {"id": 2}] }
```

SOLUTION

```
payload.orders match {
  case a is Array -> sizeOf(a) as String ++ " orders received"
  case o is Object -> "1 order received"
  case n is Null -> "No orders received"
}
```

OUTPUT

```
"2 orders received"
```

Type matchers turn an unpredictable payload shape from a source of bugs into a small, explicit, exhaustive set of handled cases.

SECTION 9 — default & Conditional Coalescing (Problems 281–290)

#281 Apply a Simple Default Value

Fall back to `"UNASSIGNED"` when a driver field is `null`.

INPUT

```
{ "driver": null }
```

SOLUTION

```
payload.driver default "UNASSIGNED"
```

OUTPUT

```
"UNASSIGNED"
```

`default` supplies a fallback value only when the left-hand side is `null` — otherwise it passes the original value through unchanged.

#282 Combine default With an if Downstream

Default a missing priority flag to `false`, then branch on it as usual.

INPUT

```
{ "priority": null }
```

SOLUTION

```
if (payload.priority default false) "Priority" else "Standard"
```

OUTPUT

```
"Standard"
```

Applying `default` before the condition guarantees `if` always receives a real Boolean, never a `null`.

#283 Default a Deeply Nested Missing Field

Safely default a nested `contact.phone` field that may not exist at all.

INPUT

```
{ "contact": {"email": "maria@cargolink.com"} }
```

SOLUTION

```
payload.contact.phone default "No phone on file"
```

OUTPUT

```
"No phone on file"
```

A missing selector step resolves to `null` rather than throwing an error, so `default` catches it exactly like an explicit `null` value.

#284 Default a Missing Array to Empty

Default a missing `items` array to an empty array so downstream code never breaks.

INPUT

```
{ "orderId": "ORD-1042" }
```

SOLUTION

```
payload.items default []
```

OUTPUT

```
[]
```

Defaulting a missing collection field to an empty Array (or empty Object) is a standard defensive pattern before any `map` or `filter` runs on it.

#285 Default Before Mapping Over a Possibly Missing Array

Safely extract item SKUs even when the `items` field is entirely missing.

INPUT

```
{ "orderId": "ORD-1042" }
```

SOLUTION

```
(payload.items default []) map (item) -> item.sku
```

OUTPUT

```
[]
```

Because `map` over an empty Array simply produces an empty Array, defaulting first avoids a runtime error on a `null` collection.

#286 Choose Between default and if/else

Show the difference: `default` only replaces `null`, while `if/else` can check any condition, such as an empty String.

INPUT

```
{ "note": "" }
```

SOLUTION

```
{
  usingDefault: payload.note default "No note",
  usingIfElse: if (payload.note == "") "No note" else payload.note
}
```

OUTPUT

```
{ "usingDefault": "", "usingIfElse": "No note" }
```

`default` only ever triggers on `null` — an empty String is still a real value, so only the explicit `if/else` check catches it.

#287 Default a Numeric Field Before Arithmetic

Default a possibly missing `discount` field to `0` before subtracting it from a total.

INPUT

```
{ "total": 200 }
```

SOLUTION

```
payload.total - (payload.discount default 0)
```

OUTPUT

```
200
```

Defaulting a numeric field before it enters an arithmetic expression avoids a `null` ever reaching an operator that can't handle it.

#288 Combine default With match for Null Handling

Default a missing carrier field, then route the shipment with `match`.

INPUT

```
{ "carrier": null }
```

SOLUTION

```
(payload.carrier default "UNKNOWN") match {
  case "FEDEX" -> "Track via FedEx"
  case "UPS" -> "Track via UPS"
  else -> "Tracking unavailable"
}
```

OUTPUT

```
"Tracking unavailable"
```

Defaulting first guarantees `match` always receives a real String, so the `else` case only has to handle genuinely unrecognized carriers.

#289 Default Multiple Nested Levels at Once

Safely default a value that is nested two levels deep, where either level might be missing.

INPUT

```
{ "shipment": null }
```

SOLUTION

```
payload.shipment.trackingNumber default "Not shipped yet"
```

OUTPUT

```
"Not shipped yet"
```

Selector chaining is already null-safe at every step, so a single trailing `default` is enough even when an intermediate level is missing entirely.

#290 Build a Safe Order Summary With Multiple Defaults

Build an order summary object where every optional field falls back to a safe default value.

INPUT

```
{ "orderId": "ORD-1042", "priority": null, "discount": null, "notes": null }
```

SOLUTION

```
{
  orderId: payload.orderId,
  priority: payload.priority default false,
  discount: payload.discount default 0,
  notes: payload.notes default "No notes"
}
```

OUTPUT

```
{ "orderId": "ORD-1042", "priority": false, "discount": 0, "notes": "No notes"
}
```

Applying `default` field-by-field when building a summary object guarantees the output is always complete and predictable, no matter which optional fields the source data left out.

SECTION 10 — Mixed Real-World Conditional Logic Challenges (Problems 291–300)

#291 Route a CargoLink Shipment by Status

Route a CargoLink shipment to the correct internal queue based on its status.

INPUT

```
{ "shipmentId": "SHP-501", "status": "EXCEPTION" }
```

SOLUTION

```
payload.status match {
  case "PENDING" -> "FULFILLMENT_QUEUE"
  case "EXCEPTION" -> "SUPPORT_QUEUE"
  case "DELIVERED" -> "ARCHIVE_QUEUE"
  else -> "REVIEW_QUEUE"
}
```

OUTPUT

```
"SUPPORT_QUEUE"
```

A status-to-queue router is one of the most common real uses of `match` in a MuleSoft integration flow.

#292 Evaluate CargoLink Discount Eligibility

A CargoLink customer qualifies for a loyalty discount if they are a Gold member and not on the fraud watchlist, or if this is their 10th+ order.

INPUT

```
{ "tier": "SILVER", "onWatchlist": false, "orderCount": 11 }
```

SOLUTION

```
((payload.tier == "GOLD") and (not payload.onWatchlist)) or
(payload.orderCount >= 10)
```

OUTPUT

```
true
```

Real eligibility rules are almost always this shape: a compound `and` condition, joined by `or` to a separate qualifying path.

#293 Calculate a CargoLink Late Delivery Fee

Calculate a late fee: $5 per day late, capped at $50 maximum, and $0 if not late at all.

INPUT

```
{ "daysLate": 14 }
```

SOLUTION

```
if (payload.daysLate <= 0) 0
else if ((payload.daysLate * 5) > 50) 50
else payload.daysLate * 5
```

OUTPUT

```
50
```

Capping a calculated value is naturally expressed as a nested `if/else`: one branch for "not applicable," one for "over the cap," and one for the plain formula.

#294 Classify a CargoLink Shipment's Weight Tier

Classify a shipment into a billing weight tier: `"LTL"` (under 150kg), `"TL"` (150-2000kg), or `"BULK"` (over 2000kg).

INPUT

```
{ "weightKg": 900 }
```

SOLUTION

```
if (payload.weightKg < 150) "LTL"
else if (payload.weightKg <= 2000) "TL"
else "BULK"
```

OUTPUT

```
"TL"
```

Real freight billing tiers are almost always defined by consecutive numeric ranges, which map directly onto a nested `if/else` chain.

#295 Calculate a CargoLink Driver Performance Bonus

Calculate a driver's monthly bonus: $200 if on-time rate is 98%+, $75 if 90%+, otherwise $0.

INPUT

```
{ "onTimeRate": 94 }
```

SOLUTION

```
if (payload.onTimeRate >= 98) 200
else if (payload.onTimeRate >= 90) 75
else 0
```

OUTPUT

```
75
```

Performance-based bonus tiers are a direct real-world application of the same ordered-threshold pattern used for grading and billing tiers.

#296 Validate a CargoLink Order Before Submission

An order is valid only if it has at least one item, a positive total, and a non-empty shipping country.

INPUT

```
{ "items": ["PALLET-01"], "total": 240, "shippingCountry": "US" }
```

SOLUTION

```
(sizeOf(payload.items) > 0) and (payload.total > 0) and
(payload.shippingCountry != "")
```

OUTPUT

```
true
```

Chaining several `and` conditions is the standard way to express a multi-field "is this record valid" check in one expression.

#297 Match a CargoLink Customer Tier to Perks

Match a customer's tier to the perks description shown on their invoice.

INPUT

```
{ "tier": "PLATINUM" }
```

SOLUTION

```
payload.tier match {
  case "PLATINUM" -> "Free express shipping + dedicated support"
  case "GOLD" -> "Free standard shipping"
  case "SILVER" -> "10% shipping discount"
  else -> "Standard rates apply"
}
```

OUTPUT

```
"Free express shipping + dedicated support"
```

A perks lookup table like this is exactly the kind of small, closed set of values `match` was designed for.

#298 Apply a CargoLink Item Update With Merge Operators

Apply a partial update to a shipment record, and remove its temporary `_pendingSync` flag once applied.

INPUT

```
{ "shipment": {"shipmentId": "SHP-501", "status": "PENDING", "_pendingSync": true}, "update": {"status": "IN_TRANSIT"} }
```

SOLUTION

```
(payload.shipment ++ payload.update) -- "_pendingSync"
```

OUTPUT

```
{ "shipmentId": "SHP-501", "status": "IN_TRANSIT" }
```

This merge-then-clean pattern is exactly how a real update endpoint would apply a partial change while stripping internal bookkeeping fields from the response.

#299 Safely Route a CargoLink Order With Missing Data

Default a possibly-missing shipping method, then route the order — combining `default` with `match`.

INPUT

```
{ "shippingMethod": null }
```

SOLUTION

```
(payload.shippingMethod default "GROUND") match {
  case "AIR" -> "Route to air cargo hub"
  case "GROUND" -> "Route to ground fleet"
  else -> "Route to manual review"
}
```

OUTPUT

```
"Route to ground fleet"
```

Defaulting immediately before a `match` is the most reliable way to guarantee the routing logic never has to deal with a raw `null`.

#300 CargoLink Order Processing Capstone

Process a full CargoLink order: default missing fields, validate it, calculate a total with a conditional discount and surcharge, and assign a routing queue — all in one expression.

INPUT

```
{ "orderId": "ORD-1042", "items": ["PALLET-01", "CRATE-07"], "subtotal": 320,
"tier": "GOLD", "international": true, "discount": null }
```

SOLUTION

```
{
  orderId: payload.orderId,
  valid: (sizeOf(payload.items) > 0) and (payload.subtotal > 0),
  total:
    (payload.subtotal - (payload.discount default 0))
    * (if (payload.tier == "GOLD") 0.9 else 1)
    + (if (payload.international) 25 else 0),
  queue: payload.tier match {
    case "GOLD" -> "PRIORITY_QUEUE"
    case "SILVER" -> "STANDARD_QUEUE"
    else -> "STANDARD_QUEUE"
  }
}
```

OUTPUT

```
{ "orderId": "ORD-1042", "valid": true, "total": 313, "queue":
"PRIORITY_QUEUE" }
```

This capstone combines nearly every idea from the volume — arithmetic, comparisons, `and`, `if/else`, `default`, and `match` — into a single realistic order-processing transform.

END OF VOLUME 3

300 Down, 700 to Go

You've now worked through selectors and basic data access in Volume 1, a complete tour of type coercion in Volume 2, and every operator and conditional structure DataWeave offers in Volume 3.

Volume 4 moves to Array Transformations: another 100 problems on `map`, `filter`, `reduce`, `orderBy`, `groupBy`, and every other array-shaping tool in the language.

MULESOFT · PRACTICE PROBLEMS · WITH FULL SOLUTIONS

1000 DataWeave Problems With Solutions

Volume 4 — Array Transformations. 100 hands-on problems on map, filter, reduce, orderBy, groupBy, and every other array-shaping tool in DataWeave.

Problems 301–400

Volume 4 of 10

2026 EDITION

How to Use This Volume

Volume 4 is about **arrays** — the collection functions that let a DataWeave script reshape, filter, sort, group, and summarize lists of data without writing a single loop. Same format as Volumes 1-3:

- **A short problem statement**, the **input** payload, a complete **solution** in a black code window, the exact **output**, and a one-line explanation.
- Ten sections of ten problems each, moving from `map` and `filter` through `reduce`, `orderBy`, `groupBy`, `distinctBy`, `flatten`, `some`/`every`, and mixed real-world challenges that chain several of these together.

Almost every real integration script spends most of its logic reshaping arrays — this volume is built to make these functions feel automatic.

VOLUME 4

Array Transformations

100 problems on `map`, `filter`, `reduce`, `orderBy`, `groupBy`, `distinctBy`, `flatten`, `some`, `every`, and the array functions that turn raw lists into exactly the shape a downstream system needs.

SECTION 1 — map Basics (Problems 301–310)

#301 Extract One Field From Every Item

Extract just the SKU from every item in an order's item array.

INPUT

```
{ "items": [{"sku": "PALLET-01", "qty": 2}, {"sku": "CRATE-07", "qty": 1}] }
```

SOLUTION

```
payload.items map (item) -> item.sku
```

OUTPUT

```
["PALLET-01", "CRATE-07"]
```

`map` transforms an array of one shape into an array of another shape, running its lambda once per element.

#302 Double Every Number in an Array

Double every quantity in an array of Numbers.

INPUT

```
{ "quantities": [2, 5, 10] }
```

SOLUTION

```
payload.quantities map (q) -> q * 2
```

OUTPUT

```
[4, 10, 20]
```

The shorthand `$` could replace `q` here too — both refer to the current element inside the lambda.

#303 Reshape Each Item Into a New Object

Convert each item into a simplified object with just `code` and `units` fields.

INPUT

```
{ "items": [{"sku": "PALLET-01", "qty": 2}] }
```

SOLUTION

```
payload.items map (item) -> {
  code: item.sku,
  units: item.qty
}
```

OUTPUT

```
[{"code": "PALLET-01", "units": 2}]
```

`map`'s lambda can return a brand-new object literal, letting the output shape differ completely from the input shape.

#304 Coerce Every Element While Mapping

Convert an array of price Strings into an array of formatted currency Strings.

INPUT

```
{ "prices": ["45", "120.5"] }
```

SOLUTION

```
payload.prices map (p) -> (p as Number) as String {format: "$0.00"}
```

OUTPUT

```
["$45.00", "$120.50"]
```

Coercion and formatting logic can be combined inside a single `map` lambda without needing a separate pass.

#305 Use $ as the Implicit Lambda Parameter

Uppercase every carrier name in an array using the shorthand $ reference.

INPUT

```
{ "carriers": ["fedex", "ups"] }
```

SOLUTION

```
payload.carriers map upper($)
```

OUTPUT

```
["FEDEX", "UPS"]
```

When a `map` lambda is a single short expression, DataWeave lets you skip the named parameter and use `$` for the current element.

#306 Map Over an Array of Objects to Build Labels

Build a display label String for every order in an array.

INPUT

```
{ "orders": [{"id": "ORD-1042", "total": 240}, {"id": "ORD-1043", "total": 90}] }
```

SOLUTION

```
payload.orders map (o) -> o.id ++ " — $" ++ (o.total as String)
```

OUTPUT

```
["ORD-1042 — $240", "ORD-1043 — $90"]
```

Building a String label per item is one of the most common uses of `map` in report and email-generation transforms.

#307 Nest a map Inside Another map

For each order, uppercase every item code inside its item array.

INPUT

```
{ "orders": [{"id": "ORD-1042", "items": ["pallet-01", "crate-07"]}] }
```

SOLUTION

```
payload.orders map (o) -> {
  id: o.id,
  items: o.items map upper($)
}
```

OUTPUT

```
[{"id": "ORD-1042", "items": ["PALLET-01", "CRATE-07"]}]
```

A `map` lambda can itself contain another `map`, which is exactly how nested arrays get transformed.

#308 Apply Conditional Logic Inside map

Flag every order over $200 as `"HIGH_VALUE"` and every other order as `"STANDARD"`.

INPUT

```
{ "orders": [{"total": 90}, {"total": 350}] }
```

SOLUTION

```
payload.orders map (o) -> if (o.total > 200) "HIGH_VALUE" else "STANDARD"
```

OUTPUT

```
["STANDARD", "HIGH_VALUE"]
```

Combining `map` with `if/else` is the standard way to classify every element of an array in one pass.

#309 Map an Array of Strings to Numbers

Convert an array of numeric ID Strings into real Numbers for sorting later.

INPUT

```
{ "ids": ["104", "12", "58"] }
```

SOLUTION

```
payload.ids map (id) -> id as Number
```

OUTPUT

```
[104, 12, 58]
```

A coercion-only `map` is a common first step before applying a Number-aware function like `orderBy`.

#310 Build a Summary Object per Item

Build a per-item summary that includes a computed line total field, not present in the original data.

INPUT

```
{ "items": [{"sku": "PALLET-01", "unitPrice": 45, "qty": 2}] }
```

SOLUTION

```
payload.items map (item) -> item ++ { lineTotal: item.unitPrice * item.qty }
```

OUTPUT

```
[{"sku": "PALLET-01", "unitPrice": 45, "qty": 2, "lineTotal": 90}]
```

Merging the original item with a computed object using ++ keeps every original field while adding new derived ones.

SECTION 2 — map With Index (Problems 311–320)

#311 Attach a Position Number to Each Item

Add a 1-based `position` field to every item in an array using the index.

INPUT

```
{ "items": [{"sku": "PALLET-01"}, {"sku": "CRATE-07"}] }
```

SOLUTION

```
payload.items map (item, index) -> item ++ { position: index + 1 }
```

OUTPUT

```
[{"sku": "PALLET-01", "position": 1}, {"sku": "CRATE-07", "position": 2}]
```

The second lambda parameter of `map` is always the zero-based index of the current element.

#312 Build a Line Number Prefix

Build an invoice line description that starts with a 1-based line number.

INPUT

```
{ "lines": ["Freight", "Insurance", "Handling"] }
```

SOLUTION

```
payload.lines map (line, i) -> "Line " ++ ((i + 1) as String) ++ ": " ++ line
```

OUTPUT

```
["Line 1: Freight", "Line 2: Insurance", "Line 3: Handling"]
```

Combining the index with the element itself lets a single `map` produce a fully formatted, numbered list.

#313 Only Transform Even-Indexed Items

Tag every item at an even index (0, 2, 4...) as `"checked"` and every other item as `"skipped"`.

INPUT

```
{ "items": ["A", "B", "C", "D"] }
```

SOLUTION

```
payload.items map (item, i) -> if ((i mod 2) == 0) "checked" else "skipped"
```

OUTPUT

```
["checked", "skipped", "checked", "skipped"]
```

Combining the index with `mod` is a common way to apply alternating logic across an array, such as zebra-striping rows.

#314 Compare Each Item to the Previous Item's Index

Flag the very first item in an array (index 0) as the `"primary"` item.

INPUT

```
{ "contacts": ["Maria Chen", "James Cole"] }
```

SOLUTION

```
payload.contacts map (c, i) -> { name: c, role: if (i == 0) "primary" else
"secondary" }
```

OUTPUT

```
[{"name": "Maria Chen", "role": "primary"}, {"name": "James Cole", "role":
"secondary"}]
```

Checking `i == 0` is the standard way to single out the first element of an array while still processing every element with `map`.

#315 Generate a Zero-Padded Sequence Code

Generate a zero-padded sequence code like `"ITEM-001"` for every item using its index.

INPUT

```
{ "items": ["Pallet", "Crate"] }
```

SOLUTION

```
payload.items map (item, i) -> "ITEM-" ++ ((i + 1) as String {format: "000"})
```

OUTPUT

```
["ITEM-001", "ITEM-002"]
```

A Number format pattern like `"000"` zero-pads the index-derived sequence number to a fixed width.

#316 Identify the Last Element by Index

Mark the last item in an array as `isLast: true`, using `sizeOf` and the index.

INPUT

```
{ "steps": ["Pack", "Label", "Ship"] }
```

SOLUTION

```
payload.steps map (step, i) -> { step: step, isLast: i ==
(sizeOf(payload.steps) - 1) }
```

OUTPUT

```
[{"step": "Pack", "isLast": false}, {"step": "Label", "isLast": false},
{"step": "Ship", "isLast": true}]
```

Since arrays are zero-indexed, the last valid index is always `sizeOf(array) - 1`.

#317 Pair Each Item With the Next Item's Value

Build a String showing each stop and the next stop on a route, using the index to look ahead.

INPUT

```
{ "stops": ["Dallas", "Austin", "Houston"] }
```

SOLUTION

```
payload.stops map (stop, i) ->
  if (i < (sizeOf(payload.stops) - 1))
    stop ++ " -> " ++ payload.stops[i + 1]
  else
    stop ++ " (final stop)"
```

OUTPUT

```
["Dallas -> Austin", "Austin -> Houston", "Houston (final stop)"]
```

Because the whole source array is still in scope inside the lambda, the index can be used to look up a neighboring element directly.

#318 Assign a Priority Weight by Position

Assign each queued task a priority weight that decreases the further back it is in the queue.

INPUT

```
{ "tasks": ["Inspect", "Load", "Dispatch"] }
```

SOLUTION

```
payload.tasks map (task, i) -> { task: task, weight: 100 - (i * 10) }
```

OUTPUT

```
[{"task": "Inspect", "weight": 100}, {"task": "Load", "weight": 90}, {"task":
"Dispatch", "weight": 80}]
```

Turning the index into a numeric score is a simple way to encode "earlier means more important" without a separate sorting step.

#319 Combine map With Index and a Type Check

Build a row number label only for items that are Numbers, skipping any non-Number entries with a placeholder.

INPUT

```
{ "values": [10, "n/a", 30] }
```

SOLUTION

```
payload.values map (v, i) ->
  "Row " ++ ((i + 1) as String) ++ ": " ++
  (v match {
    case n is Number -> n as String
    else -> "invalid"
  })
```

OUTPUT

```
["Row 1: 10", "Row 2: invalid", "Row 3: 30"]
```

Index-aware labeling and per-element type checking combine cleanly, since both are just expressions inside the same lambda.

#320 Number a CargoLink Multi-Stop Manifest

Build a numbered delivery manifest, marking the first stop as `"origin"` and all later stops with their stop number.

INPUT

```
{ "stops": ["CargoLink Depot", "Austin Hub", "Customer Site"] }
```

SOLUTION

```
payload.stops map (stop, i) -> {
  location: stop,
  label: if (i == 0) "origin" else "stop " ++ (i as String)
}
```

OUTPUT

```
[{"location": "CargoLink Depot", "label": "origin"}, {"location": "Austin Hub", "label": "stop 1"}, {"location": "Customer Site", "label": "stop 2"}]
```

This is exactly the shape a manifest-generation transform takes in a real logistics integration: index-aware labeling on top of a straightforward field reshape.

SECTION 3 — filter Basics (Problems 321–330)

#321 Keep Only In-Stock Items

Filter an array of items down to only the ones currently in stock.

INPUT

```
{ "items": [{"sku": "A", "inStock": true}, {"sku": "B", "inStock": false}] }
```

SOLUTION

```
payload.items filter ($.inStock)
```

OUTPUT

```
[{"sku": "A", "inStock": true}]
```

`filter` keeps only the elements for which the lambda returns `true`, dropping everything else.

#322 Filter by a Numeric Threshold

Keep only orders with a total greater than or equal to $100.

INPUT

```
{ "orders": [{"id": 1, "total": 40}, {"id": 2, "total": 150}] }
```

SOLUTION

```
payload.orders filter ($.total >= 100)
```

OUTPUT

```
[{"id": 2, "total": 150}]
```

Any Boolean-producing expression can be the `filter` condition, including a plain comparison on a field.

#323 Filter With a Named Lambda Parameter

Filter an array of drivers to only those based in the `"TX"` region, using a named parameter.

INPUT

```
{ "drivers": [{"name": "Sam", "region": "TX"}, {"name": "Lee", "region": "CA"}] }
```

SOLUTION

```
payload.drivers filter (driver) -> driver.region == "TX"
```

OUTPUT

```
[{"name": "Sam", "region": "TX"}]
```

A named parameter reads more clearly than `$` once the condition itself grows more complex or is checked more than once.

#324 Filter Using the Index

Keep only the items at even index positions.

INPUT

```
{ "items": ["A", "B", "C", "D", "E"] }
```

SOLUTION

```
payload.items filter (item, i) -> (i mod 2) == 0
```

OUTPUT

```
["A", "C", "E"]
```

Just like `map`, `filter`'s lambda accepts an optional second parameter for the current index.

#325 Filter With a Combined and Condition

Keep only shipments that are both international and over 50kg.

INPUT

```
{ "shipments": [{"international": true, "weightKg": 60}, {"international": false, "weightKg": 80}] }
```

SOLUTION

```
payload.shipments filter ($.international and ($.weightKg > 50))
```

OUTPUT

```
[{"international": true, "weightKg": 60}]
```

Logical operators combine with `filter` exactly as they do anywhere else, letting one condition express multiple requirements.

#326 Filter Out Null Values From an Array

Remove `null` entries from an array before further processing.

INPUT

```
{ "values": [10, null, 20, null, 30] }
```

SOLUTION

```
payload.values filter ($ != null)
```

OUTPUT

```
[10, 20, 30]
```

This "compact the array" pattern is one of the most common data-cleaning uses of `filter` in real integration flows.

#327 Filter Strings by Length

Keep only tracking codes that are exactly 8 characters long.

INPUT

```
{ "codes": ["CL881301", "CL99", "SHIP4512"] }
```

SOLUTION

```
payload.codes filter (sizeOf($) == 8)
```

OUTPUT

```
["CL881301", "SHIP4512"]
```

`sizeOf` returns a String's character count, making it easy to filter by exact or minimum length.

#328 Filter Objects by a Nested Field

Keep only orders whose shipping address is in the `"US"`.

INPUT

```
{ "orders": [{"id": 1, "address": {"country": "US"}}, {"id": 2, "address":
{"country": "CA"}}] }
```

SOLUTION

```
payload.orders filter ($.address.country == "US")
```

OUTPUT

```
[{"id": 1, "address": {"country": "US"}}]
```

A `filter` condition can chain into a nested selector just like any other expression.

#329 Filter an Array Using match Inside the Condition

Keep only orders whose status is one of a small set of "active" statuses.

INPUT

```
{ "orders": [{"status": "PENDING"}, {"status": "DELIVERED"}, {"status":
"PROCESSING"}] }
```

SOLUTION

```
payload.orders filter ($.status match {
  case "PENDING" -> true
  case "PROCESSING" -> true
  else -> false
})
```

OUTPUT

```
[{"status": "PENDING"}, {"status": "PROCESSING"}]
```

A `match` expression can act as the Boolean condition for `filter` whenever the rule is a small closed list of values.

#330 Chain filter Directly After a Selector

From a CargoLink shipment batch, keep only shipments that have not yet been delivered.

INPUT

```
{ "shipments": [{"id": "SHP-501", "status": "IN_TRANSIT"}, {"id": "SHP-502",
"status": "DELIVERED"}] }
```

SOLUTION

```
payload.shipments filter ($.status != "DELIVERED")
```

OUTPUT

```
[{"id": "SHP-501", "status": "IN_TRANSIT"}]
```

Filtering out one specific completed state is a common way to build an "active work remaining" view over a larger dataset.

SECTION 4 — reduce Basics (Problems 331–340)

#331 Sum an Array With reduce

Add up an array of shipment costs into a single grand total using `reduce`.

INPUT

```
{ "costs": [50, 75, 25] }
```

SOLUTION

```
payload.costs reduce ((cost, total = 0) -> total + cost)
```

OUTPUT

```
150
```

`reduce` walks the array once, carrying an accumulator forward from element to element until a single final value remains.

#332 Find the Maximum Value With reduce

Find the heaviest shipment weight in an array using `reduce`.

INPUT

```
{ "weights": [40, 120, 65, 90] }
```

SOLUTION

```
payload.weights reduce ((w, max = 0) -> if (w > max) w else max)
```

OUTPUT

```
120
```

The accumulator does not have to be a running sum — here it tracks the largest value seen so far.

#333 Build a String With reduce

Join an array of stop names into a single dash-separated route String using `reduce`.

INPUT

```
{ "stops": ["Dallas", "Austin", "Houston"] }
```

SOLUTION

```
payload.stops reduce ((stop, route = "") -> if (route == "") stop else route
++ " - " ++ stop)
```

OUTPUT

```
"Dallas - Austin - Houston"
```

An accumulator can just as easily be a String as a Number — `reduce` works with any type of running result.

#334 Build an Object With reduce

Turn an array of items into a lookup Object keyed by SKU, using `reduce`.

INPUT

```
{ "items": [{"sku": "A", "qty": 2}, {"sku": "B", "qty": 5}] }
```

SOLUTION

```
payload.items reduce ((item, lookup = {}) -> lookup ++ { (item.sku): item.qty
})
```

OUTPUT

```
{ "A": 2, "B": 5 }
```

Using `++` to merge a new single-key object into the accumulator on each step is the standard way to build a lookup table with `reduce`.

#335 Count Matching Elements With reduce

Count how many orders in an array are marked as priority, using `reduce`.

INPUT

```
{ "orders": [{"priority": true}, {"priority": false}, {"priority": true}] }
```

SOLUTION

```
payload.orders reduce ((o, count = 0) -> if (o.priority) count + 1 else
count)
```

OUTPUT

```
2
```

A conditional accumulator update is how `reduce` expresses a counting operation, one match at a time.

#336 Flatten Nested Arrays With reduce

Combine several orders' item arrays into a single flat array using `reduce`.

INPUT

```
{ "orders": [{"items": ["A", "B"]}, {"items": ["C"]}] }
```

SOLUTION

```
payload.orders reduce ((o, all = []) -> all ++ o.items)
```

OUTPUT

```
["A", "B", "C"]
```

Merging each element's own array into a growing accumulator with ++ is a manual, `reduce`-based version of what `flatten` does automatically.

#337 Compute a Running Average With reduce

Compute the average of an array of delivery times using `reduce` for the sum and `sizeOf` for the count.

INPUT

```
{ "deliveryDays": [2, 4, 3, 5] }
```

SOLUTION

```
(payload.deliveryDays reduce ((d, total = 0) -> total + d)) /
sizeOf(payload.deliveryDays)
```

OUTPUT

```
3.5
```

`reduce` handles the summation, and a plain division by `sizeOf` finishes the average calculation.

#338 Track the Minimum With reduce

Find the lowest-priced item in an array using `reduce`, starting from the first item's price as the seed.

INPUT

```
{ "prices": [45, 12, 80, 30] }
```

SOLUTION

```
payload.prices reduce ((p, min = payload.prices[0]) -> if (p < min) p else
min)
```

OUTPUT

```
12
```

Seeding the accumulator with the first element itself (instead of a fixed constant like `0`) avoids assumptions about the value range.

#339 Build a Running Total Array With reduce

Build an array of running totals, where each entry is the cumulative sum up to that point.

INPUT

```
{ "amounts": [10, 20, 30] }
```

SOLUTION

```
payload.amounts reduce ((amt, acc = []) ->
  acc ++ [(if (sizeOf(acc) == 0) 0 else acc[-1]) + amt]
)
```

OUTPUT

```
[10, 30, 60]
```

Because the accumulator itself is an Array here, each step both reads the last running total and appends the next one.

#340 Summarize a CargoLink Order With reduce

Summarize a CargoLink order's items into a single object with total quantity and total cost, using one `reduce` pass.

INPUT

```
{ "items": [{"qty": 2, "unitPrice": 45}, {"qty": 1, "unitPrice": 90}] }
```

SOLUTION

```
payload.items reduce ((item, summary = {totalQty: 0, totalCost: 0}) -> {
  totalQty: summary.totalQty + item.qty,
  totalCost: summary.totalCost + (item.qty * item.unitPrice)
})
```

OUTPUT

```
{ "totalQty": 3, "totalCost": 180 }
```

An Object accumulator lets a single `reduce` pass compute several related totals at once, instead of scanning the array multiple times.

SECTION 5 — orderBy (Problems 341–350)

#341 Sort Numbers in Ascending Order

Sort an array of shipment weights from lightest to heaviest.

INPUT

```
{ "weights": [80, 20, 45] }
```

SOLUTION

```
payload.weights orderBy $
```

OUTPUT

```
[20, 45, 80]
```

`orderBy` *sorts ascending by default, using whatever key expression you give it — here, the value itself.*

#342 Sort Objects by a Numeric Field

Sort an array of orders by total, from lowest to highest.

INPUT

```
{ "orders": [{"id": 1, "total": 200}, {"id": 2, "total": 50}] }
```

SOLUTION

```
payload.orders orderBy $.total
```

OUTPUT

```
[{"id": 2, "total": 50}, {"id": 1, "total": 200}]
```

The key expression can select any field, and the whole objects are reordered — not just the field values.

#343 Sort Strings Alphabetically

Sort an array of customer names alphabetically.

INPUT

```
{ "names": ["Maria Chen", "Ana Ruiz", "James Cole"] }
```

SOLUTION

```
payload.names orderBy $
```

OUTPUT

```
["Ana Ruiz", "James Cole", "Maria Chen"]
```

`orderBy` uses lexicographic order for Strings, the same ordering used by the < and > operators.

#344 Sort in Descending Order

Sort an array of order totals from highest to lowest.

INPUT

```
{ "totals": [50, 300, 120] }
```

SOLUTION

```
reverse(payload.totals orderBy $)
```

OUTPUT

```
[300, 120, 50]
```

DataWeave's `orderBy` only sorts ascending, so descending order is produced by wrapping the sorted result in `reverse`.

#345 Sort Objects by a String Field

Sort an array of warehouses by their region code alphabetically.

INPUT

```
{ "warehouses": [{"id": "WH-1", "region": "WEST"}, {"id": "WH-2", "region": "EAST"}] }
```

SOLUTION

```
payload.warehouses orderBy $.region
```

OUTPUT

```
[{"id": "WH-2", "region": "EAST"}, {"id": "WH-1", "region": "WEST"}]
```

The same `orderBy $.field` pattern works regardless of whether the field's values are Numbers or Strings.

#346 Sort by a Date Field

Sort an array of shipments by their ship Date, earliest first.

INPUT

```
{ "shipments": [{"id": "SHP-1", "shipDate": "2026-03-20"}, {"id": "SHP-2", "shipDate": "2026-03-05"}] }
```

SOLUTION

```
payload.shipments orderBy ($.shipDate as Date)
```

OUTPUT

```
[{"id": "SHP-2", "shipDate": "2026-03-05"}, {"id": "SHP-1", "shipDate": "2026-03-20"}]
```

Coercing the key expression to `Date` ensures the sort follows chronological order rather than alphabetical String order.

#347 Sort Numeric Strings Correctly

Sort an array of numeric ID Strings in true numeric order, not alphabetic order.

INPUT

```
{ "ids": ["9", "10", "2"] }
```

SOLUTION

```
payload.ids orderBy ($ as Number)
```

OUTPUT

```
["2", "9", "10"]
```

The key expression coerces each element for comparison purposes only — the original String values are still what appears in the output.

#348 Sort by a Computed Key

Sort items by their computed line total (unit price times quantity), not by a stored field.

INPUT

```
{ "items": [{"sku": "A", "price": 10, "qty": 5}, {"sku": "B", "price": 40,
"qty": 1}] }
```

SOLUTION

```
payload.items orderBy ($.price * $.qty)
```

OUTPUT

```
[{"sku": "B", "price": 40, "qty": 1}, {"sku": "A", "price": 10, "qty": 5}]
```

The `orderBy` key does not have to be an existing field — any expression that produces a comparable value will work.

#349 Sort, Then Take the Top Result

Find the single most expensive order by sorting descending and taking the first element.

INPUT

```
{ "orders": [{"id": 1, "total": 90}, {"id": 2, "total": 300}, {"id": 3, "total": 150}] }
```

SOLUTION

```
reverse(payload.orders orderBy $.total)[0]
```

OUTPUT

```
{ "id": 2, "total": 300 }
```

Sorting the full array and indexing into the result is a simple, reliable way to find a single "top" or "bottom" record.

#350 Build a Ranked CargoLink Driver Leaderboard

Build a driver leaderboard sorted by on-time percentage descending, with a 1-based rank attached to each entry.

INPUT

```
{ "drivers": [{"name": "Sam", "onTimePct": 92}, {"name": "Lee", "onTimePct": 98}, {"name": "Kai", "onTimePct": 85}] }
```

SOLUTION

```
reverse(payload.drivers orderBy $.onTimePct) map (d, i) -> d ++ { rank: i + 1 }
```

OUTPUT

```
[{"name": "Lee", "onTimePct": 98, "rank": 1}, {"name": "Sam", "onTimePct": 92, "rank": 2}, {"name": "Kai", "onTimePct": 85, "rank": 3}]
```

Chaining `orderBy`, `reverse`, and an index-aware `map` is exactly how a real leaderboard transform is built in three clean steps.

SECTION 6 — groupBy (Problems 351–360)

#351 Group Orders by Region

Group an array of orders into an Object keyed by shipping region.

INPUT

```
{ "orders": [{"id": 1, "region": "TX"}, {"id": 2, "region": "CA"}, {"id": 3, "region": "TX"}] }
```

SOLUTION

```
payload.orders groupBy $.region
```

OUTPUT

```
{ "TX": [{"id": 1, "region": "TX"}, {"id": 3, "region": "TX"}], "CA": [{"id": 2, "region": "CA"}] }
```

`groupBy` *turns an Array into an Object, where each key is a distinct value of the key expression and each value is the array of matching elements.*

#352 Group Shipments by Status

Group an array of shipments by their current status.

INPUT

```
{ "shipments": [{"id": "SHP-1", "status": "IN_TRANSIT"}, {"id": "SHP-2",
"status": "DELIVERED"}, {"id": "SHP-3", "status": "IN_TRANSIT"}] }
```

SOLUTION

```
payload.shipments groupBy $.status
```

OUTPUT

```
{ "IN_TRANSIT": [{"id": "SHP-1", "status": "IN_TRANSIT"}, {"id": "SHP-3",
"status": "IN_TRANSIT"}], "DELIVERED": [{"id": "SHP-2", "status":
"DELIVERED"}] }
```

Grouping by status is the standard first step before generating a per-status summary report.

#353 Count Items per Group

Count how many orders fall into each region, instead of keeping the full grouped arrays.

INPUT

```
{ "orders": [{"region": "TX"}, {"region": "CA"}, {"region": "TX"}] }
```

SOLUTION

```
payload.orders groupBy $.region mapObject (value, key) -> { (key):
sizeOf(value) }
```

OUTPUT

```
{ "TX": 2, "CA": 1 }
```

Chaining `mapObject` right after `groupBy` is the standard way to turn grouped arrays into per-group counts.

#354 Group by a Computed Key

Group shipments into weight tiers by computing the tier inside the `groupBy` key expression.

INPUT

```
{ "shipments": [{"id": 1, "weightKg": 30}, {"id": 2, "weightKg": 300}] }
```

SOLUTION

```
payload.shipments groupBy (s) -> if (s.weightKg >= 150) "TL" else "LTL"
```

OUTPUT

```
{ "LTL": [{"id": 1, "weightKg": 30}], "TL": [{"id": 2, "weightKg": 300}] }
```

The key expression is not limited to a plain field selector — it can be any expression, including a full `if/else`.

#355 Extract Just the Group Names

Get the distinct list of regions present in an order array, without the grouped data itself.

INPUT

```
{ "orders": [{"region": "TX"}, {"region": "CA"}, {"region": "TX"}] }
```

SOLUTION

```
namesOf(payload.orders groupBy $.region)
```

OUTPUT

```
["TX", "CA"]
```

`namesOf` returns an Object's key names as an Array — combined with `groupBy`, it's a quick way to get a distinct-values list.

#356 Group by a Boolean Field

Split orders into two groups based on their priority flag.

INPUT

```
{ "orders": [{"id": 1, "priority": true}, {"id": 2, "priority": false}, {"id": 3, "priority": true}] }
```

SOLUTION

```
payload.orders groupBy $.priority
```

OUTPUT

```
{ "true": [{"id": 1, "priority": true}, {"id": 3, "priority": true}], "false": [{"id": 2, "priority": false}] }
```

Grouping by a Boolean field produces at most two groups — `groupBy` coerces the key to a String for use as an Object key.

#357 Group by a Nested Field

Group orders by the country nested inside their shipping address.

INPUT

```
{ "orders": [{"id": 1, "address": {"country": "US"}}, {"id": 2, "address": {"country": "MX"}}] }
```

SOLUTION

```
payload.orders groupBy $.address.country
```

OUTPUT

```
{ "US": [{"id": 1, "address": {"country": "US"}}], "MX": [{"id": 2, "address": {"country": "MX"}}] }
```

The key expression can chain as deep into each element's structure as needed.

#358 Extract a Single Group After Grouping

Group shipments by carrier, then pull out just the FedEx group.

INPUT

```
{ "shipments": [{"id": 1, "carrier": "FEDEX"}, {"id": 2, "carrier": "UPS"},
{"id": 3, "carrier": "FEDEX"}] }
```

SOLUTION

```
(payload.shipments groupBy $.carrier).FEDEX
```

OUTPUT

```
[{"id": 1, "carrier": "FEDEX"}, {"id": 3, "carrier": "FEDEX"}]
```

Since `groupBy` produces a plain Object, a normal dot selector can pull out any one group by name directly afterward.

#359 Sort Within Each Group

Group orders by region, and within each region's array, sort the orders by total descending.

INPUT

```
{ "orders": [{"region": "TX", "total": 50}, {"region": "TX", "total": 300}] }
```

SOLUTION

```
payload.orders groupBy $.region mapObject (value, key) -> { (key):
reverse(value orderBy $.total) }
```

OUTPUT

```
{ "TX": [{"region": "TX", "total": 300}, {"region": "TX", "total": 50}] }
```

Because each group's value is itself an Array, any array function — including `orderBy` — can be applied to it inside the `mapObject` step.

#360 Summarize CargoLink Shipments by Carrier

Group CargoLink shipments by carrier and compute the total weight shipped per carrier.

INPUT

```
{ "shipments": [{"carrier": "FEDEX", "weightKg": 40}, {"carrier": "UPS",
"weightKg": 25}, {"carrier": "FEDEX", "weightKg": 15}] }
```

SOLUTION

```
payload.shipments groupBy $.carrier mapObject (value, key) -> {
  (key): value reduce ((s, total = 0) -> total + s.weightKg)
}
```

OUTPUT

```
{ "FEDEX": 55, "UPS": 25 }
```

Chaining `groupBy` into `mapObject` and `reduce` is exactly how a real per-carrier volume report is built from a raw shipment list.

SECTION 7 — distinctBy & flatten (Problems 361–370)

#361 Remove Duplicate SKUs

Remove duplicate item objects from an array, keyed by SKU.

INPUT

```
{ "items": [{"sku": "A", "qty": 1}, {"sku": "B", "qty": 2}, {"sku": "A",
"qty": 5}] }
```

SOLUTION

```
payload.items distinctBy $.sku
```

OUTPUT

```
[{"sku": "A", "qty": 1}, {"sku": "B", "qty": 2}]
```

`distinctBy` keeps only the first element seen for each distinct key value, dropping every later duplicate.

#362 Deduplicate Using a Computed Key

Deduplicate a list of customer contacts by their lowercase email, so case differences don't count as distinct.

INPUT

```
{ "contacts": [{"email": "Maria@CargoLink.com"}, {"email":
"maria@cargolink.com"}] }
```

SOLUTION

```
payload.contacts distinctBy (lower($.email))
```

OUTPUT

```
[{"email": "Maria@CargoLink.com"}]
```

The key expression can transform the value before comparing it, which is essential for case-insensitive or format-insensitive deduplication.

#363 Confirm distinctBy Keeps the First Occurrence

Deduplicate a list of status updates by order ID, keeping the earliest update for each order.

INPUT

```
{ "updates": [{"orderId": "ORD-1", "status": "PENDING"}, {"orderId": "ORD-1", "status": "SHIPPED"}] }
```

SOLUTION

```
payload.updates distinctBy $.orderId
```

OUTPUT

```
[{"orderId": "ORD-1", "status": "PENDING"}]
```

Because `distinctBy` always keeps the first match in array order, the array should already be sorted correctly before deduplicating if order matters.

#364 Deduplicate, Then Reshape

Deduplicate items by SKU, then map the result down to just the SKU values.

INPUT

```
{ "items": [{"sku": "A"}, {"sku": "B"}, {"sku": "A"}] }
```

SOLUTION

```
(payload.items distinctBy $.sku) map $.sku
```

OUTPUT

```
["A", "B"]
```

Chaining `distinctBy` into `map` is the standard pattern for producing a clean, unique list of values from a larger dataset.

#365 Flatten an Array of Arrays

Flatten a nested array of item lists into a single flat array.

INPUT

```
{ "batches": [["A", "B"], ["C"], ["D", "E"]] }
```

SOLUTION

```
flatten(payload.batches)
```

OUTPUT

```
["A", "B", "C", "D", "E"]
```

`flatten` collapses one level of array nesting, merging every inner array's elements into a single outer array.

#366 Map Then Flatten

Get a single flat array of every item SKU across all of a customer's orders.

INPUT

```
{ "orders": [{"items": ["A", "B"]}, {"items": ["C"]}] }
```

SOLUTION

```
flatten(payload.orders map $.items)
```

OUTPUT

```
["A", "B", "C"]
```

This "map then flatten" combination is one of the most common two-step patterns in DataWeave — first extract, then merge.

#367 Flatten Just One Level

Confirm that `flatten` merges only the outer array of arrays — inner array structure below that level is preserved.

INPUT

```
{ "groups": [[["A", "B"]], [["C"]]] }
```

SOLUTION

```
flatten(payload.groups)
```

OUTPUT

```
[["A", "B"], ["C"]]
```

A doubly-nested array only loses its outermost layer of nesting after one `flatten` call — a second `flatten` call would be needed to go one level deeper.

#368 Merge Multiple Manifests Into One

Combine several trucks' individual manifests into a single combined shipment list.

INPUT

```
{ "trucks": [{"manifest": ["SHP-1", "SHP-2"]}, {"manifest": ["SHP-3"]}] }
```

SOLUTION

```
flatten(payload.trucks map $.manifest)
```

OUTPUT

```
["SHP-1", "SHP-2", "SHP-3"]
```

Merging many small lists into one combined list is exactly the kind of transform `flatten` was built for.

#369 Flatten, Then Deduplicate

Merge two routes' stop lists into one, removing any stop that appears on both routes.

INPUT

```
{ "routes": [["Dallas", "Austin"], ["Austin", "Houston"]] }
```

SOLUTION

```
flatten(payload.routes) distinctBy $
```

OUTPUT

```
["Dallas", "Austin", "Houston"]
```

Flattening first and deduplicating second is the standard order of operations — deduplication only makes sense once everything is in one array.

#370 Build a Combined, Deduplicated CargoLink Route

Combine three trucks' stop lists into a single deduplicated route plan, preserving the order stops were first seen.

INPUT

```
{ "trucks": [{"stops": ["Depot", "Austin"]}, {"stops": ["Austin", "Houston"]},
{"stops": ["Depot", "Waco"]}] }
```

SOLUTION

```
flatten(payload.trucks map $.stops) distinctBy $
```

OUTPUT

```
["Depot", "Austin", "Houston", "Waco"]
```

This three-truck route merge is a realistic example of the "extract, flatten, deduplicate" pipeline used to consolidate overlapping datasets from multiple sources.

SECTION 8 — some, every & indexOf (Problems 371–380)

#371 Check if Any Item Matches a Condition

Check whether any item in a shipment is marked as fragile.

INPUT

```
{ "items": [{"sku": "A", "fragile": false}, {"sku": "B", "fragile": true}] }
```

SOLUTION

```
payload.items some ((item) -> item.fragile)
```

OUTPUT

```
true
```

`some` returns `true` as soon as one element satisfies the condition, without needing to build a filtered array first.

#372 Check if Every Item Matches a Condition

Check whether every item in an order is currently in stock.

INPUT

```
{ "items": [{"sku": "A", "inStock": true}, {"sku": "B", "inStock": true}] }
```

SOLUTION

```
payload.items every ((item) -> item.inStock)
```

OUTPUT

```
true
```

`every` returns `true` only if every single element satisfies the condition — one failure makes the whole result `false`.

#373 Check for a High-Value Item Anywhere in an Order

Flag an order for extra insurance if any single item is worth more than $500.

INPUT

```
{ "items": [{"sku": "A", "value": 120}, {"sku": "B", "value": 650}] }
```

SOLUTION

```
payload.items some ((item) -> item.value > 500)
```

OUTPUT

```
true
```

This is the classic use case for `some`: answering a yes/no question about a whole collection based on just one qualifying element.

#374 Validate That All Items Pass a Rule

Confirm that every item in a shipment has a non-empty SKU before allowing it to be processed.

INPUT

```
{ "items": [{"sku": "A"}, {"sku": "B"}] }
```

SOLUTION

```
payload.items every ((item) -> item.sku != "")
```

OUTPUT

```
true
```

Using `every` as a validation gate before further processing is a common defensive pattern in integration flows.

#375 Find the Position of a Value

Find the index of a specific tracking code within an array of tracking codes.

INPUT

```
{ "codes": ["CL001", "CL002", "CL003"] }
```

SOLUTION

```
indexOf(payload.codes, "CL002")
```

OUTPUT

```
1
```

`indexOf` returns the zero-based position of the first match, or `-1` if the value is not found at all.

#376 Check Existence Using indexOf

Check whether a given item code exists anywhere in an order's item array.

INPUT

```
{ "items": ["PALLET-01", "CRATE-07"], "search": "CRATE-07" }
```

SOLUTION

```
indexOf(payload.items, payload.search) >= 0
```

OUTPUT

```
true
```

Checking that `indexOf` did not return `-1` is a direct way to answer "does this array contain this value?"

#377 Compare some to filter for a Yes/No Check

Show that checking `some` gives the same Boolean answer as filtering and checking the result is non-empty, for a "does a cancelled item exist" question.

INPUT

```
{ "items": [{"status": "ACTIVE"}, {"status": "CANCELLED"}] }
```

SOLUTION

```
{
  usingSome: payload.items some ((i) -> i.status == "CANCELLED"),
  usingFilter: sizeOf(payload.items filter ($.status == "CANCELLED")) > 0
}
```

OUTPUT

```
{ "usingSome": true, "usingFilter": true }
```

Both approaches agree, but `some` is the clearer and more efficient choice when all you need is a yes/no answer, not the matching elements themselves.

#378 Check every on an Empty Array

Confirm what `every` returns when applied to an empty array of items.

INPUT

```
{ "items": [] }
```

SOLUTION

```
payload.items every ((item) -> item.inStock)
```

OUTPUT

```
true
```

`every` on an empty array is `true` by definition — there are no elements to fail the check, a rule known as vacuous truth.

#379 Build a Combined Validation Summary

Build a validation summary object using both `some` and `every` to check different rules on the same order.

INPUT

```
{ "items": [{"inStock": true, "fragile": false}, {"inStock": true, "fragile": true}] }
```

SOLUTION

```
{
  allInStock: payload.items every ((i) -> i.inStock),
  hasFragileItem: payload.items some ((i) -> i.fragile)
}
```

OUTPUT

```
{ "allInStock": true, "hasFragileItem": true }
```

`some` and `every` answer different questions about the same array and combine naturally into a single validation report.

#380 Run a CargoLink Customs Compliance Check

Confirm that every item in an international CargoLink shipment has a completed customs declaration.

INPUT

```
{ "items": [{"sku": "A", "customsDeclared": true}, {"sku": "B", "customsDeclared": true}] }
```

SOLUTION

```
payload.items every ((item) -> item.customsDeclared)
```

OUTPUT

```
true
```

A compliance gate like this is a realistic pre-shipment check: the whole shipment can only proceed if `every` item passes.

SECTION 9 — Chaining map, filter & reduce (Problems 381–390)

#381 Filter, Then Map

Get the SKUs of only the in-stock items in an order.

INPUT

```
{ "items": [{"sku": "A", "inStock": true}, {"sku": "B", "inStock": false}] }
```

SOLUTION

```
(payload.items filter $.inStock) map $.sku
```

OUTPUT

```
["A"]
```

Filtering before mapping keeps each step focused on one job — first narrow the array, then reshape what's left.

#382 Map, Then Filter

Convert item prices to numbers, then keep only the ones over $50.

INPUT

```
{ "prices": ["30", "80", "45", "120"] }
```

SOLUTION

```
(payload.prices map ($ as Number)) filter ($ > 50)
```

OUTPUT

```
[80, 120]
```

The order matters here — coercing to Number first means the `filter` comparison works correctly against real numeric values.

#383 Filter, Then Reduce

Sum the totals of only the delivered orders in a batch.

INPUT

```
{ "orders": [{"total": 100, "status": "DELIVERED"}, {"total": 50, "status": "PENDING"}, {"total": 200, "status": "DELIVERED"}] }
```

SOLUTION

```
(payload.orders filter ($.status == "DELIVERED")) reduce ((o, total = 0) -> total + o.total)
```

OUTPUT

```
300
```

Filtering to the relevant subset before reducing avoids having to add conditional logic inside the `reduce` lambda itself.

#384 Map, Then Reduce

Compute the total weight of a shipment by first extracting each item's weight, then summing.

INPUT

```
{ "items": [{"sku": "A", "weightKg": 5}, {"sku": "B", "weightKg": 12}] }
```

SOLUTION

```
(payload.items map $.weightKg) reduce ((w, total = 0) -> total + w)
```

OUTPUT

```
17
```

This "extract, then sum" chain is one of the most common patterns in DataWeave for turning a list of objects into a single aggregate.

#385 Filter, Map, and Reduce in One Chain

Calculate the total value of only the fragile items in a shipment.

INPUT

```
{ "items": [{"sku": "A", "value": 50, "fragile": true}, {"sku": "B", "value": 200, "fragile": false}, {"sku": "C", "value": 30, "fragile": true}] }
```

SOLUTION

```
((payload.items filter $.fragile) map $.value) reduce ((v, total = 0) -> total + v)
```

OUTPUT

```
80
```

Chaining `filter`, `map`, and `reduce` together is the same three-step shape as a SQL `WHERE` + `SELECT` + `SUM` query, expressed in DataWeave.

#386 Sort, Filter, and Map to Build a Top-N List

Get the names of the two highest-spending customers.

INPUT

```
{ "customers": [{"name": "Ana", "spend": 400}, {"name": "Sam", "spend": 900}, {"name": "Lee", "spend": 650}] }
```

SOLUTION

```
(reverse(payload.customers orderBy $.spend))[0 to 1] map $.name
```

OUTPUT

```
["Sam", "Lee"]
```

Sorting descending, taking a range with `[0 to 1]`, and mapping to a single field is the standard three-step recipe for a "top N" report.

#387 Group, Then Summarize Each Group

Group orders by region and compute the average order total for each region.

INPUT

```
{ "orders": [{"region": "TX", "total": 100}, {"region": "TX", "total": 200},
{"region": "CA", "total": 50}] }
```

SOLUTION

```
payload.orders groupBy $.region mapObject (value, key) -> {
  (key): (value reduce ((o, t = 0) -> t + o.total)) / sizeOf(value)
}
```

OUTPUT

```
{ "TX": 150, "CA": 50 }
```

This grouped-average pattern — `groupBy`, then `reduce` and `sizeOf` inside `mapObject` — is exactly how per-category KPI dashboards are built.

#388 Deduplicate, Then Sort

Get a sorted, deduplicated list of carrier names from a shipment batch.

INPUT

```
{ "shipments": [{"carrier": "UPS"}, {"carrier": "FEDEX"}, {"carrier": "UPS"}]
}
```

SOLUTION

```
((payload.shipments distinctBy $.carrier) map $.carrier) orderBy $
```

OUTPUT

```
["FEDEX", "UPS"]
```

Deduplicating before sorting is slightly more efficient than the reverse order, since the sort then has fewer elements to work through.

#389 Flatten, Filter, and Reduce Across Multiple Sources

Combine several trucks' item lists, keep only fragile items, and count how many there are in total.

INPUT

```
{ "trucks": [{"items": [{"sku": "A", "fragile": true}]}, {"items": [{"sku":
"B", "fragile": false}, {"sku": "C", "fragile": true}]}] }
```

SOLUTION

```
sizeOf(flatten(payload.trucks map $.items) filter $.fragile)
```

OUTPUT

```
2
```

A four-step chain like this — `map`, `flatten`, `filter`, `sizeOf` — is common once data starts nested two levels deep across multiple source records.

#390 Build a Full CargoLink Region Summary Report

Build a report grouping orders by region, with each region showing its order count and total revenue, sorted by revenue descending.

INPUT

```
{ "orders": [{"region": "TX", "total": 100}, {"region": "CA", "total": 500},
{"region": "TX", "total": 300}] }
```

SOLUTION

```
var byRegion = payload.orders groupBy $.region mapObject (value, key) -> {
  region: key,
  orderCount: sizeOf(value),
  revenue: value reduce ((o, t = 0) -> t + o.total)
}
---
reverse((byRegion pluck (v) -> v) orderBy $.revenue)
```

OUTPUT

```
[{"region": "CA", "orderCount": 1, "revenue": 500}, {"region": "TX",
"orderCount": 2, "revenue": 400}]
```

This end-to-end report — group, summarize, convert back to an array with `pluck`, then sort and reverse — is close to the exact shape a real regional sales dashboard transform would take, with the highest-revenue region listed first.

SECTION 10 — Mixed Real-World Array Challenges (Problems 391–400)

#391 Build a CargoLink Picking List

Build a warehouse picking list: only in-stock items, sorted by warehouse aisle number.

INPUT

```
{ "items": [{"sku": "A", "aisle": 3, "inStock": true}, {"sku": "B", "aisle": 1, "inStock": false}, {"sku": "C", "aisle": 2, "inStock": true}] }
```

SOLUTION

```
(payload.items filter $.inStock) orderBy $.aisle
```

OUTPUT

```
[{"sku": "C", "aisle": 2, "inStock": true}, {"sku": "A", "aisle": 3, "inStock": true}]
```

A warehouse picking list is a textbook "filter to what's actionable, then sort by physical location" transform.

#392 Flag CargoLink Orders Needing Manual Review

From a batch of orders, extract just the IDs of orders that are over $1,000 or have any single item over $500.

INPUT

```
{ "orders": [{"id": "O1", "total": 1200, "items": [{"value": 100}]}, {"id": "O2", "total": 300, "items": [{"value": 600}]}, {"id": "O3", "total": 200, "items": [{"value": 50}]}] }
```

SOLUTION

```
(payload.orders filter (($.total > 1000) or ($.items some ((i) -> i.value > 500)))) map $.id
```

OUTPUT

```
["O1", "O2"]
```

Combining `filter` with a nested `some` check lets a single expression flag orders based on either an order-level or an item-level condition.

#393 Build a CargoLink Driver Availability List

From a list of drivers, get the names of everyone available today, sorted alphabetically.

INPUT

```
{ "drivers": [{"name": "Sam", "available": true}, {"name": "Ana", "available": true}, {"name": "Lee", "available": false}] }
```

SOLUTION

```
((payload.drivers filter $.available) map $.name) orderBy $
```

OUTPUT

```
["Ana", "Sam"]
```

This filter-map-sort chain builds exactly the kind of clean, ready-to-display list a dispatch dashboard would need.

#394 Compute CargoLink Fleet Utilization

Calculate what percentage of the fleet's trucks are currently active.

INPUT

```
{ "trucks": [{"active": true}, {"active": true}, {"active": false}, {"active": true}] }
```

SOLUTION

```
((sizeOf(payload.trucks filter $.active) / sizeOf(payload.trucks)) * 100) as String {format: "0.0"}
```

OUTPUT

```
"75.0"
```

Combining `filter` and `sizeOf` before and after is the standard way to turn a raw list into a utilization percentage.

#395 Deduplicate and Count CargoLink Delivery Zones

Count how many distinct delivery zones appear across a batch of orders.

INPUT

```
{ "orders": [{"zone": "Z1"}, {"zone": "Z2"}, {"zone": "Z1"}, {"zone": "Z3"}] }
```

SOLUTION

```
sizeOf(payload.orders distinctBy $.zone)
```

OUTPUT

```
3
```

Chaining `distinctBy` straight into `sizeOf` is the quickest way to answer "how many unique values are there?"

#396 Build a CargoLink Late Shipment Report

Extract a report of only the late shipments, showing their ID and how many days late they are.

INPUT

```
{ "shipments": [{"id": "SHP-1", "promisedDays": 3, "actualDays": 5}, {"id": "SHP-2", "promisedDays": 2, "actualDays": 2}] }
```

SOLUTION

```
(payload.shipments filter ($.actualDays > $.promisedDays)) map (s) -> {
  id: s.id,
  daysLate: s.actualDays - s.promisedDays
}
```

OUTPUT

```
[{"id": "SHP-1", "daysLate": 2}]
```

Filtering on a comparison between two fields, then reshaping the survivors, is a common way to build an exceptions report.

#397 Rank CargoLink Warehouses by Order Volume

Group orders by warehouse, count each warehouse's order volume, and return the warehouses ranked from busiest to quietest.

INPUT

```
{ "orders": [{"warehouse": "WH-1"}, {"warehouse": "WH-2"}, {"warehouse": "WH-1"}, {"warehouse": "WH-1"}] }
```

SOLUTION

```
var byWarehouse = payload.orders groupBy $.warehouse
var counts = namesOf(byWarehouse) map (wh) -> { warehouse: wh, orders: sizeOf(byWarehouse[wh]) }
---
reverse(counts orderBy $.orders)
```

OUTPUT

```
[{"warehouse": "WH-1", "orders": 3}, {"warehouse": "WH-2", "orders": 1}]
```

This is a full ranking pipeline: group to count, convert group names back into a plain array with `map`, then sort and reverse for a highest-first leaderboard.

#398 Validate a Full CargoLink Shipment Batch

Confirm a whole batch of shipments is ready to dispatch: every shipment must have at least one item, and none may be flagged for customs hold.

INPUT

```
{ "shipments": [{"items": ["A"], "customsHold": false}, {"items": ["B", "C"], "customsHold": false}] }
```

SOLUTION

```
payload.shipments every ((s) -> (sizeOf(s.items) > 0) and (not s.customsHold))
```

OUTPUT

```
true
```

A batch-level readiness gate like this combines a compound condition inside `every`, so a single failing shipment blocks the entire batch.

#399 Build a CargoLink Cost Breakdown by Shipping Method

Group shipments by method and compute both the count and total cost for each method.

INPUT

```
{ "shipments": [{"method": "AIR", "cost": 200}, {"method": "GROUND", "cost": 50}, {"method": "AIR", "cost": 150}] }
```

SOLUTION

```
payload.shipments groupBy $.method mapObject (value, key) -> {
  (key): {
    count: sizeOf(value),
    totalCost: value reduce ((s, t = 0) -> t + s.cost)
  }
}
```

OUTPUT

```
{ "AIR": {"count": 2, "totalCost": 350}, "GROUND": {"count": 1, "totalCost": 50} }
```

Nesting a small summary object inside `mapObject` is how a single `groupBy` pass can report several metrics per group at once.

#400 CargoLink Fleet Dispatch Capstone

Build a full dispatch report: only active trucks, each with its total assigned weight, sorted by weight descending, with a rank attached.

INPUT

```
{ "trucks": [
  {"id": "T1", "active": true, "shipments": [{"weightKg": 200}, {"weightKg": 100}]},
  {"id": "T2", "active": false, "shipments": [{"weightKg": 500}]},
  {"id": "T3", "active": true, "shipments": [{"weightKg": 50}]}
] }
```

SOLUTION

```
var loaded = (payload.trucks filter $.active) map (t) -> {
  id: t.id,
  totalWeight: t.shipments reduce ((s, w = 0) -> w + s.weightKg)
}
var ranked = reverse(loaded orderBy $.totalWeight)
---
ranked map (t, i) -> t ++ { rank: i + 1 }
```

OUTPUT

```
[{"id": "T1", "totalWeight": 300, "rank": 1}, {"id": "T3", "totalWeight": 50, "rank": 2}]
```

This capstone chains nearly every idea from the volume — `filter`, `map`, `reduce`, `orderBy`, `reverse`, and index-aware ranking — into one realistic fleet dispatch transform.

END OF VOLUME 4

400 Down, 600 to Go

You've now worked through selectors, type coercion, operators and conditionals, and a complete tour of array transformations — `map`, `filter`, `reduce`, `orderBy`, `groupBy`, `distinctBy`, `flatten`, `some`, and `every`.

Volume 5 moves to Object Transformations: another 100 problems on `mapObject`, `pluck`, `filterObject`, `namesOf`/`valuesOf`, `entrySet`, and every other tool for reshaping Objects.

MULESOFT · PRACTICE PROBLEMS · WITH FULL SOLUTIONS

1000 DataWeave Problems With Solutions

Volume 5 — Object Transformations. 100 hands-on problems on mapObject, pluck, filterObject, namesOf/valuesOf, entrySet, and dynamic keys.

Problems 401–500

Volume 5 of 10

2026 EDITION

How to Use This Volume

Volume 5 is the mirror image of Volume 4 — instead of reshaping Arrays, it's about reshaping **Objects**: renaming keys, filtering fields, building dynamic key names, and converting between Objects and Arrays. Same format as every prior volume:

- **A short problem statement**, the **input** payload, a complete **solution** in a black code window, the exact **output**, and a one-line explanation.
- Ten sections of ten problems each, moving from `mapObject` and `pluck` through `filterObject`, `namesOf`/`valuesOf`, `entrySet`, dynamic keys, nested object reshaping, and mixed real-world challenges.

Object reshaping is the other half of almost every real transform — this volume pairs directly with Volume 4 to cover both sides of DataWeave's collection-handling toolkit.

VOLUME 5

Object Transformations

100 problems on `mapObject`, `pluck`, `filterObject`, `namesOf`, `valuesOf`, `entrySet`, dynamic key construction, and the tools that turn one Object shape into exactly the Object shape a target system expects.

SECTION 1 — mapObject Basics (Problems 401–410)

#401 Rename Every Key in an Object

Rename an object's keys from snake_case to camelCase using `mapObject`.

INPUT

```
{ "order_id": "ORD-1042", "ship_date": "2026-03-10" }
```

SOLUTION

```
payload mapObject (value, key) -> {
  (if (key == "order_id") "orderId" else "shipDate"): value
}
```

OUTPUT

```
{ "orderId": "ORD-1042", "shipDate": "2026-03-10" }
```

`mapObject` runs its lambda once per key-value pair, and the returned single-key object for each pair is merged together into the final result.

#402 Transform Every Value, Keep the Keys

Uppercase every String value in an object while leaving the keys untouched.

INPUT

```
{ "carrier": "fedex", "status": "shipped" }
```

SOLUTION

```
payload mapObject (value, key) -> { (key): upper(value) }
```

OUTPUT

```
{ "carrier": "FEDEX", "status": "SHIPPED" }
```

Wrapping the original `key` in parentheses inside the returned object reuses it as a dynamic key name, unchanged.

#403 Uppercase Every Key

Convert every key in an object to uppercase, keeping the values unchanged.

INPUT

```
{ "sku": "PALLET-01", "qty": 2 }
```

SOLUTION

```
payload mapObject (value, key) -> { (upper(key)): value }
```

OUTPUT

```
{ "SKU": "PALLET-01", "QTY": 2 }
```

Because DataWeave keys can be any expression wrapped in parentheses, `upper(key)` is a valid dynamic key just like a literal name.

#404 Transform Both Key and Value Together

Prefix every key with `"legacy_"` and coerce every Number value to a String.

INPUT

```
{ "qty": 5, "weight": 12 }
```

SOLUTION

```
payload mapObject (value, key) -> { ("legacy_" ++ key): value as String }
```

OUTPUT

```
{ "legacy_qty": "5", "legacy_weight": "12" }
```

Both the key expression and the value expression can be transformed independently within the same `mapObject` lambda.

#405 Use the Index Parameter of mapObject

Rename each key to a generic `field1`, `field2`... using the key-value pair's position.

INPUT

```
{ "a": 10, "b": 20 }
```

SOLUTION

```
payload mapObject (value, key, index) -> { ("field" ++ ((index + 1) as
String)): value }
```

OUTPUT

```
{ "field1": 10, "field2": 20 }
```

`mapObject`'s lambda accepts an optional third parameter for the zero-based index of the key-value pair, mirroring `map`'s array index.

#406 Apply Conditional Logic Inside mapObject

Double every numeric value in an object, leaving non-numeric values unchanged.

INPUT

```
{ "qty": 4, "note": "fragile" }
```

SOLUTION

```
payload mapObject (value, key) -> {
  (key): value match {
    case n is Number -> n * 2
    else -> value
  }
}
```

OUTPUT

```
{ "qty": 8, "note": "fragile" }
```

A type matcher inside the value expression lets `mapObject` apply logic selectively, based on the actual type of each value.

#407 Build a Nested Object Inside mapObject

Wrap every value in a small metadata object showing its original key name.

INPUT

```
{ "qty": 4 }
```

SOLUTION

```
payload mapObject (value, key) -> { (key): { originalKey: key, value: value }
}
```

OUTPUT

```
{ "qty": { "originalKey": "qty", "value": 4 } }
```

The inner object's `value` key name is just a String literal, so it does not collide with the lambda parameter named `value` — DataWeave still resolves the parameter correctly on the right-hand side.

#408 Round Every Numeric Field to Two Decimals

Format every Number value in a financial summary object to exactly two decimal places.

INPUT

```
{ "subtotal": 199.5, "tax": 15.999 }
```

SOLUTION

```
payload mapObject (value, key) -> { (key): value as String {format: "0.00"}
as Number }
```

OUTPUT

```
{ "subtotal": 199.5, "tax": 16 }
```

Formatting to a String and coercing back to Number is a common way to normalize precision across every field in one pass.

#409 Skip Certain Keys While Mapping

Rename all keys except `id`, which should pass through unchanged.

INPUT

```
{ "id": "ORD-1042", "total": 240 }
```

SOLUTION

```
payload mapObject (value, key) -> {
  (if (key == "id") key else "order_" ++ key): value
}
```

OUTPUT

```
{ "id": "ORD-1042", "order_total": 240 }
```

Since the key itself is just an expression, an `if/else` can decide the key name conditionally per pair.

#410 Standardize a CargoLink Order Object's Keys

Standardize a legacy CargoLink order object: lowercase every key and prefix Number values as formatted currency where the key contains "price".

INPUT

```
{ "ORDERID": "ORD-1042", "UNITPRICE": 45 }
```

SOLUTION

```
payload mapObject (value, key) -> {
  (lower(key)): if (lower(key) contains "price") ("$" ++ (value as String))
else value
}
```

OUTPUT

```
{ "orderid": "ORD-1042", "unitprice": "$45" }
```

Combining key inspection with conditional value formatting is exactly how a legacy-system adapter would normalize an inconsistent payload.

SECTION 2 — pluck Basics (Problems 411–420)

#411 Extract All Values as an Array

Convert an object's values into a plain Array using `pluck`.

INPUT

```
{ "a": 10, "b": 20, "c": 30 }
```

SOLUTION

```
payload pluck (value) -> value
```

OUTPUT

```
[10, 20, 30]
```

`pluck` converts an Object into an Array, running its lambda once per key-value pair and collecting the results in order.

#412 Extract All Keys as an Array

Convert an object's keys into an Array using `pluck`.

INPUT

```
{ "sku": "A", "qty": 2 }
```

SOLUTION

```
payload pluck (value, key) -> key
```

OUTPUT

```
["sku", "qty"]
```

`pluck`'s second lambda parameter is the key of the current pair, letting it extract keys just as easily as values.

#413 Combine Key and Value Into a Label

Build an array of `"key: value"` label Strings from an object.

INPUT

```
{ "carrier": "FedEx", "weight": 12 }
```

SOLUTION

```
payload pluck (value, key) -> key ++ ": " ++ (value as String)
```

OUTPUT

```
["carrier: FedEx", "weight: 12"]
```

Because both `key` and `value` are available in the same lambda, they combine naturally into a single formatted String per pair.

#414 Build an Array of Small Objects With pluck

Convert an object into an array of `{field, value}` objects.

INPUT

```
{ "sku": "A", "qty": 2 }
```

SOLUTION

```
payload pluck (value, key) -> { field: key, value: value }
```

OUTPUT

```
[{"field": "sku", "value": "A"}, {"field": "qty", "value": 2}]
```

This is one of the most common uses of `pluck`: turning a fixed-shape Object into a generic array of field descriptors, useful for rendering a dynamic form or table.

#415 Filter, Then Pluck

Get the values of only the Number-typed fields in a mixed object.

INPUT

```
{ "sku": "A", "qty": 2, "note": "fragile", "weight": 12 }
```

SOLUTION

```
(payload filterObject (value, key) -> value is Number) pluck (value) -> value
```

OUTPUT

```
[2, 12]
```

Chaining `filterObject` before `pluck` narrows the object down first, so only the relevant values end up in the final Array.

#416 Use pluck's Index Parameter

Number each field in an object based on its position using `pluck`'s third parameter.

INPUT

```
{ "sku": "A", "qty": 2 }
```

SOLUTION

```
payload pluck (value, key, index) -> ((index + 1) as String) ++ ". " ++ key
```

OUTPUT

```
["1. sku", "2. qty"]
```

Just like `mapObject`, `pluck`'s lambda accepts an optional index parameter reflecting each pair's position in the object.

#417 Sum All Values With pluck and reduce

Sum every numeric value in an object of quantities.

INPUT

```
{ "pallets": 4, "crates": 7, "boxes": 12 }
```

SOLUTION

```
(payload pluck (value) -> value) reduce ((v, total = 0) -> total + v)
```

OUTPUT

```
23
```

Converting an Object to an Array with `pluck` is what unlocks the rest of the Array toolkit — here, `reduce` for a total.

#418 Find the Largest Value's Key

Find which field in an object holds the largest numeric value.

INPUT

```
{ "pallets": 4, "crates": 12, "boxes": 9 }
```

SOLUTION

```
var entries = payload pluck (value, key) -> { key: key, value: value }
---
(reverse(entries orderBy $.value))[0].key
```

OUTPUT

```
"crates"
```

Once `pluck` turns the Object into an Array of key-value pairs, ordinary array sorting can answer questions the raw Object shape can't easily express.

#419 Convert Object to CSV-Style Row

Join an object's values into a single comma-separated row String, in field order.

INPUT

```
{ "sku": "A", "qty": 2, "price": 45 }
```

SOLUTION

```
(payload pluck (value) -> value as String) joinBy ","
```

OUTPUT

```
"A,2,45"
```

`joinBy` concatenates every element of a String array with the given separator — the standard last step for building a CSV row from an Object.

#420 Build a CargoLink Field Audit List

Build an audit list showing every field name and whether its value is currently `null`.

INPUT

```
{ "orderId": "ORD-1042", "discountCode": null, "notes": null }
```

SOLUTION

```
payload pluck (value, key) -> { field: key, isMissing: value == null }
```

OUTPUT

```
[{"field": "orderId", "isMissing": false}, {"field": "discountCode",
"isMissing": true}, {"field": "notes", "isMissing": true}]
```

A field-by-field audit like this is a realistic use of `pluck` for data-quality reporting before a record is passed downstream.

SECTION 3 — filterObject (Problems 421–430)

#421 Filter Out Null Fields

Remove every field whose value is `null` from an object.

INPUT

```
{ "orderId": "ORD-1042", "discountCode": null, "notes": "handle with care" }
```

SOLUTION

```
payload filterObject (value, key) -> value != null
```

OUTPUT

```
{ "orderId": "ORD-1042", "notes": "handle with care" }
```

`filterObject` keeps only the key-value pairs for which the lambda returns `true`, exactly like `filter` does for arrays.

#422 Filter by Key Name

Keep only the fields whose key starts with `"ship"`.

INPUT

```
{ "shipDate": "2026-03-10", "shipMethod": "GROUND", "orderId": "ORD-1042" }
```

SOLUTION

```
payload filterObject (value, key) -> key startsWith "ship"
```

OUTPUT

```
{ "shipDate": "2026-03-10", "shipMethod": "GROUND" }
```

The `filterObject` condition can inspect the key just as easily as the value, using String functions like `startsWith`.

#423 Filter by Value Type

Keep only the fields whose value is a Number, discarding Strings and Booleans.

INPUT

```
{ "qty": 4, "sku": "A", "active": true, "weight": 12 }
```

SOLUTION

```
payload filterObject (value, key) -> value is Number
```

OUTPUT

```
{ "qty": 4, "weight": 12 }
```

A type check inside `filterObject` is a clean way to keep only fields of one particular kind before running further calculations on them.

#424 Filter by a Numeric Threshold

Keep only inventory fields where the on-hand count is above zero.

INPUT

```
{ "pallets": 4, "crates": 0, "boxes": 9 }
```

SOLUTION

```
payload filterObject (value, key) -> value > 0
```

OUTPUT

```
{ "pallets": 4, "boxes": 9 }
```

This "strip zero-value fields" pattern is common when preparing a stock report that should only show items actually on hand.

#425 Filter Using the Index Parameter

Keep only the first two fields in an object, based on their position.

INPUT

```
{ "a": 1, "b": 2, "c": 3, "d": 4 }
```

SOLUTION

```
payload filterObject (value, key, index) -> index < 2
```

OUTPUT

```
{ "a": 1, "b": 2 }
```

`filterObject`'s third parameter, the index, makes position-based filtering possible in addition to key- and value-based filtering.

#426 Remove Every Internal Field by Prefix

Remove every field whose key starts with an underscore, a common convention for internal-only fields.

INPUT

```
{ "orderId": "ORD-1042", "_cacheKey": "abc", "_debug": true }
```

SOLUTION

```
payload filterObject (value, key) -> not (key startsWith "_")
```

OUTPUT

```
{ "orderId": "ORD-1042" }
```

This is a much more scalable version of chaining several `- -` calls — one condition strips every internal field regardless of how many there are.

#427 Combine filterObject With a Logical Condition

Keep only fields that are both Numbers and greater than 10.

INPUT

```
{ "qty": 4, "weight": 22, "sku": "A" }
```

SOLUTION

```
payload filterObject (value, key) -> (value is Number) and (value > 10)
```

OUTPUT

```
{ "weight": 22 }
```

Combining a type check with a value check inside `and` avoids a runtime error from comparing a non-Number to `10`.

#428 Filter, Then Count Remaining Fields

Count how many fields in an object have a non-null value.

INPUT

```
{ "orderId": "ORD-1042", "notes": null, "priority": true }
```

SOLUTION

```
sizeOf(payload filterObject (value, key) -> value != null)
```

OUTPUT

```
2
```

`sizeOf` works on Objects too, counting the number of key-value pairs — here, after `filterObject` has already trimmed the nulls.

#429 Filter, Then mapObject in a Chain

Keep only Number fields, then double each of their values.

INPUT

```
{ "qty": 4, "sku": "A", "weight": 12 }
```

SOLUTION

```
(payload filterObject (value, key) -> value is Number) mapObject (value, key)
-> { (key): value * 2 }
```

OUTPUT

```
{ "qty": 8, "weight": 24 }
```

Chaining `filterObject` into `mapObject` is the Object-shaped equivalent of the array `filter`-then-`map` pattern from Volume 4.

#430 Build a Clean CargoLink API Response

Strip every internal field and every null field from an order object before returning it as an API response.

INPUT

```
{ "orderId": "ORD-1042", "_internalRetryCount": 2, "notes": null, "total": 240
}
```

SOLUTION

```
payload filterObject (value, key) -> (not (key startsWith "_")) and (value !=
null)
```

OUTPUT

```
{ "orderId": "ORD-1042", "total": 240 }
```

This is exactly the kind of response-cleanup rule a real API layer applies before sending an internal record out to an external client.

SECTION 4 — namesOf & valuesOf (Problems 431–440)

#431 Get an Object's Key Names

Get the list of field names present on an order object.

INPUT

```
{ "orderId": "ORD-1042", "total": 240, "status": "OPEN" }
```

SOLUTION

```
namesOf(payload)
```

OUTPUT

```
["orderId", "total", "status"]
```

`namesOf` returns an Object's keys as an Array, in the same order they appear in the Object.

#432 Get an Object's Values

Get the list of values from an object, discarding the key names.

INPUT

```
{ "orderId": "ORD-1042", "total": 240 }
```

SOLUTION

```
valuesOf(payload)
```

OUTPUT

```
["ORD-1042", 240]
```

`valuesOf` is the counterpart to `namesOf`, returning just the values in the same order as the keys.

#433 Check if a Key Exists

Check whether a `discountCode` field exists on an order object at all, regardless of its value.

INPUT

```
{ "orderId": "ORD-1042", "discountCode": null }
```

SOLUTION

```
namesOf(payload) contains "discountCode"
```

OUTPUT

```
true
```

This distinguishes "the key exists with a `null` value" from "the key is missing entirely" — a distinction a plain selector default can't make.

#434 Count an Object's Fields

Count how many fields an order object has.

INPUT

```
{ "orderId": "ORD-1042", "total": 240, "status": "OPEN" }
```

SOLUTION

```
sizeOf(namesOf(payload))
```

OUTPUT

```
3
```

`sizeOf(payload)` alone would give the same count directly — combining it with `namesOf` is useful mainly when you need the key list for something else too.

#435 Check if All Values Are Non-Null

Confirm that every value in an object is non-null, using `valuesOf` and `every`.

INPUT

```
{ "orderId": "ORD-1042", "total": 240 }
```

SOLUTION

```
valuesOf(payload) every ((v) -> v != null)
```

OUTPUT

```
true
```

Converting to an Array with `valuesOf` first is what makes an Array-only function like `every` usable against an Object's contents.

#436 Compare Two Objects' Key Sets

Check whether two objects have exactly the same set of keys, regardless of values.

INPUT

```
{ "a": {"x": 1, "y": 2}, "b": {"y": 5, "x": 9} }
```

SOLUTION

```
(namesOf(payload.a) orderBy $) == (namesOf(payload.b) orderBy $)
```

OUTPUT

```
true
```

Sorting both key lists before comparing avoids a false mismatch caused only by the two objects declaring their keys in different orders.

#437 Find the Sum of All Numeric Values

Sum every value in a quantities object, using `valuesOf` and `reduce`.

INPUT

```
{ "pallets": 4, "crates": 7, "boxes": 12 }
```

SOLUTION

```
valuesOf(payload) reduce ((v, total = 0) -> total + v)
```

OUTPUT

```
23
```

This is functionally identical to the earlier `pluck (value) -> value` approach — `valuesOf` is simply the more direct built-in for this exact job.

#438 Build a Key-Count Summary

Build a small report showing how many fields an object has and what their names are.

INPUT

```
{ "sku": "A", "qty": 2, "weight": 12 }
```

SOLUTION

```
{
  fieldCount: sizeOf(namesOf(payload)),
  fieldNames: namesOf(payload)
}
```

OUTPUT

```
{ "fieldCount": 3, "fieldNames": ["sku", "qty", "weight"] }
```

Combining `namesOf` and `sizeOf` in one summary object is a common way to describe a payload's shape for logging or debugging.

#439 Check for Any Missing Required Fields

Given a list of required field names, check whether an order object is missing any of them.

INPUT

```
{ "order": {"orderId": "ORD-1042", "total": 240}, "required": ["orderId",
"total", "shippingCountry"] }
```

SOLUTION

```
payload.required filter (not (namesOf(payload.order) contains $))
```

OUTPUT

```
["shippingCountry"]
```

Filtering the required-fields list down to the ones missing from `namesOf` produces exactly the report a validation step needs.

#440 Build a Full CargoLink Schema Audit

Audit a shipment object: report its field names, field count, and whether every value is non-null.

INPUT

```
{ "shipmentId": "SHP-501", "carrier": "FEDEX", "trackingNumber": null }
```

SOLUTION

```
{
  fields: namesOf(payload),
  fieldCount: sizeOf(namesOf(payload)),
  allComplete: valuesOf(payload) every ((v) -> v != null)
}
```

OUTPUT

```
{ "fields": ["shipmentId", "carrier", "trackingNumber"], "fieldCount": 3,
"allComplete": false }
```

Combining `namesOf`, `sizeOf`, and `valuesOf` with `every` in one object builds a complete, self-contained schema audit in a single expression.

SECTION 5 — entrySet & Object Construction (Problems 441–450)

#441 Convert an Object to Key-Value Pair Objects

Use `entrySet` to convert an object into an array of explicit key-value pair objects.

INPUT

```
{ "sku": "A", "qty": 2 }
```

SOLUTION

```
entrySet(payload)
```

OUTPUT

```
[{"key": "sku", "value": "A"}, {"key": "qty", "value": 2}]
```

`entrySet` is a shortcut for what `pluck (value, key) -> {key: key, value: value}` builds manually — both keys and values, already paired up.

#442 Filter entrySet Results Like Any Array

Get only the entries from an object whose value is a Number, using `entrySet` and `filter`.

INPUT

```
{ "sku": "A", "qty": 2, "weight": 12 }
```

SOLUTION

```
entrySet(payload) filter ($.value is Number)
```

OUTPUT

```
[{"key": "qty", "value": 2}, {"key": "weight", "value": 12}]
```

Once converted with `entrySet`, the result is a plain Array, so every array function from Volume 4 becomes available.

#443 Sort an Object's Entries by Value

Get an object's entries sorted by value, descending.

INPUT

```
{ "pallets": 4, "crates": 12, "boxes": 9 }
```

SOLUTION

```
reverse(entrySet(payload) orderBy $.value)
```

OUTPUT

```
[{"key": "crates", "value": 12}, {"key": "boxes", "value": 9}, {"key":
"pallets", "value": 4}]
```

This is exactly how the earlier "find the key with the largest value" problem could be generalized to a full sorted ranking of every field.

#444 Convert entrySet Back Into an Object

Round-trip an object through `entrySet` and back into an object using dynamic keys.

INPUT

```
{ "sku": "A", "qty": 2 }
```

SOLUTION

```
entrySet(payload) reduce ((entry, obj = {}) -> obj ++ { (entry.key):
entry.value })
```

OUTPUT

```
{ "sku": "A", "qty": 2 }
```

Rebuilding an Object from an Array of entries with `reduce` and a dynamic key is the reverse operation of `entrySet`.

#445 Build an Object Literal With a Dynamic Key

Build a single-field object where the key name itself comes from a variable.

INPUT

```
{ "fieldName": "sku", "fieldValue": "PALLET-01" }
```

SOLUTION

```
{ (payload.fieldName): payload.fieldValue }
```

OUTPUT

```
{ "sku": "PALLET-01" }
```

Any expression wrapped in parentheses can serve as an object literal's key — this is what makes dynamic key construction possible throughout this volume.

#446 Conditionally Include a Field

Build an order summary where the `discountCode` field is only included at all when a discount was actually applied.

INPUT

```
{ "orderId": "ORD-1042", "discountCode": "SAVE10", "hasDiscount": true }
```

SOLUTION

```
{
  orderId: payload.orderId,
  discountCode: payload.discountCode if payload.hasDiscount
}
```

OUTPUT

```
{ "orderId": "ORD-1042", "discountCode": "SAVE10" }
```

The trailing `if condition` syntax on an object field omits the key entirely from the output when the condition is `false`, rather than including it with a `null` value.

#447 Confirm a Conditionally Omitted Field Is Really Gone

Build the same conditional field, but with the condition false, and confirm the key does not appear at all.

INPUT

```
{ "orderId": "ORD-1042", "discountCode": null, "hasDiscount": false }
```

SOLUTION

```
{
  orderId: payload.orderId,
  discountCode: payload.discountCode if payload.hasDiscount
}
```

OUTPUT

```
{ "orderId": "ORD-1042" }
```

This is the key difference from `default`: a conditional field with a false condition removes the key entirely, rather than replacing its value.

#448 Build an Object From an Array Using Dynamic Keys

Convert an array of items into a lookup object keyed by SKU, using a dynamic key inside a `map`-then-merge pattern.

INPUT

```
{ "items": [{"sku": "A", "qty": 2}, {"sku": "B", "qty": 5}] }
```

SOLUTION

```
payload.items reduce ((item, lookup = {}) -> lookup ++ { (item.sku): item.qty
})
```

OUTPUT

```
{ "A": 2, "B": 5 }
```

This is the same Array-to-Object conversion seen earlier in Volume 4 — it's included here because it relies entirely on the dynamic-key feature this section introduces.

#449 Build Multiple Dynamic Keys in One Object

Build a summary object where both the key name and a nested field name are computed dynamically from input values.

INPUT

```
{ "region": "TX", "metric": "orderCount", "value": 42 }
```

SOLUTION

```
{ (payload.region): { (payload.metric): payload.value } }
```

OUTPUT

```
{ "TX": { "orderCount": 42 } }
```

Dynamic keys can be nested at any depth in an object literal, letting a single expression build an arbitrarily shaped result.

#450 Build a Clean, Conditional CargoLink Order Summary

Build a CargoLink order summary where `rushFee` is only included when the order is marked priority, and every field with a dynamic key is properly cased.

INPUT

```
{ "orderId": "ORD-1042", "priority": true, "total": 240 }
```

SOLUTION

```
{
  orderId: payload.orderId,
  total: payload.total,
  rushFee: 15 if payload.priority
}
```

OUTPUT

```
{ "orderId": "ORD-1042", "total": 240, "rushFee": 15 }
```

Conditional fields are the cleanest way to build an output object whose shape itself depends on the input, rather than always including every possible key.

SECTION 6 — Dynamic Keys & Computed Field Names (Problems 451–460)

#451 Build a Key From Two Concatenated Fields

Build a single-entry lookup object where the key combines a prefix and a numeric ID.

INPUT

```
{ "prefix": "ORD", "id": 1042, "status": "OPEN" }
```

SOLUTION

```
{ (payload.prefix ++ "-" ++ (payload.id as String)): payload.status }
```

OUTPUT

```
{ "ORD-1042": "OPEN" }
```

Any expression that resolves to a String can serve as a dynamic key, including one built from several concatenated fields.

#452 Build a Key From a Coerced Number

Build a key that embeds a Number field, coerced to String first.

INPUT

```
{ "year": 2026, "revenue": 50000 }
```

SOLUTION

```
{ ("revenue_" ++ (payload.year as String)): payload.revenue }
```

OUTPUT

```
{ "revenue_2026": 50000 }
```

Object keys must be Strings, so a Number used inside a dynamic key expression always needs an explicit `as String` coercion first.

#453 Zip Two Parallel Arrays Into an Object

Combine a keys array and a values array into a single object, matched by position.

INPUT

```
{ "keys": ["sku", "qty", "price"], "values": ["PALLET-01", 2, 45] }
```

SOLUTION

```
var pairs = payload.keys map ((k, i) -> { (k): payload.values[i] })
---
pairs reduce ((pair, obj = {}) -> obj ++ pair)
```

OUTPUT

```
{ "sku": "PALLET-01", "qty": 2, "price": 45 }
```

`map`'s index parameter lines up each key with its matching value from the second array, and `reduce` merges the resulting single-key objects together.

#454 Build a Key From a Boolean Condition

Route a count into a `priorityCount` or `standardCount` key based on a Boolean flag.

INPUT

```
{ "isPriority": true, "count": 5 }
```

SOLUTION

```
{ (if (payload.isPriority) "priorityCount" else "standardCount"):
payload.count }
```

OUTPUT

```
{ "priorityCount": 5 }
```

An `if/else` expression works as a dynamic key exactly like any other expression, since it evaluates to a plain String.

#455 Build Two Levels of Dynamic Keys

Build a two-level nested object where both the outer and inner key names are computed from input fields.

INPUT

```
{ "region": "TX", "warehouse": "WH-1", "stock": 120 }
```

SOLUTION

```
var regionKey = payload.region
var warehouseKey = payload.warehouse
---
{ (regionKey): { (warehouseKey): payload.stock } }
```

OUTPUT

```
{ "TX": { "WH-1": 120 } }
```

Storing each dynamic key in a named `var` first keeps a multi-level dynamic object readable, rather than nesting the key expressions inline.

#456 Convert a Dotted Path String Into a Nested Object

Convert a dotted field path like `"address.city"` into a genuinely nested object with that shape.

INPUT

```
{ "path": "address.city", "value": "Austin" }
```

SOLUTION

```
var parts = payload.path splitBy "."
---
reverse(parts) reduce ((part, acc = payload.value) -> { (part): acc })
```

OUTPUT

```
{ "address": { "city": "Austin" } }
```

Reducing over the path parts in reverse order builds the nesting from the inside out — the last part wraps the raw value first, and each earlier part wraps the previous result.

#457 Rename Keys Using a Lookup Table

Rename an object's keys using a separate mapping table, falling back to the original key name if no mapping exists.

INPUT

```
{ "data": {"order_id": "ORD-1042", "total": 240}, "mapping": {"order_id": "orderId"} }
```

SOLUTION

```
payload.data mapObject ((value, key) -> {
  ((payload.mapping[(key)]) default key): value
})
```

OUTPUT

```
{ "orderId": "ORD-1042", "total": 240 }
```

The parenthesized bracket selector `[(key)]` looks up a dynamic key name at runtime, and `default` supplies the original key whenever no mapping entry exists.

#458 Sanitize Key Names Before Building an Object

Replace spaces in every key name with underscores, producing safe identifier-style keys.

INPUT

```
{ "Order Total": 240, "Ship Date": "2026-03-10" }
```

SOLUTION

```
payload mapObject ((value, key) -> { (key replace " " with "_"): value })
```

OUTPUT

```
{ "Order_Total": 240, "Ship_Date": "2026-03-10" }
```

DataWeave's `replace ... with ...` operator works on a key expression just as easily as on any other String, making key sanitization a one-line operation.

#459 Build a Key Suffix From the Value's Type

Append a type-indicating suffix to every key, based on whether its value is a Number or a String.

INPUT

```
{ "a": 10, "b": "text" }
```

SOLUTION

```
payload mapObject ((value, key) -> {
  (key ++ (if (value is Number) "_num" else "_str")): value
})
```

OUTPUT

```
{ "a_num": 10, "b_str": "text" }
```

Type-tagging keys like this can be a useful debugging aid when a payload's field types are otherwise not obvious from the key names alone.

#460 Build a Nested CargoLink Warehouse Inventory Report

Build a nested inventory report keyed by warehouse, then by SKU, from a flat array of item records.

INPUT

```
{ "items": [{"warehouse": "WH-1", "sku": "A", "qty": 2}, {"warehouse": "WH-1",
"sku": "B", "qty": 5}, {"warehouse": "WH-2", "sku": "C", "qty": 1}] }
```

SOLUTION

```
payload.items reduce ((item, report = {}) ->
  report ++ { (item.warehouse): ((report[(item.warehouse)]) default {}) ++ {
(item.sku): item.qty } }
)
```

OUTPUT

```
{ "WH-1": { "A": 2, "B": 5 }, "WH-2": { "C": 1 } }
```

This capstone combines dynamic keys at two nesting levels with `reduce` and `default`, building a full nested report from a flat list in a single pass.

SECTION 7 — Merging & Restructuring Objects (Problems 461–470)

#461 Deep-Merge a Nested Field Manually

Merge an override into a nested `contact` object without losing the fields the override doesn't mention.

INPUT

```
{ "base": {"contact": {"phone": "555-1000", "email": "a@x.com"}}, "override":
{"contact": {"phone": "555-2000"}} }
```

SOLUTION

```
payload.base ++ { contact: payload.base.contact ++ payload.override.contact }
```

OUTPUT

```
{ "contact": { "phone": "555-2000", "email": "a@x.com" } }
```

A plain top-level `++` would have replaced the whole `contact` object and lost `email` — merging the nested field explicitly preserves it.

#462 Layer Three Configuration Objects

Layer a default config, a regional override, and a customer-specific override, in that priority order.

INPUT

```
{ "defaults": {"carrier": "STANDARD", "insured": false}, "regional":
{"carrier": "EXPRESS"}, "customer": {"insured": true} }
```

SOLUTION

```
payload.defaults ++ payload.regional ++ payload.customer
```

OUTPUT

```
{ "carrier": "EXPRESS", "insured": true }
```

Chaining `++` across three objects layers them left to right, with each later object's fields taking priority over the ones before it.

#463 Group Flat Fields Into a Nested Object

Restructure a flat customer record by grouping its address-related fields into a nested `address` object.

INPUT

```
{ "name": "Maria Chen", "street": "123 Main St", "city": "Austin", "zip": "78701" }
```

SOLUTION

```
{
  name: payload.name,
  address: {
    street: payload.street,
    city: payload.city,
    zip: payload.zip
  }
}
```

OUTPUT

```
{ "name": "Maria Chen", "address": { "street": "123 Main St", "city": "Austin", "zip": "78701" } }
```

Building a fresh object literal that reuses the flat fields as nested values is the most direct way to introduce structure that wasn't there originally.

#464 Flatten a Nested Object One Level

Flatten a nested `address` object back out into prefixed top-level fields.

INPUT

```
{ "name": "Maria Chen", "address": {"street": "123 Main St", "city": "Austin"}
}
```

SOLUTION

```
{ name: payload.name } ++ (payload.address mapObject ((value, key) -> {
("address_" ++ key): value }))
```

OUTPUT

```
{ "name": "Maria Chen", "address_street": "123 Main St", "address_city":
"Austin" }
```

Prefixing each nested key with its parent field's name before merging it back to the top level avoids naming collisions between sibling nested objects.

#465 Remove a Dynamic List of Fields

Remove every field named in a separate list of keys to strip, using `reduce` and `--`.

INPUT

```
{ "data": {"orderId": "ORD-1042", "_debug": "x", "_cacheKey": "y", "total":
240}, "keysToRemove": ["_debug", "_cacheKey"] }
```

SOLUTION

```
payload.keysToRemove reduce ((k, obj = payload.data) -> obj -- k)
```

OUTPUT

```
{ "orderId": "ORD-1042", "total": 240 }
```

Seeding the accumulator with the original object and applying `--` once per key to remove scales to any number of keys, unlike chaining `--` calls by hand.

#466 Apply a Sequence of Partial Updates

Apply an ordered array of partial update objects to a base record, one after another.

INPUT

```
{ "base": {"status": "PENDING", "priority": false}, "updates": [{"status": "PROCESSING"}, {"priority": true}] }
```

SOLUTION

```
payload.updates reduce ((u, acc = payload.base) -> acc ++ u)
```

OUTPUT

```
{ "status": "PROCESSING", "priority": true }
```

Seeding `reduce`'s accumulator with the base record and merging each update in turn replays a whole update history in the order it happened.

#467 Rename a Field While Moving It Into a Nested Group

Move a flat `phoneNumber` field into a nested `contact` object, renaming it to `phone` at the same time.

INPUT

```
{ "name": "Maria Chen", "phoneNumber": "555-1000" }
```

SOLUTION

```
{
  name: payload.name,
  contact: { phone: payload.phoneNumber }
}
```

OUTPUT

```
{ "name": "Maria Chen", "contact": { "phone": "555-1000" } }
```

Renaming and restructuring are really the same operation — a fresh object literal that reads from the old field and writes to a new location and name.

#468 Merge Two Objects While Concatenating an Array Field

Merge a base record and an update, but for the `tags` field specifically, combine both arrays instead of letting one replace the other.

INPUT

```
{ "base": {"tags": ["fragile"]}, "update": {"tags": ["urgent"]} }
```

SOLUTION

```
payload.base ++ payload.update ++ { tags: payload.base.tags ++
payload.update.tags }
```

OUTPUT

```
{ "tags": ["fragile", "urgent"] }
```

A plain `++` merge would have let `update.tags` fully replace `base.tags` — appending one more explicit override recombines the two arrays instead.

#469 Build a Field-Level Diff Between Two Objects

Keep only the fields that actually changed between an old and a new version of the same object.

INPUT

```
{ "oldVersion": {"status": "PENDING", "total": 240, "carrier": "UPS"},
"newVersion": {"status": "SHIPPED", "total": 240, "carrier": "FEDEX"} }
```

SOLUTION

```
payload.newVersion filterObject ((value, key) -> value !=
(payload.oldVersion[(key)]))
```

OUTPUT

```
{ "status": "SHIPPED", "carrier": "FEDEX" }
```

Comparing each field in the new version against the same key looked up dynamically in the old version is a compact way to compute a whole-object diff.

#470 Merge and Clean a CargoLink Shipment Record

Apply a carrier-specific override to a base shipment record, then strip any internal fields before the result leaves the flow.

INPUT

```
{ "base": {"shipmentId": "SHP-501", "carrier": "STANDARD", "_internal": "x"},
"carrierOverrides": {"carrier": "FEDEX_EXPRESS"} }
```

SOLUTION

```
var merged = payload.base ++ payload.carrierOverrides
---
merged filterObject ((value, key) -> not (key startsWith "_"))
```

OUTPUT

```
{ "shipmentId": "SHP-501", "carrier": "FEDEX_EXPRESS" }
```

Merging first and cleaning second is the standard two-step order for combining an update with a downstream-safe output shape.

SECTION 8 — Nested Object Transformations (Problems 471–480)

#471 Add a Field to Every Item in a Nested Array

Add a `received: false` field to every item in an order's nested items array.

INPUT

```
{ "orderId": "ORD-1042", "items": [{"sku": "A", "qty": 2}] }
```

SOLUTION

```
payload ++ { items: payload.items map ((item) -> item ++ { received: false })
}
```

OUTPUT

```
{ "orderId": "ORD-1042", "items": [{"sku": "A", "qty": 2, "received": false}]
}
```

Overriding just the `items` field with `++` at the top level, while mapping over it internally, updates the nested array without disturbing any sibling fields.

#472 Update a Deeply Nested Field

Update a customer's tier, nested two levels deep inside an order, without touching any other field along the way.

INPUT

```
{ "order": {"id": "ORD-1042", "customer": {"name": "Maria Chen", "tier": "SILVER"}} }
```

SOLUTION

```
payload ++ { order: payload.order ++ { customer: payload.order.customer ++ { tier: "GOLD" } } }
```

OUTPUT

```
{ "order": { "id": "ORD-1042", "customer": { "name": "Maria Chen", "tier": "GOLD" } } }
```

Updating a deeply nested field means merging at every level along the path — each `++` layer preserves the sibling fields at that level.

#473 Apply mapObject to Only a Nested Field

Multiply every value inside a nested `meta` object by 10, leaving the rest of the payload untouched.

INPUT

```
{ "orderId": "ORD-1042", "meta": {"a": 1, "b": 2} }
```

SOLUTION

```
payload ++ { meta: payload.meta mapObject ((value, key) -> { (key): value * 10 }) }
```

OUTPUT

```
{ "orderId": "ORD-1042", "meta": { "a": 10, "b": 20 } }
```

Scoping `mapObject` to just the nested field, then merging the result back in, keeps the transformation isolated to the part of the payload that actually needs it.

#474 Uppercase Every String Across Two Levels

Uppercase every String value at both the top level and inside the nested `customer` object, reusing the same logic for both.

INPUT

```
{ "status": "open", "customer": {"name": "maria chen", "region": "tx"} }
```

SOLUTION

```
var upperStrings = (obj) -> obj mapObject ((value, key) -> {
  (key): if (value is String) upper(value) else value
})
---
upperStrings(payload) ++ { customer: upperStrings(payload.customer) }
```

OUTPUT

```
{ "status": "OPEN", "customer": { "name": "MARIA CHEN", "region": "TX" } }
```

Storing a small transform as a named function in a `var` lets the same logic be reused at each nesting level without repeating the lambda.

#475 Extract a Field Nested Inside Every Array Element

Extract just the customer names from an array of orders, where each name is nested inside a `customer` sub-object.

INPUT

```
{ "orders": [{"customer": {"name": "Maria Chen"}}, {"customer": {"name":
"James Cole"}}] }
```

SOLUTION

```
payload.orders map $.customer.name
```

OUTPUT

```
["Maria Chen", "James Cole"]
```

A `map` lambda's selector can chain as many levels deep as the source data requires, right down to the field you actually need.

#476 Apply a Discount to Every Nested Item's Price

Apply a 10% discount to the price of every item in a nested items array.

INPUT

```
{ "orderId": "ORD-1042", "items": [{"sku": "A", "price": 50}, {"sku": "B", "price": 100}] }
```

SOLUTION

```
payload ++ { items: payload.items map ((item) -> item ++ { price: item.price * 0.9 }) }
```

OUTPUT

```
{ "orderId": "ORD-1042", "items": [{"sku": "A", "price": 45}, {"sku": "B", "price": 90}] }
```

Overriding just the `price` field inside each item with ++ keeps every other item field, like `sku`, exactly as it was.

#477 Update One Nested Sub-Object, Leave Siblings Untouched

Apply an update to only the `shipping` sub-object of an order, leaving the sibling `billing` sub-object exactly as it was.

INPUT

```
{ "order": {"shipping": {"method": "GROUND"}, "billing": {"method": "INVOICE"}}, "shippingUpdate": {"method": "AIR"} }
```

SOLUTION

```
payload.order ++ { shipping: payload.order.shipping ++ payload.shippingUpdate }
```

OUTPUT

```
{ "shipping": { "method": "AIR" }, "billing": { "method": "INVOICE" } }
```

Because the override only names the `shipping` key, the outer ++ leaves every other sibling key — including `billing` — completely untouched.

#478 Add a Nested Field Computed From a Sibling

Add a nested `flags` object with an `insuranceRequired` field, computed from the order's total.

INPUT

```
{ "order": {"total": 1200} }
```

SOLUTION

```
payload.order ++ { flags: { insuranceRequired: payload.order.total > 1000 } }
```

OUTPUT

```
{ "total": 1200, "flags": { "insuranceRequired": true } }
```

A newly added nested field can reference any other field already in scope, including one from the same object it's being merged into.

#479 Transform Nested Items Across an Array of Orders

Add a tax-inclusive price to every item, nested inside every order, across an entire array of orders.

INPUT

```
{ "orders": [{"id": "O1", "items": [{"price": 50}, {"price": 20}]}] }
```

SOLUTION

```
payload.orders map ((order) -> order ++ {
  items: order.items map ((item) -> item ++ { priceWithTax: item.price * 1.08
})
})
```

OUTPUT

```
[{"id": "O1", "items": [{"price": 50, "priceWithTax": 54}, {"price": 20,
"priceWithTax": 21.6}]}]
```

Nesting one `map` inside another handles two levels of arrays at once — the outer `map` walks the orders, and the inner `map` walks each order's items.

#480 CargoLink Full Nested Order Transform

Upgrade a customer's tier, add a line total to every item, and add a computed grand total — all in one order transform.

INPUT

```
{ "orderId": "ORD-1042", "customer": {"name": "Maria Chen", "tier": "SILVER"},
"items": [{"sku": "A", "price": 50, "qty": 2}, {"sku": "B", "price": 30,
"qty": 1}] }
```

SOLUTION

```
var itemsWithTotals = payload.items map ((item) -> item ++ { lineTotal:
item.price * item.qty })
var grandTotal = itemsWithTotals reduce ((item, t = 0) -> t + item.lineTotal)
---
payload ++ {
  customer: payload.customer ++ { tier: "GOLD" },
  items: itemsWithTotals,
  grandTotal: grandTotal
}
```

OUTPUT

```
{ "orderId": "ORD-1042", "customer": { "name": "Maria Chen", "tier": "GOLD" },
"items": [{"sku": "A", "price": 50, "qty": 2, "lineTotal": 100}, {"sku": "B",
"price": 30, "qty": 1, "lineTotal": 30}], "grandTotal": 130 }
```

Breaking the work into named `var` steps before the final merge keeps a multi-part nested transform readable, even though it touches three different parts of the payload at once.

SECTION 9 — Chaining Object Functions Together (Problems 481–490)

#481 Filter to String Fields, Then Lowercase Them

Keep only the String-valued fields in an object, then lowercase every one of them.

INPUT

```
{ "sku": "A", "note": "FRAGILE", "qty": 2 }
```

SOLUTION

```
(payload filterObject ((value, key) -> value is String)) mapObject ((value, key) -> { (key): lower(value) })
```

OUTPUT

```
{ "sku": "a", "note": "fragile" }
```

Chaining `filterObject` into `mapObject` is the Object-shaped version of the array `filter`-then-`map` pattern from Volume 4.

#482 Drop Empty Fields via entrySet

Remove every field with an empty String value, using `entrySet`, `filter`, and `reduce` to rebuild the object.

INPUT

```
{ "a": "", "b": "x", "c": "" }
```

SOLUTION

```
var kept = entrySet(payload) filter ($.value != "")
---
kept reduce ((e, obj = {}) -> obj ++ { (e.key): e.value })
```

OUTPUT

```
{ "b": "x" }
```

This three-step pipeline — `entrySet`, `filter`, `reduce` — achieves exactly what `filterObject` does directly, and is worth knowing for cases where the filtering logic needs the full Array toolkit.

#483 Find the Field With the Lowest Value

Find which field in an object holds the smallest numeric value.

INPUT

```
{ "pallets": 4, "crates": 12, "boxes": 9 }
```

SOLUTION

```
var entries = payload pluck ((value, key) -> { key: key, value: value })
---
(entries orderBy $.value)[0].key
```

OUTPUT

```
"pallets"
```

Sorting ascending and taking the first entry finds the minimum, mirroring the same technique used for arrays in Volume 4.

#484 Transform, Then Strip Falsy Fields

After a pass-through transform, remove any field whose value is `0`, an empty String, or `false`.

INPUT

```
{ "qty": 0, "weight": 12, "note": "" }
```

SOLUTION

```
(payload mapObject ((value, key) -> { (key): value })) filterObject ((value,
key) ->
  (value != 0) and (value != "") and (value != false)
)
```

OUTPUT

```
{ "weight": 12 }
```

Chaining a transform into a cleanup filter is a common last step before an object is considered ready to send downstream.

#485 Build a Key-Presence Report

Build a report where every field name maps to `true`, showing which fields are present regardless of their actual values.

INPUT

```
{ "orderId": "ORD-1042", "total": 240 }
```

SOLUTION

```
namesOf(payload) reduce ((k, report = {}) -> report ++ { (k): true })
```

OUTPUT

```
{ "orderId": true, "total": true }
```

Chaining `namesOf` into `reduce` with a dynamic key is a quick way to turn a field list back into an Object shape.

#486 Round-Trip Through entrySet With a Transform

Double every value in an object by converting to entries, transforming, and rebuilding the object.

INPUT

```
{ "a": 1, "b": 2 }
```

SOLUTION

```
var doubled = entrySet(payload) map ((e) -> { key: e.key, value: e.value * 2
})
---
doubled reduce ((e, obj = {}) -> obj ++ { (e.key): e.value })
```

OUTPUT

```
{ "a": 2, "b": 4 }
```

While `mapObject` would do this more directly, the `entrySet-map-reduce` chain is useful whenever the transform needs the full Array toolkit along the way.

#487 Build a CSV Header Row of Numeric Fields

Build a comma-separated header row listing only the Number-typed field names in an object.

INPUT

```
{ "sku": "A", "qty": 2, "weight": 12 }
```

SOLUTION

```
((payload filterObject ((value, key) -> value is Number)) pluck ((value, key)
-> key)) joinBy ","
```

OUTPUT

```
"qty,weight"
```

Filtering by type, extracting just the key names, and joining into a String is a three-function chain that builds a CSV header directly from a live payload.

#488 Check if Any Field Equals a Target Value

Check whether any field in an object holds the exact value `"HIGH"`, regardless of which field it is.

INPUT

```
{ "status": "OPEN", "priority": "HIGH", "region": "TX" }
```

SOLUTION

```
valuesOf(payload) some ((v) -> v == "HIGH")
```

OUTPUT

```
true
```

Converting to an Array with `valuesOf` first is what makes `some` — an Array function — usable to answer a question about an Object's contents.

#489 Rebuild an Object From a Filtered Key List

Build an object containing only the fields whose key starts with `"ship"`, using `namesOf` and `reduce` instead of `filterObject`.

INPUT

```
{ "shipDate": "2026-03-10", "shipMethod": "GROUND", "orderId": "ORD-1042" }
```

SOLUTION

```
var shipKeys = namesOf(payload) filter ($ startsWith "ship")
---
shipKeys reduce ((k, obj = {}) -> obj ++ { (k): (payload[(k)]) })
```

OUTPUT

```
{ "shipDate": "2026-03-10", "shipMethod": "GROUND" }
```

This manual reconstruction shows what `filterObject` does under the hood — filtering the key list, then looking each surviving key back up dynamically.

#490 Build a CargoLink Missing-Field Issue List

Build a list of field names whose values are `null`, using `entrySet` to inspect every field at once.

INPUT

```
{ "orderId": "ORD-1042", "discountCode": null, "notes": null, "total": 240 }
```

SOLUTION

```
(entrySet(payload) filter ($.value == null)) map ((e) -> e.key)
```

OUTPUT

```
["discountCode", "notes"]
```

This exact chain — `entrySet`, `filter`, `map` — is the standard way to build a "which fields need attention" report from any Object.

SECTION 10 — Mixed Real-World Object Challenges (Problems 491–500)

#491 Strip PII Fields Before Logging

Remove sensitive personal fields from a customer record before it is written to an application log.

INPUT

```
{ "name": "Maria Chen", "ssn": "xxx-xx-1234", "dob": "1990-01-01", "tier": "GOLD" }
```

SOLUTION

```
payload filterObject ((value, key) -> not (["ssn", "dob"] contains key))
```

OUTPUT

```
{ "name": "Maria Chen", "tier": "GOLD" }
```

Checking the key against a small denylist Array with `contains` is a clean way to strip a known set of sensitive fields before logging.

#492 Build a Clean CargoLink API Response

Build an API-ready order response: rename `order_id`, keep `total`, and only include `rushFee` when the order is marked rush.

INPUT

```
{ "order_id": "ORD-1042", "_internal": "x", "rush": true, "total": 240 }
```

SOLUTION

```
{
  orderId: payload.order_id,
  total: payload.total,
  rushFee: 15 if payload.rush
}
```

OUTPUT

```
{ "orderId": "ORD-1042", "total": 240, "rushFee": 15 }
```

Building the response as a fresh object literal — rather than transforming the input in place — automatically drops any field the literal doesn't mention, like `_internal`.

#493 Apply a CargoLink Status Update

Apply an incoming status update to an existing order record.

INPUT

```
{ "order": {"status": "PENDING", "lastUpdated": "2026-03-01"}, "update":
{"status": "SHIPPED", "lastUpdated": "2026-03-14"} }
```

SOLUTION

```
payload.order ++ payload.update
```

OUTPUT

```
{ "status": "SHIPPED", "lastUpdated": "2026-03-14" }
```

A single ++ is often all a status-update handler needs, as long as the update payload only contains the fields that actually changed.

#494 Restructure a Flat CargoLink Order Into a Shipping Address

Group an order's flat address fields into a nested `shippingAddress` object.

INPUT

```
{ "orderId": "ORD-1042", "street": "200 Depot Rd", "city": "Austin", "state": "TX", "zip": "78701" }
```

SOLUTION

```
{
  orderId: payload.orderId,
  shippingAddress: {
    street: payload.street,
    city: payload.city,
    state: payload.state,
    zip: payload.zip
  }
}
```

OUTPUT

```
{ "orderId": "ORD-1042", "shippingAddress": { "street": "200 Depot Rd", "city": "Austin", "state": "TX", "zip": "78701" } }
```

This kind of flat-to-nested restructuring is one of the most common transforms at an integration boundary, where two systems model the same data at different levels of nesting.

#495 Build a Dynamic CargoLink Surcharge Table

Build a surcharge lookup object from a list of carrier names, where every carrier has a $10 surcharge except ground shipping.

INPUT

```
{ "carriers": ["FEDEX", "UPS", "GROUND"] }
```

SOLUTION

```
payload.carriers reduce ((c, table = {}) -> table ++ { (c): if (c ==
"GROUND") 0 else 10 })
```

OUTPUT

```
{ "FEDEX": 10, "UPS": 10, "GROUND": 0 }
```

Building a lookup table from a plain Array of names with `reduce` and a dynamic key is a common way to generate configuration objects programmatically.

#496 Validate Required CargoLink Order Fields

Report which required fields are either missing entirely or present but `null`.

INPUT

```
{ "order": {"orderId": "ORD-1042", "total": 240, "carrier": null}, "required":
["orderId", "total", "carrier", "shippingCountry"] }
```

SOLUTION

```
payload.required filter ((f) ->
  (not (namesOf(payload.order) contains f)) or ((payload.order[(f)]) == null)
)
```

OUTPUT

```
["carrier", "shippingCountry"]
```

Checking both "does the key exist" and "is its value null" together catches both ways a required field can effectively be missing.

#497 Flatten a Nested CargoLink Inventory Object Into Rows

Convert a nested warehouse-then-SKU inventory object into a flat array of individual rows.

INPUT

```
{ "inventory": {"WH-1": {"A": 4, "B": 2}, "WH-2": {"C": 9}} }
```

SOLUTION

```
namesOf(payload.inventory) reduce ((wh, rows = []) ->
  rows ++ (entrySet(payload.inventory[(wh)]) map ((e) -> { warehouse: wh,
sku: e.key, qty: e.value }))
)
```

OUTPUT

```
[{"warehouse": "WH-1", "sku": "A", "qty": 4}, {"warehouse": "WH-1", "sku":
"B", "qty": 2}, {"warehouse": "WH-2", "sku": "C", "qty": 9}]
```

This is the inverse of the nested inventory report built earlier in the volume — `entrySet` and `reduce` together unwind two levels of nesting into flat, tabular rows.

#498 Normalize CargoLink Settings Key Names

Normalize a legacy settings object's key names into lowercase, underscore-separated form.

INPUT

```
{ "Max Weight Kg": 2000, "Default Carrier": "GROUND" }
```

SOLUTION

```
payload mapObject ((value, key) -> { (lower(key replace " " with "_")): value
})
```

OUTPUT

```
{ "max_weight_kg": 2000, "default_carrier": "GROUND" }
```

Combining `replace ... with ...` and `lower` inside a single dynamic key expression handles both normalization steps in one pass.

#499 Build a CargoLink Order Version Diff Report

Build a readable diff report showing the field name, old value, and new value for every field that changed between two order versions.

INPUT

```
{ "oldVersion": {"status": "PENDING", "total": 240}, "newVersion": {"status": "SHIPPED", "total": 240} }
```

SOLUTION

```
(namesOf(payload.newVersion) filter ((k) -> (payload.newVersion[(k)]) != (payload.oldVersion[(k)])))
  map ((k) -> { field: k, old: payload.oldVersion[(k)], new: payload.newVersion[(k)] })
```

OUTPUT

```
[{"field": "status", "old": "PENDING", "new": "SHIPPED"}]
```

Building a structured diff report like this — rather than just the changed object itself — is what a real audit-log transform for order changes would produce.

#500 CargoLink Order Transformation Capstone

Transform a raw legacy order payload into a clean, API-ready shape: rename fields, nest the customer and address, compute item and order totals, and only include a rush fee when applicable — all while dropping internal fields.

INPUT

```
{ "order_id": "ORD-1042", "_syncFlag": true, "customer_name": "Maria Chen",
"rush": true, "items": [{"sku": "A", "price": 50, "qty": 2}], "street": "200
Depot Rd", "city": "Austin" }
```

SOLUTION

```
var itemsWithTotals = payload.items map ((item) -> item ++ { lineTotal:
item.price * item.qty })
var grandTotal = itemsWithTotals reduce ((item, t = 0) -> t + item.lineTotal)
---
{
  orderId: payload.order_id,
  customer: { name: payload.customer_name },
  shippingAddress: { street: payload.street, city: payload.city },
  items: itemsWithTotals,
  grandTotal: grandTotal,
  rushFee: 15 if payload.rush
}
```

OUTPUT

```
{ "orderId": "ORD-1042", "customer": { "name": "Maria Chen" },
"shippingAddress": { "street": "200 Depot Rd", "city": "Austin" }, "items":
[{"sku": "A", "price": 50, "qty": 2, "lineTotal": 100}], "grandTotal": 100,
"rushFee": 15 }
```

This capstone combines nearly every idea from the volume — renaming, nested restructuring, dynamic `var` steps, item-level transforms, and conditional fields — into one realistic order-normalization transform, with `_syncFlag` dropped simply by never being referenced.

END OF VOLUME 5

500 Down, 500 to Go

Halfway there. You've now worked through selectors, type coercion, operators and conditionals, array transformations, and a complete tour of object transformations — `mapObject`, `pluck`, `filterObject`, `namesOf`/`valuesOf`, `entrySet`, and dynamic key construction.

Volume 6 moves to String Functions: another 100 problems on `upper`/`lower`, `trim`, `splitBy`/`joinBy`, `replace`, regular expressions, padding, and every other tool for shaping text.

MULESOFT · PRACTICE PROBLEMS · WITH FULL SOLUTIONS

1000 DataWeave Problems With Solutions

Volume 6 — String Functions. 100 hands-on problems on case conversion, trimming, splitting/joining, replacing, slicing, and regular expressions.

Problems 501–600

Volume 6 of 10

2026 EDITION

How to Use This Volume

Volume 6 is about **text** — cleaning it, splitting it, searching it, and reshaping it into exactly the format a downstream field expects. Same format as every prior volume:

- **A short problem statement**, the **input** payload, a complete **solution** in a black code window, the exact **output**, and a one-line explanation.
- Ten sections of ten problems each, moving from `upper`/`lower` and `trim` through `splitBy`/`joinBy`, `replace`, substring slicing, `contains`/`startsWith`/`endsWith`, case-and-padding helpers, and regular expressions.

Most of this volume's functions — `upper`, `lower`, `trim`, `contains`, `startsWith`, `endsWith`, `splitBy`, `joinBy`, `replace`, `matches`, `scan` — are available globally with no setup. A handful of extra helpers used in Section 7, like `capitalize` and `leftPad`, live in the `dw::core::Strings` module and need one line at the top of a real script: `import * from dw::core::Strings`. Each problem that needs it shows that line explicitly.

VOLUME 6

String Functions

100 problems on `upper`, `lower`, `trim`, `splitBy`, `joinBy`, `replace`, substring slicing, `contains`, `startsWith`, `endsWith`, and the regular expression tools DataWeave uses to validate and extract text.

SECTION 1 — upper/lower Basics (Problems 501–510)

#501 Uppercase a Carrier Name

Convert a carrier name to uppercase for consistent display.

INPUT

```
{ "carrier": "fedex" }
```

SOLUTION

```
upper(payload.carrier)
```

OUTPUT

```
"FEDEX"
```

`upper` converts every letter in a String to uppercase, leaving non-letter characters untouched.

#502 Normalize an Email to Lowercase

Lowercase a customer's email address before storing it, so lookups are case-insensitive.

INPUT

```
{ "email": "Maria@CargoLink.com" }
```

SOLUTION

```
lower(payload.email)
```

OUTPUT

```
"maria@cargolink.com"
```

Normalizing case before storage is one of the most common reasons to reach for `lower` in a real integration.

#503 Build a Shout-Style Alert Label

Build an uppercase alert label by combining `upper` with String concatenation.

INPUT

```
{ "status": "delayed" }
```

SOLUTION

```
"ALERT: " ++ upper(payload.status)
```

OUTPUT

```
"ALERT: DELAYED"
```

`upper` can be applied to just part of a String before concatenation, letting the rest of the label keep its original casing.

#504 Compare Two Strings Case-Insensitively

Check whether two carrier codes match, ignoring case differences.

INPUT

```
{ "a": "FedEx", "b": "fedex" }
```

SOLUTION

```
lower(payload.a) == lower(payload.b)
```

OUTPUT

```
true
```

Lowercasing both sides before == is the standard way to compare Strings without their casing affecting the result.

#505 Uppercase Every Element of an Array

Uppercase every warehouse code in an array.

INPUT

```
{ "codes": ["wh-1", "wh-2"] }
```

SOLUTION

```
payload.codes map upper($)
```

OUTPUT

```
["WH-1", "WH-2"]
```

`upper` combines naturally with `map` to normalize casing across an entire array at once.

#506 Compare Ignoring Case Inside a filter

Keep only the orders whose status equals `"open"`, regardless of how it was cased in the source data.

INPUT

```
{ "orders": [{"status": "OPEN"}, {"status": "Closed"}, {"status": "open"}] }
```

SOLUTION

```
payload.orders filter (lower($.status) == "open")
```

OUTPUT

```
[{"status": "OPEN"}, {"status": "open"}]
```

Normalizing with `lower` inside a `filter` condition protects against inconsistent source-system casing.

#507 Build Initials From Two Name Fields

Build a two-letter initials String from a first and last name.

INPUT

```
{ "firstName": "maria", "lastName": "chen" }
```

SOLUTION

```
upper(payload.firstName[0]) ++ upper(payload.lastName[0])
```

OUTPUT

```
"MC"
```

Strings support the same `[index]` selector as Arrays, so `firstName[0]` grabs the first character before it gets uppercased.

#508 Normalize a Country Code Before Comparison

Check whether a shipment's country code is the US, regardless of casing in the source field.

INPUT

```
{ "country": "us" }
```

SOLUTION

```
upper(payload.country) == "US"
```

OUTPUT

```
true
```

It doesn't matter whether you normalize both sides to upper or both to lower — what matters is normalizing both sides the same way.

#509 Uppercase Only the String Values in a Mixed Array

Uppercase every String in a mixed array, leaving Numbers untouched.

INPUT

```
{ "values": ["fedex", 12, "ups"] }
```

SOLUTION

```
payload.values map (v) -> v match {
  case s is String -> upper(s)
  else -> v
}
```

OUTPUT

```
["FEDEX", 12, "UPS"]
```

A type matcher guards `upper` from ever being called on a non-String value, which would otherwise raise a runtime error.

#510 Normalize a CargoLink Carrier Code Lookup

Build a normalized lookup Array of carrier codes so downstream matching never breaks on inconsistent casing.

INPUT

```
{ "carriers": ["FedEx", "ups", "USPS"] }
```

SOLUTION

```
payload.carriers map upper($)
```

OUTPUT

```
["FEDEX", "UPS", "USPS"]
```

Normalizing an entire lookup list up front, once, is more reliable than trying to normalize every comparison against it individually later.

SECTION 2 — trim & Whitespace Cleanup (Problems 511–520)

#511 Trim a Padded Code

Remove leading and trailing whitespace from a tracking code.

INPUT

```
{ "code": "  CL88213  " }
```

SOLUTION

```
trim(payload.code)
```

OUTPUT

```
"CL88213"
```

`trim` removes whitespace only from the start and end of a String, leaving any internal spacing untouched.

#512 Compare a Trimmed Value to an Expected Value

Check whether a trimmed input status matches an expected value exactly.

INPUT

```
{ "status": " OPEN " }
```

SOLUTION

```
trim(payload.status) == "OPEN"
```

OUTPUT

```
true
```

Without `trim`, stray whitespace from a source system would make an otherwise-correct equality check fail silently.

#513 Trim Every Element of an Array

Clean up an array of item codes that each have stray leading or trailing whitespace.

INPUT

```
{ "codes": [" A ", "B  ", "  C"] }
```

SOLUTION

```
payload.codes map trim($)
```

OUTPUT

```
["A", "B", "C"]
```

Combining `trim` with `map` is the standard first cleanup pass over an array of raw, user-entered or CSV-sourced Strings.

#514 Detect a Blank-but-Not-Null Field

Check whether a notes field is effectively blank, even though it contains only whitespace rather than being empty or null.

INPUT

```
{ "notes": "   " }
```

SOLUTION

```
isEmpty(trim(payload.notes))
```

OUTPUT

```
true
```

Trimming before checking `isEmpty` catches whitespace-only Strings, which plain `isEmpty` or a `== ""` check would otherwise miss.

#515 Trim Before Splitting a Delimited List

Clean up a comma-separated list that has stray whitespace around the whole String before splitting it.

INPUT

```
{ "raw": "  A,B,C  " }
```

SOLUTION

```
trim(payload.raw) splitBy ","
```

OUTPUT

```
["A", "B", "C"]
```

Trimming the whole String first avoids an unwanted leading or trailing empty-ish fragment appearing after the split.

#516 Trim a Field While Building an Object

Build a clean customer record where the notes field has been trimmed inline.

INPUT

```
{ "name": "Maria Chen", "notes": "  handle with care  " }
```

SOLUTION

```
{
  name: payload.name,
  notes: trim(payload.notes)
}
```

OUTPUT

```
{ "name": "Maria Chen", "notes": "handle with care" }
```

Applying `trim` directly inside an object literal's field value keeps the cleanup step co-located with the field it affects.

#517 Filter Out Blank Strings From an Array

Remove every entry from an array that is blank after trimming, keeping only genuinely meaningful entries.

INPUT

```
{ "notes": ["Fragile", "  ", "", "Handle with care"] }
```

SOLUTION

```
payload.notes filter (not isEmpty(trim($)))
```

OUTPUT

```
["Fragile", "Handle with care"]
```

This "trim, then check empty" combination is the most reliable way to distinguish real content from whitespace-only noise.

#518 Combine trim With default

Provide a fallback message when a notes field, after trimming, turns out to be blank.

INPUT

```
{ "notes": "   " }
```

SOLUTION

```
if (isEmpty(trim(payload.notes))) "No notes provided" else
trim(payload.notes)
```

OUTPUT

```
"No notes provided"
```

`default` alone only catches `null`, so a whitespace-only String needs this explicit `if/else` check instead.

#519 Trim Key Names While Rebuilding an Object

Clean up an object whose key names have stray whitespace, a common artifact of poorly generated CSV headers.

INPUT

```
{ " sku ": "A", " qty": 2 }
```

SOLUTION

```
payload mapObject ((value, key) -> { (trim(key)): value })
```

OUTPUT

```
{ "sku": "A", "qty": 2 }
```

`trim` applies to a dynamic key expression exactly as it would to any other String, cleaning up malformed key names from messy source headers.

#520 Clean a Batch of CargoLink Tracking Codes

Clean an entire batch of tracking codes: trim whitespace and uppercase each one in a single pass.

INPUT

```
{ "codes": [" cl001 ", "cl002  ", "  CL003"] }
```

SOLUTION

```
payload.codes map upper(trim($))
```

OUTPUT

```
["CL001", "CL002", "CL003"]
```

Nesting `upper` around `trim` applies both cleanup steps to every element in one combined `map` pass.

SECTION 3 — splitBy & joinBy (Problems 521–530)

#521 Split a CSV Row Into Fields

Split a comma-separated row into an Array of individual fields.

INPUT

```
{ "row": "PALLET-01,2,45.00" }
```

SOLUTION

```
payload.row splitBy ","
```

OUTPUT

```
["PALLET-01", "2", "45.00"]
```

`splitBy` divides a String into an Array wherever the separator appears, discarding the separator itself.

#522 Join an Array Into a Delimited String

Join an array of item codes into a single comma-separated String.

INPUT

```
{ "items": ["PALLET-01", "CRATE-07", "BOX-12"] }
```

SOLUTION

```
payload.items joinBy ","
```

OUTPUT

```
"PALLET-01,CRATE-07,BOX-12"
```

`joinBy` is the exact inverse of `splitBy` — it stitches an Array of Strings back into one String with the separator in between.

#523 Split on a Multi-Character Separator

Split a route description on the multi-character separator `" -> "`.

INPUT

```
{ "route": "Dallas -> Austin -> Houston" }
```

SOLUTION

```
payload.route splitBy " -> "
```

OUTPUT

```
["Dallas", "Austin", "Houston"]
```

The separator passed to `splitBy` can be any length — DataWeave matches it as a literal substring, not just a single character.

#524 Split, Transform, and Rejoin

Uppercase every stop name in a route String, keeping the same overall format.

INPUT

```
{ "route": "dallas-austin-houston" }
```

SOLUTION

```
(payload.route splitBy "-") map upper($) joinBy "-"
```

OUTPUT

```
"DALLAS-AUSTIN-HOUSTON"
```

Splitting, mapping a transform over the pieces, and rejoining is the standard three-step pattern for editing a delimited String in place.

#525 Split a Full Name Into First and Last

Split a full name on the space character to get first and last name separately.

INPUT

```
{ "fullName": "Maria Chen" }
```

SOLUTION

```
var parts = payload.fullName splitBy " "
---
{ firstName: parts[0], lastName: parts[1] }
```

OUTPUT

```
{ "firstName": "Maria", "lastName": "Chen" }
```

Once split into an Array, ordinary index selectors pull out each named part of the original String.

#526 Reformat a Delimiter

Convert a pipe-delimited String into a comma-and-space-delimited String.

INPUT

```
{ "raw": "PALLET-01|CRATE-07|BOX-12" }
```

SOLUTION

```
(payload.raw splitBy "|") joinBy ", "
```

OUTPUT

```
"PALLET-01, CRATE-07, BOX-12"
```

Splitting on one separator and joining with another is the simplest way to convert between two delimiter formats.

#527 Get the Last Segment of a Path-Like String

Get just the file name from a slash-delimited file path.

INPUT

```
{ "path": "shipments/2026/03/manifest.csv" }
```

SOLUTION

```
(payload.path splitBy "/")[-1]
```

OUTPUT

```
"manifest.csv"
```

The `[-1]` index selector — already familiar from Array selectors — grabs the final segment after a split, regardless of how many segments there are.

#528 Count Items in a Delimited List

Count how many item codes are present in a comma-separated list.

INPUT

```
{ "items": "A,B,C,D" }
```

SOLUTION

```
sizeOf(payload.items splitBy ",")
```

OUTPUT

```
4
```

Chaining `splitBy` into `sizeOf` is a quick way to count entries in a delimited String without building an intermediate variable.

#529 Build a Summary Line With joinBy After a map

Build a single summary line listing each order's ID and total, joined with semicolons.

INPUT

```
{ "orders": [{"id": "O1", "total": 100}, {"id": "O2", "total": 50}] }
```

SOLUTION

```
(payload.orders map ((o) -> o.id ++ ": $" ++ (o.total as String))) joinBy "; "
```

OUTPUT

```
"O1: $100; O2: $50"
```

Mapping to formatted label Strings first, then joining, is exactly how a one-line summary report is generated from structured data.

#530 Parse and Rebuild a CargoLink Multi-Stop Route

Parse a raw multi-stop route String, trim each stop, and rebuild it with a cleaner separator.

INPUT

```
{ "raw": " Dallas , Austin ,Houston " }
```

SOLUTION

```
((payload.raw splitBy ",") map trim($)) joinBy " -> "
```

OUTPUT

```
"Dallas -> Austin -> Houston"
```

This split-trim-rejoin chain is exactly the kind of cleanup a real route-import transform applies to messy, manually entered stop lists.

SECTION 4 — replace: Literal Find-and-Replace (Problems 531–540)

#531 Replace a Literal Substring

Replace the word `"Old"` with `"New"` in a warehouse name.

INPUT

```
{ "name": "Old Austin Depot" }
```

SOLUTION

```
payload.name replace "Old" with "New"
```

OUTPUT

```
"New Austin Depot"
```

The `replace ... with ...` operator swaps every occurrence of the target substring for the replacement.

#532 Confirm All Occurrences Are Replaced

Replace every dash in a code with an underscore, confirming more than one occurrence is handled.

INPUT

```
{ "code": "CL-2026-03-14" }
```

SOLUTION

```
payload.code replace "-" with "_"
```

OUTPUT

```
"CL_2026_03_14"
```

For a literal substring target, `replace` swaps every matching occurrence in the String, not just the first one.

#533 Replace With a Dynamically Built Value

Replace a placeholder in a message template with a real order ID.

INPUT

```
{ "template": "Your order {ID} has shipped.", "orderId": "ORD-1042" }
```

SOLUTION

```
payload.template replace "{ID}" with payload.orderId
```

OUTPUT

```
"Your order ORD-1042 has shipped."
```

The replacement side of `replace ... with ...` can be any String expression, not just a literal — including a value pulled from elsewhere in the payload.

#534 Strip a Special Character

Remove every **#** character from a reference code by replacing it with an empty String.

INPUT

```
{ "ref": "#CL-8821#" }
```

SOLUTION

```
payload.ref replace "#" with ""
```

OUTPUT

```
"CL-8821"
```

Replacing a character with an empty String is the standard way to delete it entirely, rather than swap it for something else.

#535 Chain Two Replace Calls

Clean up a code by replacing two different unwanted characters in sequence.

INPUT

```
{ "code": "CL_88#21" }
```

SOLUTION

```
(payload.code replace "_" with "-") replace "#" with ""
```

OUTPUT

```
"CL-8821"
```

Each `replace` call returns a new String, so chaining a second `replace` onto the result handles cleanup steps one at a time.

#536 Normalize a Date Separator

Convert a slash-separated date into a dash-separated date, ready for `as Date` coercion.

INPUT

```
{ "rawDate": "2026/03/14" }
```

SOLUTION

```
payload.rawDate replace "/" with "-"
```

OUTPUT

```
"2026-03-14"
```

Normalizing a separator with `replace` before coercion is a common fix for a source system that formats dates differently than DataWeave's default `Date` pattern expects.

#537 Redact a Known Fixed Prefix

Redact a known area code prefix from a phone number, replacing it with `"XXX"`.

INPUT

```
{ "phone": "512-555-1000" }
```

SOLUTION

```
payload.phone replace "512" with "XXX"
```

OUTPUT

```
"XXX-555-1000"
```

A literal `replace` only matches an exact known substring — masking a varying pattern like "any 3 digits" needs a regular expression instead, covered later in this volume.

#538 Apply replace Across an Array

Replace every space with a dash across an array of route names.

INPUT

```
{ "routes": ["North Texas", "South Bay"] }
```

SOLUTION

```
payload.routes map ($ replace " " with "-")
```

OUTPUT

```
["North-Texas", "South-Bay"]
```

Combining `replace` with `map` applies the same substitution rule consistently across an entire array.

#539 Build a URL-Safe Slug

Build a URL-safe slug from a route name by lowercasing it and replacing spaces with dashes.

INPUT

```
{ "routeName": "North Texas Express" }
```

SOLUTION

```
lower(payload.routeName) replace " " with "-"
```

OUTPUT

```
"north-texas-express"
```

This two-step "lower, then replace" pattern is the simplest way to build a basic URL slug directly in DataWeave.

#540 Normalize a Batch of CargoLink Order Codes

Normalize a batch of order codes so every separator character is a dash, regardless of whether the source used dots or underscores.

INPUT

```
{ "codes": ["ORD.1042", "ORD_1043"] }
```

SOLUTION

```
payload.codes map (($ replace "." with "-") replace "_" with "-")
```

OUTPUT

```
["ORD-1042", "ORD-1043"]
```

Chaining two `replace` calls inside a `map` handles both known separator variants in a single normalization pass.

SECTION 5 — Substring Access & Slicing (Problems 541–550)

#541 Get the First Character of a String

Get just the first character of an order status code.

INPUT

```
{ "code": "OPEN" }
```

SOLUTION

```
payload.code[0]
```

OUTPUT

```
"O"
```

Strings support the same `[index]` selector as Arrays — index `0` is always the first character.

#542 Get the Last Character of a String

Get just the last character of a tracking code.

INPUT

```
{ "code": "CL88213" }
```

SOLUTION

```
payload.code[-1]
```

OUTPUT

```
"3"
```

Just like Arrays, a negative index counts backward from the end of the String, so `[-1]` is always the last character.

#543 Extract a Substring Range

Extract the first three characters of a warehouse code.

INPUT

```
{ "code": "WH-1-AUSTIN" }
```

SOLUTION

```
payload.code[0 to 2]
```

OUTPUT

```
"WH-"
```

A range selector `[a to b]` on a String is inclusive of both ends, exactly like it is on an Array.

#544 Extract Everything After a Fixed Prefix

Extract everything after the first 3 characters of a code, regardless of the code's total length.

INPUT

```
{ "code": "CL-88213" }
```

SOLUTION

```
payload.code[3 to -1]
```

OUTPUT

```
"88213"
```

Combining a fixed start index with `-1` as the end index extracts "everything from here to the end" without needing to know the String's exact length.

#545 Extract a Fixed-Width Prefix Code

Extract a 2-character carrier prefix from the start of a longer tracking code.

INPUT

```
{ "trackingCode": "FX88213000" }
```

SOLUTION

```
payload.trackingCode[0 to 1]
```

OUTPUT

```
"FX"
```

Extracting a fixed-width prefix by index range is common when parsing legacy codes that pack multiple fields into one fixed-format String.

#546 Extract a Fixed-Width Suffix

Extract the last 3 digits of a tracking code as a checksum suffix.

INPUT

```
{ "trackingCode": "FX88213000" }
```

SOLUTION

```
payload.trackingCode[-3 to -1]
```

OUTPUT

```
"000"
```

A range built entirely from negative indices extracts a fixed-width slice counted from the end of the String.

#547 Parse a Fixed-Width Numeric Field

Extract a fixed-width quantity field from a legacy record and coerce it to a real Number.

INPUT

```
{ "record": "SKU001QTY012" }
```

SOLUTION

```
payload.record[-3 to -1] as Number
```

OUTPUT

```
12
```

Slicing and coercion combine naturally — the slice extracts the raw digits, and `as Number` turns them into a usable value.

#548 Build a Truncated Preview String

Build a preview of a notes field, truncated to 10 characters with an ellipsis if it's longer than that.

INPUT

```
{ "notes": "Handle with extreme care during transit" }
```

SOLUTION

```
if (sizeOf(payload.notes) > 10) (payload.notes[0 to 9] ++ "...") else
payload.notes
```

OUTPUT

```
"Handle wit..."
```

Checking length with `sizeOf` before slicing avoids an unnecessarily truncated ellipsis on Strings that were already short enough.

#549 Extract a Shared Prefix From Every Code in an Array

Extract the 2-character carrier prefix from every tracking code in a batch.

INPUT

```
{ "codes": ["FX88213", "UP99201"] }
```

SOLUTION

```
payload.codes map ($[0 to 1])
```

OUTPUT

```
["FX", "UP"]
```

A slice expression works inside a `map` lambda exactly like any other expression, applying the same extraction rule to every element.

#550 Parse a CargoLink Fixed-Format Tracking Code

Parse a fixed-format tracking code into its carrier prefix, batch number, and checksum digit using slicing.

INPUT

```
{ "code": "FX882130007" }
```

SOLUTION

```
{
  carrierPrefix: payload.code[0 to 1],
  batchNumber: payload.code[2 to -2],
  checksum: payload.code[-1]
}
```

OUTPUT

```
{ "carrierPrefix": "FX", "batchNumber": "88213000", "checksum": "7" }
```

Combining several slice ranges in one object literal is exactly how a fixed-width legacy format gets parsed into named fields.

SECTION 6 — contains, startsWith & endsWith (Problems 551–560)

#551 Check if a String Contains a Substring

Check whether a shipping note mentions the word `"FRAGILE"` anywhere within it.

INPUT

```
{ "note": "Handle with care - FRAGILE" }
```

SOLUTION

```
payload.note contains "FRAGILE"
```

OUTPUT

```
true
```

Unlike `matches`, `contains` only needs the substring to appear anywhere within the larger String.

#552 Check if a String Starts With a Prefix

Check whether an order code starts with the expected `"ORD-"` prefix.

INPUT

```
{ "code": "ORD-1042" }
```

SOLUTION

```
payload.code startsWith "ORD-"
```

OUTPUT

```
true
```

`startsWith` checks only the beginning of a String, ignoring everything that comes after the prefix.

#553 Check if a String Ends With a Suffix

Check whether a file name has the expected `.csv` extension.

INPUT

```
{ "file": "manifest.csv" }
```

SOLUTION

```
payload.file endsWith ".csv"
```

OUTPUT

```
true
```

`endsWith` is the mirror image of `startsWith`, checking only the tail end of the String.

#554 Filter an Array Using contains

Keep only the notes that mention `"fragile"` anywhere in their text.

INPUT

```
{ "notes": ["fragile item", "standard box", "handle with fragile care"] }
```

SOLUTION

```
payload.notes filter ($ contains "fragile")
```

OUTPUT

```
["fragile item", "handle with fragile care"]
```

Combining `contains` with `filter` is a simple, effective way to search free-text fields across an array.

#555 Check a String Against a List of Allowed Prefixes

Check whether a tracking code starts with any one of several recognized carrier prefixes.

INPUT

```
{ "code": "UPS-88213", "prefixes": ["UPS-", "FDX-", "USPS-"] }
```

SOLUTION

```
payload.prefixes some ((p) -> payload.code startsWith p)
```

OUTPUT

```
true
```

Wrapping `startsWith` inside `some` checks a String against a whole list of acceptable prefixes in one expression.

#556 Validate a File Extension

Confirm a file name ends with the expected extension before processing it.

INPUT

```
{ "file": "manifest.txt" }
```

SOLUTION

```
payload.file endsWith ".txt"
```

OUTPUT

```
true
```

A quick `endsWith` check like this is a common early gate in a file-processing flow, before anything more expensive runs.

#557 Check contains Case-Insensitively

Check whether a message mentions `"urgent"` anywhere, regardless of how it was cased.

INPUT

```
{ "message": "URGENT delivery required" }
```

SOLUTION

```
lower(payload.message) contains "urgent"
```

OUTPUT

```
true
```

`contains`, like `==`, is case-sensitive by default — normalizing with `lower` first is what makes the check case-insensitive.

#558 Combine startsWith and endsWith in One Validation

Validate that a shipment code starts with `"SHP-"` and does not have a trailing space.

INPUT

```
{ "code": "SHP-501 " }
```

SOLUTION

```
(payload.code startsWith "SHP-") and (not (payload.code endsWith " "))
```

OUTPUT

```
false
```

Even though the prefix is correct, the trailing space fails the second condition — combining both checks with `and` catches format problems a single check would miss.

#559 Route a Message Based on Keyword Presence

Route a free-text status message to a queue based on which keyword it contains.

INPUT

```
{ "message": "the shipment was cancelled by customer" }
```

SOLUTION

```
if (payload.message contains "cancelled") "CANCELLATION_QUEUE"
else if (payload.message contains "delayed") "EXCEPTION_QUEUE"
else "STANDARD_QUEUE"
```

OUTPUT

```
"CANCELLATION_QUEUE"
```

Checking free text for known keywords with `contains`, ordered from most to least specific, is a lightweight way to route unstructured messages.

#560 Flag CargoLink Orders With Urgent Notes

Flag every order in a batch whose notes field mentions `"urgent"`, regardless of case.

INPUT

```
{ "orders": [{"id": "O1", "notes": "Urgent - handle first"}, {"id": "O2",
"notes": "standard delivery"}] }
```

SOLUTION

```
payload.orders filter (lower($.notes) contains "urgent")
```

OUTPUT

```
[{"id": "O1", "notes": "Urgent - handle first"}]
```

This exact pattern — normalize with `lower`, then `contains`, then `filter` — is the standard way to keyword-search free text across a whole array of records.

SECTION 7 — Case Conversion & Padding (Problems 561–570)

#561 Capitalize the First Letter of a Name

Capitalize just the first letter of a lowercase name.

INPUT

```
{ "name": "maria" }
```

SOLUTION

```
import * from dw::core::Strings
---
capitalize(payload.name)
```

OUTPUT

```
"Maria"
```

`capitalize` lives in the `dw::core::Strings` module, so a real script needs the `import` line shown here before it can be called by name.

#562 Fix ALL-CAPS Input Into Proper Case

Convert an ALL-CAPS name into proper case: lowercase everything, then capitalize the first letter.

INPUT

```
{ "name": "MARIA" }
```

SOLUTION

```
import * from dw::core::Strings
---
capitalize(lower(payload.name))
```

OUTPUT

```
"Maria"
```

Lowercasing first avoids `capitalize` leaving the rest of an ALL-CAPS String uppercase, since it only ever touches the very first character.

#563 Zero-Pad a Numeric Code

Pad a numeric ID to a fixed 5-character width using leading zeros.

INPUT

```
{ "id": 42 }
```

SOLUTION

```
import * from dw::core::Strings
---
leftPad(payload.id as String, 5, "0")
```

OUTPUT

```
"00042"
```

`leftPad` pads a String on the left until it reaches the target length, repeating the given pad character as needed.

#564 Pad a String With Spaces

Right-align a short code within a fixed 5-character field using space padding.

INPUT

```
{ "code": "A1" }
```

SOLUTION

```
import * from dw::core::Strings
---
leftPad(payload.code, 5, " ")
```

OUTPUT

```
"   A1"
```

The pad character passed to `leftPad` doesn't have to be a zero — any single character works, including a plain space.

#565 Pad Every Number in an Array

Format every invoice line number in an array as a zero-padded 4-digit String.

INPUT

```
{ "lineNumbers": [1, 23, 456] }
```

SOLUTION

```
import * from dw::core::Strings
---
payload.lineNumbers map ((n) -> leftPad(n as String, 4, "0"))
```

OUTPUT

```
["0001", "0023", "0456"]
```

Combining `leftPad` with `map` applies the same fixed-width formatting rule to every Number in the array.

#566 Capitalize Every Name in an Array

Capitalize every name in an array of lowercase driver names.

INPUT

```
{ "names": ["maria", "james", "ana"] }
```

SOLUTION

```
import * from dw::core::Strings
---
payload.names map capitalize($)
```

OUTPUT

```
["Maria", "James", "Ana"]
```

`capitalize` works inside `map` exactly like `upper` or `lower` do, once it has been imported.

#567 Combine capitalize and leftPad Into One Label

Build a formatted driver label combining a capitalized name and a zero-padded badge number.

INPUT

```
{ "name": "sam", "badgeNumber": 7 }
```

SOLUTION

```
import * from dw::core::Strings
---
capitalize(payload.name) ++ "-" ++ leftPad(payload.badgeNumber as String, 3,
"0")
```

OUTPUT

```
"Sam-007"
```

Once imported, both functions combine with ++ just like any other String-producing expression.

#568 Extract the Part of a String Before a Separator

Extract the username portion of an email address, before the @.

INPUT

```
{ "email": "maria@cargolink.com" }
```

SOLUTION

```
import * from dw::core::Strings
---
substringBefore(payload.email, "@")
```

OUTPUT

```
"maria"
```

`substringBefore` returns everything up to — but not including — the first occurrence of the given separator.

#569 Extract the Part of a String After a Separator

Extract the domain portion of an email address, after the @.

INPUT

```
{ "email": "maria@cargolink.com" }
```

SOLUTION

```
import * from dw::core::Strings
---
substringAfter(payload.email, "@")
```

OUTPUT

```
"cargolink.com"
```

`substringAfter` is the mirror of `substringBefore`, returning everything following the first occurrence of the separator.

#570 Format a CargoLink Driver Roster

Format a driver roster where every name is capitalized and every badge number is zero-padded to 4 digits.

INPUT

```
{ "drivers": [{"name": "sam", "badge": 7}, {"name": "ana", "badge": 23}] }
```

SOLUTION

```
import * from dw::core::Strings
---
payload.drivers map ((d) -> {
  name: capitalize(d.name),
  badge: leftPad(d.badge as String, 4, "0")
})
```

OUTPUT

```
[{"name": "Sam", "badge": "0007"}, {"name": "Ana", "badge": "0023"}]
```

This is exactly the kind of formatting pass a real roster or badge-printing transform applies before the data reaches its final display.

SECTION 8 — Regular Expressions: matches, scan & replace (Problems 571–580)

#571 Validate a Code Format With matches

Validate that an order code is exactly `"ORD-"` followed by 4 digits.

INPUT

```
{ "code": "ORD-1042" }
```

SOLUTION

```
payload.code matches /ORD-\d{4}/
```

OUTPUT

```
true
```

*`matches` checks the pattern against the **entire** String, not just part of it — unlike `contains`, a partial match anywhere in the middle is not enough.*

#572 Validate a Simple Email-Like Pattern

Check whether a String has the basic shape of an email address.

INPUT

```
{ "email": "maria@cargolink.com" }
```

SOLUTION

```
payload.email matches /[a-zA-Z0-9.]+@[a-zA-Z0-9.]+/
```

OUTPUT

```
true
```

Because `matches` requires a full match, the pattern must account for every character in the input, including the domain's dot.

#573 Filter an Array to Valid-Format Codes

Keep only the codes that match the exact `"ORD-"` plus 4-digit format.

INPUT

```
{ "codes": ["ORD-1042", "BAD CODE", "ORD-99"] }
```

SOLUTION

```
payload.codes filter ($ matches /ORD-\d{4}/)
```

OUTPUT

```
["ORD-1042"]
```

`"ORD-99"` fails because `matches` requires the whole String to fit the pattern, and it only has 2 digits where the pattern requires exactly 4.

#574 Extract Every Digit Sequence From a String

Extract every run of digits found anywhere within a free-text String.

INPUT

```
{ "text": "Order 1042 has 3 items, total $45" }
```

SOLUTION

```
scan(payload.text, /\d+/)
```

OUTPUT

```
[["1042"], ["3"], ["45"]]
```

`scan` finds every match anywhere in the String and returns them as an array of arrays — one inner array per match, holding the full match plus any capture groups.

#575 Extract a Capture Group From a Match

Extract just the numeric part of an order reference embedded in a sentence.

INPUT

```
{ "text": "Please process order O-1042 immediately" }
```

SOLUTION

```
scan(payload.text, /O-(\d+)/)
```

OUTPUT

```
[["O-1042", "1042"]]
```

Wrapping part of a regex pattern in parentheses creates a capture group — each match's inner array holds the full match first, then each capture group in order.

#576 Remove All Non-Digit Characters

Strip everything except digits from a formatted phone number.

INPUT

```
{ "phone": "(512) 555-1000" }
```

SOLUTION

```
payload.phone replace /[^0-9]/ with ""
```

OUTPUT

```
"5125551000"
```

A negated character class like `[^0-9]` matches "anything that is not a digit," making it a quick way to strip all formatting from a numeric field.

#577 Reformat a Date Using Capture Groups

Reformat a date from `YYYY-MM-DD` to `MM/DD/YYYY` using regex capture groups in the replacement.

INPUT

```
{ "date": "2026-03-14" }
```

SOLUTION

```
payload.date replace /(\d{4})-(\d{2})-(\d{2})/ with "$2/$3/$1"
```

OUTPUT

```
"03/14/2026"
```

The replacement String can reference captured groups as `$1`, `$2`, `$3`, letting a single regex both parse and reorder the matched pieces.

#578 Route Based on a Regex Match

Route a code to a priority queue if it matches an urgent-code format.

INPUT

```
{ "code": "URGENT-99" }
```

SOLUTION

```
if (payload.code matches /URGENT-\d+/) "PRIORITY_QUEUE" else "STANDARD_QUEUE"
```

OUTPUT

```
"PRIORITY_QUEUE"
```

`matches` returns a plain Boolean, so it drops directly into an `if` condition just like any other Boolean-producing expression.

#579 Count Pattern Occurrences With scan

Count how many times a repeating pattern appears in a String.

INPUT

```
{ "text": "item1, item2, item3, item4" }
```

SOLUTION

```
sizeOf(scan(payload.text, /item\d/))
```

OUTPUT

```
4
```

Chaining `scan` into `sizeOf` is a direct way to count how many times a pattern occurs, without caring about the matched text itself.

#580 Extract Structured Data From a Raw CargoLink String

Extract the tracking code and ship date embedded in a raw, semi-structured tracking String.

INPUT

```
{ "raw": "TRACK:CL-88213;DATE:2026-03-14" }
```

SOLUTION

```
{
  trackingCode: (scan(payload.raw, /TRACK:([A-Z0-9-]+);/))[0][1],
  shipDate: (scan(payload.raw, /DATE:(\d{4}-\d{2}-\d{2})/))[0][1]
}
```

OUTPUT

```
{ "trackingCode": "CL-88213", "shipDate": "2026-03-14" }
```

Indexing into `scan`'s result with `[0][1]` grabs the first match's first capture group — the standard way to pull one specific value out of a larger raw String.

SECTION 9 — Chaining String Functions Together (Problems 581–590)

#581 Trim, Then Uppercase

Clean up a code by trimming whitespace and uppercasing it in one expression.

INPUT

```
{ "code": " cl-001 " }
```

SOLUTION

```
upper(trim(payload.code))
```

OUTPUT

```
"CL-001"
```

Nesting function calls, innermost first, is the standard way to apply several single-purpose String functions in sequence.

#582 Clean a Messy CSV Row

Split a CSV row, trim each field, and rejoin it into a clean row.

INPUT

```
{ "row": " A , B ,C " }
```

SOLUTION

```
((payload.row splitBy ",") map trim($)) joinBy ","
```

OUTPUT

```
"A,B,C"
```

This split-clean-rejoin chain is the standard fix for CSV data that has inconsistent spacing around its delimiters.

#583 Normalize a Separator, Then Split

Normalize a pipe-delimited String to commas, then split it into an Array.

INPUT

```
{ "raw": "A|B|C|D" }
```

SOLUTION

```
(payload.raw replace "|" with ",") splitBy ","
```

OUTPUT

```
["A", "B", "C", "D"]
```

Normalizing a delimiter before splitting is useful when a source system might send either format inconsistently.

#584 Filter and Rebuild a Summary Line

Keep only the fragile-tagged items and rebuild them into a single comma-separated summary.

INPUT

```
{ "items": ["fragile-glass", "standard-box", "fragile-electronics"] }
```

SOLUTION

```
(payload.items filter ($ contains "fragile")) joinBy ", "
```

OUTPUT

```
"fragile-glass, fragile-electronics"
```

Filtering first and joining second is the standard two-step pattern for turning a keyword search into a readable summary line.

#585 Strip Punctuation, Lowercase, and Split Into Words

Clean a title by removing punctuation, lowercasing it, and splitting it into individual words.

INPUT

```
{ "title": "CargoLink: Fast, Reliable Shipping!" }
```

SOLUTION

```
(lower(payload.title replace /[^a-zA-Z ]/ with "")) splitBy " "
```

OUTPUT

```
["cargolink", "fast", "reliable", "shipping"]
```

Removing anything that isn't a letter or space with a regex `replace`, before lowercasing and splitting, is a simple way to tokenize free text into clean words.

#586 Validate, Then Reformat

Keep only correctly formatted order codes, then reformat each one into a friendlier label.

INPUT

```
{ "codes": ["ORD-1042", "BAD", "ORD-2201"] }
```

SOLUTION

```
(payload.codes filter ($ matches /ORD-\d{4}/)) map ((c) -> c replace "ORD-"
with "Order #")
```

OUTPUT

```
["Order #1042", "Order #2201"]
```

Validating with `matches` before transforming avoids applying formatting logic to data that was never in the expected shape to begin with.

#587 Parse a Fixed-Width Record and Calculate

Parse a fixed-width record into a quantity and a price, then calculate their product.

INPUT

```
{ "record": "012045" }
```

SOLUTION

```
(payload.record[0 to 2] as Number) * (payload.record[3 to 5] as Number)
```

OUTPUT

```
540
```

Slicing out each fixed-width field and coercing it to Number before the calculation is the standard way to work with legacy positional records.

#588 Parse a CSV String and Sum the Values

Parse a comma-separated String of numbers and compute their total.

INPUT

```
{ "csv": "10,20,30" }
```

SOLUTION

```
((payload.csv splitBy ",") map ((s) -> s as Number)) reduce ((n, total = 0) -
> total + n)
```

OUTPUT

```
60
```

Splitting, coercing every piece, and reducing to a sum is exactly the pipeline needed to turn a raw numeric CSV String into a usable total.

#589 Build a Multi-Part Formatted Report Line

Build a single formatted report line combining a cleaned name, an order count, and a formatted revenue figure.

INPUT

```
{ "name": "maria chen", "orders": 5, "revenue": 1200.5 }
```

SOLUTION

```
upper(trim(payload.name)) ++ " | Orders: " ++ (payload.orders as String) ++ "
| Revenue: $" ++ (payload.revenue as String {format: "0.00"})
```

OUTPUT

```
"MARIA CHEN | Orders: 5 | Revenue: $1200.50"
```

Report lines like this typically combine four or five small String operations — case conversion, coercion, and formatting — into one concatenated expression.

#590 Clean and Reformat Raw CargoLink Carrier Data

Clean up a raw carrier field: trim whitespace, replace underscores with spaces, and uppercase the result.

INPUT

```
{ "raw": "  fedex_express  " }
```

SOLUTION

```
upper(trim(payload.raw) replace "_" with " ")
```

OUTPUT

```
"FEDEX EXPRESS"
```

Trimming before replacing avoids the outer whitespace interfering with the underscore replacement, and uppercasing last produces the final display format.

SECTION 10 — Mixed Real-World String Challenges (Problems 591–600)

#591 Normalize a CargoLink Tracking Number Format

Normalize an inconsistently formatted tracking number: strip anything that isn't a letter or digit, uppercase it, then insert a dash after the 2-letter carrier prefix.

INPUT

```
{ "raw": "fx 882 13" }
```

SOLUTION

```
var clean = upper(payload.raw replace /[^a-zA-Z0-9]/ with "")
---
clean[0 to 1] ++ "-" ++ clean[2 to -1]
```

OUTPUT

```
"FX-88213"
```

Cleaning first with a regex, then slicing the result into fixed parts, is a reliable way to normalize inconsistent tracking number formats from different source systems.

#592 Validate a CargoLink Order Code Format

Validate that an order code exactly matches the expected `"ORD-"` plus 4-digit format.

INPUT

```
{ "code": "ORD-1042" }
```

SOLUTION

```
payload.code matches /ORD-\d{4}/
```

OUTPUT

```
true
```

A quick `matches` gate like this at the top of a flow catches malformed order codes before they reach any downstream logic.

#593 Build a CargoLink SMS Notification

Fill in a message template with the customer name, order ID, and ship date.

INPUT

```
{ "template": "Hi {name}, your order {id} ships {date}.", "name": "Maria",
"id": "ORD-1042", "date": "March 14" }
```

SOLUTION

```
((payload.template replace "{name}" with payload.name) replace "{id}" with
payload.id) replace "{date}" with payload.date
```

OUTPUT

```
"Hi Maria, your order ORD-1042 ships March 14."
```

Chaining one `replace` call per placeholder is a simple, dependency-free way to fill a template without a dedicated templating library.

#594 Parse a Combined City, State Zip String

Parse a combined `"City, State Zip"` String into three separate fields.

INPUT

```
{ "location": "Austin, TX 78701" }
```

SOLUTION

```
var parts = payload.location splitBy ", "
var stateZip = parts[1] splitBy " "
---
{ city: parts[0], state: stateZip[0], zip: stateZip[1] }
```

OUTPUT

```
{ "city": "Austin", "state": "TX", "zip": "78701" }
```

A two-stage split — first on the comma, then on the space within the second piece — handles this common combined-address format cleanly.

#595 Mask a Card Number for Display

Mask a card number for display, showing only the last 4 digits.

INPUT

```
{ "card": "4111111111111234" }
```

SOLUTION

```
"************" ++ payload.card[-4 to -1]
```

OUTPUT

```
"************1234"
```

Combining a fixed literal mask with a negative-index slice for the last 4 characters is a simple, dependable way to redact sensitive fixed-length data.

#596 Build a CargoLink Route Slug

Build a URL-safe slug from a route name, removing punctuation and using dashes between words.

INPUT

```
{ "routeName": "Austin Dallas Express!" }
```

SOLUTION

```
lower(payload.routeName replace /[^a-zA-Z0-9 ]/ with "") replace " " with "-"
```

OUTPUT

```
"austin-dallas-express"
```

Stripping punctuation first, then converting spaces to dashes, keeps the two cleanup concerns — unwanted characters and separators — handled as two clear, separate steps.

#597 Extract Driver Initials for a Badge

Extract two-letter initials from a driver's full name for use on a badge.

INPUT

```
{ "fullName": "Samuel Ortiz" }
```

SOLUTION

```
var parts = payload.fullName splitBy " "
---
upper(parts[0][0]) ++ upper(parts[1][0])
```

OUTPUT

```
"SO"
```

Splitting on the space first, then slicing the first character of each resulting name, is the reliable way to build initials from names of any length.

#598 Collapse Duplicate Whitespace

Collapse any run of multiple spaces in a free-text notes field down to a single space.

INPUT

```
{ "notes": "Handle   with    care" }
```

SOLUTION

```
payload.notes replace / +/ with " "
```

OUTPUT

```
"Handle with care"
```

The regex `/ +/` matches any run of one or more spaces, so both single and repeated spaces collapse to exactly one.

#599 Build a Multi-Field CargoLink Shipment Label

Build a formatted shipment label combining the carrier, tracking code, and weight into one readable line.

INPUT

```
{ "carrier": "fedex", "trackingCode": "fx88213000", "weight": 12.5 }
```

SOLUTION

```
"Carrier: " ++ upper(payload.carrier) ++ " | Tracking: " ++
upper(payload.trackingCode) ++ " | Weight: " ++ (payload.weight as String) ++
"kg"
```

OUTPUT

```
"Carrier: FEDEX | Tracking: FX88213000 | Weight: 12.5kg"
```

Building a label like this from several normalized fields is exactly what a shipping-label print transform does before handing text off to a label printer.

#600 Parse a Raw CargoLink Order Line

Parse a raw pipe-delimited order line into a validated, typed object.

INPUT

```
{ "raw": " ORD-1042 | Maria Chen | 240.50 | 2026-03-14 " }
```

SOLUTION

```
var fields = (trim(payload.raw) splitBy "|") map trim($)
---
{
  orderId: fields[0],
  isValidOrderId: fields[0] matches /ORD-\d{4}/,
  customerName: fields[1],
  total: fields[2] as Number,
  shipDate: fields[3] as Date as String
}
```

OUTPUT

```
{ "orderId": "ORD-1042", "isValidOrderId": true, "customerName": "Maria Chen",
"total": 240.5, "shipDate": "2026-03-14" }
```

This capstone chains nearly every idea from the volume — trimming, splitting, regex validation, and type coercion — into one realistic raw-text order parser.

END OF VOLUME 6

600 Down, 400 to Go

You've now worked through selectors, type coercion, operators and conditionals, array transformations, object transformations, and a complete tour of string handling — case conversion, trimming, splitting/joining, replacing, slicing, and regular expressions.

Volume 7 moves to Date & Time Functions: another 100 problems on `Date`, `DateTime`, `Time`, formatting patterns, date arithmetic, and time zone handling.

MULESOFT · PRACTICE PROBLEMS · WITH FULL SOLUTIONS

1000 DataWeave Problems With Solutions

Volume 7 — Date & Time Functions. 100 hands-on problems on Date, DateTime, Time, format patterns, date arithmetic, and comparisons.

Problems 601–700

Volume 7 of 10

2026 EDITION

How to Use This Volume

Volume 7 is about **time** — parsing it, formatting it, comparing it, and doing arithmetic on it. Same format as every prior volume:

- **A short problem statement**, the **input** payload, a complete **solution** in a black code window, the exact **output**, and a one-line explanation.
- Ten sections of ten problems each, moving from `Date`/`DateTime`/`Time` basics through custom format patterns, `Period`-based arithmetic, comparing and sorting dates, and mixed real-world scheduling challenges.

Every date and calendar fact used in this volume's inputs and outputs — day-of-week values, day counts between dates, leap-year rollovers — was independently verified against a real calendar rather than assumed, so you can trust the arithmetic as well as the DataWeave syntax.

VOLUME 7

Date & Time Functions

100 problems on `Date`, `DateTime`, `Time`, custom format patterns, `Period`-based arithmetic, and the comparison tools DataWeave uses to reason about schedules, deadlines, and elapsed time.

SECTION 1 — Date Basics: Literals & Coercion (Problems 601–610)

#601 Write a Date Literal

Write a `Date` literal directly for March 14th, 2026, without coercing from a String.

INPUT

```
{}
```

SOLUTION

```
|2026-03-14|
```

OUTPUT

```
|2026-03-14|
```

Wrapping an ISO-format date String in pipe characters creates a real `Date` value directly, without needing an `as Date` coercion.

#602 Coerce an ISO String to a Date

Coerce a standard ISO-format date String into a real `Date` value.

INPUT

```
{ "shipDate": "2026-03-14" }
```

SOLUTION

```
payload.shipDate as Date
```

OUTPUT

```
|2026-03-14|
```

Without a `{format: "..."}` option, `as Date` expects the String to already be in ISO-8601 format, `yyyy-MM-dd`.

#603 Coerce a Date Back to a String

Coerce a `Date` value back into its default String representation.

INPUT

```
{ "shipDate": "2026-03-14" }
```

SOLUTION

```
(payload.shipDate as Date) as String
```

OUTPUT

```
"2026-03-14"
```

The default String form of a `Date` is the same ISO-8601 `yyyy-MM-dd` format used for parsing it.

#604 Coerce a Non-ISO Date String

Coerce a date String in `MM/dd/yyyy` format into a real `Date` value.

INPUT

```
{ "rawDate": "03/14/2026" }
```

SOLUTION

```
payload.rawDate as Date {format: "MM/dd/yyyy"}
```

OUTPUT

```
|2026-03-14|
```

The `{format: "..."}` option tells the coercion exactly how to read a non-ISO source format, using the same Java-style pattern letters used for Number formatting.

#605 Compare Two Date Values for Equality

Check whether two Dates, written in different source formats, represent the same calendar day.

INPUT

```
{ "a": "2026-03-14", "b": "03/14/2026" }
```

SOLUTION

```
(payload.a as Date) == (payload.b as Date {format: "MM/dd/yyyy"})
```

OUTPUT

```
true
```

Once both Strings are coerced to real `Date` values, `==` compares the actual calendar dates, ignoring how each one was originally formatted.

#606 Add Days to a Date

Add 7 days to a ship date to estimate a delivery date.

INPUT

```
{ "shipDate": "2026-03-14" }
```

SOLUTION

```
(payload.shipDate as Date) + |P7D|
```

OUTPUT

```
|2026-03-21|
```

A `Period` literal like `|P7D|` ("7 days" in ISO-8601 duration notation) can be added directly to a `Date` with `+`.

#607 Subtract Days From a Date

Subtract 10 days from a delivery date to find when the order must have shipped.

INPUT

```
{ "deliveryDate": "2026-03-14" }
```

SOLUTION

```
(payload.deliveryDate as Date) - |P10D|
```

OUTPUT

```
|2026-03-04|
```

- *works with a `Period` the same way `+` does, just moving the Date backward instead of forward.*

#608 Check the Type of a Coerced Value

Confirm that a coerced field is genuinely a `Date`, not left as a String.

INPUT

```
{ "shipDate": "2026-03-14" }
```

SOLUTION

```
(payload.shipDate as Date) is Date
```

OUTPUT

```
true
```

Just like `Number`, `String`, and `Boolean`, `Date` is a real type that `is` and `match` type matchers can check against.

#609 Coerce an Array of Date Strings

Coerce every element of an array of date Strings into real `Date` values.

INPUT

```
{ "dates": ["2026-03-14", "2026-03-21"] }
```

SOLUTION

```
payload.dates map ((d) -> d as Date)
```

OUTPUT

```
[|2026-03-14|, |2026-03-21|]
```

Coercing every element with `map` is the standard first step before any Date-aware comparison or arithmetic can run on the array.

#610 Normalize a Batch of CargoLink Order Dates

Normalize a batch of order dates from a legacy `MM-dd-yyyy` format into standard ISO Dates.

INPUT

```
{ "orderDates": ["03-14-2026", "03-21-2026"] }
```

SOLUTION

```
payload.orderDates map ((d) -> d as Date {format: "MM-dd-yyyy"})
```

OUTPUT

```
[|2026-03-14|, |2026-03-21|]
```

This is exactly the kind of one-time normalization pass a real integration applies when connecting to a legacy system with its own date convention.

SECTION 2 — DateTime Basics & Time Zones (Problems 611–620)

#611 Write a DateTime Literal

Write a `DateTime` literal for March 14th, 2026 at 10:30 AM UTC.

INPUT

```
{}
```

SOLUTION

```
|2026-03-14T10:30:00Z|
```

OUTPUT

```
|2026-03-14T10:30:00Z|
```

The trailing `Z` marks the offset as UTC — `DateTime` always carries an offset, unlike the plain `Date` type.

#612 Coerce a String to a DateTime

Coerce an ISO-8601 timestamp String into a real `DateTime` value.

INPUT

```
{ "timestamp": "2026-03-14T10:30:00Z" }
```

SOLUTION

```
payload.timestamp as DateTime
```

OUTPUT

```
|2026-03-14T10:30:00Z|
```

Just like `Date`, an unadorned `as DateTime` expects the source String to already be in ISO-8601 format.

#613 Coerce a DateTime With a Non-UTC Offset

Coerce a timestamp that carries a specific, non-UTC offset instead of `Z`.

INPUT

```
{ "timestamp": "2026-03-14T10:30:00-05:00" }
```

SOLUTION

```
payload.timestamp as DateTime
```

OUTPUT

```
|2026-03-14T10:30:00-05:00|
```

A `DateTime` preserves whatever offset it was given rather than automatically normalizing to UTC.

#614 Extract the Date Portion of a DateTime

Extract just the calendar date from a full timestamp, discarding the time-of-day.

INPUT

```
{ "timestamp": "2026-03-14T22:45:00Z" }
```

SOLUTION

```
(payload.timestamp as DateTime) as Date
```

OUTPUT

```
|2026-03-14|
```

Coercing a `DateTime` directly to `Date` keeps only the calendar date, dropping the time-of-day and offset entirely.

#615 Combine a Date and a Time Into a DateTime

Combine a separately stored Date String and Time String into a single `DateTime` value.

INPUT

```
{ "date": "2026-03-14", "time": "17:00:00" }
```

SOLUTION

```
(payload.date ++ "T" ++ payload.time ++ "Z") as DateTime
```

OUTPUT

```
|2026-03-14T17:00:00Z|
```

Building the ISO-8601 String by hand with `++` before coercing is a reliable way to merge two separately sourced fields into one `DateTime`.

#616 Compare Two DateTimes With Different Offsets

Confirm that two timestamps written with different offsets can still represent the exact same instant.

INPUT

```
{ "a": "2026-03-14T10:00:00-05:00", "b": "2026-03-14T15:00:00Z" }
```

SOLUTION

```
(payload.a as DateTime) == (payload.b as DateTime)
```

OUTPUT

```
true
```

`==` on two `DateTime` values compares the actual instant in time, so a `-05:00` offset and its UTC equivalent five hours later are equal.

#617 Add Hours Across a Midnight Boundary

Add 3 hours to a late-night timestamp and confirm it correctly rolls over into the next calendar day.

INPUT

```
{ "timestamp": "2026-03-14T22:00:00Z" }
```

SOLUTION

```
(payload.timestamp as DateTime) + |PT3H|
```

OUTPUT

```
|2026-03-15T01:00:00Z|
```

The `T` inside a Period literal like `|PT3H|` marks the start of the time portion — DataWeave correctly rolls the date forward when the hour total crosses midnight.

#618 Coerce a DateTime With a Custom Format

Coerce a non-standard timestamp format into a real `DateTime` value.

INPUT

```
{ "raw": "14/03/2026 17:00" }
```

SOLUTION

```
payload.raw as DateTime {format: "dd/MM/yyyy HH:mm"}
```

OUTPUT

```
|2026-03-14T17:00:00Z|
```

A custom format pattern for `DateTime` combines date and time pattern letters in one String, exactly matching the shape of the source data.

#619 Set a Fixed Deadline Time on a Given Date

Build a same-day 5:00 PM deadline `DateTime` from a given `Date`.

INPUT

```
{ "orderDate": "2026-03-14" }
```

SOLUTION

```
((payload.orderDate as Date) as String) ++ "T17:00:00Z" as DateTime
```

OUTPUT

```
|2026-03-14T17:00:00Z|
```

Coercing the Date back to its ISO String and appending a fixed time is a simple way to attach a standard cutoff time to any date field.

#620 Parse a Raw Legacy CargoLink Timestamp

Parse a raw timestamp from a legacy CargoLink dispatch system into a standard `DateTime`.

INPUT

```
{ "raw": "20260314173000" }
```

SOLUTION

```
payload.raw as DateTime {format: "yyyyMMddHHmmss"}
```

OUTPUT

```
|2026-03-14T17:30:00Z|
```

Compact, separator-free timestamp formats like this are common in older batch systems, and a matching pattern String parses them just as easily as a nicely formatted one.

SECTION 3 — Time Basics (Problems 621–630)

#621 Write a Time Literal

Write a `Time` literal for 5:00 PM, with no date attached.

INPUT

```
{}
```

SOLUTION

```
|17:00:00|
```

OUTPUT

```
|17:00:00|
```

The `Time` type represents a time-of-day on its own, independent of any calendar date.

#622 Coerce a String to a Time

Coerce a time String into a real `Time` value.

INPUT

```
{ "cutoff": "17:00:00" }
```

SOLUTION

```
payload.cutoff as Time
```

OUTPUT

```
|17:00:00|
```

An unadorned `as Time` expects the standard `HH:mm:ss` ISO time format.

#623 Compare Two Time Values

Check whether a delivery attempt's time is before a warehouse's 6:00 PM closing time.

INPUT

```
{ "attemptTime": "16:30:00" }
```

SOLUTION

```
(payload.attemptTime as Time) < |18:00:00|
```

OUTPUT

```
true
```

Comparison operators work on `Time` values exactly as they do on `Date` and `DateTime`, ordering them by time-of-day.

#624 Add Minutes to a Time

Add 45 minutes to a departure time to estimate an arrival time.

INPUT

```
{ "departure": "22:30:00" }
```

SOLUTION

```
(payload.departure as Time) + |PT45M|
```

OUTPUT

```
|23:15:00|
```

Adding a `Period` to a `Time` works the same way it does for `Date` and `DateTime`, using `PT` for a time-only duration.

#625 Extract the Hour From a Time

Extract just the hour component from a delivery window's start time.

INPUT

```
{ "windowStart": "09:15:00" }
```

SOLUTION

```
(payload.windowStart as Time).hour
```

OUTPUT

```
9
```

A dot-selector like `.hour` reads a component directly off a `Time` value as a plain Number, on a 24-hour clock.

#626 Extract the Minutes From a Time

Extract just the minutes component from a timestamp.

INPUT

```
{ "time": "09:15:42" }
```

SOLUTION

```
(payload.time as Time).minutes
```

OUTPUT

```
15
```

`.minutes` and `.seconds` follow the same pattern as `.hour`, each returning one component as a plain Number.

#627 Format a Time in 12-Hour Notation

Format a 24-hour Time value as a 12-hour clock String with an AM/PM marker.

INPUT

```
{ "time": "17:00:00" }
```

SOLUTION

```
(payload.time as Time) as String {format: "hh:mm a"}
```

OUTPUT

```
"05:00 PM"
```

Lowercase `hh` requests the 12-hour clock hour, and `a` adds the AM/PM marker — both standard Java-style pattern letters.

#628 Coerce a Time Back to Its Default String

Coerce a `Time` value back to its default ISO String form.

INPUT

```
{ "time": "09:15:00" }
```

SOLUTION

```
(payload.time as Time) as String
```

OUTPUT

```
"09:15:00"
```

Without a custom format, coercing a `Time` back to String uses the same 24-hour `HH:mm:ss` shape it was parsed from.

#629 Check if a Time Falls Within Business Hours

Check whether a delivery attempt time falls within a 9 AM to 5 PM business-hours window.

INPUT

```
{ "attemptTime": "14:30:00" }
```

SOLUTION

```
var t = payload.attemptTime as Time
---
(t >= |09:00:00|) and (t <= |17:00:00|)
```

OUTPUT

```
true
```

Combining two comparisons with `and` against `Time` literals is exactly how a business-hours validation gate is built.

#630 Validate a CargoLink Delivery Window

Validate that a requested delivery window's start time is before its end time.

INPUT

```
{ "windowStart": "13:00:00", "windowEnd": "16:00:00" }
```

SOLUTION

```
(payload.windowStart as Time) < (payload.windowEnd as Time)
```

OUTPUT

```
true
```

This kind of start-before-end sanity check is a standard validation step before a delivery window is accepted into a schedule.

SECTION 4 — Custom Format Patterns (Problems 631–640)

#631 Format a Date as MM/DD/YYYY

Format a `Date` value into the common US `MM/dd/yyyy` display format.

INPUT

```
{ "shipDate": "2026-03-14" }
```

SOLUTION

```
(payload.shipDate as Date) as String {format: "MM/dd/yyyy"}
```

OUTPUT

```
"03/14/2026"
```

The same pattern letters used for parsing work identically for formatting — just applied in the output direction.

#632 Format a Date as a Full Written Date

Format a Date as a fully written-out String like `"March 14, 2026"`.

INPUT

```
{ "shipDate": "2026-03-14" }
```

SOLUTION

```
(payload.shipDate as Date) as String {format: "MMMM d, yyyy"}
```

OUTPUT

```
"March 14, 2026"
```

`MMMM` spells out the full month name, while a single `d` prints the day of month without leading zeros.

#633 Format a DateTime as a Full Timestamp

Format a DateTime as a space-separated `yyyy-MM-dd HH:mm:ss` timestamp String, common in log files.

INPUT

```
{ "timestamp": "2026-03-14T17:30:00Z" }
```

SOLUTION

```
(payload.timestamp as DateTime) as String {format: "yyyy-MM-dd HH:mm:ss"}
```

OUTPUT

```
"2026-03-14 17:30:00"
```

Dropping the `T` separator and the offset from the pattern produces a plain, log-friendly timestamp format.

#634 Parse a Date With a Two-Digit Year

Parse a legacy date format that uses a two-digit year.

INPUT

```
{ "rawDate": "14-03-26" }
```

SOLUTION

```
payload.rawDate as Date {format: "dd-MM-yy"}
```

OUTPUT

```
|2026-03-14|
```

A lowercase two-letter `yy` pattern matches a two-digit year, which DataWeave expands into a full four-digit year using standard century rules.

#635 Parse a Date With an Abbreviated Month Name

Parse a date String that spells out an abbreviated month name, like `"14-Mar-2026"`.

INPUT

```
{ "rawDate": "14-Mar-2026" }
```

SOLUTION

```
payload.rawDate as Date {format: "dd-MMM-yyyy"}
```

OUTPUT

```
|2026-03-14|
```

A three-letter `MMM` pattern matches an abbreviated month name like `"Mar"`, distinct from the two-digit numeric `MM`.

#636 Format a Date With Its Day-of-Week Name

Format a Date to include the full day-of-week name alongside the calendar date.

INPUT

```
{ "shipDate": "2026-03-14" }
```

SOLUTION

```
(payload.shipDate as Date) as String {format: "EEEE, MMMM d, yyyy"}
```

OUTPUT

```
"Saturday, March 14, 2026"
```

`EEEE` spells out the full weekday name — March 14th, 2026 is independently verified to fall on a Saturday.

#637 Build a Zero-Padded Month/Day Code

Build a compact `MMDD` code from a Date, useful as part of a generated reference number.

INPUT

```
{ "shipDate": "2026-03-04" }
```

SOLUTION

```
(payload.shipDate as Date) as String {format: "MMdd"}
```

OUTPUT

```
"0304"
```

Two-letter `MM` and `dd` patterns always zero-pad single-digit months and days, which is exactly what a fixed-width reference code needs.

#638 Round-Trip a Custom Format Into a Different One

Parse a date in one custom format and immediately reformat it into a different custom format.

INPUT

```
{ "rawDate": "2026.03.14" }
```

SOLUTION

```
(payload.rawDate as Date {format: "yyyy.MM.dd"}) as String {format: "dd-MMM-yyyy"}
```

OUTPUT

```
"14-Mar-2026"
```

Parsing with one `{format: ...}` and formatting with another in the same expression is the standard way to convert between two arbitrary date formats.

#639 Format Every Date in an Array Consistently

Format every Date in an array into the same `MMM d` short display format.

INPUT

```
{ "dates": ["2026-03-14", "2026-04-01"] }
```

SOLUTION

```
payload.dates map ((d) -> (d as Date) as String {format: "MMM d"})
```

OUTPUT

```
["Mar 14", "Apr 1"]
```

Combining `map` with a format coercion applies the same display rule to every Date in a list, ready for a UI or report.

#640 Generate a CargoLink Invoice Date Footer

Build an invoice footer line combining a written-out invoice date with a generated timestamp.

INPUT

```
{ "invoiceDate": "2026-03-14" }
```

SOLUTION

```
"Invoice Date: " ++ ((payload.invoiceDate as Date) as String {format: "MMMM d, yyyy"})
```

OUTPUT

```
"Invoice Date: March 14, 2026"
```

Combining a formatted Date with a literal label String is exactly how a real invoice or report footer is generated from a raw Date field.

SECTION 5 — Date Arithmetic With Period (Problems 641–650)

#641 Add a Period of Days to a Date

Add a 21-day Period to a ship date to calculate a follow-up date.

INPUT

```
{ "shipDate": "2026-03-14" }
```

SOLUTION

```
(payload.shipDate as Date) + |P21D|
```

OUTPUT

```
|2026-04-04|
```

A larger day count in a Period literal works exactly the same way as a small one, correctly rolling over month boundaries.

#642 Subtract a Period of Days From a Date

Subtract 30 days from a due date to find the reminder date.

INPUT

```
{ "dueDate": "2026-04-14" }
```

SOLUTION

```
(payload.dueDate as Date) - |P30D|
```

OUTPUT

```
|2026-03-15|
```

Subtracting a Period rolls backward across month boundaries exactly as reliably as adding one rolls forward.

#643 Add a Period of Months to a Date

Add one month to a subscription start date to find the next renewal date.

INPUT

```
{ "startDate": "2026-03-14" }
```

SOLUTION

```
(payload.startDate as Date) + |P1M|
```

OUTPUT

```
|2026-04-14|
```

A Period expressed in months, `P1M`, advances the calendar month while keeping the same day of month whenever possible.

#644 Add a Period of Years to a Date

Add one year to a contract start date to find its anniversary date.

INPUT

```
{ "startDate": "2026-03-14" }
```

SOLUTION

```
(payload.startDate as Date) + |P1Y|
```

OUTPUT

```
|2027-03-14|
```

`P1Y` advances the year component by one, leaving the month and day unchanged whenever the resulting date is valid.

#645 Combine Days and Hours in One Period

Add a combined Period of 1 day and 2 hours to a DateTime.

INPUT

```
{ "timestamp": "2026-03-14T10:00:00Z" }
```

SOLUTION

```
(payload.timestamp as DateTime) + |P1DT2H|
```

OUTPUT

```
|2026-03-15T12:00:00Z|
```

A single Period literal can combine a date part and a time part, separated by `T`, applying both components in one addition.

#646 Calculate a Date 90 Days in the Future

Calculate a warranty expiration date, 90 days after purchase.

INPUT

```
{ "purchaseDate": "2026-01-01" }
```

SOLUTION

```
(payload.purchaseDate as Date) + |P90D|
```

OUTPUT

```
|2026-04-01|
```

There's no need to convert 90 days into "roughly 3 months" — a day-based Period handles the exact calendar math correctly on its own.

#647 Confirm Correct Month-End Rollover

Add one day to February 28th in a non-leap year and confirm it rolls into March correctly.

INPUT

```
{ "date": "2026-02-28" }
```

SOLUTION

```
(payload.date as Date) + |P1D|
```

OUTPUT

```
|2026-03-01|
```

2026 is not a leap year, so February has only 28 days — adding one day correctly rolls straight into March 1st.

#648 Confirm Correct Leap Year Handling

Add one day to February 28th in a leap year and confirm it lands on February 29th instead of rolling over.

INPUT

```
{ "date": "2028-02-28" }
```

SOLUTION

```
(payload.date as Date) + |P1D|
```

OUTPUT

```
|2028-02-29|
```

2028 is a leap year, so DataWeave's calendar-aware Date arithmetic correctly recognizes February 29th as a valid date that year.

#649 Add a Period to Every Date in an Array

Add a 7-day grace period to every due date in an array of invoices.

INPUT

```
{ "dueDates": ["2026-03-14", "2026-03-21"] }
```

SOLUTION

```
payload.dueDates map ((d) -> (d as Date) + |P7D|)
```

OUTPUT

```
[|2026-03-21|, |2026-03-28|]
```

A Period addition drops into a `map` lambda exactly like any other expression, applying the same offset to every Date in the array.

#650 Calculate a CargoLink Estimated Delivery Date

Calculate an estimated delivery date by adding a carrier's typical transit days to the ship date.

INPUT

```
{ "shipDate": "2026-03-10", "transitDays": 5 }
```

SOLUTION

```
(payload.shipDate as Date) + ("P" ++ (payload.transitDays as String) ++ "D"
as Period)
```

OUTPUT

```
|2026-03-15|
```

A Period doesn't have to be a fixed literal — building the ISO-8601 duration String dynamically with ++ and coercing it with `as Period` lets the day count come from any field.

SECTION 6 — Period Literals & Between-Date Calculations (Problems 651–660)

#651 Compute the Period Between Two Dates

Subtract one Date from another to compute the Period between them.

INPUT

```
{ "start": "2026-03-10", "end": "2026-03-15" }
```

SOLUTION

```
(payload.end as Date) - (payload.start as Date)
```

OUTPUT

```
|P5D|
```

Subtracting one `Date` from another produces a `Period` value, DataWeave's type for representing a span of time.

#652 Extract the Day Count From a Period

Extract the number of days from a Period as a plain Number.

INPUT

```
{ "start": "2026-03-10", "end": "2026-03-15" }
```

SOLUTION

```
((payload.end as Date) - (payload.start as Date)).days
```

OUTPUT

```
5
```

Just like `Date` and `Time`, a `Period` exposes its components — `.days`, `.hours`, `.minutes` — through dot-selectors.

#653 Count Days Between Two Dates in One Expression

Count the number of days between an order date and today's reference date.

INPUT

```
{ "orderDate": "2026-01-01", "today": "2026-03-14" }
```

SOLUTION

```
((payload.today as Date) - (payload.orderDate as Date)).days
```

OUTPUT

```
72
```

Subtracting and immediately selecting `.days` in one expression is the standard idiom for a "how many days between" calculation.

#654 Compare a Day Gap Against a Threshold

Check whether the gap between two dates exceeds a given threshold number of days.

INPUT

```
{ "shipDate": "2026-03-10", "today": "2026-03-15", "thresholdDays": 3 }
```

SOLUTION

```
(((payload.today as Date) - (payload.shipDate as Date)).days) >
payload.thresholdDays
```

OUTPUT

```
true
```

Extracting `.days` to a plain Number first is what makes the result directly comparable to an ordinary numeric threshold.

#655 Report How Late a Delivery Was

Build a message reporting how many days late a delivery was, or that it was on time.

INPUT

```
{ "promised": "2026-03-10", "actual": "2026-03-15" }
```

SOLUTION

```
var daysLate = ((payload.actual as Date) - (payload.promised as Date)).days
---
if (daysLate > 0) ((daysLate as String) ++ " days late") else "On time"
```

OUTPUT

```
"5 days late"
```

Storing the day count in a `var` first keeps the message-building logic that follows easy to read.

#656 Compute an Order's Age in Days

Compute how many days old an order is, relative to a given reference date.

INPUT

```
{ "orderDate": "2026-02-01", "referenceDate": "2026-03-14" }
```

SOLUTION

```
((payload.referenceDate as Date) - (payload.orderDate as Date)).days
```

OUTPUT

```
41
```

Using an explicit reference date field, rather than the current system time, keeps this calculation's result reproducible and testable.

#657 Compute a Countdown to a Target Date

Compute how many days remain until a target date.

INPUT

```
{ "today": "2026-03-14", "targetDate": "2026-04-01" }
```

SOLUTION

```
((payload.targetDate as Date) - (payload.today as Date)).days
```

OUTPUT

```
18
```

Swapping the order of the two operands compared to a "days since" calculation is all it takes to turn it into a "days until" countdown.

#658 Compare a Computed Period to a Period Literal

Confirm that a computed Period between two dates equals an expected Period literal.

INPUT

```
{ "start": "2026-03-01", "end": "2026-03-15" }
```

SOLUTION

```
((payload.end as Date) - (payload.start as Date)) == |P14D|
```

OUTPUT

```
true
```

Two `Period` values can be compared with == directly, without needing to first extract and compare a specific component.

#659 Combine Two Period Calculations in One Object

Build a summary showing both how many days old an order is and how many days remain until a deadline.

INPUT

```
{ "orderDate": "2026-02-01", "today": "2026-03-14", "deadline": "2026-04-01" }
```

SOLUTION

```
{
  daysSinceOrder: ((payload.today as Date) - (payload.orderDate as
Date)).days,
  daysUntilDeadline: ((payload.deadline as Date) - (payload.today as
Date)).days
}
```

OUTPUT

```
{ "daysSinceOrder": 41, "daysUntilDeadline": 18 }
```

Building both calculations as separate object fields is a common way to give a report both "how long ago" and "how soon" context at once.

#660 Determine if a CargoLink Shipment Is Overdue

Check whether a shipment's promised date has already passed, relative to a given reference date.

INPUT

```
{ "promisedDate": "2026-03-10", "today": "2026-03-14" }
```

SOLUTION

```
(payload.today as Date) > (payload.promisedDate as Date)
```

OUTPUT

```
true
```

A plain > comparison, without needing a Period at all, is often the simplest way to answer a yes/no overdue question.

SECTION 7 — Extracting Date Components (Problems 661–670)

#661 Extract the Year From a Date

Extract just the year from a Date value.

INPUT

```
{ "shipDate": "2026-03-14" }
```

SOLUTION

```
(payload.shipDate as Date).year
```

OUTPUT

```
2026
```

The `.year` selector returns a plain Number, ready for arithmetic or comparison without any further coercion.

#662 Extract the Month From a Date

Extract just the month from a Date value.

INPUT

```
{ "shipDate": "2026-03-14" }
```

SOLUTION

```
(payload.shipDate as Date).month
```

OUTPUT

```
3
```

Unlike some programming languages, DataWeave's `.month` is 1-based — March is `3`, not `2`.

#663 Extract the Day of Month From a Date

Extract just the day-of-month from a Date value.

INPUT

```
{ "shipDate": "2026-03-14" }
```

SOLUTION

```
(payload.shipDate as Date).day
```

OUTPUT

```
14
```

`.day` returns the day-of-month component only — it does not indicate the day of the week.

#664 Extract Hour, Minutes, and Seconds From a DateTime

Extract the hour, minutes, and seconds components from a DateTime, all at once.

INPUT

```
{ "timestamp": "2026-03-14T17:45:30Z" }
```

SOLUTION

```
var dt = payload.timestamp as DateTime
---
{ hour: dt.hour, minutes: dt.minutes, seconds: dt.seconds }
```

OUTPUT

```
{ "hour": 17, "minutes": 45, "seconds": 30 }
```

A `DateTime` exposes the same time components as `Time`, in addition to its date components.

#665 Extract the Day of the Week

Extract the day of the week from a Date, as a Number.

INPUT

```
{ "shipDate": "2026-03-14" }
```

SOLUTION

```
(payload.shipDate as Date).dayOfWeek
```

OUTPUT

```
6
```

`.dayOfWeek` follows the ISO-8601 convention: `1` is Monday and `7` is Sunday — March 14th, 2026 independently checks out as a Saturday, day `6`.

#666 Extract the Day of the Year

Extract which day of the calendar year a Date falls on.

INPUT

```
{ "shipDate": "2026-03-14" }
```

SOLUTION

```
(payload.shipDate as Date).dayOfYear
```

OUTPUT

```
73
```

Counting from January 1st as day 1, March 14th, 2026 is independently verified to be the 73rd day of the year.

#667 Extract the Calendar Quarter

Extract which calendar quarter a Date falls in.

INPUT

```
{ "shipDate": "2026-03-14" }
```

SOLUTION

```
(payload.shipDate as Date).quarter
```

OUTPUT

```
1
```

`.quarter` returns `1` through `4` — March falls in Q1, the January-through-March quarter.

#668 Build a "Q1 2026" Label From Components

Build a short quarter-and-year label String from a Date's components.

INPUT

```
{ "shipDate": "2026-03-14" }
```

SOLUTION

```
var d = payload.shipDate as Date
---
"Q" ++ (d.quarter as String) ++ " " ++ (d.year as String)
```

OUTPUT

```
"Q1 2026"
```

Combining two extracted Number components with ++ after coercing each to String is exactly how a short reporting label like this gets built.

#669 Check if a Date Falls on a Weekend

Check whether a Date falls on a Saturday or Sunday.

INPUT

```
{ "date": "2026-03-14" }
```

SOLUTION

```
var dow = (payload.date as Date).dayOfWeek
---
(dow == 6) or (dow == 7)
```

OUTPUT

```
true
```

Since ISO-8601 numbers Saturday as `6` and Sunday as `7`, checking for either value identifies a weekend date.

#670 Flag CargoLink Shipments Scheduled on a Weekend

Flag every shipment in a batch whose ship date falls on a weekend.

INPUT

```
{ "shipments": [{"id": "S1", "shipDate": "2026-03-14"}, {"id": "S2",
"shipDate": "2026-03-16"}] }
```

SOLUTION

```
payload.shipments filter ((s) -> ((s.shipDate as Date).dayOfWeek) >= 6)
```

OUTPUT

```
[{"id": "S1", "shipDate": "2026-03-14"}]
```

Since Saturday and Sunday are the two highest ISO day-of-week numbers, a single `>= 6` check catches both without an explicit `or`.

SECTION 8 — Comparing & Sorting Dates (Problems 671–680)

#671 Compare Two Dates Directly

Check whether one Date is later than another.

INPUT

```
{ "a": "2026-03-14", "b": "2026-03-10" }
```

SOLUTION

```
(payload.a as Date) > (payload.b as Date)
```

OUTPUT

```
true
```

Comparison operators on `Date` order values chronologically, exactly as they did back in Volume 3.

#672 Find the Earliest Date in an Array

Find the earliest Date in an array of Date Strings.

INPUT

```
{ "dates": ["2026-03-21", "2026-03-10", "2026-03-14"] }
```

SOLUTION

```
((payload.dates map ((d) -> d as Date)) orderBy $)[0]
```

OUTPUT

```
|2026-03-10|
```

Sorting ascending and taking the first element is the same "find the minimum" pattern used for Numbers and Strings, applied here to `Date` values.

#673 Find the Latest Date in an Array

Find the latest Date in the same array.

INPUT

```
{ "dates": ["2026-03-21", "2026-03-10", "2026-03-14"] }
```

SOLUTION

```
(reverse((payload.dates map ((d) -> d as Date)) orderBy $))[0]
```

OUTPUT

```
|2026-03-21|
```

Reversing the ascending sort before taking the first element finds the maximum, the same "find the latest" pattern from earlier volumes.

#674 Sort an Array of Objects by a Date Field

Sort an array of orders by their ship date, earliest first.

INPUT

```
{ "orders": [{"id": "O1", "shipDate": "2026-03-21"}, {"id": "O2", "shipDate": "2026-03-10"}] }
```

SOLUTION

```
payload.orders orderBy ((o) -> o.shipDate as Date)
```

OUTPUT

```
[{"id": "O2", "shipDate": "2026-03-10"}, {"id": "O1", "shipDate": "2026-03-21"}]
```

Coercing inside the `orderBy` key expression ensures the sort compares real `Date` values, not the underlying Strings.

#675 Filter Dates Within a Range

Keep only the dates that fall within a given start and end range, inclusive.

INPUT

```
{ "dates": ["2026-03-05", "2026-03-14", "2026-03-25"], "rangeStart": "2026-03-10", "rangeEnd": "2026-03-20" }
```

SOLUTION

```
payload.dates filter ((d) ->
  ((d as Date) >= (payload.rangeStart as Date)) and ((d as Date) <= (payload.rangeEnd as Date))
)
```

OUTPUT

```
["2026-03-14"]
```

Two inclusive comparisons combined with `and` is the standard way to express a closed date range check.

#676 Check if a Date Is in the Past

Check whether an event date has already passed, relative to a given reference date.

INPUT

```
{ "eventDate": "2026-03-01", "today": "2026-03-14" }
```

SOLUTION

```
(payload.eventDate as Date) < (payload.today as Date)
```

OUTPUT

```
true
```

This exact one-line check is the building block behind every "is this overdue / has this expired" rule elsewhere in this volume.

#677 Find the Nearest Upcoming Date

Find the soonest upcoming date among several candidates, ignoring any that have already passed.

INPUT

```
{ "today": "2026-03-14", "candidates": ["2026-03-21", "2026-04-01", "2026-03-16"] }
```

SOLUTION

```
var future = (payload.candidates map ((d) -> d as Date)) filter ($ > (payload.today as Date))
---
(future orderBy $)[0]
```

OUTPUT

```
|2026-03-16|
```

Filtering out past dates before sorting ensures the "nearest" result is always genuinely still upcoming.

#678 Find Shipments Due Within the Next 7 Days

Find every shipment whose ship date falls within the next 7 days from today, inclusive.

INPUT

```
{ "today": "2026-03-14", "shipments": [{"id": "S1", "shipDate": "2026-03-16"}, {"id": "S2", "shipDate": "2026-03-25"}] }
```

SOLUTION

```
var todayDate = payload.today as Date
---
payload.shipments filter ((s) -> (
  (((s.shipDate as Date) - todayDate).days >= 0) and (((s.shipDate as Date) - todayDate).days <= 7)
))
```

OUTPUT

```
[{"id": "S1", "shipDate": "2026-03-16"}]
```

Checking that the day gap is both non-negative and within the window catches only genuinely upcoming, near-term shipments.

#679 Sort DateTimes With Different Offsets

Sort two DateTimes written with different offsets by their actual chronological instant.

INPUT

```
{ "timestamps": ["2026-03-14T09:00:00-05:00", "2026-03-14T10:00:00Z"] }
```

SOLUTION

```
(payload.timestamps map ((t) -> t as DateTime)) orderBy $
```

OUTPUT

```
[|2026-03-14T10:00:00Z|, |2026-03-14T09:00:00-05:00|]
```

Even though "`09:00-05:00`" reads earlier as a raw String, it actually represents 14:00 UTC — later than `10:00Z` — so `orderBy` correctly places it second.

#680 Build a Sorted CargoLink Delivery Schedule

Build a delivery schedule from a batch of orders, sorted by ship date.

INPUT

```
{ "orders": [{"id": "O1", "shipDate": "2026-03-21"}, {"id": "O2", "shipDate": "2026-03-10"}, {"id": "O3", "shipDate": "2026-03-15"}] }
```

SOLUTION

```
payload.orders orderBy ((o) -> o.shipDate as Date)
```

OUTPUT

```
[{"id": "O2", "shipDate": "2026-03-10"}, {"id": "O3", "shipDate": "2026-03-15"}, {"id": "O1", "shipDate": "2026-03-21"}]
```

This single line is exactly what powers a real dispatch dashboard's "next up" delivery ordering.

SECTION 9 — Chaining Date Functions Together (Problems 681–690)

#681 Parse One Format, Then Format Into Another

Parse a date in `dd/MM/yyyy` format and reformat it as just the month and year.

INPUT

```
{ "raw": "14/03/2026" }
```

SOLUTION

```
(payload.raw as Date {format: "dd/MM/yyyy"}) as String {format: "MMMM yyyy"}
```

OUTPUT

```
"March 2026"
```

Parsing and reformatting in the same expression is a compact way to normalize a raw date field straight to its final display form.

#682 Compute and Format a Due Date

Compute an invoice's due date from its payment terms, then format it for display.

INPUT

```
{ "invoiceDate": "2026-03-14", "termsDays": 30 }
```

SOLUTION

```
var due = (payload.invoiceDate as Date) + ("P" ++ (payload.termsDays as
String) ++ "D" as Period)
---
due as String {format: "MMMM d, yyyy"}
```

OUTPUT

```
"April 13, 2026"
```

Computing first, then formatting the final result, keeps the arithmetic and the display logic cleanly separated.

#683 Build a Labeled Breakdown From Date Components

Build a labeled String breaking a Date down into its year, month, and day components.

INPUT

```
{ "shipDate": "2026-03-14" }
```

SOLUTION

```
var d = payload.shipDate as Date
---
"Year " ++ (d.year as String) ++ ", Month " ++ (d.month as String) ++ ", Day
" ++ (d.day as String)
```

OUTPUT

```
"Year 2026, Month 3, Day 14"
```

Once a Date is stored in a `var`, its components can be reused across a whole expression without recomputing the coercion each time.

#684 Filter to Future Orders, Then Sort Them

Keep only the orders that ship after today, sorted with the soonest first.

INPUT

```
{ "today": "2026-03-14", "orders": [{"id": "O1", "shipDate": "2026-03-10"},
{"id": "O2", "shipDate": "2026-03-21"}, {"id": "O3", "shipDate": "2026-03-
16"}] }
```

SOLUTION

```
var todayDate = payload.today as Date
---
(payload.orders filter ((o) -> (o.shipDate as Date) > todayDate)) orderBy
((o) -> o.shipDate as Date)
```

OUTPUT

```
[{"id": "O3", "shipDate": "2026-03-16"}, {"id": "O2", "shipDate": "2026-03-
21"}]
```

Filtering out past-due orders before sorting is what turns a raw order list into a genuinely useful "what's coming up" view.

#685 Coerce and Format Every Date in an Array

Coerce an array of raw date Strings to Dates, then format every one for display.

INPUT

```
{ "dates": ["2026-03-14", "2026-04-01"] }
```

SOLUTION

```
(payload.dates map ((d) -> d as Date)) map ((d) -> d as String {format: "MMM
d, yyyy"})
```

OUTPUT

```
["Mar 14, 2026", "Apr 1, 2026"]
```

Two `map` passes — one to coerce, one to format — keeps each transformation step simple and easy to reason about independently.

#686 Classify an Order Into an Age Bucket

Classify an order's age into a `"NEW"`, `"RECENT"`, or `"AGED"` bucket, based on days since it was placed.

INPUT

```
{ "orderDate": "2026-02-01", "today": "2026-03-14" }
```

SOLUTION

```
var ageDays = ((payload.today as Date) - (payload.orderDate as Date)).days
---
if (ageDays > 30) "AGED"
else if (ageDays > 7) "RECENT"
else "NEW"
```

OUTPUT

```
"AGED"
```

Computing the day count once, then branching on it, is more efficient and readable than recomputing the Period subtraction in every branch.

#687 Validate a Date Falls Within a Year, Then Format It

Confirm a date falls within a given calendar range before formatting it for display.

INPUT

```
{ "raw": "2026-03-14", "rangeStart": "2026-01-01", "rangeEnd": "2026-12-31" }
```

SOLUTION

```
var d = payload.raw as Date
---
if ((d >= (payload.rangeStart as Date)) and (d <= (payload.rangeEnd as
Date)))
  (d as String {format: "MMMM d, yyyy"})
else
  "Out of range"
```

OUTPUT

```
"March 14, 2026"
```

Validating before formatting avoids presenting a nicely formatted date that actually falls outside the range a report is supposed to cover.

#688 Build a Multi-Field Date Summary Object

Build a summary object combining a Date's ISO form, a display form, its day of week, and its quarter.

INPUT

```
{ "shipDate": "2026-03-14" }
```

SOLUTION

```
var d = payload.shipDate as Date
---
{
  iso: d as String,
  display: d as String {format: "MMMM d, yyyy"},
  dayOfWeek: d.dayOfWeek,
  quarter: d.quarter
}
```

OUTPUT

```
{ "iso": "2026-03-14", "display": "March 14, 2026", "dayOfWeek": 6, "quarter": 1 }
```

Combining several derived views of the same underlying Date into one object is exactly how a real date-picker or scheduling API response is built.

#689 Build a "Days Ago" Relative Label

Build a relative label like `"13 days ago"` from an event date and a reference date.

INPUT

```
{ "eventDate": "2026-03-01", "today": "2026-03-14" }
```

SOLUTION

```
var daysAgo = ((payload.today as Date) - (payload.eventDate as Date)).days
---
(daysAgo as String) ++ " days ago"
```

OUTPUT

```
"13 days ago"
```

This is the exact calculation behind the "posted 13 days ago" style labels common in activity feeds and order histories.

#690 Parse and Sort a Batch of Legacy CargoLink Order Dates

Parse a batch of legacy-formatted order dates, attach the parsed Date to each record, and sort the batch chronologically.

INPUT

```
{ "rawOrders": [{"id": "O1", "rawDate": "03-14-2026"}, {"id": "O2", "rawDate": "03-05-2026"}] }
```

SOLUTION

```
(payload.rawOrders map ((o) -> o ++ { shipDate: o.rawDate as Date {format: "MM-dd-yyyy"} })) orderBy ((o) -> o.shipDate)
```

OUTPUT

```
[{"id": "O2", "rawDate": "03-05-2026", "shipDate": |2026-03-05|}, {"id": "O1", "rawDate": "03-14-2026", "shipDate": |2026-03-14|}]
```

Attaching the parsed Date as a new field before sorting keeps the original raw String intact for audit purposes, while still enabling a correct chronological sort.

SECTION 10 — Mixed Real-World Date Challenges (Problems 691–700)

#691 Calculate a CargoLink Estimated Delivery Date

Calculate an estimated delivery date from a ship date and a carrier's transit days.

INPUT

```
{ "shipDate": "2026-03-10", "transitDays": 5 }
```

SOLUTION

```
(payload.shipDate as Date) + |P5D|
```

OUTPUT

```
|2026-03-15|
```

This one-line calculation is the foundation nearly every other problem in this section builds on.

#692 Flag Overdue CargoLink Shipments

Flag every shipment in a batch whose promised date has already passed.

INPUT

```
{ "today": "2026-03-14", "shipments": [{"id": "S1", "promisedDate": "2026-03-10"}, {"id": "S2", "promisedDate": "2026-03-20"}] }
```

SOLUTION

```
payload.shipments filter ((s) -> (s.promisedDate as Date) < (payload.today as Date))
```

OUTPUT

```
[{"id": "S1", "promisedDate": "2026-03-10"}]
```

This is the exact query that would power an "overdue shipments" alert list on a real operations dashboard.

#693 Format a CargoLink Delivery Window Message

Build a customer-facing message describing a delivery window in 12-hour time.

INPUT

```
{ "windowStart": "2026-03-14T09:00:00Z", "windowEnd": "2026-03-14T12:00:00Z" }
```

SOLUTION

```
var s = payload.windowStart as DateTime
var e = payload.windowEnd as DateTime
---
"Delivery window: " ++ (s as String {format: "h:mm a"}) ++ " - " ++ (e as
String {format: "h:mm a"}) ++ " on " ++ (s as String {format: "MMMM d"})
```

OUTPUT

```
"Delivery window: 9:00 AM - 12:00 PM on March 14"
```

A single lowercase `h` (instead of `hh`) omits the leading zero on single-digit hours, which reads more naturally in a customer-facing message.

#694 Compute a CargoLink Driver's Shift Duration

Compute how long a driver's shift lasted, from a start and end Time.

INPUT

```
{ "shiftStart": "08:00:00", "shiftEnd": "16:30:00" }
```

SOLUTION

```
var diff = (payload.shiftEnd as Time) - (payload.shiftStart as Time)
---
(diff.hours as String) ++ "h " ++ (diff.minutes as String) ++ "m"
```

OUTPUT

```
"8h 30m"
```

Subtracting two `Time` values produces a `Period` just like subtracting two Dates does, with its own `.hours` and `.minutes` components.

#695 Build a CargoLink Order Aging Report

Bucket a batch of orders by age: `"0-7"`, `"8-14"`, or `"15+"` days old.

INPUT

```
{ "today": "2026-03-14", "orders": [{"id": "O1", "orderDate": "2026-03-10"},
{"id": "O2", "orderDate": "2026-03-01"}, {"id": "O3", "orderDate": "2026-02-
01"}] }
```

SOLUTION

```
var todayDate = payload.today as Date
---
payload.orders map ((o) -> {
  id: o.id,
  ageDays: (todayDate - (o.orderDate as Date)).days,
  bucket:
    if ((todayDate - (o.orderDate as Date)).days <= 7) "0-7"
    else if ((todayDate - (o.orderDate as Date)).days <= 14) "8-14"
    else "15+"
})
```

OUTPUT

```
[{"id": "O1", "ageDays": 4, "bucket": "0-7"}, {"id": "O2", "ageDays": 13,
"bucket": "8-14"}, {"id": "O3", "ageDays": 41, "bucket": "15+"}]
```

This bucketed aging view is exactly what a real backlog-management report shows to help prioritize which orders need attention first.

#696 Validate a CargoLink Promised Date Is a Business Day

Check whether a promised delivery date falls on a weekday rather than a weekend.

INPUT

```
{ "promisedDate": "2026-03-14" }
```

SOLUTION

```
var dow = (payload.promisedDate as Date).dayOfWeek
---
(dow != 6) and (dow != 7)
```

OUTPUT

```
false
```

March 14th, 2026 falls on a Saturday, so this promised date fails the business-day check and would need to be flagged for adjustment.

#697 Build a CargoLink Quarterly Revenue Report

Group a batch of orders by the calendar quarter their order date falls in.

INPUT

```
{ "orders": [{"total": 100, "orderDate": "2026-01-15"}, {"total": 200,
"orderDate": "2026-02-10"}, {"total": 150, "orderDate": "2026-06-15"}] }
```

SOLUTION

```
payload.orders groupBy ((o) -> "Q" ++ ((o.orderDate as Date).quarter as
String))
```

OUTPUT

```
{ "Q1": [{"total": 100, "orderDate": "2026-01-15"}, {"total": 200,
"orderDate": "2026-02-10"}], "Q2": [{"total": 150, "orderDate": "2026-06-15"}]
}
```

Using `.quarter` directly as a `groupBy` key is a quick way to build a quarter-over-quarter breakdown without writing month-range logic by hand.

#698 Reformat a Batch of Legacy CargoLink Timestamps

Reformat a batch of compact legacy timestamps into standard, readable form.

INPUT

```
{ "raw": ["20260314093000", "20260315170000"] }
```

SOLUTION

```
payload.raw map ((t) -> (t as DateTime {format: "yyyyMMddHHmmss"}) as String
{format: "yyyy-MM-dd HH:mm:ss"})
```

OUTPUT

```
["2026-03-14 09:30:00", "2026-03-15 17:00:00"]
```

Batch-reformatting a whole array of legacy timestamps in one `map` is a common one-time migration step when replacing an old dispatch system.

#699 Calculate Time Remaining Until a CargoLink Deadline

Calculate how many hours remain until a same-day delivery deadline.

INPUT

```
{ "now": "2026-03-14T09:00:00Z", "deadline": "2026-03-14T17:00:00Z" }
```

SOLUTION

```
((payload.deadline as DateTime) - (payload.now as DateTime)).hours
```

OUTPUT

```
8
```

Subtracting two `DateTime` values and reading `.hours` gives a same-day countdown, exactly like the day-based countdown pattern used earlier for Dates.

#700 CargoLink Shipment Timeline Capstone

Calculate a shipment's estimated delivery date from its transit time, and confirm whether that delivery date is a valid business day.

INPUT

```
{ "shipDate": "2026-03-13", "transitDays": 3 }
```

SOLUTION

```
var estDelivery = (payload.shipDate as Date) + ("P" ++ (payload.transitDays
as String) ++ "D" as Period)
var dow = estDelivery.dayOfWeek
---
{
  estimatedDelivery: estDelivery as String {format: "EEEE, MMMM d, yyyy"},
  isBusinessDay: (dow != 6) and (dow != 7)
}
```

OUTPUT

```
{ "estimatedDelivery": "Monday, March 16, 2026", "isBusinessDay": true }
```

This capstone combines nearly every idea from the volume — dynamic Period construction, Date arithmetic, component extraction, and formatting — into one realistic shipment-timeline calculation.

END OF VOLUME 7

700 Down, 300 to Go

You've now worked through selectors, type coercion, operators and conditionals, array transformations, object transformations, string handling, and a complete tour of dates, times, and durations.

Volume 8 moves to Custom Functions, Recursion & Pattern Matching: another 100 problems on `fun`, recursive functions, higher-order functions, and advanced `match` patterns.

MULESOFT · PRACTICE PROBLEMS · WITH FULL SOLUTIONS

1000 DataWeave Problems With Solutions

Volume 8 — Custom Functions, Recursion & Pattern Matching. 100 hands-on problems on `fun`, higher-order functions, recursion, and advanced `match` patterns.

Problems 701–800

Volume 8 of 10

2026 EDITION

How to Use This Volume

Volume 8 moves past DataWeave's built-in functions and into the tools you use to build your own: `fun` definitions, functions passed as values, and recursion. Same format as every prior volume:

- **A short problem statement**, the **input** payload, a complete **solution** in a black code window, the exact **output**, and a one-line explanation.
- Ten sections of ten problems each, moving from basic `fun` definitions and default parameters through higher-order functions, recursion, accumulator-based recursion, and a deep dive into `match`'s pattern-matching power — including the `[head ~ tail]` array decomposition pattern.

A note on style: every recursive function in this volume guards its base case explicitly (checking `isEmpty` or `sizeOf` before slicing) rather than relying on an array or String range selector to quietly return an empty result at a boundary. This avoids an edge case that isn't consistently documented, and it happens to be good defensive habit for production DataWeave code too. Likewise, every `match` expression in this volume ends in an explicit `else` — Section 6 explains exactly why that matters.

VOLUME 8

Custom Functions, Recursion & Pattern Matching

100 problems on defining functions with `fun`, default and overloaded parameters, passing functions as values, recursion (with and without accumulators), and the full depth of DataWeave's `match` pattern matching.

SECTION 1 — Custom Functions Basics (Problems 701–710)

#701 Define and Call a Simple Function

Define a function that doubles a number, then call it on a value from the payload.

INPUT

```
{ "n": 21 }
```

SOLUTION

```
fun double(n) = n * 2
---
double(payload.n)
```

OUTPUT

```
42
```

A `fun` definition goes in the header, before the `---` separator, exactly like a `var`. Once defined, it's called just like any built-in function.

#702 Define a Function With Two Parameters

Define a function that calculates a tax amount from a price and a rate.

INPUT

```
{ "price": 200, "rate": 0.25 }
```

SOLUTION

```
fun calculateTax(price: Number, rate: Number): Number = price * rate
---
calculateTax(payload.price, payload.rate)
```

OUTPUT

```
50.0
```

Type annotations on parameters (`price: Number`) are optional but document the function's expected shape and catch mismatches early.

#703 Give a Function an Explicit Return Type

Define a function that checks whether a number is even, with an explicit `Boolean` return type.

INPUT

```
{ "quantity": 18 }
```

SOLUTION

```
fun isEven(n: Number): Boolean = (n mod 2) == 0
---
isEven(payload.quantity)
```

OUTPUT

```
true
```

The `: Boolean` after the parameter list annotates the function's return type, the same way `: Number` or `: String` would.

#704 Call One Function From Another

Define a helper function, then use it inside a second function.

INPUT

```
{ "a": 3, "b": 4 }
```

SOLUTION

```
fun square(n) = n * n
fun sumOfSquares(a, b) = square(a) + square(b)
---
sumOfSquares(payload.a, payload.b)
```

OUTPUT

```
25
```

Multiple `fun` definitions can stack in the header, and any of them can call any other one defined alongside it.

#705 Write a String-Processing Function

Define a function that uppercases a message and adds emphasis.

INPUT

```
{ "message": "dock the truck" }
```

SOLUTION

```
fun shout(msg: String): String = upper(msg) ++ "!!!"
---
shout(payload.message)
```

OUTPUT

```
"DOCK THE TRUCK!!!"
```

Custom functions can freely call built-in functions like `upper` inside their bodies.

#706 Write a Function That Returns an Object

Define a function that builds a full order summary object from an id and a total.

INPUT

```
{ "orderId": "ORD-9001", "total": 249.5 }
```

SOLUTION

```
fun makeOrderSummary(id: String, total: Number) =
  { orderId: id, total: total, status: "PENDING" }
---
makeOrderSummary(payload.orderId, payload.total)
```

OUTPUT

```
{ "orderId": "ORD-9001", "total": 249.5, "status": "PENDING" }
```

A function's body can be any expression, including an object literal — there's no separate "return" keyword needed.

#707 Call a Function From Inside a map

Define a Celsius-to-Fahrenheit converter, then apply it to every value in an array.

INPUT

```
{ "temps": [0, 20, 100] }
```

SOLUTION

```
fun celsiusToFahrenheit(c: Number): Number = (c * 9 / 5) + 32
---
payload.temps map ((c) -> celsiusToFahrenheit(c))
```

OUTPUT

```
[32.0, 68.0, 212.0]
```

Wrapping the call in a lambda, `(c) -> celsiusToFahrenheit(c)`, guarantees the function runs once per array element with the element bound to `c`.

#708 Use if/else Inside a Function Body

Define a function that applies a member discount only when a Boolean flag is true.

INPUT

```
{ "price": 100, "isMember": true }
```

SOLUTION

```
fun applyDiscount(price: Number, isMember: Boolean): Number =
  if (isMember) price * 0.9 else price
---
applyDiscount(payload.price, payload.isMember)
```

OUTPUT

```
90.0
```

A function body is free to use any expression form, including `if/else`, exactly as it would appear anywhere else in a DataWeave script.

#709 Write a Function That Validates Input

Define a function that returns a validity object, including an error message for invalid input.

INPUT

```
{ "qty": -3 }
```

SOLUTION

```
fun validateOrder(qty: Number) =
  if (qty > 0) { valid: true }
  else { valid: false, error: "Quantity must be positive" }
---
validateOrder(payload.qty)
```

OUTPUT

```
{ "valid": false, "error": "Quantity must be positive" }
```

Because DataWeave functions aren't statically bound to one return shape, the two branches can return objects with different fields.

#710 Use a Function Inside an Object Literal

Define a SKU-formatting function and use it directly as an object field's value.

INPUT

```
{ "prefix": "CRG", "num": 4471 }
```

SOLUTION

```
fun formatSku(prefix: String, num: Number): String = prefix ++ "-" ++ (num as String)
---
{ sku: formatSku(payload.prefix, payload.num) }
```

OUTPUT

```
{ "sku": "CRG-4471" }
```

A function call is just an expression, so it can sit anywhere an expression is allowed — including as an object field's value.

SECTION 2 — Default Parameters & Overloading (Problems 711–720)

#711 Give a Parameter a Default Value

Define a greeting function whose greeting word defaults to "Hello" when not supplied.

INPUT

```
{ "name": "Priya" }
```

SOLUTION

```
fun greet(name: String, greeting: String = "Hello") = greeting ++ ", " ++
name ++ "!"
---
greet(payload.name)
```

OUTPUT

```
"Hello, Priya!"
```

A parameter written as `greeting: String = "Hello"` becomes optional — callers that omit it get the default value.

#712 Override a Default Parameter

Call the same greeting function, this time explicitly supplying a custom greeting.

INPUT

```
{ "name": "Priya", "greeting": "Welcome back" }
```

SOLUTION

```
fun greet(name: String, greeting: String = "Hello") = greeting ++ ", " ++
name ++ "!"
---
greet(payload.name, payload.greeting)
```

OUTPUT

```
"Welcome back, Priya!"
```

Supplying a second argument simply overrides the default — the function itself doesn't need to change.

#713 Default a Numeric Parameter

Define a function that adds a processing surcharge, defaulting to a 5% rate.

INPUT

```
{ "total": 200 }
```

SOLUTION

```
fun withSurcharge(total: Number, feeRate: Number = 0.05): Number = total +
(total * feeRate)
---
withSurcharge(payload.total)
```

OUTPUT

```
210.0
```

Default values aren't limited to Strings — any type, including `Number`, can have a default expression.

#714 Default a Boolean Parameter

Define a weight formatter that defaults to metric units.

INPUT

```
{ "kg": 10 }
```

SOLUTION

```
fun formatWeight(kg: Number, useMetric: Boolean = true): String =
  if (useMetric) (kg as String) ++ " kg" else ((kg * 2.20462) as String) ++ "
lb"
---
formatWeight(payload.kg)
```

OUTPUT

```
"10 kg"
```

With `useMetric` left at its default of `true`, the function takes the metric branch without the caller needing to say so.

#715 Override the Boolean Default

Call the same weight formatter, explicitly requesting imperial units.

INPUT

```
{ "kg": 10 }
```

SOLUTION

```
fun formatWeight(kg: Number, useMetric: Boolean = true): String =
  if (useMetric) (kg as String) ++ " kg" else ((kg * 2.20462) as String) ++ "
lb"
---
formatWeight(payload.kg, false)
```

OUTPUT

```
"22.0462 lb"
```

Passing `false` explicitly for `useMetric` routes into the imperial-conversion branch instead.

#716 Define Two Functions With the Same Name (Overloading)

Define a pricing function with a one-argument and a two-argument version, then call the simpler one.

INPUT

```
{ "base": 150 }
```

SOLUTION

```
fun price(base: Number) = base
fun price(base: Number, discountPct: Number) = base - (base * discountPct)
---
price(payload.base)
```

OUTPUT

```
150
```

DataWeave allows multiple `fun` definitions to share a name as long as their parameter lists differ — the call site's argument count picks the right one.

#717 Call the Overload With a Second Argument

Call the same overloaded pricing function, this time supplying a discount percentage.

INPUT

```
{ "base": 150, "discountPct": 0.2 }
```

SOLUTION

```
fun price(base: Number) = base
fun price(base: Number, discountPct: Number) = base - (base * discountPct)
---
price(payload.base, payload.discountPct)
```

OUTPUT

```
120.0
```

Adding a second argument automatically routes the call to the two-parameter overload instead of the one-parameter version.

#718 Overload a Function by Parameter Type

Define a labeling function overloaded by type, then call it with a String.

INPUT

```
{ "value": "hello" }
```

SOLUTION

```
fun label(v: String) = "String: " ++ v
fun label(v: Number) = "Number: " ++ (v as String)
---
label(payload.value)
```

OUTPUT

```
"String: hello"
```

*Overloads can also be distinguished by parameter **type** rather than just count, letting `label` behave differently for a String versus a Number.*

#719 Call the Type-Based Overload With a Number

Call the same overloaded labeling function with a Number instead of a String.

INPUT

```
{ "value": 42 }
```

SOLUTION

```
fun label(v: String) = "String: " ++ v
fun label(v: Number) = "Number: " ++ (v as String)
---
label(payload.value)
```

OUTPUT

```
"Number: 42"
```

DataWeave picks the overload whose parameter type matches the argument's runtime type — no explicit dispatch logic needed.

#720 Combine a Default Parameter With Type Annotations

Define a tracking-URL builder with a default base URL.

INPUT

```
{ "carrier": "fedex", "trackingId": "884921" }
```

SOLUTION

```
fun buildTrackingUrl(carrier: String, trackingId: String, baseUrl: String =
"https://track.cargolink.com"): String =
  baseUrl ++ "/" ++ carrier ++ "/" ++ trackingId
---
buildTrackingUrl(payload.carrier, payload.trackingId)
```

OUTPUT

```
"https://track.cargolink.com/fedex/884921"
```

Type annotations and default values combine freely on the same parameter list, giving callers a documented, low-friction API.

SECTION 3 — Functions That Accept Functions (Problems 721–730)

#721 Write a Function That Takes Another Function as a Parameter

Define a function that applies a passed-in function to a value twice.

INPUT

```
{ "start": 10 }
```

SOLUTION

```
fun applyTwice(f, x) = f(f(x))
---
applyTwice((n) -> n + 5, payload.start)
```

OUTPUT

```
20
```

A parameter like `f` can be a function value; calling it inside the body with `f(x)` works exactly like calling any named function.

#722 Pass a Named Function's Logic Through a Lambda Wrapper

Reuse the same "apply twice" helper, this time driving it with a separately defined function.

INPUT

```
{ "start": 2 }
```

SOLUTION

```
fun triple(n) = n * 3
fun applyTwice(f, x) = f(f(x))
---
applyTwice((n) -> triple(n), payload.start)
```

OUTPUT

```
18
```

Wrapping `triple` in a lambda, `(n) -> triple(n)`, lets a named function be handed off anywhere a function value is expected.

#723 Write a Function That Filters Using a Passed-In Condition

Define a reusable filter helper that accepts the filtering condition as a function.

INPUT

```
{ "orders": [{"id": "O1", "total": 80}, {"id": "O2", "total": 150}] }
```

SOLUTION

```
fun keepWhere(arr: Array, cond) = arr filter ((item) -> cond(item))
---
keepWhere(payload.orders, (o) -> o.total > 100)
```

OUTPUT

```
[{ "id": "O2", "total": 150 }]
```

`keepWhere` doesn't know anything about orders — the caller's lambda supplies the actual business rule.

#724 Write a Function That Transforms Using a Passed-In Function

Define a reusable transform helper that accepts the transformation as a function.

INPUT

```
{ "prices": [100, 200] }
```

SOLUTION

```
fun transformAll(arr: Array, f) = arr map ((item) -> f(item))
---
transformAll(payload.prices, (p) -> p * 1.08)
```

OUTPUT

```
[108.0, 216.0]
```

The same `transformAll` helper could apply any transformation at all — tax, discounting, formatting — just by swapping the lambda passed in.

#725 Pass a Conditional Lambda Into a Custom Function

Use a higher-order helper to label each order, using an if/else lambda as the decision logic.

INPUT

```
{ "orders": [{"total": 90}, {"total": 250}] }
```

SOLUTION

```
fun runFor(items, action) = items map ((i) -> action(i))
---
runFor(payload.orders, (o) -> if (o.total > 200) "HIGH" else "STANDARD")
```

OUTPUT

```
["STANDARD", "HIGH"]
```

A lambda passed as an argument can contain any expression, including a full `if/else`.

#726 Build a Generic Validator Function

Define a validator that checks whether every item in an array satisfies a passed-in rule.

INPUT

```
{ "orders": [{"total": 10}, {"total": 25}] }
```

SOLUTION

```
fun validateAll(items: Array, rule): Boolean = sizeOf(items filter ((i) ->
not rule(i))) == 0
---
validateAll(payload.orders, (o) -> o.total > 0)
```

OUTPUT

```
true
```

Counting the items that fail the rule and checking for zero is a simple, readable way to implement an "all match" check.

#727 Reuse the Same Higher-Order Function With a Stricter Rule

Call the same generic validator again, this time with a stricter passed-in rule.

INPUT

```
{ "orders": [{"total": 10}, {"total": 25}] }
```

SOLUTION

```
fun validateAll(items: Array, rule): Boolean = sizeOf(items filter ((i) ->
not rule(i))) == 0
---
validateAll(payload.orders, (o) -> o.total > 50)
```

OUTPUT

```
false
```

Nothing about `validateAll` changed — only the rule passed into it did, which is the whole point of accepting a function as a parameter.

#728 Combine Two Passed-In Functions Inside One Custom Function

Define a function that takes two other functions and combines their results.

INPUT

```
{ "n": 5 }
```

SOLUTION

```
fun applyBoth(x, f, g) = f(x) + g(x)
---
applyBoth(payload.n, (n) -> n * 2, (n) -> n + 1)
```

OUTPUT

```
16
```

A function parameter list can hold more than one function value at once — `f` and `g` here are two entirely independent pieces of logic.

#729 Use a Passed-In Function to Build a Dynamic Field Name

Use a function parameter to compute an object's key name dynamically.

INPUT

```
{ "status": "OPEN" }
```

SOLUTION

```
fun tagField(status: String, keyFn) = { (keyFn(status)): true }
---
tagField(payload.status, (s) -> lower(s) ++ "Flag")
```

OUTPUT

```
{ "openFlag": true }
```

The parenthesized key selector `(keyFn(status))` can hold any expression, including a call to a function passed in from outside.

#730 Pass a Custom Comparison Function Into a Filter Helper

Use a higher-order helper to find CargoLink orders matching a multi-condition business rule.

INPUT

```
{ "orders": [{"id": "O1", "region": "WEST", "total": 50}, {"id": "O2",
"region": "WEST", "total": 150}, {"id": "O3", "region": "EAST", "total": 200}]
}
```

SOLUTION

```
fun findMatching(orders: Array, matcher) = orders filter ((o) -> matcher(o))
---
findMatching(payload.orders, (o) -> o.region == "WEST" and o.total > 100)
```

OUTPUT

```
[{ "id": "O2", "region": "WEST", "total": 150 }]
```

The passed-in `matcher` function can combine as many conditions as needed with `and`/`or`, keeping `findMatching` itself completely generic.

SECTION 4 — Recursion Basics (Problems 731–740)

#731 Write a Recursive Countdown Function

Write a function that recursively builds an array counting down from n to 0.

INPUT

```
{ "n": 5 }
```

SOLUTION

```
fun countdown(n: Number): Array = if (n <= 0) [0] else [n] ++ countdown(n - 1)
---
countdown(payload.n)
```

OUTPUT

```
[5, 4, 3, 2, 1, 0]
```

A recursive function calls itself with a smaller input each time; the `if (n <= 0)` check is the base case that stops the recursion.

#732 Write a Recursive Factorial Function

Write the classic recursive factorial function.

INPUT

```
{ "n": 6 }
```

SOLUTION

```
fun factorial(n: Number): Number = if (n <= 1) 1 else n * factorial(n - 1)
---
factorial(payload.n)
```

OUTPUT

```
720
```

Each call multiplies `n` by the factorial of everything smaller, until `n <= 1` stops the chain and returns `1`.

#733 Write a Recursive Sum-to-N Function

Write a function that recursively sums every whole number from 1 to n.

INPUT

```
{ "n": 10 }
```

SOLUTION

```
fun sumToN(n: Number): Number = if (n <= 0) 0 else n + sumToN(n - 1)
---
sumToN(payload.n)
```

OUTPUT

```
55
```

This follows the same shape as `factorial` — a base case at zero, and each call adding one more value into the total.

#734 Write a Recursive String Reversal Function

Write a function that reverses a String recursively, one character at a time.

INPUT

```
{ "text": "cargo" }
```

SOLUTION

```
fun reverseStr(s: String): String = if (sizeOf(s) <= 1) s else reverseStr(s[1
to -1]) ++ s[0]
---
reverseStr(payload.text)
```

OUTPUT

```
"ograc"
```

Checking `sizeOf(s) <= 1` before slicing guarantees `s[1 to -1]` is only ever evaluated on a String with at least two characters.

#735 Write a Recursive Function to Compute a Power

Write a function that recursively raises a base to an exponent.

INPUT

```
{ "base": 2, "exp": 8 }
```

SOLUTION

```
fun power(base: Number, exp: Number): Number = if (exp == 0) 1 else base *
power(base, exp - 1)
---
power(payload.base, payload.exp)
```

OUTPUT

```
256
```

The exponent counts down toward the base case of `0`, the same pattern used by `factorial` and `sumToN`.

#736 Write a Recursive Function to Find the Greatest Common Divisor

Implement Euclid's algorithm recursively to find the GCD of two numbers.

INPUT

```
{ "a": 48, "b": 18 }
```

SOLUTION

```
fun gcd(a: Number, b: Number): Number = if (b == 0) a else gcd(b, a mod b)
---
gcd(payload.a, payload.b)
```

OUTPUT

```
6
```

Each call replaces `(a, b)` with `(b, a mod b)`; once `b` reaches `0`, `a` holds the greatest common divisor.

#737 Write a Recursive Fibonacci Function

Write a function that recursively computes the nth Fibonacci number.

INPUT

```
{ "n": 9 }
```

SOLUTION

```
fun fib(n: Number): Number = if (n <= 1) n else fib(n - 1) + fib(n - 2)
---
fib(payload.n)
```

OUTPUT

```
34
```

*Fibonacci is the classic example of a function that recurses **twice** per call, once for each of the two preceding terms.*

#738 Write a Recursive Function to Count Vowels in a String

Write a function that recursively counts the vowels in a String.

INPUT

```
{ "text": "cargolink" }
```

SOLUTION

```
fun countVowels(s: String): Number =
  if (sizeOf(s) == 0) 0
  else if (sizeOf(s) == 1) (if (contains("aeiouAEIOU", s[0])) 1 else 0)
  else (if (contains("aeiouAEIOU", s[0])) 1 else 0) + countVowels(s[1 to -1])
---
countVowels(payload.text)
```

OUTPUT

```
3
```

Handling lengths `0` and `1` as separate base cases means the recursive branch only ever slices a String with two or more characters left.

#739 Write a Recursive Function to Sum an Array

Write a function that recursively sums the numbers in an Array, without using `reduce`.

INPUT

```
{ "values": [10, 20, 30, 15] }
```

SOLUTION

```
fun sumArr(arr: Array): Number =
  if (isEmpty(arr)) 0
  else if (sizeOf(arr) == 1) arr[0]
  else arr[0] + sumArr(arr[1 to -1])
---
sumArr(payload.values)
```

OUTPUT

```
75
```

Arrays follow the same recursive shape as Strings: peel off the first element, recurse on the rest, and combine the results.

#740 Write a Recursive Function That Tracks Depth

Write a countdown function that also tracks how many recursive calls it took to finish.

INPUT

```
{ "start": 4 }
```

SOLUTION

```
fun countdownWithDepth(n: Number, depth: Number): Object =
  if (n <= 0) { finalValue: n, depth: depth }
  else countdownWithDepth(n - 1, depth + 1)
---
countdownWithDepth(payload.start, 0)
```

OUTPUT

```
{ "finalValue": 0, "depth": 4 }
```

An extra parameter like `depth` can carry information forward through the recursion — this is the idea Section 5 builds on with dedicated accumulator parameters.

SECTION 5 — Recursion With Accumulator Parameters (Problems 741–750)

#741 Convert sumToN to Use an Accumulator

Rewrite the sum-to-n function to carry its running total in an accumulator parameter.

INPUT

```
{ "n": 10 }
```

SOLUTION

```
fun sumToNAcc(n: Number, acc: Number = 0): Number = if (n <= 0) acc else
sumToNAcc(n - 1, acc + n)
---
sumToNAcc(payload.n)
```

OUTPUT

```
55
```

*Instead of adding `n` after the recursive call returns, an accumulator adds it **before** recursing — the total is already complete by the time the base case is hit.*

#742 Convert factorial to Use an Accumulator

Rewrite the factorial function to carry its running product in an accumulator parameter.

INPUT

```
{ "n": 6 }
```

SOLUTION

```
fun factorialAcc(n: Number, acc: Number = 1): Number = if (n <= 1) acc else
factorialAcc(n - 1, acc * n)
---
factorialAcc(payload.n)
```

OUTPUT

```
720
```

The accumulator's default value, `1`, doubles as the correct starting point for a product — the same way `0` is the right starting point for a sum.

#743 Build an Array Using an Accumulator Parameter

Write a function that recursively builds an array of numbers from a starting point up to 5.

INPUT

```
{ "start": 1 }
```

SOLUTION

```
fun buildRange(n: Number, acc: Array = []): Array = if (n > 5) acc else
buildRange(n + 1, acc ++ [n])
---
buildRange(payload.start)
```

OUTPUT

```
[1, 2, 3, 4, 5]
```

An accumulator doesn't have to be a Number — here it's an Array that grows by one element on every recursive call.

#744 Accumulate a Running Total Across an Array

Write a recursive version of `reduce` that sums an Array using an accumulator.

INPUT

```
{ "values": [5, 10, 15] }
```

SOLUTION

```
fun sumWithAcc(arr: Array, acc: Number = 0): Number =
  if (isEmpty(arr)) acc
  else sumWithAcc(if (sizeOf(arr) == 1) [] else arr[1 to -1], acc + arr[0])
---
sumWithAcc(payload.values)
```

OUTPUT

```
30
```

This is the accumulator-based mirror of `sumArr` from Section 4 — the running total is passed forward instead of built up as calls return.

#745 Accumulate a String by Concatenation

Write a recursive version of `joinBy` that builds up a separated String using an accumulator.

INPUT

```
{ "codes": ["A1", "B2", "C3"] }
```

SOLUTION

```
fun joinWithAcc(arr: Array, sep: String, acc: String = ""): String =
  if (isEmpty(arr)) acc
  else joinWithAcc(
    if (sizeOf(arr) == 1) [] else arr[1 to -1],
    sep,
    if (acc == "") (arr[0] as String) else acc ++ sep ++ (arr[0] as String)
  )
---
joinWithAcc(payload.codes, "-")
```

OUTPUT

```
"A1-B2-C3"
```

The `if (acc == "")` check avoids a leading separator on the very first element, matching how `joinBy` itself behaves.

#746 Track Both a Count and a Sum in One Accumulator

Write a function that recursively computes both a count and a sum in a single Object accumulator.

INPUT

```
{ "values": [4, 8, 15, 16] }
```

SOLUTION

```
fun statsAcc(arr: Array, acc: Object = {count: 0, sum: 0}): Object =
  if (isEmpty(arr)) acc
  else statsAcc(
    if (sizeOf(arr) == 1) [] else arr[1 to -1],
    { count: acc.count + 1, sum: acc.sum + arr[0] }
  )
---
statsAcc(payload.values)
```

OUTPUT

```
{ "count": 4, "sum": 43 }
```

An accumulator can be a whole Object, letting one recursive pass track several running values — here a count and a sum — at once.

#747 Use an Accumulator to Reverse an Array Recursively

Write a recursive array-reversal function using an accumulator, without `reverse`.

INPUT

```
{ "values": [1, 2, 3, 4] }
```

SOLUTION

```
fun reverseArr(arr: Array, acc: Array = []): Array =
  if (isEmpty(arr)) acc
  else reverseArr(if (sizeOf(arr) == 1) [] else arr[1 to -1], [arr[0]] ++
acc)
---
reverseArr(payload.values)
```

OUTPUT

```
[4, 3, 2, 1]
```

Prepending each element to the front of the accumulator, `[arr[0]] ++ acc`, naturally builds the reversed order as the recursion proceeds.

#748 Use an Accumulator to Find the Maximum Value

Write a recursive function that tracks the largest value seen so far in an accumulator.

INPUT

```
{ "values": [42, 17, 89, 63] }
```

SOLUTION

```
fun maxAcc(arr: Array, acc: Number = 0): Number =
  if (isEmpty(arr)) acc
  else maxAcc(if (sizeOf(arr) == 1) [] else arr[1 to -1], if (arr[0] > acc)
arr[0] else acc)
---
maxAcc(payload.values)
```

OUTPUT

```
89
```

At each step, the accumulator either keeps its current value or updates to the new element, whichever is larger.

#749 Combine a Default Parameter With Recursive Validation

Write a function that recursively finds the first order with an invalid (non-positive) total.

INPUT

```
{ "orders": [{"id": "O1", "total": 50}, {"id": "O2", "total": -10}, {"id":
"O3", "total": 30}] }
```

SOLUTION

```
fun findFirstInvalid(orders: Array, idx: Number = 0): Object =
  if (idx >= sizeOf(orders)) { found: false }
  else if (orders[idx].total <= 0) { found: true, index: idx, orderId:
orders[idx].id }
  else findFirstInvalid(orders, idx + 1)
---
findFirstInvalid(payload.orders)
```

OUTPUT

```
{ "found": true, "index": 1, "orderId": "O2" }
```

Here the accumulator, `idx`, defaults to `0` and walks forward through the array by index instead of slicing it on every call.

#750 CargoLink Capstone: Total Weight and Overweight Flags

Recursively compute a shipment batch's total weight and flag any package over a weight limit, in a single pass.

INPUT

```
{ "items": [{"id": "PKG-1", "weight": 30}, {"id": "PKG-2", "weight": 65},
{"id": "PKG-3", "weight": 48}] }
```

SOLUTION

```
fun weighShipments(items: Array, limit: Number, acc: Object = {totalWeight:
0, overLimit: []}): Object =
  if (isEmpty(items)) acc
  else weighShipments(
    if (sizeOf(items) == 1) [] else items[1 to -1],
    limit,
    {
      totalWeight: acc.totalWeight + items[0].weight,
      overLimit: if (items[0].weight > limit) acc.overLimit ++ [items[0].id]
else acc.overLimit
    }
  )
---
weighShipments(payload.items, 50)
```

OUTPUT

```
{ "totalWeight": 143, "overLimit": ["PKG-2"] }
```

This closes out the section by combining every idea so far — a default Object accumulator, conditional accumulation, and a guarded base case — into one realistic weigh-in report.

SECTION 6 — match/case Deep Dive (Problems 751–760)

#751 Bind a Matched Literal to a Name

Match a carrier code literally, binding the matched value to a name for reuse in the result.

INPUT

```
{ "carrier": "fedex" }
```

SOLUTION

```
payload.carrier match {
  case c: "fedex" -> "Carrier: " ++ upper(c)
  else -> "Unknown carrier"
}
```

OUTPUT

```
"Carrier: FEDEX"
```

Writing `case c: "fedex"` both checks for the exact literal and binds it to `c`, ready to use on the right of `->`.

#752 Add a Guard Condition to a match Case

Classify an order's value into a tier using guard conditions on the match cases.

INPUT

```
{ "total": 550 }
```

SOLUTION

```
payload.total match {
  case n if n > 1000 -> "HIGH VALUE"
  case n if n > 100 -> "STANDARD"
  else -> "LOW VALUE"
}
```

OUTPUT

```
"STANDARD"
```

A guard, written as `case n if <condition>`, only matches when both the binding and the condition succeed — cases are still tried top to bottom.

#753 Combine a Type Check With a Guard

Match a value as a Number and additionally require it to be positive.

INPUT

```
{ "quantity": 12 }
```

SOLUTION

```
payload.quantity match {
  case n is Number if n > 0 -> "Valid: " ++ (n as String)
  case n is Number -> "Invalid: not positive"
  else -> "Not a number"
}
```

OUTPUT

```
"Valid: 12"
```

A type matcher and a guard can stack on the same case — `n is Number` binds and checks the type, then `if n > 0` adds a further condition.

#754 Use a Guard With a String Function Inside match

Use a guard condition that calls `contains` to do a lightweight email sanity check.

INPUT

```
{ "email": "ops@cargolink.com" }
```

SOLUTION

```
payload.email match {
  case e if (e contains "@") -> "Looks like a valid email"
  else -> "Missing @ symbol"
}
```

OUTPUT

```
"Looks like a valid email"
```

A guard's condition is just a Boolean expression, so it can call any built-in function, not just compare literals.

#755 Use matches Inside a Guard for Pattern Validation

Use a guard with `matches` to confirm a tracking ID follows the expected format.

INPUT

```
{ "trackingId": "CRG-4471" }
```

SOLUTION

```
payload.trackingId match {
  case t if (t matches /[A-Z]{3}-\d{4}/) -> "Well-formed tracking ID"
  else -> "Malformed tracking ID"
}
```

OUTPUT

```
"Well-formed tracking ID"
```

Because `matches` requires the whole String to match, no anchors are needed in the pattern — `"CRG-4471"` matches the pattern in full.

#756 Nest a match Expression Inside Another match's Case

Route a return request by nesting a second match inside the first case's result.

INPUT

```
{ "type": "return", "reason": "damaged" }
```

SOLUTION

```
payload match {
  case p if p.type == "return" -> p.reason match {
    case "damaged" -> "Route to inspection"
    case "wrong item" -> "Route to restock"
    else -> "Route to review"
  }
  else -> "Not a return"
}
```

OUTPUT

```
"Route to inspection"
```

A `match` expression's result can be any expression, including another complete `match`.

#757 Use match Inside a Custom Function's Body

Define a function whose entire body is a `match` expression classifying a number's sign.

INPUT

```
{ "value": -5 }
```

SOLUTION

```
fun classify(n: Number): String = n match {
  case x if x < 0 -> "negative"
  case 0 -> "zero"
  else -> "positive"
}
---
classify(payload.value)
```

OUTPUT

```
"negative"
```

`match` and `fun` combine naturally — a function body that's "classify this value" is often clearer as a `match` than as nested `if/else`.

#758 Match Against Multiple Related Literals

Route two related status values to the same result using two separate cases.

INPUT

```
{ "status": "PENDING" }
```

SOLUTION

```
payload.status match {
  case "OPEN" -> "active"
  case "PENDING" -> "active"
  case "CLOSED" -> "inactive"
  else -> "unknown"
}
```

OUTPUT

```
"active"
```

DataWeave's `match` doesn't support combining literals into one case, so two related values are matched with two separate cases that share a result.

#759 Use match to Safely Handle a Value That Might Be Null

Use type matchers to handle a field that might be a String or might be null.

INPUT

```
{ "notes": null }
```

SOLUTION

```
payload.notes match {
  case n is String -> n
  case is Null -> "No notes provided"
}
```

OUTPUT

```
"No notes provided"
```

`case is Null` matches DataWeave's explicit Null type, giving a clean, readable alternative to a `default` null check.

#760 Guarantee a match Expression Never Fails

Route a priority code to a numeric rank, safely handling any value the cases don't expect.

INPUT

```
{ "priority": "URGENT" }
```

SOLUTION

```
payload.priority match {
  case "HIGH" -> 1
  case "MEDIUM" -> 2
  case "LOW" -> 3
  else -> 99
}
```

OUTPUT

```
99
```

DataWeave's `match` has no implicit fallthrough: if no case matches and there is no `else`, the expression throws a runtime error instead of quietly returning `null`. An unexpected value like `"URGENT"` arriving from an upstream system is exactly the scenario a catch-all `else` protects against — every `match` in this book, and every one you write, should end with one.

SECTION 7 — Array Pattern Matching & Recursion (Problems 761–770)

#761 Match the First Element of an Array

Use an array decomposition pattern to pull out the first element of a queue.

INPUT

```
{ "queue": ["ORD-1", "ORD-2", "ORD-3"] }
```

SOLUTION

```
payload.queue match {
  case [head ~ tail] -> "First: " ++ head
  case [] -> "Queue is empty"
}
```

OUTPUT

```
"First: ORD-1"
```

The pattern `[head ~ tail]` decomposes an Array into its first element, `head`, and every remaining element as an Array, `tail`.

#762 Match and Return the Tail of an Array

Use the same decomposition pattern to return everything except the first element.

INPUT

```
{ "queue": ["ORD-1", "ORD-2", "ORD-3"] }
```

SOLUTION

```
payload.queue match {
  case [head ~ tail] -> tail
  case [] -> []
}
```

OUTPUT

```
["ORD-2", "ORD-3"]
```

`tail` is bound to a real Array, so it can be returned, transformed, or passed straight into another recursive call.

#763 Match an Empty Array Explicitly

Handle the empty-queue case explicitly alongside the head/tail pattern.

INPUT

```
{ "queue": [] }
```

SOLUTION

```
payload.queue match {
  case [head ~ tail] -> "Has " ++ ((sizeOf(tail) + 1) as String) ++ " items"
  case [] -> "Nothing to process"
}
```

OUTPUT

```
"Nothing to process"
```

The empty-array pattern `[]` is what makes `[head ~ tail]` safe to use — every Array is either empty or has a head, so together they cover all cases.

#764 Recursively Process an Array Using Head/Tail Matching

Write a function that uppercases every String in an Array using recursive head/tail matching.

INPUT

```
{ "queue": ["dock-a", "dock-b"] }
```

SOLUTION

```
fun processQueue(items: Array): Array = items match {
  case [head ~ tail] -> [upper(head)] ++ processQueue(tail)
  case [] -> []
}
---
processQueue(payload.queue)
```

OUTPUT

```
["DOCK-A", "DOCK-B"]
```

This is the same "process the head, recurse on the tail" idea as Section 4's recursion, expressed through `match` instead of `if/else` and range selectors.

#765 Write a Recursive Sum Using match Instead of if/else

Rewrite the array-summing function from Section 4 to use `match`.

INPUT

```
{ "values": [5, 10, 15, 20] }
```

SOLUTION

```
fun sumViaMatch(arr: Array): Number = arr match {
  case [head ~ tail] -> head + sumViaMatch(tail)
  case [] -> 0
}
---
sumViaMatch(payload.values)
```

OUTPUT

```
50
```

Compared to `sumArr` from Section 4, this version needs no `sizeOf` guard at all — `[]` and `[head ~ tail]` already cover every possible Array shape.

#766 Write a Recursive Count Using match

Write a function that recursively counts the elements of an Array using `match`.

INPUT

```
{ "values": [1, 2, 3, 4, 5, 6] }
```

SOLUTION

```
fun countViaMatch(arr: Array): Number = arr match {
  case [head ~ tail] -> 1 + countViaMatch(tail)
  case [] -> 0
}
---
countViaMatch(payload.values)
```

OUTPUT

```
6
```

Every recursive call adds exactly `1` for the current `head`, so the total accumulates automatically as the calls return.

#767 Write a Recursive Search Using match

Write a function that recursively searches an Array of orders for a matching id.

INPUT

```
{ "orders": [{"id": "O1"}, {"id": "O2"}, {"id": "O3"}], "target": "O2" }
```

SOLUTION

```
fun containsId(orders: Array, target: String): Boolean = orders match {
  case [head ~ tail] -> if (head.id == target) true else containsId(tail,
target)
  case [] -> false
}
---
containsId(payload.orders, payload.target)
```

OUTPUT

```
true
```

The moment a match is found, the recursion returns `true` immediately without walking the rest of the Array.

#768 Match a Fixed-Length Array Alongside a Head/Tail Fallback

Give a coordinate pair special handling, but fall back to head/tail matching for any other length.

INPUT

```
{ "coords": [1, 2, 3, 4] }
```

SOLUTION

```
payload.coords match {
  case [x, y] -> "Point: (" ++ (x as String) ++ ", " ++ (y as String) ++ ")"
  case [head ~ tail] -> "Path with " ++ ((sizeOf(tail) + 1) as String) ++ "
points"
  case [] -> "No coordinates"
}
```

OUTPUT

```
"Path with 4 points"
```

Cases are tried in order: the exact two-element pattern `[x, y]` only matches arrays of exactly two, so a four-element Array falls through to `[head ~ tail]`.

#769 Recursively Build a New Array With Filtering Baked In

Write a recursive function that keeps only the positive numbers from an Array.

INPUT

```
{ "values": [5, -3, 8, -1, 2] }
```

SOLUTION

```
fun keepPositive(arr: Array): Array = arr match {
  case [head ~ tail] -> if (head > 0) [head] ++ keepPositive(tail) else
keepPositive(tail)
  case [] -> []
}
---
keepPositive(payload.values)
```

OUTPUT

```
[5, 8, 2]
```

A recursive function can build a shorter Array than it was given, simply by choosing not to include `head` in the result on some calls.

#770 CargoLink Capstone: Recursively Validate a Shipment Queue

Recursively check a queue of shipments for the first one that exceeds a weight limit.

INPUT

```
{ "items": [{"id": "PKG-1", "weight": 40}, {"id": "PKG-2", "weight": 150},
{"id": "PKG-3", "weight": 20}] }
```

SOLUTION

```
fun validateQueue(items: Array): Object = items match {
  case [head ~ tail] -> if (head.weight > 100) { valid: false, failedId:
head.id } else validateQueue(tail)
  case [] -> { valid: true }
}
---
validateQueue(payload.items)
```

OUTPUT

```
{ "valid": false, "failedId": "PKG-2" }
```

This closes out the section's array-pattern ideas in one realistic check: stop and report as soon as a problem is found, otherwise recurse into the rest of the queue.

SECTION 8 — Recursive Functions on Arrays (Problems 771–780)

#771 Reimplement map Recursively

Build your own version of `map` using recursion and a passed-in function.

INPUT

```
{ "values": [1, 2, 3] }
```

SOLUTION

```
fun myMap(arr: Array, f): Array = arr match {
  case [head ~ tail] -> [f(head)] ++ myMap(tail, f)
  case [] -> []
}
---
myMap(payload.values, (n) -> n * 10)
```

OUTPUT

```
[10, 20, 30]
```

This section rebuilds several of the array functions from Volume 4 from scratch, combining everything the volume has covered: recursion, `match`, and functions as parameters.

#772 Reimplement filter Recursively

Build your own version of `filter` using recursion and a passed-in predicate.

INPUT

```
{ "values": [1, 2, 3, 4, 5, 6] }
```

SOLUTION

```
fun myFilter(arr: Array, pred): Array = arr match {
  case [head ~ tail] -> if (pred(head)) [head] ++ myFilter(tail, pred) else
myFilter(tail, pred)
  case [] -> []
}
---
myFilter(payload.values, (n) -> (n mod 2) == 0)
```

OUTPUT

```
[2, 4, 6]
```

Every element still gets visited exactly once; the predicate just decides whether it's included in the result.

#773 Reimplement reduce Recursively

Build your own version of `reduce` using recursion, a passed-in function, and an accumulator.

INPUT

```
{ "values": [10, 20, 30] }
```

SOLUTION

```
fun myReduce(arr: Array, f, acc) = arr match {
  case [head ~ tail] -> myReduce(tail, f, f(acc, head))
  case [] -> acc
}
---
myReduce(payload.values, (acc, n) -> acc + n, 0)
```

OUTPUT

```
60
```

`myReduce` is really just Section 5's accumulator pattern with the combining logic pulled out into a passed-in function, `f`.

#774 Reimplement sizeOf for Arrays Recursively

Build your own version of `sizeOf` for Arrays using recursion.

INPUT

```
{ "values": ["a", "b", "c", "d"] }
```

SOLUTION

```
fun myLength(arr: Array): Number = arr match {
  case [head ~ tail] -> 1 + myLength(tail)
  case [] -> 0
}
---
myLength(payload.values)
```

OUTPUT

```
4
```

This is identical in shape to `countViaMatch` from Section 7 — counting elements is really just a special case of summing `1` for each one.

#775 Reimplement Array Max Recursively

Build your own version of a maximum-finder for Arrays using recursion.

INPUT

```
{ "values": [23, 89, 4, 67] }
```

SOLUTION

```
fun myMax(arr: Array): Number = arr match {
  case [single] -> single
  case [head ~ tail] -> if (head > myMax(tail)) head else myMax(tail)
  case [] -> 0
}
---
myMax(payload.values)
```

OUTPUT

```
89
```

The single-element pattern `[single]` is checked before the general `[head ~ tail]` pattern, giving the recursion a well-defined value to compare against at the very end of the Array.

#776 Reimplement contains for Arrays Recursively

Build your own version of `contains` for Arrays using recursion.

INPUT

```
{ "values": ["A1", "B2", "C3"], "target": "B2" }
```

SOLUTION

```
fun myContains(arr: Array, target): Boolean = arr match {
  case [head ~ tail] -> if (head == target) true else myContains(tail,
target)
  case [] -> false
}
---
myContains(payload.values, payload.target)
```

OUTPUT

```
true
```

Leaving off a type annotation on `target` lets this helper compare elements of any type, mirroring how the built-in `contains` behaves.

#777 Reimplement flatten Recursively

Build your own one-level `flatten` for an Array of Arrays using recursion.

INPUT

```
{ "groups": [[1, 2], [3, 4], [5]] }
```

SOLUTION

```
fun myFlatten(arr: Array): Array = arr match {
  case [head ~ tail] -> head ++ myFlatten(tail)
  case [] -> []
}
---
myFlatten(payload.groups)
```

OUTPUT

```
[1, 2, 3, 4, 5]
```

Since `head` is itself an Array here, `++` merges it directly into the result instead of wrapping it in another set of brackets.

#778 Reimplement Array Uniqueness Recursively

Build your own version of a duplicate-removal function for Arrays using recursion.

INPUT

```
{ "values": ["A", "B", "A", "C", "B"] }
```

SOLUTION

```
fun inArr(arr: Array, v): Boolean = arr match {
  case [head ~ tail] -> if (head == v) true else inArr(tail, v)
  case [] -> false
}
fun myDistinct(arr: Array, seen: Array = []): Array = arr match {
  case [head ~ tail] -> if (inArr(seen, head)) myDistinct(tail, seen) else
[head] ++ myDistinct(tail, seen ++ [head])
  case [] -> []
}
---
myDistinct(payload.values)
```

OUTPUT

```
["A", "B", "C"]
```

Two functions work together here: `inArr` is a small recursive helper, and `myDistinct` calls it to decide whether each element has already been `seen`.

#779 Reimplement every Recursively

Build your own version of `every` using recursion and a passed-in predicate.

INPUT

```
{ "orders": [{"total": 10}, {"total": 25}] }
```

SOLUTION

```
fun myEvery(arr: Array, pred): Boolean = arr match {
  case [head ~ tail] -> if (not pred(head)) false else myEvery(tail, pred)
  case [] -> true
}
---
myEvery(payload.orders, (o) -> o.total > 0)
```

OUTPUT

```
true
```

The base case for "every element satisfies the predicate" is `true` — an empty Array vacuously satisfies any condition, since there's nothing to violate it.

#780 CargoLink Capstone: Recursive Weighted Sum With a Custom Weight Function

Compute a shipment batch's total value using a recursive helper and a passed-in per-item pricing function.

INPUT

```
{ "items": [{"price": 10, "qty": 3}, {"price": 5, "qty": 4}] }
```

SOLUTION

```
fun weightedTotal(items: Array, weightFn): Number = items match {
  case [head ~ tail] -> weightFn(head) + weightedTotal(tail, weightFn)
  case [] -> 0
}
---
weightedTotal(payload.items, (i) -> i.price * i.qty)
```

OUTPUT

```
50
```

This closes out the section by combining recursive traversal with a higher-order function, so the same `weightedTotal` helper could total any per-item metric just by swapping `weightFn`.

SECTION 9 — Recursive Functions on Nested Data (Problems 781–790)

#781 Recursively Flatten a Deeply Nested Array

Write a function that flattens an Array nested to any depth, not just one level.

INPUT

```
{ "nested": [1, [2, 3, [4, 5]], 6] }
```

SOLUTION

```
fun deepFlatten(arr: Array): Array = arr match {
  case [head ~ tail] -> (if (head is Array) deepFlatten(head) else [head]) ++
deepFlatten(tail)
  case [] -> []
}
---
deepFlatten(payload.nested)
```

OUTPUT

```
[1, 2, 3, 4, 5, 6]
```

Checking `head is Array` lets the function recurse into nested Arrays wherever it finds them, however deep they go — unlike the built-in `flatten`, which only unwraps one level.

#782 Recursively Sum All Numbers in a Deeply Nested Array

Write a function that sums every Number in an Array, however deeply it's nested.

INPUT

```
{ "nested": [1, [2, 3], [4, [5, 6]]] }
```

SOLUTION

```
fun deepSum(arr: Array): Number = arr match {
  case [head ~ tail] -> (if (head is Array) deepSum(head) else head) +
deepSum(tail)
  case [] -> 0
}
---
deepSum(payload.nested)
```

OUTPUT

```
21
```

The same `is Array` check from `deepFlatten` reappears here — recursing into nested structures is a repeatable pattern, not a one-off trick.

#783 Recursively Find the Maximum Nesting Depth of an Array

Write a function that measures how many levels of Array nesting a structure contains.

INPUT

```
{ "nested": [1, [2, [3, 4]], 5] }
```

SOLUTION

```
fun maxOf2(a: Number, b: Number): Number = if (a > b) a else b
fun maxDepth(arr: Array): Number = arr match {
  case [head ~ tail] -> maxOf2((if (head is Array) maxDepth(head) else 0) +
1, maxDepth(tail))
  case [] -> 0
}
---
maxDepth(payload.nested)
```

OUTPUT

```
2
```

Each element contributes `0` to the depth if it isn't an Array, or `1` plus its own nested depth if it is; `maxOf2` keeps the largest contribution seen across the whole Array.

#784 Recursively Sum a Field Across a Nested Object Tree

Sum a `value` field across a tree of nested category objects, each with a `children` Array.

INPUT

```
{ "name": "Warehouse", "value": 10, "children": [{"name": "A", "value": 5,
"children": []}, {"name": "B", "value": 8, "children": [{"name": "B1",
"value": 3, "children": []}]}] }
```

SOLUTION

```
fun sumTree(node: Object): Number = node.value + sumChildren(node.children)
fun sumChildren(nodes: Array): Number = nodes match {
  case [head ~ tail] -> sumTree(head) + sumChildren(tail)
  case [] -> 0
}
---
sumTree(payload)
```

OUTPUT

```
26
```

Two mutually recursive functions handle this cleanly: `sumTree` processes one node, and `sumChildren` loops it back over that node's Array of children.

#785 Recursively Count All Nodes in a Tree

Count every node in the same category tree structure, including the root.

INPUT

```
{ "name": "Warehouse", "value": 10, "children": [{"name": "A", "value": 5, "children": []}, {"name": "B", "value": 8, "children": [{"name": "B1", "value": 3, "children": []}]}] }
```

SOLUTION

```
fun countNodes(node: Object): Number = 1 + countChildren(node.children)
fun countChildren(nodes: Array): Number = nodes match {
  case [head ~ tail] -> countNodes(head) + countChildren(tail)
  case [] -> 0
}
---
countNodes(payload)
```

OUTPUT

```
4
```

The `1 +` in `countNodes` counts the current node itself, before `countChildren` recurses into whatever is beneath it.

#786 Recursively Find the Deepest Node in a Tree

Find the name and depth of the deepest node in the category tree.

INPUT

```
{ "name": "Warehouse", "value": 10, "children": [{"name": "A", "value": 5,
"children": []}, {"name": "B", "value": 8, "children": [{"name": "B1",
"value": 3, "children": []}]}] }
```

SOLUTION

```
fun deeper(a: Object, b: Object): Object = if (a.depth >= b.depth) a else b
fun deepestAmong(nodes: Array, depth: Number): Object = nodes match {
  case [head ~ tail] -> deeper(deepestName(head, depth), deepestAmong(tail,
depth))
  case [] -> { name: "", depth: -1 }
}
fun deepestName(node: Object, depth: Number = 0): Object =
  if (isEmpty(node.children)) { name: node.name, depth: depth }
  else deepestAmong(node.children, depth + 1)
---
deepestName(payload)
```

OUTPUT

```
{ "name": "B1", "depth": 2 }
```

Three small functions divide the work: `deepestName` walks down one node, `deepestAmong` compares across a list of siblings, and `deeper` picks the winner between any two candidates.

#787 Recursively Rename Every Key in a Nested Object

Recursively apply a key-renaming function to every level of a nested Object, including nested Arrays.

INPUT

```
{ "name": "cargolink", "address": {"city": "Reno", "zip": "89501"} }
```

SOLUTION

```
fun applyToKeys(element, func) =
  if (element is Object) element mapObject (value, key) -> { (func(key)):
applyToKeys(value, func) }
  else if (element is Array) element map ((item) -> applyToKeys(item, func))
  else element
---
applyToKeys(payload, (key) -> upper(key))
```

OUTPUT

```
{ "NAME": "cargolink", "ADDRESS": { "CITY": "Reno", "ZIP": "89501" } }
```

This function recurses on three fronts at once: it walks into nested Objects with `mapObject`, into nested Arrays with `map`, and leaves every other value untouched.

#788 Recursively Remove Null-Valued Fields From a Nested Object

Recursively strip out every field whose value is null, at every level of nesting.

INPUT

```
{ "id": "O1", "note": null, "meta": {"city": "Reno", "state": null} }
```

SOLUTION

```
fun stripNulls(element) =
  if (element is Object)
    (element filterObject ((value, key) -> value != null)) mapObject ((value,
key) -> { (key): stripNulls(value) })
  else if (element is Array) element map ((item) -> stripNulls(item))
  else element
---
stripNulls(payload)
```

OUTPUT

```
{ "id": "O1", "meta": { "city": "Reno" } }
```

Parenthesizing the `filterObject` step before chaining `mapObject` onto it keeps the two-step Object transformation unambiguous, the same discipline used throughout this series.

#789 Recursively Extract All Leaf Values From a Nested Object

Recursively collect every non-container value in a nested Object into one flat Array.

INPUT

```
{ "a": 1, "b": {"c": 2, "d": 3}, "e": [4, 5] }
```

SOLUTION

```
fun leaves(element): Array =
  if (element is Object) leavesArr(valuesOf(element))
  else if (element is Array) leavesArr(element)
  else [element]
fun leavesArr(arr: Array): Array = arr match {
  case [head ~ tail] -> leaves(head) ++ leavesArr(tail)
  case [] -> []
}
---
leaves(payload)
```

OUTPUT

```
[1, 2, 3, 4, 5]
```

`valuesOf` turns an Object into an Array of its field values, which lets `leavesArr` walk Objects and Arrays with exactly the same recursive machinery.

#790 CargoLink Capstone: Total Value of a Nested Shipment Manifest

Recursively compute the total value of a manifest whose boxes can themselves contain other boxes and packages.

INPUT

```
{ "id": "MANIFEST-1", "value": 0, "items": [{"id": "PKG-1", "value": 120, "items": []}, {"id": "BOX-A", "value": 0, "items": [{"id": "PKG-2", "value": 45, "items": []}, {"id": "PKG-3", "value": 80, "items": []}]}] }
```

SOLUTION

```
fun manifestTotal(node: Object): Number = node.value + itemsTotal(node.items)
fun itemsTotal(items: Array): Number = items match {
  case [head ~ tail] -> manifestTotal(head) + itemsTotal(tail)
  case [] -> 0
}
---
manifestTotal(payload)
```

OUTPUT

```
245
```

This closes out the section with a real-world shape — a manifest of boxes within boxes — that the same two mutually recursive functions handle no matter how deeply it's nested.

SECTION 10 — Mixed Real-World Custom Function & Recursion Challenges (Problems 791–800)

#791 Build a Reusable Discount Function With a Default Tier

Combine a default parameter with `match` to build a tiered discount function.

INPUT

```
{ "total": 200, "tier": "GOLD" }
```

SOLUTION

```
fun discountFor(total: Number, tier: String = "STANDARD"): Number = tier
match {
  case "GOLD" -> total * 0.8
  case "SILVER" -> total * 0.9
  else -> total
}
---
discountFor(payload.total, payload.tier)
```

OUTPUT

```
160.0
```

Default parameters and `match` combine naturally: a caller that omits `tier` entirely still gets a safe, defined `"STANDARD"` result.

#792 Simulate a Retry Counter Recursively

Model a retry loop recursively, succeeding on a specific attempt number or giving up after a max.

INPUT

```
{ "maxAttempts": 5, "succeedsOn": 3 }
```

SOLUTION

```
fun attempt(n: Number, maxAttempts: Number, succeedsOn: Number): Object =
  if (n == succeedsOn) { success: true, attempts: n }
  else if (n >= maxAttempts) { success: false, attempts: n }
  else attempt(n + 1, maxAttempts, succeedsOn)
---
attempt(1, payload.maxAttempts, payload.succeedsOn)
```

OUTPUT

```
{ "success": true, "attempts": 3 }
```

Recursion is a natural fit for modeling a bounded retry sequence — each call is one attempt, and the two `if` checks are the two ways the sequence can end.

#793 Recursively Chunk an Array Into Fixed-Size Batches

Write a function that recursively splits an Array into batches of a given size.

INPUT

```
{ "items": ["a", "b", "c", "d", "e"] }
```

SOLUTION

```
fun chunk(arr: Array, size: Number): Array =
  if (isEmpty(arr)) []
  else if (sizeOf(arr) <= size) [arr]
  else [arr[0 to (size - 1)]] ++ chunk(arr[size to -1], size)
---
chunk(payload.items, 2)
```

OUTPUT

```
[["a", "b"], ["c", "d"], ["e"]]
```

The `sizeOf(arr) <= size` base case handles the final, possibly-shorter batch, so the last chunk doesn't need any special-case logic of its own.

#794 Recursively Compute Running Totals Across an Array

Write a function that recursively builds an Array of running (prefix) totals.

INPUT

```
{ "values": [10, 20, 30] }
```

SOLUTION

```
fun runningTotals(arr: Array, acc: Number = 0): Array = arr match {
  case [head ~ tail] -> [acc + head] ++ runningTotals(tail, acc + head)
  case [] -> []
}
---
runningTotals(payload.values)
```

OUTPUT

```
[10, 30, 60]
```

The accumulator carries the running total forward, and each recursive call both emits it into the result Array and passes the updated value deeper into the recursion.

#795 Recursively Validate a Chain of Approval Steps

Write a function that recursively confirms every step in an approval chain has status "APPROVED".

INPUT

```
{ "steps": [{"name": "manager", "status": "APPROVED"}, {"name": "finance",
"status": "APPROVED"}] }
```

SOLUTION

```
fun allApproved(steps: Array): Boolean = steps match {
  case [head ~ tail] -> if (head.status != "APPROVED") false else
allApproved(tail)
  case [] -> true
}
---
allApproved(payload.steps)
```

OUTPUT

```
true
```

This is the same "every element must satisfy a condition" shape as `myEvery` from Section 8, applied to a realistic approval-workflow payload.

#796 Recursively Build a Formatted Report String

Write a function that recursively builds a multi-line text report from an Array of orders.

INPUT

```
{ "orders": [{"id": "O1", "total": 50}, {"id": "O2", "total": 75}] }
```

SOLUTION

```
fun buildReport(orders: Array, acc: String = ""): String = orders match {
  case [head ~ tail] -> buildReport(tail, acc ++ "Order " ++ head.id ++ ": $"
++ (head.total as String) ++ "\n")
  case [] -> acc
}
---
buildReport(payload.orders)
```

OUTPUT

```
"Order O1: $50\nOrder O2: $75\n"
```

Building up a text report by String accumulation is a common real-world use for the accumulator pattern from Section 5, applied to plain-text output instead of JSON.

#797 Combine a Higher-Order Function With Recursion to Rank Items

Find the highest-scoring order in a non-empty Array, using a passed-in scoring function.

INPUT

```
{ "orders": [{"id": "O1", "total": 80}, {"id": "O2", "total": 150}, {"id": "O3", "total": 60}] }
```

SOLUTION

```
fun better(a: Object, b: Object, scoreFn): Object = if (scoreFn(a) >= scoreFn(b)) a else b
fun bestOf(items: Array, scoreFn): Object = items match {
  case [single] -> single
  case [head ~ tail] -> better(head, bestOf(tail, scoreFn), scoreFn)
}
---
bestOf(payload.orders, (o) -> o.total)
```

OUTPUT

```
{ "id": "O2", "total": 150 }
```

This assumes a non-empty Array, so `bestOf` deliberately has no `case []` — a reminder that an incomplete pattern like this should only be used where the missing case genuinely can't occur.

#798 Combine Default Parameters and a Passed-In Function in One Recursive Helper

Write a recursive helper that applies a passed-in transform to every element, using a default Array accumulator.

INPUT

```
{ "prices": [100, 200, 300] }
```

SOLUTION

```
fun processAll(items: Array, transform, acc: Array = []): Array = items match
{
  case [head ~ tail] -> processAll(tail, transform, acc ++ [transform(head)])
  case [] -> acc
}
---
processAll(payload.prices, (p) -> p + 10)
```

OUTPUT

```
[110, 210, 310]
```

This single helper now combines every idea from the volume at once: a default Array parameter, a passed-in function, and `match`-based recursion.

#799 Aggregate Nested Category Totals With a Custom Aggregator Function

Recursively total a numeric field across a category tree, using a passed-in function to select which field to total.

INPUT

```
{ "name": "Warehouse", "units": 10, "children": [{"name": "A", "units": 5, "children": []}, {"name": "B", "units": 8, "children": [{"name": "B1", "units": 3, "children": []}]}] }
```

SOLUTION

```
fun aggregate(node: Object, valueFn): Number = valueFn(node) + aggregateChildren(node.children, valueFn)
fun aggregateChildren(nodes: Array, valueFn): Number = nodes match {
  case [head ~ tail] -> aggregate(head, valueFn) + aggregateChildren(tail, valueFn)
  case [] -> 0
}
---
aggregate(payload, (n) -> n.units)
```

OUTPUT

```
26
```

Because the field to total is passed in as `valueFn`, the exact same pair of functions from Section 9's `sumTree` now works for any numeric field, on any tree shape.

#800 Series Capstone: Build a Classified Shipment Manifest

Combine a classifier function, `match`, and recursion to build a manifest that tags every order with a priority tier.

INPUT

```
{ "orders": [{"id": "O1", "total": 620}, {"id": "O2", "total": 150}, {"id": "O3", "total": 45}] }
```

SOLUTION

```
fun classify(total: Number): String = total match {
  case n if n >= 500 -> "PRIORITY"
  case n if n >= 100 -> "STANDARD"
  else -> "ECONOMY"
}
fun buildManifest(orders: Array, acc: Array = []): Array = orders match {
  case [head ~ tail] -> buildManifest(tail, acc ++ [{ id: head.id, tier: classify(head.total) }])
  case [] -> acc
}
---
buildManifest(payload.orders)
```

OUTPUT

```
[{ "id": "O1", "tier": "PRIORITY" }, { "id": "O2", "tier": "STANDARD" }, { "id": "O3", "tier": "ECONOMY" }]
```

The volume's capstone ties everything together: a guarded default-parameter accumulator drives the recursion, and a separate `classify` function — built on a guarded `match` — supplies the business logic for each step.

END OF VOLUME 8

800 Down, 200 to Go

You've now worked through selectors, type coercion, operators and conditionals, array and object transformations, string handling, dates and times, and a full tour of custom functions, recursion, and pattern matching.

Volume 9 moves to Format Conversions: another 100 problems on reading and writing JSON, XML, CSV, and other formats, and the DataWeave features that make cross-format transformation possible.

MULESOFT · PRACTICE PROBLEMS · WITH FULL SOLUTIONS

1000 DataWeave Problems With Solutions

Volume 9 — Format Conversions. 100 hands-on problems on JSON, XML, CSV, the `read` and `write` functions, and converting cleanly between formats.

Problems 801–900

Volume 9 of 10

2026 EDITION

How to Use This Volume

Volume 9 is about the boundary between DataWeave and the outside world: reading and writing JSON, XML, and CSV. Same format as every prior volume:

- **A short problem statement**, the **input** payload, a complete **solution** in a black code window, the exact **output**, and a one-line explanation.
- Ten sections of ten problems each, moving from output directives and the `read`/`write` functions through XML basics, attributes, and namespaces, CSV basics, cross-format conversion, and the small but important gotchas each format has of its own.

A note on inputs: earlier volumes always showed a JSON payload. From here on, the **Input** box shows whatever format the problem actually starts from — plain JSON, a raw XML document, or raw CSV text — exactly as it would look coming from a real system.

VOLUME 9

Format Conversions

100 problems on reading and writing JSON, XML, and CSV — output directives, the `read` and `write` functions, XML attributes and namespaces, CSV reader and writer properties, and the small type differences that matter when data crosses format boundaries.

SECTION 1 — JSON Basics & Output Directives (Problems 801–810)

#801 Select a Subset of Fields and Output as JSON

Output only a chosen subset of an incoming JSON payload's fields.

INPUT

```
{ "id": "O1", "total": 50, "internalNote": "do not ship" }
```

SOLUTION

```
output application/json
---
{ id: payload.id, total: payload.total }
```

OUTPUT

```
{ "id": "O1", "total": 50 }
```

The `output application/json` directive is what tells DataWeave to serialize the result as JSON — it goes in the header, alongside any `fun` or `var` definitions.

#802 Turn Off Pretty-Printing With the indent Property

Produce compact, single-line JSON output instead of the default pretty-printed form.

INPUT

```
{ "id": "O1", "total": 50 }
```

SOLUTION

```
output application/json {indent: false}
---
payload
```

OUTPUT

```
{"id":"O1","total":50}
```

Writer properties go in curly braces right after the MIME type; `{indent: false}` strips the extra whitespace, which is useful for minimizing payload size on the wire.

#803 Declare the Input MIME Type Explicitly

Explicitly declare the input payload's MIME type instead of relying on auto-detection.

INPUT

```
{ "total": 25 }
```

SOLUTION

```
input payload application/json
output application/json
---
payload.total * 2
```

OUTPUT

```
50
```

An `input` directive is only strictly necessary when DataWeave can't infer the format on its own — for example, a raw String with no reliable Content-Type metadata attached.

#804 Output an Array of Objects as JSON

Reshape and output an array of orders, keeping only the fields needed downstream.

INPUT

```
{ "orders": [{"id": "O1", "total": 50, "extra": "x"}, {"id": "O2", "total": 75, "extra": "y"}] }
```

SOLUTION

```
output application/json
---
payload.orders map ((o) -> { id: o.id, total: o.total })
```

OUTPUT

```
[{ "id": "O1", "total": 50 }, { "id": "O2", "total": 75 }]
```

Every transformation you've written across this whole series ends up here — reshaped, then handed to an `output` directive to become bytes on the wire.

#805 Produce a Nested JSON Object From Flat Input

Wrap flat input fields into a nested JSON structure.

INPUT

```
{ "id": "SHIP-1", "weight": 42 }
```

SOLUTION

```
output application/json
---
{ shipment: { id: payload.id, weight: payload.weight } }
```

OUTPUT

```
{ "shipment": { "id": "SHIP-1", "weight": 42 } }
```

JSON's nesting is just DataWeave's own object literal nesting, serialized directly — no extra ceremony required.

#806 Explicitly Coerce a Value's Type Before Writing JSON

Ensure a numeric-looking String field is written to JSON as a real number, not a quoted String.

INPUT

```
{ "total": "49.99" }
```

SOLUTION

```
output application/json
---
{ total: payload.total as Number }
```

OUTPUT

```
{ "total": 49.99 }
```

Without the coercion, the field would be written back out with quotes, `"49.99"`, since DataWeave preserves the source String type unless told otherwise.

#807 Write null Explicitly for a Missing Field

Guarantee an optional field always appears in the output, even when it's missing from the source.

INPUT

```
{ "id": "O1" }
```

SOLUTION

```
output application/json
---
{ id: payload.id, note: payload.note default null }
```

OUTPUT

```
{ "id": "O1", "note": null }
```

`default null` guarantees `note` appears in the output with an explicit null, instead of being silently absent when the source field doesn't exist.

#808 Remove Null Fields Before Writing JSON Output

Strip out every field whose value is null before producing the JSON output.

INPUT

```
{ "id": "O1", "note": null, "total": 50 }
```

SOLUTION

```
output application/json
---
payload filterObject ((value, key) -> value != null)
```

OUTPUT

```
{ "id": "O1", "total": 50 }
```

This is the opposite of #807 — instead of guaranteeing a null field is present, `filterObject` guarantees one never is.

#809 Output JSON With a Specific Field Order

Produce JSON output whose field order doesn't depend on the order fields appeared in the source.

INPUT

```
{ "total": 50, "id": "O1", "status": "OPEN" }
```

SOLUTION

```
output application/json
---
{ id: payload.id, status: payload.status, total: payload.total }
```

OUTPUT

```
{ "id": "O1", "status": "OPEN", "total": 50 }
```

JSON's field order in the output is whatever order the DataWeave object literal defines, regardless of the field order in the source payload.

#810 Combine Multiple Source Fields Into One Output Field

Combine two separate source fields into one computed output field.

INPUT

```
{ "prefix": "CRG", "num": 4471 }
```

SOLUTION

```
output application/json
---
{ fullId: payload.prefix ++ "-" ++ (payload.num as String) }
```

OUTPUT

```
{ "fullId": "CRG-4471" }
```

Format conversion is rarely a straight pass-through — most real transformations reshape and combine fields on the way to their new format.

SECTION 2 — Reading Data With the read Function (Problems 811–820)

#811 Parse a JSON String With read

Parse a JSON String embedded inside a payload field into a real DataWeave value.

INPUT

```
{ "rawJson": "{\"id\":\"O1\",\"total\":50}" }
```

SOLUTION

```
output application/json
---
read(payload.rawJson, "application/json")
```

OUTPUT

```
{ "id": "O1", "total": 50 }
```

`read(content, mimeType)` parses a String using whatever reader matches the given MIME type — here, the standard JSON reader.

#812 Parse a JSON String and Immediately Select a Field

Parse an embedded JSON String and select a single field from the result.

INPUT

```
{ "rawJson": "{\"id\":\"O1\",\"total\":50}" }
```

SOLUTION

```
output application/json
---
(read(payload.rawJson, "application/json")).total
```

OUTPUT

```
50
```

Since `read` returns a real DataWeave value, parenthesizing the call lets a selector like `.total` chain directly onto its result.

#813 Parse an XML String With read

Parse an XML String embedded inside a JSON payload field.

INPUT

```
{ "rawXml": "<order><id>O1</id><total>50</total></order>" }
```

SOLUTION

```
output application/json
---
read(payload.rawXml, "application/xml")
```

OUTPUT

```
{ "order": { "id": "O1", "total": "50" } }
```

Notice `total` comes back as the String `"50"`, not the Number `50` — XML has no native number type, a theme this whole volume returns to.

#814 Parse a CSV String With Headers

Parse an embedded CSV String, using its first line as field names.

INPUT

```
{ "rawCsv": "id,total\nO1,50\nO2,75\n" }
```

SOLUTION

```
output application/json
---
read(payload.rawCsv, "application/csv", {header: true})
```

OUTPUT

```
[{ "id": "O1", "total": "50" }, { "id": "O2", "total": "75" }]
```

The third argument to `read` is a reader-properties Object; `{header: true}` tells the CSV reader to use the first line as field names for every row.

#815 Parse a CSV String Without Headers

Parse a headerless CSV String, accessing each row's fields by position instead of by name.

INPUT

```
{ "rawCsv": "O1,50\nO2,75\n" }
```

SOLUTION

```
output application/json
---
read(payload.rawCsv, "application/csv", {header: false})
```

OUTPUT

```
[["O1", "50"], ["O2", "75"]]
```

With `header: false`, each row becomes a plain Array of Strings instead of an Object — fields are accessed by index, like `row[0]`, rather than by name.

#816 Parse CSV With a Custom Separator

Parse a semicolon-separated CSV String instead of the default comma-separated form.

INPUT

```
{ "rawCsv": "id;total\nO1;50\nO2;75\n" }
```

SOLUTION

```
output application/json
---
read(payload.rawCsv, "application/csv", {header: true, separator: ";"})
```

OUTPUT

```
[{ "id": "O1", "total": "50" }, { "id": "O2", "total": "75" }]
```

The `separator` reader property tells the CSV reader which character divides fields — useful for the many real-world CSV variants that don't use a comma.

#817 Coerce Fields After Reading CSV

Parse a CSV String, then coerce its numeric-looking fields into real Numbers.

INPUT

```
{ "rawCsv": "id,total\nO1,50\nO2,75\n" }
```

SOLUTION

```
output application/json
---
read(payload.rawCsv, "application/csv", {header: true}) map ((row) -> row ++
{total: row.total as Number})
```

OUTPUT

```
[{ "id": "O1", "total": 50 }, { "id": "O2", "total": 75 }]
```

Since every CSV value reads in as a String, coercing the fields you'll do math on is a normal, expected extra step right after `read`.

#818 Use read Inside a Larger Transformation

Use a parsed value's size as one field inside a larger result object.

INPUT

```
{ "rawJson": "[{\"id\":\"O1\"},{\"id\":\"O2\"},{\"id\":\"O3\"}]" }
```

SOLUTION

```
output application/json
---
{ orderCount: sizeOf(read(payload.rawJson, "application/json")) }
```

OUTPUT

```
{ "orderCount": 3 }
```

`read`'s result is just a value like any other — it can be passed straight into `sizeOf`, used as a field, or transformed further.

#819 Read Two Different Embedded Formats in One Transformation

Parse two payload fields that hold two different embedded formats.

INPUT

```
{ "metaJson": "{\"source\":\"warehouse-3\"}", "dataCsv": "id,qty\nPKG-1,5\nPKG-2,2\n" }
```

SOLUTION

```
output application/json
---
{
  meta: read(payload.metaJson, "application/json"),
  rows: read(payload.dataCsv, "application/csv", {header: true})
}
```

OUTPUT

```
{ "meta": { "source": "warehouse-3" }, "rows": [{ "id": "PKG-1", "qty": "5" }, { "id": "PKG-2", "qty": "2" }] }
```

`read` doesn't care what format the rest of the script is using — each call picks its own MIME type independently.

#820 CargoLink Capstone: Parse and Aggregate an Embedded CSV Manifest

Parse an embedded CSV manifest and compute both a row count and a total weight from it.

INPUT

```
{ "manifestCsv": "id,weight\nPKG-1,30\nPKG-2,45\nPKG-3,20\n" }
```

SOLUTION

```
var rows = read(payload.manifestCsv, "application/csv", {header: true})
---
{
  itemCount: sizeOf(rows),
  totalWeight: sum(rows map ((row) -> row.weight as Number))
}
```

OUTPUT

```
{ "itemCount": 3, "totalWeight": 95 }
```

Parsing once into a `var` and reusing it — rather than calling `read` twice — keeps this capstone both correct and efficient.

SECTION 3 — Writing Data With the write Function (Problems 821–830)

#821 Convert a Value to a Plain Text String With write

Convert a Number field into a plain text String using `write`.

INPUT

```
{ "total": 50 }
```

SOLUTION

```
output application/json
---
{ label: write(payload.total, "text/plain") }
```

OUTPUT

```
{ "label": "50" }
```

`write(value, mimeType)` is the mirror image of `read` — instead of parsing a String into a value, it serializes a value into a String.

#822 Embed a CSV String Inside a JSON Field

Produce a CSV-formatted String and embed it as a field inside a larger JSON response.

INPUT

```
{ "orders": [{"id": "O1", "total": 50}, {"id": "O2", "total": 75}] }
```

SOLUTION

```
output application/json
---
{ csvExport: write(payload.orders, "application/csv") }
```

OUTPUT

```
{ "csvExport": "id,total\nO1,50\nO2,75\n" }
```

The whole script's own `output` stays JSON — `write` only affects the one value it's called on, embedding a CSV-formatted String inside the JSON result.

#823 Embed an XML String Inside a JSON Field

Produce a compact XML String and embed it as a field inside a JSON response.

INPUT

```
{ "id": "O1", "total": 50 }
```

SOLUTION

```
output application/json
---
{ xmlExport: write({ order: payload }, "application/xml", {writeDeclaration:
false, indent: false}) }
```

OUTPUT

```
{ "xmlExport": "<order><id>O1</id><total>50</total></order>" }
```

Turning off both `writeDeclaration` and `indent` keeps the embedded XML String compact and free of an unnecessary declaration line.

#824 Use write With Custom CSV Properties

Produce a semicolon-separated CSV String using `write`'s properties argument.

INPUT

```
{ "orders": [{"id": "O1", "total": 50}] }
```

SOLUTION

```
output application/json
---
{ csvExport: write(payload.orders, "application/csv", {separator: ";",
header: true}) }
```

OUTPUT

```
{ "csvExport": "id;total\nO1;50\n" }
```

`write`'s third argument is a writer-properties Object, exactly parallel to the properties Object `read` takes for its reader settings.

#825 Compare read and write as Inverses

Confirm that writing a value and then reading it back reproduces the original value.

INPUT

```
{ "id": "O1", "total": 50 }
```

SOLUTION

```
output application/json
---
{ roundTrip: read(write(payload, "application/json"), "application/json") ==
payload }
```

OUTPUT

```
{ "roundTrip": true }
```

JSON round-trips cleanly because it shares DataWeave's own type system — Numbers stay Numbers, Booleans stay Booleans, unlike the trip through XML or CSV.

#826 Write Only a Subset of Fields to a Log-Friendly String

Build a compact, single-line JSON String suitable for a log line, omitting sensitive fields.

INPUT

```
{ "id": "O1", "status": "OPEN", "internalNotes": "vip customer" }
```

SOLUTION

```
output application/json
---
{ logLine: write({ id: payload.id, status: payload.status },
"application/json", {indent: false}) }
```

OUTPUT

```
{ "logLine": "{\"id\":\"O1\",\"status\":\"OPEN\"}" }
```

Selecting fields before calling `write` is how you keep sensitive data like `internalNotes` out of a log line entirely.

#827 Write Headerless CSV From an Array of Arrays

Write a headerless CSV String directly from an Array of Arrays.

INPUT

```
{ "rows": [["O1", "50"], ["O2", "75"]] }
```

SOLUTION

```
output application/json
---
{ fileContent: write(payload.rows, "application/csv", {header: false}) }
```

OUTPUT

```
{ "fileContent": "O1,50\nO2,75\n" }
```

With `header: false`, the CSV writer expects (and directly serializes) an Array of Arrays instead of an Array of Objects.

#828 Write the Same Source Data in Two Different Formats

Produce both a JSON String and a CSV String from the exact same source value.

INPUT

```
{ "id": "O1", "total": 50 }
```

SOLUTION

```
output application/json
---
{
  asJson: write(payload, "application/json", {indent: false}),
  asCsv: write([payload], "application/csv")
}
```

OUTPUT

```
{ "asJson": "{\"id\":\"O1\",\"total\":50}", "asCsv": "id,total\nO1,50\n" }
```

The CSV writer needs an Array of rows, so a single Object has to be wrapped in `[payload]` first — CSV has no concept of a single standalone row.

#829 Chain read and a Transformation Before write

Parse a CSV String, coerce and transform it, then write the result back out as a compact JSON String.

INPUT

```
{ "rawCsv": "id,total\nO1,50\nO2,75\n" }
```

SOLUTION

```
var parsed = read(payload.rawCsv, "application/csv", {header: true}) map
((row) -> row ++ {total: row.total as Number})
---
{ report: write(parsed, "application/json", {indent: false}) }
```

OUTPUT

```
{ "report": "[{\"id\":\"O1\",\"total\":50},{\"id\":\"O2\",\"total\":75}]" }
```

`read`, a transformation, and `write` chain together into a complete mini pipeline: parse, reshape, re-serialize.

#830 CargoLink Capstone: Build a CSV Export String From a Filtered JSON Array

Filter a JSON array of orders and write only the qualifying rows out as a CSV String.

INPUT

```
{ "orders": [{"id": "O1", "total": 50}, {"id": "O2", "total": 75}, {"id":
"O3", "total": 90}] }
```

SOLUTION

```
output application/json
---
{ export: write(payload.orders filter ((o) -> o.total > 60),
"application/csv") }
```

OUTPUT

```
{ "export": "id,total\nO2,75\nO3,90\n" }
```

This closes out the section by combining a filter from Volume 4 with the `write` function to produce a ready-to-download export String.

SECTION 4 — XML Basics (Problems 831–840)

#831 Set the Output MIME Type to XML

Output a JSON payload as XML by changing only the output directive.

INPUT

```
{ "id": "O1", "total": 50 }
```

SOLUTION

```
output application/xml
---
{ order: payload }
```

OUTPUT

```
<?xml version='1.0' encoding='UTF-8'?>
<order>
  <id>O1</id>
  <total>50</total>
</order>
```

Simply changing `output` from `application/json` to `application/xml` re-serializes the exact same value shape into XML, declaration and all.

#832 Understand XML's Single-Root Requirement

Wrap output fields so the result has exactly one top-level XML element.

INPUT

```
{ "id": "SHIP-1", "weight": 42 }
```

SOLUTION

```
output application/xml
---
{ shipment: { id: payload.id, weight: payload.weight } }
```

OUTPUT

```
<?xml version='1.0' encoding='UTF-8'?>
<shipment>
  <id>SHIP-1</id>
  <weight>42</weight>
</shipment>
```

Unlike JSON, XML output must resolve to a single root element — the whole body has to be an Object with exactly one top-level key, here `shipment`.

#833 Suppress the XML Declaration

Produce XML output without the leading `<?xml ...?>` declaration line.

INPUT

```
{ "id": "O1", "total": 50 }
```

SOLUTION

```
output application/xml {writeDeclaration: false}
---
{ order: payload }
```

OUTPUT

```
<order>
  <id>O1</id>
  <total>50</total>
</order>
```

`writeDeclaration: false` is common when embedding an XML fragment inside something else that doesn't want its own declaration line.

#834 Turn Off Indentation for Compact XML

Produce compact, single-line XML with no declaration and no indentation.

INPUT

```
{ "id": "O1", "total": 50 }
```

SOLUTION

```
output application/xml {indent: false, writeDeclaration: false}
---
{ order: payload }
```

OUTPUT

```
<order><id>O1</id><total>50</total></order>
```

Writer properties combine freely — turning off both `indent` and `writeDeclaration` gives the most compact XML form possible.

#835 Produce Repeated Elements From an Array

Turn a JSON Array of Strings into repeated XML elements with the same tag name.

INPUT

```
{ "items": ["PKG-1", "PKG-2", "PKG-3"] }
```

SOLUTION

```
output application/xml {writeDeclaration: false}
---
{ items: { item: payload.items } }
```

OUTPUT

```
<items>
  <item>PKG-1</item>
  <item>PKG-2</item>
  <item>PKG-3</item>
</items>
```

An Array value written under one key becomes one repeated XML element per Array entry — this is how XML represents lists at all.

#836 Produce Repeated Elements From an Array of Objects

Turn a JSON Array of order Objects into repeated, nested XML elements.

INPUT

```
{ "orders": [{"id": "O1", "total": 50}, {"id": "O2", "total": 75}] }
```

SOLUTION

```
output application/xml {writeDeclaration: false}
---
{ orders: { order: payload.orders } }
```

OUTPUT

```
<orders>
  <order>
    <id>O1</id>
    <total>50</total>
  </order>
  <order>
    <id>O2</id>
    <total>75</total>
  </order>
</orders>
```

Each Object in the Array becomes its own `<order>` element, with that Object's own fields nested inside — the array-of-objects pattern that shows up in almost every real XML feed.

#837 Convert Parsed XML Back to JSON

Convert an incoming XML document straight to JSON with no reshaping.

INPUT

```
<order><id>O1</id><total>50</total></order>
```

SOLUTION

```
output application/json
---
payload
```

OUTPUT

```
{ "order": { "id": "O1", "total": "50" } }
```

With no transformation at all, the root element becomes the single top-level JSON key, and every child element becomes a nested field.

#838 Select a Nested Value From Parsed XML

Select one nested field out of an incoming XML document.

INPUT

```
<shipment><id>SHIP-1</id><status>IN_TRANSIT</status></shipment>
```

SOLUTION

```
output application/json
---
payload.shipment.status
```

OUTPUT

```
"IN_TRANSIT"
```

Selecting into parsed XML works exactly like selecting into parsed JSON — `.shipment.status` walks down the same way either way.

#839 Remember: XML Text Content Is Always a String

Read a numeric XML field both as-is and explicitly coerced, to see the difference.

INPUT

```
<order><total>50</total></order>
```

SOLUTION

```
output application/json
---
{ total: payload.order.total, totalAsNumber: payload.order.total as Number }
```

OUTPUT

```
{ "total": "50", "totalAsNumber": 50 }
```

Unlike JSON, XML has no native number type — every text node parses as a String until it's explicitly coerced.

#840 CargoLink Capstone: Convert a Flat JSON Shipment Into XML

Convert a flat JSON shipment record into a clean XML document.

INPUT

```
{ "id": "SHIP-42", "carrier": "fedex", "weight": 18, "status": "IN_TRANSIT" }
```

SOLUTION

```
output application/xml {writeDeclaration: false}
---
{ shipment: { id: payload.id, carrier: payload.carrier, weight:
payload.weight, status: payload.status } }
```

OUTPUT

```
<shipment>
  <id>SHIP-42</id>
  <carrier>fedex</carrier>
  <weight>18</weight>
  <status>IN_TRANSIT</status>
</shipment>
```

This closes out the section with the most common real conversion of all: a flat JSON record becoming a flat XML document, field for field.

SECTION 5 — XML Attributes (Problems 841–850)

#841 Read a Single XML Attribute

Read one attribute's value off a parsed XML element.

INPUT

```
<order id="O1"><total>50</total></order>
```

SOLUTION

```
output application/json
---
payload.order.@id
```

OUTPUT

```
"O1"
```

The `@` selector reads an XML attribute the same way a plain selector reads a child element — `.@id` reaches the `id` attribute directly.

#842 Read All Attributes of an Element at Once

Read every attribute on an element in a single expression.

INPUT

```
<order id="O1" status="OPEN"><total>50</total></order>
```

SOLUTION

```
output application/json
---
payload.order.@
```

OUTPUT

```
{ "id": "O1", "status": "OPEN" }
```

A bare `.@` with no name returns every attribute on that element as its own Object.

#843 Combine an Attribute and a Child Element in One Result

Build a JSON object combining an attribute value and a coerced child element value.

INPUT

```
<order id="O1"><total>50</total></order>
```

SOLUTION

```
output application/json
---
{ orderId: payload.order.@id, total: payload.order.total as Number }
```

OUTPUT

```
{ "orderId": "O1", "total": 50 }
```

Attributes and child elements are read side by side with no special ceremony — the `@` only changes which one you're selecting.

#844 Write a Static XML Attribute

Write an XML element with a fixed attribute value.

INPUT

```
{ "total": 50 }
```

SOLUTION

```
output application/xml {writeDeclaration: false}
---
{ order @(id: "O1"): { total: payload.total } }
```

OUTPUT

```
<order id="O1">
  <total>50</total>
</order>
```

The `@(...)` block right after a key name attaches attributes to that element — it sits between the key and its `:` value.

#845 Write a Dynamic XML Attribute From the Payload

Write an XML attribute whose value comes from the incoming payload.

INPUT

```
{ "id": "O1", "total": 50 }
```

SOLUTION

```
output application/xml {writeDeclaration: false}
---
{ order @(id: payload.id): { total: payload.total } }
```

OUTPUT

```
<order id="O1">
  <total>50</total>
</order>
```

Anything that can go on the right of a `:` in a normal field can go inside `@(...)` too, including a selector into the payload.

#846 Write Multiple Attributes on the Same Element

Write an XML element carrying two separate attributes at once.

INPUT

```
{ "id": "O1", "status": "OPEN", "total": 50 }
```

SOLUTION

```
output application/xml {writeDeclaration: false}
---
{ order @(id: payload.id, status: payload.status): { total: payload.total } }
```

OUTPUT

```
<order id="O1" status="OPEN">
  <total>50</total>
</order>
```

Just like an object literal, `@(...)` can hold as many comma-separated key-value pairs as needed.

#847 Copy All Source Attributes Onto a New Element

Copy every attribute from a source element straight onto a differently-named output element.

INPUT

```
<order id="O1" status="OPEN"><total>50</total></order>
```

SOLUTION

```
output application/xml {writeDeclaration: false}
---
{ shipment @((payload.order.@)): { total: payload.order.total } }
```

OUTPUT

```
<shipment id="O1" status="OPEN">
  <total>50</total>
</shipment>
```

Wrapping an attributes Object in `@((...))` copies every attribute it contains onto the new element at once, instead of listing them one by one.

#848 Add an Attribute While Also Restructuring Children

Build a new element with one attribute and several restructured child elements.

INPUT

```
{ "id": "SHIP-1", "carrier": "fedex", "weight": 20 }
```

SOLUTION

```
output application/xml {writeDeclaration: false}
---
{ shipment @(id: payload.id): { carrier: payload.carrier, weight:
payload.weight } }
```

OUTPUT

```
<shipment id="SHIP-1">
  <carrier>fedex</carrier>
  <weight>20</weight>
</shipment>
```

Attributes and child elements are independent — nothing about adding `@(id: ...)` changes how the child fields underneath it are built.

#849 Attach an Attribute to Every Element in a Repeated Group

Give each repeated element in an array its own attribute, built from that array entry's own data.

INPUT

```
{ "items": [{"sku": "A1", "qty": 5}, {"sku": "A2", "qty": 3}] }
```

SOLUTION

```
output application/xml {writeDeclaration: false}
---
{ items: { (payload.items map ((i) -> { item @(sku: i.sku): { qty: i.qty }
})) } }
```

OUTPUT

```
<items>
  <item sku="A1">
    <qty>5</qty>
  </item>
  <item sku="A2">
    <qty>3</qty>
  </item>
</items>
```

Mapping to an Array of single-key `{ item @(...): {...} }` objects and spreading it with `{ (...) }` is how per-element attributes combine with repeated elements.

#850 CargoLink Capstone: Convert a JSON Order With Metadata Into Attributed XML

Convert a JSON order into XML, promoting two identifying fields to attributes and leaving the rest as child elements.

INPUT

```
{ "id": "O1", "priority": "HIGH", "carrier": "fedex", "total": 250 }
```

SOLUTION

```
output application/xml {writeDeclaration: false}
---
{ order @(id: payload.id, priority: payload.priority): { carrier:
payload.carrier, total: payload.total } }
```

OUTPUT

```
<order id="O1" priority="HIGH">
  <carrier>fedex</carrier>
  <total>250</total>
</order>
```

This closes out the section with the shape most real XML APIs actually use: a few identifying fields as attributes, everything else as nested child elements.

SECTION 6 — XML Namespaces (Problems 851–860)

#851 Declare and Use a Namespace Prefix

Declare a namespace and use it to build a namespaced XML element.

INPUT

```
{ "status": "IN_TRANSIT" }
```

SOLUTION

```
ns crg http://cargolink.com/ns
output application/xml {writeDeclaration: false}
---
{ crg#shipment: { crg#status: payload.status } }
```

OUTPUT

```
<crg:shipment xmlns:crg="http://cargolink.com/ns">
  <crg:status>IN_TRANSIT</crg:status>
</crg:shipment>
```

A `ns` directive in the header declares a prefix for a URI; `crg#shipment` then uses that prefix to build a namespaced element, and DataWeave adds the matching `xmlns:crg` declaration automatically.

#852 Read an Element From a Namespaced XML Document

Read a nested value out of an XML document that uses a namespace prefix.

INPUT

```
<crg:shipment xmlns:crg="http://cargolink.com/ns">
<crg:status>IN_TRANSIT</crg:status></crg:shipment>
```

SOLUTION

```
ns crg http://cargolink.com/ns
output application/json
---
payload.crg#shipment.crg#status
```

OUTPUT

```
"IN_TRANSIT"
```

The same `ns` directive used to write namespaced XML is also what lets you select into it with the matching `prefix#name` syntax when reading.

#853 Mix Namespaced and Non-Namespaced Elements

Build an XML document where only some child elements use a namespace prefix.

INPUT

```
{ "id": "SHIP-1", "status": "IN_TRANSIT" }
```

SOLUTION

```
ns crg http://cargolink.com/ns
output application/xml {writeDeclaration: false}
---
{ crg#shipment: { id: payload.id, crg#status: payload.status } }
```

OUTPUT

```
<crg:shipment xmlns:crg="http://cargolink.com/ns">
  <id>SHIP-1</id>
  <crg:status>IN_TRANSIT</crg:status>
</crg:shipment>
```

Namespaces apply per element, not to the whole document — `id` stays unprefixed while `crg#status` carries the `crg:` prefix.

#854 Use Two Different Namespaces in One Document

Build an XML document that mixes elements from two different namespaces.

INPUT

```
{ "origin": "Reno, NV", "destination": "Boise, ID" }
```

SOLUTION

```
ns crg http://cargolink.com/ns
ns geo http://cargolink.com/geo
output application/xml {writeDeclaration: false}
---
{ crg#shipment: { geo#origin: payload.origin, geo#destination:
payload.destination } }
```

OUTPUT

```
<crg:shipment xmlns:crg="http://cargolink.com/ns"
xmlns:geo="http://cargolink.com/geo">
  <geo:origin>Reno, NV</geo:origin>
  <geo:destination>Boise, ID</geo:destination>
</crg:shipment>
```

Multiple `ns` directives can coexist in the same header, and every prefix they declare gets its own `xmlns:` declaration on the root element.

#855 Extract Multiple Namespaced Fields Into a Flat JSON Object

Read two namespaced fields out of an XML document into a flat, prefix-free JSON object.

INPUT

```
<crg:shipment xmlns:crg="http://cargolink.com/ns"><crg:id>SHIP-1</crg:id>
<crg:status>IN_TRANSIT</crg:status></crg:shipment>
```

SOLUTION

```
ns crg http://cargolink.com/ns
output application/json
---
{ id: payload.crg#shipment.crg#id, status: payload.crg#shipment.crg#status }
```

OUTPUT

```
{ "id": "SHIP-1", "status": "IN_TRANSIT" }
```

Reading a namespaced field is no different from reading any other nested field — the resulting JSON is free to drop the namespace prefix entirely.

#856 Collect Every Repeated Element With the Multi-Value Selector

Safely collect every `shipment` element under a parent, regardless of how many there are, using the multi-value selector.

INPUT

```
<crg:shipments xmlns:crg="http://cargolink.com/ns"><crg:shipment><crg:id>SHIP-1</crg:id></crg:shipment><crg:shipment><crg:id>SHIP-2</crg:id></crg:shipment></crg:shipments>
```

SOLUTION

```
ns crg http://cargolink.com/ns
output application/json
---
payload.crg#shipments.*crg#shipment map ((s) -> s.crg#id)
```

OUTPUT

```
["SHIP-1", "SHIP-2"]
```

*DataWeave objects can hold more than one entry under the same key — a plain `.crg#shipment` selector would only return the **first** one. The multi-value selector, written with a `*` right before the key, collects every matching entry into one Array instead.*

#857 Convert a Namespaced XML Document Directly to JSON

Convert a namespaced XML document to JSON with no field selection at all.

INPUT

```
<crg:shipment xmlns:crg="http://cargolink.com/ns"><crg:id>SHIP-1</crg:id>
<crg:status>IN_TRANSIT</crg:status></crg:shipment>
```

SOLUTION

```
ns crg http://cargolink.com/ns
output application/json
---
payload
```

OUTPUT

```
{ "crg#shipment": { "crg#id": "SHIP-1", "crg#status": "IN_TRANSIT" } }
```

Since `prefix#name` is simply DataWeave's own internal representation of a namespaced key, it carries straight through into the JSON keys when nothing reshapes it away first.

#858 Strip Namespace Prefixes From Every Key Generically

Generically strip the namespace prefix off every field's key, without listing the fields one by one.

INPUT

```
<crg:shipment xmlns:crg="http://cargolink.com/ns"><crg:id>SHIP-1</crg:id>
<crg:status>IN_TRANSIT</crg:status></crg:shipment>
```

SOLUTION

```
ns crg http://cargolink.com/ns
output application/json
---
payload.crg#shipment mapObject ((value, key) -> { ((key as String) splitBy
"#")[1]: value })
```

OUTPUT

```
{ "id": "SHIP-1", "status": "IN_TRANSIT" }
```

Since a namespaced key's text form is literally `"prefix#localName"`, splitting each key on `"#"` and keeping the second piece strips every prefix in one generic pass.

#859 Add a Namespaced Field Alongside Existing Plain Fields

Merge a new namespaced field onto an otherwise plain, unprefixed XML element.

INPUT

```
{ "id": "SHIP-1", "carrier": "fedex" }
```

SOLUTION

```
ns crg http://cargolink.com/ns
output application/xml {writeDeclaration: false}
---
{ shipment: payload ++ { crg#trackingRef: payload.id } }
```

OUTPUT

```
<shipment xmlns:crg="http://cargolink.com/ns">
  <id>SHIP-1</id>
  <carrier>fedex</carrier>
  <crg:trackingRef>SHIP-1</crg:trackingRef>
</shipment>
```

The `++` merge operator works the same regardless of namespaces — it simply adds `crg#trackingRef` as one more field alongside the plain ones already there.

#860 CargoLink Capstone: Convert a Namespaced Partner Feed Into Clean Internal JSON

Convert a namespaced multi-shipment XML feed into a clean, prefix-free, type-coerced JSON array.

INPUT

```
<crg:shipments xmlns:crg="http://cargolink.com/ns"><crg:shipment><crg:id>SHIP-1</crg:id><crg:status>IN_TRANSIT</crg:status><crg:weight>40</crg:weight></crg:shipment><crg:shipment><crg:id>SHIP-2</crg:id><crg:status>DELIVERED</crg:status><crg:weight>22</crg:weight></crg:shipment></crg:shipments>
```

SOLUTION

```
ns crg http://cargolink.com/ns
output application/json
---
payload.crg#shipments.*crg#shipment map ((s) -> {
  id: s.crg#id,
  status: s.crg#status,
  weight: s.crg#weight as Number
})
```

OUTPUT

```
[{ "id": "SHIP-1", "status": "IN_TRANSIT", "weight": 40 }, { "id": "SHIP-2", "status": "DELIVERED", "weight": 22 }]
```

This closes out the section by combining the multi-value selector with field renaming and type coercion — the complete recipe for turning a namespaced partner feed into clean internal JSON.

SECTION 7 — CSV Basics (Problems 861–870)

#861 Convert a JSON Array of Objects Into CSV

Output a JSON array of orders directly as CSV.

INPUT

```
{ "orders": [{"id": "O1", "total": 50}, {"id": "O2", "total": 75}] }
```

SOLUTION

```
output application/csv
---
payload.orders
```

OUTPUT

```
id,total
O1,50
O2,75
```

The CSV writer takes each Object's keys as the header row and each Object's values as one data row, in the order the Objects appear.

#862 Turn Off the CSV Header Row

Produce CSV output with no header row, from an Array of Arrays.

INPUT

```
{ "orders": [{"id": "O1", "total": 50}, {"id": "O2", "total": 75}] }
```

SOLUTION

```
output application/csv {header: false}
---
payload.orders map ((o) -> [o.id, o.total])
```

OUTPUT

```
O1,50
O2,75
```

With no header row, each row is built as a plain Array in the exact column order you want, rather than relying on Object key order.

#863 Use a Custom Field Separator

Output CSV using a semicolon instead of a comma as the field separator.

INPUT

```
{ "orders": [{"id": "O1", "total": 50}] }
```

SOLUTION

```
output application/csv {separator: ";"}
---
payload.orders
```

OUTPUT

```
id;total
O1;50
```

Many real-world CSV consumers, especially in Europe, expect a semicolon separator — the `separator` writer property handles that without changing anything else.

#864 Quote Every Data Value in the Output

Quote every data value in the CSV output, while leaving the header row unquoted.

INPUT

```
{ "orders": [{"id": "O1", "total": 50}] }
```

SOLUTION

```
output application/csv {quoteValues: true}
---
payload.orders
```

OUTPUT

```
id,total
"O1","50"
```

`quoteValues` and `quoteHeader` are two separate properties — `quoteValues: true` alone only quotes the data rows, not the header.

#865 Quote the Header Row Too

Quote both the header row and every data value in the CSV output.

INPUT

```
{ "orders": [{"id": "O1", "total": 50}] }
```

SOLUTION

```
output application/csv {quoteValues: true, quoteHeader: true}
---
payload.orders
```

OUTPUT

```
"id","total"
"O1","50"
```

Adding `quoteHeader: true` extends the same quoting behavior to the header row as well.

#866 Read a CSV Payload and Convert It to JSON

Treat the whole incoming payload as CSV and convert it directly to JSON.

INPUT

```
id,total
O1,50
O2,75
```

SOLUTION

```
input payload application/csv {header: true}
output application/json
---
payload
```

OUTPUT

```
[{ "id": "O1", "total": "50" }, { "id": "O2", "total": "75" }]
```

Declaring `input payload application/csv` parses the whole incoming `payload` as CSV, the same way `read` parses a String — just applied to the script's main input instead of one embedded field.

#867 Filter Rows From a CSV Input

Filter out low-value rows from an incoming CSV payload.

INPUT

```
id,total
O1,50
O2,75
O3,90
```

SOLUTION

```
input payload application/csv {header: true}
output application/json
---
payload filter ((row) -> (row.total as Number) > 60)
```

OUTPUT

```
[{ "id": "O2", "total": "75" }, { "id": "O3", "total": "90" }]
```

Once `payload` is parsed as CSV, it behaves exactly like any other Array of Objects — `filter` works the same way it always has, as long as the comparison coerces `total` first.

#868 Reshape CSV Rows Into a Different JSON Structure

Reshape an incoming CSV payload into a summary JSON object.

INPUT

```
id,total
O1,50
O2,75
```

SOLUTION

```
input payload application/csv {header: true}
output application/json
---
{
  orderIds: payload map ((row) -> row.id),
  totalRevenue: sum(payload map ((row) -> row.total as Number))
}
```

OUTPUT

```
{ "orderIds": ["O1", "O2"], "totalRevenue": 125 }
```

CSV rows parsed with a header behave like any other Array of Objects — `map` and `sum` work on them exactly as they would on a JSON array.

#869 Convert Between CSV and XML via a Simple Reshape

Convert an incoming CSV payload into an XML document with repeated elements.

INPUT

```
id,total
O1,50
O2,75
```

SOLUTION

```
input payload application/csv {header: true}
output application/xml {writeDeclaration: false}
---
{ orders: { order: payload } }
```

OUTPUT

```
<orders>
  <order>
    <id>O1</id>
    <total>50</total>
  </order>
  <order>
    <id>O2</id>
    <total>75</total>
  </order>
</orders>
```

Once CSV is parsed into an Array of Objects, wrapping it into a repeated XML element works exactly like it did for a JSON array back in Section 4.

#870 CargoLink Capstone: Convert a CSV Manifest Into a Grouped JSON Summary

Convert a CSV shipment manifest into a JSON summary counting packages per carrier.

INPUT

```
id,carrier
PKG-1,fedex
PKG-2,ups
PKG-3,fedex
```

SOLUTION

```
input payload application/csv {header: true}
output application/json
---
(payload groupBy ((row) -> row.carrier)) mapObject ((rows, carrier) -> {
(carrier): sizeOf(rows) })
```

OUTPUT

```
{ "fedex": 2, "ups": 1 }
```

This closes out the section by combining a CSV input directive with `groupBy` and `mapObject` from Volume 4 and 5 — CSV rows really are just another Array of Objects once parsed.

SECTION 8 — Cross-Format Conversion (Problems 871–880)

#871 Convert a JSON Payload to XML

Convert a flat JSON order into an XML document.

INPUT

```
{ "id": "O1", "carrier": "fedex", "total": 50 }
```

SOLUTION

```
output application/xml {writeDeclaration: false}
---
{ order: payload }
```

OUTPUT

```
<order>
  <id>O1</id>
  <carrier>fedex</carrier>
  <total>50</total>
</order>
```

This section revisits each format pairing directly, side by side, as a quick reference for converting between any two of them.

#872 Convert a JSON Array to CSV

Convert a JSON array of orders into CSV.

INPUT

```
{ "orders": [{"id": "O1", "total": 50}, {"id": "O2", "total": 75}] }
```

SOLUTION

```
output application/csv
---
payload.orders
```

OUTPUT

```
id,total
O1,50
O2,75
```

JSON to CSV needs no reshaping at all when the source is already a flat Array of Objects.

#873 Convert an XML Document to JSON

Convert an XML order document into JSON.

INPUT

```
<order><id>O1</id><total>50</total></order>
```

SOLUTION

```
output application/json
---
payload
```

OUTPUT

```
{ "order": { "id": "O1", "total": "50" } }
```

XML to JSON is likewise a direct pass-through — the reshaping work only shows up once fields need to be renamed, flattened, or coerced.

#874 Convert an XML Document to CSV

Convert a multi-order XML document into CSV, collecting every repeated element first.

INPUT

```
<orders><order><id>O1</id><total>50</total></order><order><id>O2</id>
<total>75</total></order></orders>
```

SOLUTION

```
output application/csv
---
payload.orders.*order
```

OUTPUT

```
id,total
O1,50
O2,75
```

The multi-value selector, `.*order`, collects the repeated `<order>` elements into an Array first, which the CSV writer can then turn into rows.

#875 Convert a CSV Document to XML

Convert a CSV payload into an XML document with repeated elements.

INPUT

```
id,total
O1,50
O2,75
```

SOLUTION

```
input payload application/csv {header: true}
output application/xml {writeDeclaration: false}
---
{ orders: { order: payload } }
```

OUTPUT

```
<orders>
  <order>
    <id>O1</id>
    <total>50</total>
  </order>
  <order>
    <id>O2</id>
    <total>75</total>
  </order>
</orders>
```

CSV to XML is a two-step trip in one script: parse the CSV via the `input` directive, then wrap the resulting Array the same way Section 4 wrapped a JSON array.

#876 Round-Trip JSON Through XML and Back

Write a JSON value out as XML, then immediately read it back, to see what survives the trip.

INPUT

```
{ "id": "O1", "total": 50 }
```

SOLUTION

```
output application/json
---
read(write({ order: payload }, "application/xml"), "application/xml")
```

OUTPUT

```
{ "order": { "id": "O1", "total": "50" } }
```

Notice `total` comes back as the String `"50"`, not the original Number `50` — the round trip through XML's text-only model loses the original type, a common cross-format gotcha.

#877 Round-Trip JSON Through CSV and Back

Write a JSON array out as CSV, then read it back, to see the same type-loss gotcha in CSV.

INPUT

```
{ "orders": [{"id": "O1", "total": 50}, {"id": "O2", "total": 75}] }
```

SOLUTION

```
output application/json
---
read(write(payload.orders, "application/csv"), "application/csv", {header:
true})
```

OUTPUT

```
[{ "id": "O1", "total": "50" }, { "id": "O2", "total": "75" }]
```

CSV has no type system at all, so every value returns as a String after the round trip — the same lesson as #876, just through a different format.

#878 Choose the Output Format Based on a Condition

Pick which format to export data in, based on a flag in the payload.

INPUT

```
{ "orders": [{"id": "O1", "total": 50}], "wantsCsv": true }
```

SOLUTION

```
fun exportAs(data, format: String) = write(data, format)
---
{ export: exportAs(payload.orders, if (payload.wantsCsv) "application/csv"
else "application/json") }
```

OUTPUT

```
{ "export": "id,total\nO1,50\n" }
```

Because `write`'s MIME type is just a String argument, it can be computed at runtime — including from an `if/else` — rather than fixed in advance.

#879 Merge Data From Two Different Source Formats Into One Output

Combine an attribute from an embedded XML String with rows from an embedded CSV String into one JSON result.

INPUT

```
{ "orderXml": "<order id=\"O1\"><status>OPEN</status></order>", "pricingCsv":
"item,total\nA,10\nB,20\n" }
```

SOLUTION

```
output application/json
---
{
  orderId: (read(payload.orderXml, "application/xml")).order.@id,
  totals: read(payload.pricingCsv, "application/csv", {header: true}) map
((row) -> row.total as Number)
}
```

OUTPUT

```
{ "orderId": "O1", "totals": [10, 20] }
```

`read` makes it easy to merge data from several source formats into one script, since each call just returns an ordinary DataWeave value regardless of what format it started as.

#880 CargoLink Capstone: Full-Circle Format Conversion Report

Parse an embedded CSV manifest and report both its row count and an XML re-serialization of it.

INPUT

```
{ "manifestCsv": "id,weight\nPKG-1,30\nPKG-2,45\n" }
```

SOLUTION

```
var rows = read(payload.manifestCsv, "application/csv", {header: true})
---
{
  sourceFormat: "csv",
  parsedRows: sizeOf(rows),
  asXml: write({ manifest: { row: rows } }, "application/xml",
{writeDeclaration: false, indent: false})
}
```

OUTPUT

```
{ "sourceFormat": "csv", "parsedRows": 2, "asXml": "<manifest><row><id>PKG-
1</id><weight>30</weight></row><row><id>PKG-2</id><weight>45</weight></row>
</manifest>" }
```

This closes out the section with a single manifest passing through three formats in one script: CSV in, an internal DataWeave value in the middle, and both a JSON field and an embedded XML String out.

SECTION 9 — Format-Specific Gotchas & Edge Cases (Problems 881–890)

#881 XML Elements With No Children Parse as Empty Strings

Confirm that an empty XML element reads as an empty String, not null.

INPUT

```
<order><note></note></order>
```

SOLUTION

```
output application/json
---
{ note: payload.order.note, isEmpty: isEmpty(payload.order.note) }
```

OUTPUT

```
{ "note": "", "isEmpty": true }
```

An element that exists but has no text content parses as an empty String, `""` — it's present, just empty.

#882 A Missing XML Element Is Genuinely Null

Confirm that an XML element that doesn't exist at all reads as null, not an empty String.

INPUT

```
<order><id>O1</id></order>
```

SOLUTION

```
output application/json
---
{ note: payload.order.note, noteIsNull: payload.order.note == null }
```

OUTPUT

```
{ "note": null, "noteIsNull": true }
```

This is the opposite case from #881 — a genuinely absent element and a present-but-empty one look identical in the source XML's absence, but behave differently once parsed.

#883 CSV Fields Are Always Strings, Even When They Look Numeric

Confirm that a numeric-looking CSV field parses as a String, not a Number.

INPUT

```
id,total
O1,50
```

SOLUTION

```
input payload application/csv {header: true}
output application/json
---
{ rawTotal: payload[0].total, isString: payload[0].total is String, coerced:
payload[0].total as Number }
```

OUTPUT

```
{ "rawTotal": "50", "isString": true, "coerced": 50 }
```

Just like XML, CSV has no type system of its own — every field is a String until something in the script says otherwise.

#884 An Empty CSV Value Reads as an Empty String

Confirm that a CSV field left blank in a row reads as an empty String, not null.

INPUT

```
id,note
O1,
```

SOLUTION

```
input payload application/csv {header: true}
output application/json
---
{ note: payload[0].note, isEmptyString: payload[0].note == "" }
```

OUTPUT

```
{ "note": "", "isEmptyString": true }
```

A blank field between two separators is still a present, empty String — the CSV equivalent of #881's empty XML element.

#885 A Missing Field and an Explicit null Look the Same in JSON

Compare a field that was never present against a field explicitly set to null.

INPUT

```
{ "explicitNull": null }
```

SOLUTION

```
output application/json
---
{ a: payload.missingField == null, b: payload.explicitNull == null }
```

OUTPUT

```
{ "a": true, "b": true }
```

DataWeave doesn't distinguish between "the key was never present" and "the key was present with a null value" — both read back as null through a selector, unlike the XML case in #881 and #882.

#886 Writing null to CSV Produces an Empty Field

See what happens when a null value is written out as CSV.

INPUT

```
{ "orders": [{"id": "O1", "note": null}] }
```

SOLUTION

```
output application/csv
---
payload.orders
```

OUTPUT

```
id,note
O1,
```

A null value writes out as nothing between the separators — the field is still there, just empty, mirroring how it would read back in.

#887 Writing null to XML by Default

See what happens when a null value is written out as XML with default writer settings.

INPUT

```
{ "id": "O1", "note": null }
```

SOLUTION

```
output application/xml {writeDeclaration: false}
---
{ order: { id: payload.id, note: payload.note } }
```

OUTPUT

```
<order>
  <id>O1</id>
  <note></note>
</order>
```

By default, a null-valued field still produces an empty element, `<note></note>`, rather than being skipped.

#888 Skip Null Fields Entirely in XML Output

Configure the XML writer to omit null-valued elements entirely instead of writing them empty.

INPUT

```
{ "id": "O1", "note": null }
```

SOLUTION

```
output application/xml {writeDeclaration: false, skipNullOn: "everywhere"}
---
{ order: { id: payload.id, note: payload.note } }
```

OUTPUT

```
<order>
  <id>O1</id>
</order>
```

The `skipNullOn` writer property changes #887's default behavior, dropping any element (or attribute) whose value is null instead of writing it out empty.

#889 Booleans Look Different Across Formats

Compare how a Boolean value renders in CSV and in XML.

INPUT

```
{ "flag": true }
```

SOLUTION

```
output application/json
---
{
  asCsv: write([{flag: payload.flag}], "application/csv"),
  asXml: write({root: {flag: payload.flag}}, "application/xml",
{writeDeclaration: false, indent: false})
}
```

OUTPUT

```
{ "asCsv": "flag\ntrue\n", "asXml": "<root><flag>true</flag></root>" }
```

Both CSV and XML render every value, Booleans included, as plain text — `true` the word, not a native Boolean — since neither format has a Boolean type of its own.

#890 CargoLink Capstone: A Defensive Conversion That Handles Missing and Null Fields

Build a conversion that safely defaults every field, regardless of whether it was missing, null, or present.

INPUT

```
{ "id": "O1" }
```

SOLUTION

```
output application/json
---
{
  id: payload.id default "UNKNOWN",
  note: (payload.note default "") as String,
  weight: (payload.weight default 0) as Number
}
```

OUTPUT

```
{ "id": "O1", "note": "", "weight": 0 }
```

This closes out the section by combining every gotcha covered so far into one defensive pattern: default first, then coerce, so a missing field never breaks the conversion.

SECTION 10 — Mixed Real-World Format Conversion Challenges (Problems 891–900)

#891 Convert an Incoming XML Order Into a Simplified JSON API Response

Convert an XML order with an attribute into a simplified, type-coerced JSON response.

INPUT

```
<order id="O1"><carrier>fedex</carrier><total>249.50</total></order>
```

SOLUTION

```
output application/json
---
{
  orderId: payload.order.@id,
  carrier: payload.order.carrier,
  total: payload.order.total as Number
}
```

OUTPUT

```
{ "orderId": "O1", "carrier": "fedex", "total": 249.5 }
```

A realistic API-facing conversion almost always combines an attribute read, a plain child read, and a type coercion in one small object.

#892 Convert a JSON API Response Into a Legacy XML Format for a Partner

Convert a JSON order into an XML shape matching a partner's legacy attribute-and-capitalization conventions.

INPUT

```
{ "orderId": "O1", "carrier": "fedex", "total": 249.5 }
```

SOLUTION

```
output application/xml {writeDeclaration: false}
---
{ Order @(OrderID: payload.orderId): { Carrier: payload.carrier, Total:
payload.total } }
```

OUTPUT

```
<Order OrderID="O1">
  <Carrier>fedex</Carrier>
  <Total>249.5</Total>
</Order>
```

Element and attribute names in DataWeave's output are written exactly as given — matching a partner's exact capitalization is simply a matter of writing the keys that way.

#893 Batch-Convert an Array of Orders From JSON to a CSV Export File

Convert a JSON array of orders into a CSV export, dropping an internal-only field.

INPUT

```
{ "orders": [{"id": "O1", "carrier": "fedex", "total": 50, "internal": "x"},
{"id": "O2", "carrier": "ups", "total": 75, "internal": "y"}] }
```

SOLUTION

```
output application/csv
---
payload.orders map ((o) -> { id: o.id, carrier: o.carrier, total: o.total })
```

OUTPUT

```
id,carrier,total
O1,fedex,50
O2,ups,75
```

Selecting only the fields that belong in the export, before the CSV writer ever sees the data, is the simplest way to keep internal-only fields out of a file that leaves the system.

#894 Parse an Uploaded CSV File and Validate Every Row

Parse an uploaded CSV file and count how many rows pass a basic validity check.

INPUT

```
id,total
O1,50
O2,-10
O3,20
```

SOLUTION

```
input payload application/csv {header: true}
output application/json
---
{
  validRows: sizeOf(payload filter ((row) -> (row.total as Number) > 0)),
  invalidRows: sizeOf(payload filter ((row) -> (row.total as Number) <= 0))
}
```

OUTPUT

```
{ "validRows": 2, "invalidRows": 1 }
```

Validating an uploaded file is often the very first thing a real integration does with it, right after parsing and before anything else happens.

#895 Convert a Nested JSON Structure Into a Flat CSV-Ready Array

Flatten a nested JSON structure into a flat shape that the CSV writer can serialize.

INPUT

```
{ "orders": [{"id": "O1", "shipTo": {"city": "Reno"}, "total": 50}, {"id":
"O2", "shipTo": {"city": "Boise"}, "total": 75}] }
```

SOLUTION

```
output application/csv
---
payload.orders map ((o) -> { id: o.id, city: o.shipTo.city, total: o.total })
```

OUTPUT

```
id,city,total
O1,Reno,50
O2,Boise,75
```

CSV can only represent flat rows, so a nested field like `shipTo.city` has to be pulled up to the top level before the CSV writer ever sees it.

#896 Convert a Flat CSV Into a Nested JSON Structure

Convert flat CSV rows into a nested JSON structure, the reverse of #895.

INPUT

```
id,city
O1,Reno
O2,Boise
```

SOLUTION

```
input payload application/csv {header: true}
output application/json
---
payload map ((row) -> { id: row.id, shipTo: { city: row.city } })
```

OUTPUT

```
[{ "id": "O1", "shipTo": { "city": "Reno" } }, { "id": "O2", "shipTo": {
"city": "Boise" } }]
```

Going the other direction, a flat field like `city` gets wrapped into whatever nested shape the destination system expects.

#897 Handle an XML Feed Where an Element May or May Not Repeat

Safely read an XML element that might appear once or several times, always as an Array.

INPUT

```
<order><items><item>PKG-1</item></items></order>
```

SOLUTION

```
output application/json
---
payload.order.items.*item
```

OUTPUT

```
["PKG-1"]
```

The multi-value selector from Section 6 always returns an Array, even when there's only one matching element — exactly the defensive pattern you want when a feed's cardinality isn't guaranteed.

#898 Handle a Value Whose Type Might Vary by Source Format

Use `match` to handle a field that might already be numeric, or might still be a String from a text-based format.

INPUT

```
{ "value": "50" }
```

SOLUTION

```
output application/json
---
payload.value match {
  case n is Number -> "already numeric: " ++ (n as String)
  case s is String -> "needs coercion: " ++ s
  else -> "unexpected type"
}
```

OUTPUT

```
"needs coercion: 50"
```

A field's type can depend entirely on which format it arrived through — a `match` guard like this handles either case safely, from Volume 8's type-matching patterns.

#899 Build a Format-Agnostic Coercion Function

Define a reusable helper that safely coerces a value to a Number, whatever format it arrived from.

INPUT

```
{ "total": "125.50" }
```

SOLUTION

```
fun asNumberSafe(v) = v match {
  case n is Number -> n
  case s is String -> s as Number
  else -> 0
}
---
{ total: asNumberSafe(payload.total) }
```

OUTPUT

```
{ "total": 125.5 }
```

Wrapping the type check in a reusable `fun` means every field that needs this safe coercion can call one helper instead of repeating the same `match` everywhere.

#900 Series Capstone: A Complete Multi-Format Order Intake Pipeline

Build a complete intake transformation for an XML order feed: an attribute id, a safely-coerced total, and a defensively-collected item count.

INPUT

```
<order id="O1"><carrier>fedex</carrier><total>249.50</total><items><item>PKG-1</item><item>PKG-2</item></items></order>
```

SOLUTION

```
fun asNumberSafe(v) = v match {
  case n is Number -> n
  case s is String -> s as Number
  else -> 0
}
---
{
  orderId: payload.order.@id,
  carrier: payload.order.carrier,
  total: asNumberSafe(payload.order.total),
  itemCount: sizeOf(payload.order.items.*item)
}
```

OUTPUT

```
{ "orderId": "O1", "carrier": "fedex", "total": 249.5, "itemCount": 2 }
```

The series' 900th problem draws on all nine volumes at once: an XML attribute selector, a `match`-based coercion function, and the multi-value selector — every one of them safe against the small surprises real-world data brings across a format boundary.

END OF VOLUME 9

900 Down, 100 to Go

You've now worked through selectors, type coercion, operators and conditionals, array and object transformations, string handling, dates and times, custom functions and recursion, and a full tour of reading and writing JSON, XML, and CSV.

Volume 10, the final volume, moves to Real-World Integration & Error Handling: the last 100 problems on `try`, error handling, validation, and the end-to-end patterns that tie a MuleSoft integration together.

MULESOFT · PRACTICE PROBLEMS · WITH FULL SOLUTIONS

1000 DataWeave Problems With Solutions

Volume 10 — Real-World Integration & Error Handling. The final 100 problems on validation, `try`/`fail`, retries, pagination, and end-to-end integration patterns.

Problems 901–1000

Volume 10 of 10

2026 EDITION

How to Use This Volume

Volume 10 closes out the series with the concerns that show up once a transformation has to run in a real, unreliable system: validating input you don't fully trust, handling errors instead of letting them crash a flow, and the recurring patterns — retries, pagination, batching, idempotency — that hold a real integration together. Same format as every prior volume:

- **A short problem statement**, the **input** payload, a complete **solution** in a black code window, the exact **output**, and a one-line explanation.
- Ten sections of ten problems each, moving from defensive validation through the `try` and `fail` functions, safe fallback chains, idempotency and retry patterns, pagination and batching, schema validation, and mixed real-world capstones — ending with the series' 1000th problem.

A note on `try` and `fail`: both live in the `dw::Runtime` module and need `import * from dw::Runtime` in the header. `try(() -> expression)` runs an expression that might throw, and returns `{success: true, result: ...}` on success or `{success: false, error: {message: ...}}` on failure. This volume only ever relies on those specific fields — `success`, `result`, and `error.message` — and every `error.message` shown is one this book's own `fail(...)` calls set explicitly, never a guessed-at internal engine message.

VOLUME 10

Real-World Integration & Error Handling

The final 100 problems: defensive validation, the `try` and `fail` functions, safe fallback chains, idempotency and correlation, retries and backoff, pagination and batching, schema validation, and mixed capstones that draw on everything the series has covered.

SECTION 1 — Defensive Defaults & Validation Basics (Problems 901–910)

#901 Default a Missing Field to a Safe Value

Default a missing numeric field to zero instead of letting it stay null.

INPUT

```
{}
```

SOLUTION

```
output application/json
---
{ total: payload.total default 0 }
```

OUTPUT

```
{ "total": 0 }
```

This volume starts where every real integration should: assuming the input can't be fully trusted, and defending against it field by field.

#902 Validate a Required Field Is Present

Write a reusable function that checks a required field is present, returning a clear error if not.

INPUT

```
{ "orderId": null }
```

SOLUTION

```
fun requireField(v, name: String) = if (v == null) { valid: false, error:
name ++ " is required" } else { valid: true, value: v }
---
requireField(payload.orderId, "orderId")
```

OUTPUT

```
{ "valid": false, "error": "orderId is required" }
```

Returning an Object that describes the failure, rather than just a Boolean, gives a caller enough information to act on the result.

#903 Validate a Field's Type

Write a reusable function that checks whether a value is genuinely a Number.

INPUT

```
{ "total": "50" }
```

SOLUTION

```
fun requireNumber(v) = if (v is Number) { valid: true } else { valid: false,
error: "must be a Number" }
---
requireNumber(payload.total)
```

OUTPUT

```
{ "valid": false, "error": "must be a Number" }
```

A numeric-looking String, like `"50"`, still fails an `is Number` check — the two are different types even when they look the same.

#904 Validate a Numeric Range

Write a reusable function that checks a Number falls within a minimum and maximum.

INPUT

```
{ "quantity": 150 }
```

SOLUTION

```
fun requireRange(n: Number, min: Number, max: Number) =
  if (n >= min and n <= max) { valid: true }
  else { valid: false, error: "must be between " ++ (min as String) ++ " and
" ++ (max as String) }
---
requireRange(payload.quantity, 1, 100)
```

OUTPUT

```
{ "valid": false, "error": "must be between 1 and 100" }
```

Building the error message from the same `min` and `max` parameters used for the check keeps the message and the logic from ever drifting apart.

#905 Validate a String Against an Allowed Set

Write a reusable function that checks a String against a list of allowed values.

INPUT

```
{ "status": "CANCELLED" }
```

SOLUTION

```
fun requireOneOf(v: String, allowed: Array) =
  if (allowed contains v) { valid: true } else { valid: false, error: v ++ "
is not a recognized value" }
---
requireOneOf(payload.status, ["OPEN", "CLOSED", "PENDING"])
```

OUTPUT

```
{ "valid": false, "error": "CANCELLED is not a recognized value" }
```

Passing the allowed list as a parameter, rather than hardcoding it, means the same function works for any enum-style field.

#906 Validate a String's Format With matches

Write a reusable function that checks a String matches an expected pattern.

INPUT

```
{ "trackingId": "CRG-4471" }
```

SOLUTION

```
fun requireFormat(v: String) = if (v matches /[A-Z]{3}-\d{4}/) { valid: true
} else { valid: false, error: "does not match expected format" }
---
requireFormat(payload.trackingId)
```

OUTPUT

```
{ "valid": true }
```

Format validation with `matches` is especially useful for identifiers like tracking numbers, where "wrong shape" is a common source of downstream failures.

#907 Combine Two Validations With and

Combine two separate conditions into one validity check.

INPUT

```
{ "id": "O1", "total": -5 }
```

SOLUTION

```
fun validateOrder(o) = if (o.total > 0 and o.id != null) { valid: true } else
{ valid: false, error: "invalid order" }
---
validateOrder(payload)
```

OUTPUT

```
{ "valid": false, "error": "invalid order" }
```

A single combined `and` condition is fine when the caller only needs to know "valid or not" — Section 2 covers building a report of exactly which conditions failed.

#908 Provide a Fallback Chain of Defaults

Fall back from a primary field to a secondary field to a hardcoded default, in order.

INPUT

```
{ "defaultCarrier": "fedex" }
```

SOLUTION

```
output application/json
---
{ carrier: payload.carrier default (payload.defaultCarrier default
"UNASSIGNED") }
```

OUTPUT

```
{ "carrier": "fedex" }
```

Nesting `default` calls builds a fallback chain — each one only runs if everything before it turned out to be null.

#909 Validate That an Array Is Not Empty

Write a reusable function that rejects an empty Array where at least one item is required.

INPUT

```
{ "items": [] }
```

SOLUTION

```
fun requireNonEmpty(arr: Array) = if (isEmpty(arr)) { valid: false, error:
"array must not be empty" } else { valid: true }
---
requireNonEmpty(payload.items)
```

OUTPUT

```
{ "valid": false, "error": "array must not be empty" }
```

An empty Array is a common, easy-to-miss edge case — "at least one item" is a validation rule worth writing down explicitly.

#910 CargoLink Capstone: Validate a Complete Order Before Processing

Chain three separate validation checks into one function, stopping at the first failure.

INPUT

```
{ "id": "O1", "total": -10 }
```

SOLUTION

```
fun validate(o) =
  if (o.id == null) { valid: false, error: "id is required" }
  else if (not (o.total is Number)) { valid: false, error: "total must be a
Number" }
  else if (o.total <= 0) { valid: false, error: "total must be positive" }
  else { valid: true }
---
validate(payload)
```

OUTPUT

```
{ "valid": false, "error": "total must be positive" }
```

This closes out the section with the pattern the rest of the volume builds on: a chain of `if/else` checks, each one only reached if everything before it passed.

SECTION 2 — Building Structured Validation Results (Problems 911–920)

#911 Collect Multiple Field Errors Into One Array

Check two separate fields and collect every failure into one Array, instead of stopping at the first.

INPUT

```
{ "id": null, "total": null }
```

SOLUTION

```
fun validate(o) =
  (if (o.id == null) ["id is required"] else []) ++
  (if (o.total == null) ["total is required"] else [])
---
validate(payload)
```

OUTPUT

```
["id is required", "total is required"]
```

Each check contributes either an empty Array or a one-element Array, and ++ merges every contribution into one combined error list.

#912 Turn an Error Array Into a Validation Report Object

Wrap a list of collected errors into a report Object with an overall validity flag.

INPUT

```
{ "id": "O1", "total": null }
```

SOLUTION

```
fun validate(o) =
  (if (o.id == null) ["id is required"] else []) ++
  (if (o.total == null) ["total is required"] else [])
---
{ isValid: isEmpty(validate(payload)), errors: validate(payload) }
```

OUTPUT

```
{ "isValid": false, "errors": ["total is required"] }
```

Deriving `isValid` from whether the error list is empty keeps the two fields from ever disagreeing with each other.

#913 Validate Every Item in an Array and Collect Failures

Check every item in an Array against a rule, collecting one message per failing item.

INPUT

```
{ "items": [{"qty": 5}, {"qty": -1}, {"qty": 2}] }
```

SOLUTION

```
fun validateItem(i, idx: Number) = if (i.qty > 0) null else "item at index "
++ (idx as String) ++ " has invalid quantity"
---
(payload.items map ((i, idx) -> validateItem(i, idx))) filter ((e) -> e !=
null)
```

OUTPUT

```
["item at index 1 has invalid quantity"]
```

Mapping to either a message or `null`, then filtering out the nulls, is a compact way to collect only the failures out of a whole Array.

#914 Split an Array Into Valid and Invalid Rows

Write a reusable function that partitions an Array into rows that pass a rule and rows that don't.

INPUT

```
{ "orders": [{"id": "O1", "total": 50}, {"id": "O2", "total": -5}] }
```

SOLUTION

```
fun splitValid(rows: Array, rule) = { valid: rows filter ((r) -> rule(r)),
invalid: rows filter ((r) -> not rule(r)) }
---
splitValid(payload.orders, (o) -> o.total > 0)
```

OUTPUT

```
{ "valid": [{ "id": "O1", "total": 50 }], "invalid": [{ "id": "O2", "total":
-5 }] }
```

Taking the rule as a parameter, rather than hardcoding it, means `splitValid` works for any Array and any pass/fail condition.

#915 Attach a Field Name to Every Collected Error

Build a reusable field-checker that reports which field failed, not just that something did.

INPUT

```
{ "id": "O1", "total": "50" }
```

SOLUTION

```
fun checkField(v, name: String, rule) = if (rule(v)) null else { field: name,
message: name ++ " failed validation" }
---
[checkField(payload.id, "id", (v) -> v != null), checkField(payload.total,
"total", (v) -> v is Number)] filter ((e) -> e != null)
```

OUTPUT

```
[{ "field": "total", "message": "total failed validation" }]
```

Including the field name in every error Object is what lets a caller — or a UI — point a user at exactly the input that needs fixing.

#916 Count Errors by Severity

Group a list of issues by severity level and count each group.

INPUT

```
{ "issues": [{"level": "ERROR"}, {"level": "WARNING"}, {"level": "ERROR"}] }
```

SOLUTION

```
output application/json
---
(payload.issues groupBy ((i) -> i.level)) mapObject ((rows, level) -> {
(level): sizeOf(rows) })
```

OUTPUT

```
{ "ERROR": 2, "WARNING": 1 }
```

This is the same `groupBy`-then-`mapObject` counting pattern from earlier volumes, applied to validation issues instead of business data.

#917 Build a Pass/Fail Summary Across a Batch

Summarize how many items in a batch passed and how many failed a validity check.

INPUT

```
{ "orders": [{"total": 50}, {"total": -5}, {"total": 20}] }
```

SOLUTION

```
output application/json
---
{ passed: sizeOf(payload.orders filter ((o) -> o.total > 0)), failed:
sizeOf(payload.orders filter ((o) -> o.total <= 0)) }
```

OUTPUT

```
{ "passed": 2, "failed": 1 }
```

A batch summary like this is usually the very first thing worth logging when a validation pass finishes.

#918 Short-Circuit Validation: Stop at the First Error

Recursively walk a list of named checks and return the message of the first one that fails.

INPUT

```
{ "id": "O1", "total": -5 }
```

SOLUTION

```
fun firstError(rules: Array): String = rules match {
  case [head ~ tail] -> if (head.check) firstError(tail) else head.message
  case [] -> "no errors"
}
---
firstError([{check: payload.id != null, message: "id is required"}, {check:
payload.total > 0, message: "total must be positive"}])
```

OUTPUT

```
"total must be positive"
```

Modeling each rule as `{check, message}` lets a single recursive function walk any list of rules, stopping as soon as one fails.

#919 Separate Warnings From Errors

Build a validation result that distinguishes hard errors from softer warnings.

INPUT

```
{ "total": 50, "weight": 0 }
```

SOLUTION

```
output application/json
---
{
  errors: if (payload.total <= 0) ["total must be positive"] else [],
  warnings: if (payload.weight == 0) ["weight is zero — please confirm"] else
[]
}
```

OUTPUT

```
{ "errors": [], "warnings": ["weight is zero — please confirm"] }
```

Not every issue should block processing — separating `errors` from `warnings` lets a caller decide what's actually blocking.

#920 CargoLink Capstone: A Complete Multi-Field Validation Report

Check three fields on an order and build one combined validation report.

INPUT

```
{ "id": "O1", "total": -5, "carrier": null }
```

SOLUTION

```
fun checkField(v, name: String, rule) = if (rule(v)) null else name ++ " is
invalid"
fun allErrors(o) = [
  checkField(o.id, "id", (v) -> v != null),
  checkField(o.total, "total", (v) -> v is Number and v > 0),
  checkField(o.carrier, "carrier", (v) -> v != null)
] filter ((e) -> e != null)
---
{ isValid: isEmpty(allErrors(payload)), errors: allErrors(payload) }
```

OUTPUT

```
{ "isValid": false, "errors": ["total is invalid", "carrier is invalid"] }
```

This closes out the section by combining every idea so far — per-field checks, error collection, and a summary flag — into one report worth returning from a real validation step.

SECTION 3 — The try Function (Problems 921–930)

#921 Catch a Runtime Error Safely

Use `try` to check whether a coercion succeeds, without letting a bad value crash the script.

INPUT

```
{ "total": "not-a-number" }
```

SOLUTION

```
import * from dw::Runtime
output application/json
---
(try(() -> payload.total as Number)).success
```

OUTPUT

```
false
```

`try` takes a zero-argument function, `() -> ...`, so the risky expression only runs inside `try`'s own controlled evaluation, instead of before `try` is even called.

#922 Show Both success and result on a Successful Call

See the full shape `try` returns when the wrapped expression succeeds.

INPUT

```
{ "total": "50" }
```

SOLUTION

```
import * from dw::Runtime
output application/json
---
try(() -> payload.total as Number)
```

OUTPUT

```
{ "success": true, "result": 50 }
```

On success, `try` returns `success: true` together with the wrapped expression's actual return value under `result`.

#923 Use try's success Field to Choose a Fallback

Use a `try` result's `success` field to decide between the parsed value and a fallback.

INPUT

```
{ "total": "abc" }
```

SOLUTION

```
import * from dw::Runtime
var attempt = try(() -> payload.total as Number)
---
{ total: if (attempt.success) attempt.result else 0 }
```

OUTPUT

```
{ "total": 0 }
```

Storing the `try` result in a `var` first avoids calling the risky expression twice — once for the check, once for the value.

#924 Catch a Division-by-Zero Error

Use `try` to safely detect a division by zero.

INPUT

```
{ "divisor": 0 }
```

SOLUTION

```
import * from dw::Runtime
output application/json
---
(try(() -> 100 / payload.divisor)).success
```

OUTPUT

```
false
```

Dividing by zero is a runtime error in DataWeave, not a special "infinity" value — exactly the kind of failure `try` exists to catch.

#925 Catch an Invalid Date Coercion

Use `try` to safely detect a String that can't be parsed as a Date.

INPUT

```
{ "dateStr": "not-a-date" }
```

SOLUTION

```
import * from dw::Runtime
output application/json
---
(try(() -> payload.dateStr as Date)).success
```

OUTPUT

```
false
```

Just like a bad Number coercion, an unparseable Date String throws — `try` handles either one the exact same way.

#926 Use try Together With a Custom fail Message

Catch an error deliberately raised by a validation function.

INPUT

```
{ "amount": -5 }
```

SOLUTION

```
import * from dw::Runtime
fun validateAmount(amt: Number) = if (amt > 0) amt else fail("Amount must be
positive")
---
try(() -> validateAmount(payload.amount))
```

OUTPUT

```
{ "success": false, "error": { "message": "Amount must be positive" } }
```

When the wrapped expression fails, `try` returns `success: false` and an `error` Object holding the failure's `message` — here, exactly the String passed to `fail`.

#927 Extract Just the Error Message From a Failed try

Select just the error message out of a failed `try` result.

INPUT

```
{ "amount": -5 }
```

SOLUTION

```
import * from dw::Runtime
fun validateAmount(amt: Number) = if (amt > 0) amt else fail("Amount must be
positive")
---
(try(() -> validateAmount(payload.amount))).error.message
```

OUTPUT

```
"Amount must be positive"
```

`error.message` chains onto `try`'s result exactly like any other nested selector.

#928 Build a Safe Coercion Helper Using try

Write a reusable helper that safely coerces a value to a Number, defaulting to zero on failure.

INPUT

```
{ "total": "abc" }
```

SOLUTION

```
import * from dw::Runtime
fun safeToNumber(v): Number = (try(() -> v as Number)).result default 0
---
safeToNumber(payload.total)
```

OUTPUT

```
0
```

On failure, `try`'s result has no `result` field at all, so selecting `.result` returns null — which `default 0` then catches, exactly like any other missing field.

#929 Chain Multiple try Calls to Attempt Several Parsing Strategies

Try one date format, and fall back to a second format if the first one fails.

INPUT

```
{ "dateStr": "03/14/2026" }
```

SOLUTION

```
import * from dw::Runtime
fun attemptFormat(s: String, fmt: String) = try(() -> s as Date {format:
fmt})
fun parseFlexibleDate(s: String) =
  if ((attemptFormat(s, "yyyy-MM-dd")).success) (attemptFormat(s, "yyyy-MM-
dd")).result
  else (attemptFormat(s, "MM/dd/yyyy")).result
---
parseFlexibleDate(payload.dateStr)
```

OUTPUT

```
|2026-03-14|
```

Since the source date doesn't match ISO format, the first attempt fails and the function falls through to the second format, which parses it correctly.

#930 CargoLink Capstone: Safely Process a Batch With Mixed Valid and Invalid Rows

Parse a numeric field on every row in a batch, marking each row as parsed successfully or not.

INPUT

```
{ "rows": [{"id": "O1", "total": "50"}, {"id": "O2", "total": "bad"}] }
```

SOLUTION

```
import * from dw::Runtime
output application/json
---
payload.rows map ((r) -> {
  id: r.id,
  parsed: (try(() -> r.total as Number)).result default null,
  ok: (try(() -> r.total as Number)).success
})
```

OUTPUT

```
[{ "id": "O1", "parsed": 50, "ok": true }, { "id": "O2", "parsed": null, "ok": false }]
```

This closes out the section with the shape most real batch processing takes: run the risky step once per row, and let each row succeed or fail independently of the others.

SECTION 4 — Raising Custom Errors With fail (Problems 931–940)

#931 Raise a Custom Error for Invalid Input

Write a function that raises a custom error instead of returning quietly on invalid input.

INPUT

```
{ "n": -5 }
```

SOLUTION

```
import * from dw::Runtime
fun requirePositive(n: Number) = if (n > 0) n else fail("Value must be
positive")
---
try(() -> requirePositive(payload.n))
```

OUTPUT

```
{ "success": false, "error": { "message": "Value must be positive" } }
```

`fail(message)` immediately stops execution with a runtime error carrying that exact message — it has to be wrapped in `try` to be handled gracefully rather than crashing the script.

#932 Raise a Custom Error With a Dynamic Message

Build the error message passed to `fail` dynamically, from a parameter.

INPUT

```
{ "orderId": null }
```

SOLUTION

```
import * from dw::Runtime
fun requireField(v, name: String) = if (v != null) v else fail(name ++ " is
required")
---
try(() -> requireField(payload.orderId, "orderId"))
```

OUTPUT

```
{ "success": false, "error": { "message": "orderId is required" } }
```

Since `fail` just takes a String, that String can be built with ++ the same way any other message would be.

#933 Use fail Inside a match Expression

Use `fail` as a `match` expression's catch-all case, for a value that should never occur.

INPUT

```
{ "status": "ARCHIVED" }
```

SOLUTION

```
import * from dw::Runtime
fun classify(status: String) = status match {
  case "OPEN" -> "active"
  case "CLOSED" -> "inactive"
  else -> fail("Unrecognized status: " ++ status)
}
---
try(() -> classify(payload.status))
```

OUTPUT

```
{ "success": false, "error": { "message": "Unrecognized status: ARCHIVED" } }
```

Using `fail` instead of a normal value in `match`'s `else` case turns "unexpected value" from a silent bug into a loud, catchable error.

#934 Raise an Error From Deep Inside a Recursive Function

Raise a custom error partway through a recursive walk over an Array.

INPUT

```
{ "items": [{"id": "O1", "total": 50}, {"id": "O2", "total": -5}] }
```

SOLUTION

```
import * from dw::Runtime
fun checkAll(items: Array): Boolean = items match {
  case [head ~ tail] -> if (head.total > 0) checkAll(tail) else fail("Invalid
total for item " ++ head.id)
  case [] -> true
}
---
try(() -> checkAll(payload.items))
```

OUTPUT

```
{ "success": false, "error": { "message": "Invalid total for item O2" } }
```

`fail` propagates out of a recursive call exactly like any other error — `try` at the very top catches it regardless of how deep the recursion went.

#935 Compare fail With Returning an Error Object Instead

Contrast two valid validation styles: raising with `fail`, versus returning an error Object.

INPUT

```
{ "n": -5 }
```

SOLUTION

```
import * from dw::Runtime
fun requirePositiveThrow(n: Number) = if (n > 0) n else fail("must be
positive")
fun requirePositiveReturn(n: Number) = if (n > 0) { ok: true, value: n } else
{ ok: false, error: "must be positive" }
---
{ throwStyle: (try(() -> requirePositiveThrow(payload.n))).success,
returnStyle: requirePositiveReturn(payload.n) }
```

OUTPUT

```
{ "throwStyle": false, "returnStyle": { "ok": false, "error": "must be
positive" } }
```

`fail` stops execution and must be caught with `try`; returning an error Object keeps the script flowing normally and must be checked explicitly by the caller. Both are legitimate — the right choice depends on whether a failure should be exceptional or just another outcome.

#936 Guard a Whole Transformation With a Single fail Check

Guard an entire transformation behind one upfront `fail` check.

INPUT

```
{ "orders": [] }
```

SOLUTION

```
import * from dw::Runtime
---
try(() -> if (isEmpty(payload.orders)) fail("orders array must not be empty")
else sizeOf(payload.orders))
```

OUTPUT

```
{ "success": false, "error": { "message": "orders array must not be empty" } }
```

A single guard check at the top of a transformation is often clearer than scattering null checks throughout the rest of the logic.

#937 Raise Different Error Messages for Different Failure Reasons

Check two different conditions, raising a different message depending on which one fails.

INPUT

```
{ "id": "O1", "total": "50" }
```

SOLUTION

```
import * from dw::Runtime
fun validate(o) =
  if (o.id == null) fail("id is required")
  else if (not (o.total is Number)) fail("total must be a Number")
  else o
---
try(() -> validate(payload))
```

OUTPUT

```
{ "success": false, "error": { "message": "total must be a Number" } }
```

Since `fail` only ever runs on the branch that's actually reached, the resulting message always matches the specific reason that branch failed.

#938 Use fail as the else Branch of a match's Catch-All

Reject an unrecognized carrier code by failing inside a routing function's catch-all case.

INPUT

```
{ "code": "DHL" }
```

SOLUTION

```
import * from dw::Runtime
fun routeCarrier(code: String) = code match {
  case "FDX" -> "fedex"
  case "UPS" -> "ups"
  else -> fail("Unknown carrier code: " ++ code)
}
---
try(() -> routeCarrier(payload.code))
```

OUTPUT

```
{ "success": false, "error": { "message": "Unknown carrier code: DHL" } }
```

This is the same pattern as #933, applied to a routing lookup — an unrecognized code fails loudly instead of silently returning something wrong.

#939 Build a Validation Function That Either Returns Cleanly or Fails

Write a function that either returns a fully validated Object, or fails with a specific reason.

INPUT

```
{ "id": "O1", "total": 50 }
```

SOLUTION

```
import * from dw::Runtime
fun parseOrder(o) =
  if (o.id == null) fail("id is required")
  else if (o.total <= 0) fail("total must be positive")
  else { id: o.id, total: o.total, status: "VALIDATED" }
---
try(() -> parseOrder(payload))
```

OUTPUT

```
{ "success": true, "result": { "id": "O1", "total": 50, "status": "VALIDATED"
} }
```

With valid input, none of the `fail` branches are reached, and `try` reports success with the fully built result.

#940 CargoLink Capstone: A Fail-Fast Validator for Incoming Shipments

Validate a shipment against three separate rules, failing fast with the first violated one.

INPUT

```
{ "id": "SHIP-1", "weight": -5, "carrier": "fedex" }
```

SOLUTION

```
import * from dw::Runtime
fun validateShipment(s) =
  if (s.id == null) fail("Shipment id is required")
  else if (not (s.weight is Number) or s.weight <= 0) fail("Shipment weight
must be a positive Number")
  else if (s.carrier == null) fail("Shipment carrier is required")
  else { id: s.id, weight: s.weight, carrier: s.carrier, status: "ACCEPTED" }
---
try(() -> validateShipment(payload))
```

OUTPUT

```
{ "success": false, "error": { "message": "Shipment weight must be a positive
Number" } }
```

This closes out the section with a complete fail-fast validator — the exact pattern this volume's later capstones build directly on top of.

SECTION 5 — Safe Fallback Chains (Problems 941–950)

#941 Chain default After a Failed Coercion

Combine `try` and `default` so a failed coercion falls back to zero.

INPUT

```
{ "total": "N/A" }
```

SOLUTION

```
import * from dw::Runtime
output application/json
---
{ total: (try(() -> payload.total as Number)).result default 0 }
```

OUTPUT

```
{ "total": 0 }
```

This is the single most common `try` pattern in real integration scripts: attempt a coercion, and quietly fall back if it doesn't work.

#942 Try a Primary Source, Fall Back to a Secondary Source

Attempt a primary field first, falling back to a secondary field if the primary is unusable.

INPUT

```
{ "fallbackCarrier": "fedex" }
```

SOLUTION

```
import * from dw::Runtime
output application/json
---
{ carrier: (try(() -> payload.primaryCarrier as String)).result default
(payload.fallbackCarrier default "UNASSIGNED") }
```

OUTPUT

```
{ "carrier": "fedex" }
```

`try` and `default` combine into a chain exactly like #908's chain of defaults, just with a risky step added at the front.

#943 Build a Function That Never Throws, No Matter What

Write a division function that always returns a Number, even when the division itself would fail.

INPUT

```
{ "a": 10, "b": 0 }
```

SOLUTION

```
import * from dw::Runtime
fun safeDivide(a: Number, b: Number): Number = (try(() -> a / b)).result
default 0
---
safeDivide(payload.a, payload.b)
```

OUTPUT

```
0
```

Wrapping the risky operation and defaulting the result in one small helper means every caller gets the safety for free.

#944 Combine try With match to Handle Multiple Outcomes

Use `match` on a `try` result to produce a different message depending on the outcome.

INPUT

```
{ "total": "abc" }
```

SOLUTION

```
import * from dw::Runtime
var attempt = try(() -> payload.total as Number)
---
attempt match {
  case a if a.success -> "Parsed: " ++ (a.result as String)
  else -> "Failed to parse total"
}
```

OUTPUT

```
"Failed to parse total"
```

A guard clause on `a.success` is all it takes to route a `try` result through `match` just like any other value.

#945 Layer Three Fallbacks Deep

Fall back through three candidate fields before landing on a final hardcoded default.

INPUT

```
{ "c": "fallback-c" }
```

SOLUTION

```
output application/json
---
{ value: payload.a default (payload.b default (payload.c default "NONE")) }
```

OUTPUT

```
{ "value": "fallback-c" }
```

There's no limit to how deep a `default` chain can nest — each level only matters if every level before it came back null.

#946 Use try to Guard a Whole Object Construction

Wrap an entire multi-field object construction in `try`, so a failure anywhere inside it falls back cleanly.

INPUT

```
{ "total": "50", "count": 0 }
```

SOLUTION

```
import * from dw::Runtime
output application/json
---
(try(() -> { total: payload.total as Number, average: (payload.total as
Number) / payload.count })).result default { total: 0, average: 0 }
```

OUTPUT

```
{ "total": 0, "average": 0 }
```

Even though `total` alone would coerce fine, the `average` field's division by zero fails inside the same thunk — so the whole object falls back together, not field by field.

#947 Retry a Flaky Operation a Fixed Number of Times

Simulate retrying an operation that only succeeds on a specific attempt.

INPUT

```
{}
```

SOLUTION

```
fun retry(n: Number, maxAttempts: Number, succeedsOn: Number) =
  if (n == succeedsOn) { success: true, attempts: n }
  else if (n >= maxAttempts) { success: false, attempts: n }
  else retry(n + 1, maxAttempts, succeedsOn)
---
retry(1, 5, 4)
```

OUTPUT

```
{ "success": true, "attempts": 4 }
```

This is the retry-counter recursion from Volume 8, back in its natural home — a fallback strategy that spans multiple attempts instead of just one.

#948 Combine try With a Custom Validator Function

Use a `try`-wrapped validator to decide an order's acceptance status.

INPUT

```
{ "id": "O1", "total": -5 }
```

SOLUTION

```
import * from dw::Runtime
fun validate(o) = if (o.total > 0) o else fail("total must be positive")
---
{ order: payload.id, status: if ((try(() -> validate(payload))).success)
"ACCEPTED" else "REJECTED" }
```

OUTPUT

```
{ "order": "O1", "status": "REJECTED" }
```

The validator's internal reasoning stays hidden — the caller only needs the Boolean `success` to decide the order's fate.

#949 Provide a Detailed Fallback Report When a try Fails

Build a fallback Object with an explanatory note when a multi-field parse fails.

INPUT

```
{ "id": "O1", "total": "bad" }
```

SOLUTION

```
import * from dw::Runtime
var attempt = try(() -> { id: payload.id, total: payload.total as Number })
---
if (attempt.success) attempt.result else { id: payload.id default "UNKNOWN",
total: 0, note: "used fallback values" }
```

OUTPUT

```
{ "id": "O1", "total": 0, "note": "used fallback values" }
```

A fallback doesn't have to just be a default value — it can be a whole explanatory Object, telling downstream consumers exactly what happened.

#950 CargoLink Capstone: A Fully Defensive Shipment Cost Calculator

Calculate a shipment's cost from two fields that might not coerce cleanly, falling back to zero if anything goes wrong.

INPUT

```
{ "weight": "20", "ratePerKg": "bad" }
```

SOLUTION

```
import * from dw::Runtime
var attempt = try(() -> (payload.weight as Number) * (payload.ratePerKg as
Number))
---
{ cost: if (attempt.success) attempt.result else 0 }
```

OUTPUT

```
{ "cost": 0 }
```

This closes out the section the same way it opened: wrap the risky calculation, check `success`, and never let a bad input crash the whole script.

SECTION 6 — Idempotency & Correlation Patterns (Problems 951–960)

#951 Generate a Deterministic Idempotency Key From Order Fields

Build a repeatable idempotency key from a combination of fields on an order.

INPUT

```
{ "customerId": "CUST-1", "orderDate": "2026-03-14", "amount": 50 }
```

SOLUTION

```
output application/json
---
{ idempotencyKey: payload.customerId ++ "-" ++ payload.orderDate ++ "-" ++
(payload.amount as String) }
```

OUTPUT

```
{ "idempotencyKey": "CUST-1-2026-03-14-50" }
```

Building a key from fields that will always be the same for a "duplicate" of the same order is what makes it deterministic — the same order always produces the same key.

#952 Detect a Duplicate Request by Comparing Idempotency Keys

Check whether an incoming request's key has already been seen.

INPUT

```
{ "requestKey": "REQ-1", "seenKeys": ["REQ-1", "REQ-2"] }
```

SOLUTION

```
output application/json
---
{ isDuplicate: payload.seenKeys contains payload.requestKey }
```

OUTPUT

```
{ "isDuplicate": true }
```

A real system would look this up in a store rather than an inline Array, but the check itself — has this key been seen before — is exactly this simple.

#953 Deduplicate a Batch of Events by Correlation ID

Remove duplicate events from a batch, keeping only the first occurrence of each correlation ID.

INPUT

```
{ "events": [{"correlationId": "C1", "data": "a"}, {"correlationId": "C1",
"data": "b"}, {"correlationId": "C2", "data": "c"}] }
```

SOLUTION

```
output application/json
---
payload.events distinctBy ((e) -> e.correlationId)
```

OUTPUT

```
[{ "correlationId": "C1", "data": "a" }, { "correlationId": "C2", "data": "c"
}]
```

`distinctBy` keeps the first Object it sees for each key, so the second `"C1"` event is silently dropped.

#954 Propagate a Correlation ID Through a Transformation

Carry an incoming correlation ID through to the response, alongside the actual result.

INPUT

```
{ "headers": {"correlationId": "CORR-99"}, "body": {"id": "O1"} }
```

SOLUTION

```
output application/json
---
{ correlationId: payload.headers.correlationId, result: { orderId:
payload.body.id, status: "PROCESSED" } }
```

OUTPUT

```
{ "correlationId": "CORR-99", "result": { "orderId": "O1", "status":
"PROCESSED" } }
```

Keeping the correlation ID at the top level of the response, separate from the actual result, is what lets logs and traces tie a request to its response.

#955 Generate a Correlation ID When None Was Provided

Fall back to a generated correlation ID when the caller didn't supply one.

INPUT

```
{ "orderId": "O1" }
```

SOLUTION

```
output application/json
---
{ correlationId: payload.correlationId default ("AUTO-" ++ payload.orderId) }
```

OUTPUT

```
{ "correlationId": "AUTO-O1" }
```

Every request having some correlation ID, generated or not, keeps the same downstream logging path working either way.

#956 Tag Every Event in a Batch With a Shared Batch ID

Attach the same batch identifier to every event processed as part of that batch.

INPUT

```
{ "batchId": "BATCH-7", "events": [{"id": "E1"}, {"id": "E2"}] }
```

SOLUTION

```
output application/json
---
payload.events map ((e) -> e ++ { batchId: payload.batchId })
```

OUTPUT

```
[{ "id": "E1", "batchId": "BATCH-7" }, { "id": "E2", "batchId": "BATCH-7" }]
```

Tagging every record with a shared batch ID is what makes it possible to later ask "which events came from this run" in a downstream system or log.

#957 Detect Whether a Batch Contains Any Duplicate IDs

Check whether a batch of IDs contains any duplicates at all, before processing it.

INPUT

```
{ "ids": ["A", "B", "A", "C"] }
```

SOLUTION

```
output application/json
---
{ hasDuplicates: sizeOf(payload.ids) != sizeOf(payload.ids distinctBy ((id) -
> id)) }
```

OUTPUT

```
{ "hasDuplicates": true }
```

If deduplicating an Array changes its size, there had to be at least one duplicate in it — a quick, reusable way to test for duplicates without inspecting individual elements.

#958 Build an Idempotent Upsert Decision

Decide whether an incoming record should be inserted or updated, based on whether its ID already exists.

INPUT

```
{ "existingIds": ["O1", "O2"], "newRecord": {"id": "O3"} }
```

SOLUTION

```
output application/json
---
{ action: if (payload.existingIds contains payload.newRecord.id) "UPDATE"
else "INSERT" }
```

OUTPUT

```
{ "action": "INSERT" }
```

This "upsert" decision is what makes an operation idempotent — running it twice with the same record updates it the second time instead of creating a duplicate.

#959 Merge a Retry's Results Without Duplicating Prior Successes

Combine results from a retry attempt with previously successful results, without duplicating any IDs.

INPUT

```
{ "previousResults": [{"id": "O1", "status": "OK"}], "newResults": [{"id":
"O1", "status": "OK"}, {"id": "O2", "status": "OK"}] }
```

SOLUTION

```
output application/json
---
(payload.previousResults ++ payload.newResults) distinctBy ((r) -> r.id)
```

OUTPUT

```
[{ "id": "O1", "status": "OK" }, { "id": "O2", "status": "OK" }]
```

Merging then deduplicating is a simple, reliable way to combine results across retries without needing to track exactly what succeeded on which attempt.

#960 CargoLink Capstone: Build a Deduplicated, Correlated Event Log Entry

Build a log entry combining a correlation ID, a duplicate check, and the event itself.

INPUT

```
{ "shipmentId": "SHIP-1", "event": "DELIVERED", "seenShipmentIds": ["SHIP-1"]
}
```

SOLUTION

```
output application/json
---
{
  correlationId: payload.correlationId default ("AUTO-" ++
payload.shipmentId),
  isDuplicate: payload.seenShipmentIds contains payload.shipmentId,
  entry: { shipmentId: payload.shipmentId, event: payload.event }
}
```

OUTPUT

```
{ "correlationId": "AUTO-SHIP-1", "isDuplicate": true, "entry": {
"shipmentId": "SHIP-1", "event": "DELIVERED" } }
```

This closes out the section by combining every idea covered so far — a generated correlation ID, a duplicate check, and the event payload — into one log-ready entry.

SECTION 7 — Retry & Backoff Simulation (Problems 961–970)

#961 Simulate a Fixed Number of Retry Attempts

Simulate retrying an operation up to a maximum number of attempts.

INPUT

```
{}
```

SOLUTION

```
fun retry(n: Number, maxAttempts: Number, succeedsOn: Number) =
  if (n == succeedsOn) { success: true, attempts: n }
  else if (n >= maxAttempts) { success: false, attempts: n }
  else retry(n + 1, maxAttempts, succeedsOn)
---
retry(1, 5, 4)
```

OUTPUT

```
{ "success": true, "attempts": 4 }
```

This section builds out the retry idea from Section 5 into a full family of realistic patterns: backoff timing, state tracking, and logging.

#962 Compute an Exponential Backoff Delay Sequence

Compute a sequence of exponentially increasing delays for a series of retry attempts.

INPUT

```
{ "attempts": 4, "baseMs": 100 }
```

SOLUTION

```
fun powerOf2(n: Number): Number = if (n == 0) 1 else 2 * powerOf2(n - 1)
fun backoffDelays(attempts: Number, baseMs: Number): Array = (0 to (attempts
- 1)) map ((n) -> baseMs * powerOf2(n))
---
backoffDelays(payload.attempts, payload.baseMs)
```

OUTPUT

```
[100, 200, 400, 800]
```

Each delay doubles the one before it — the classic exponential backoff shape used to avoid hammering a struggling downstream system.

#963 Cap a Backoff Delay at a Maximum Value

Compute a backoff delay for a given attempt number, capped at a maximum.

INPUT

```
{ "attempt": 10, "baseMs": 200, "maxMs": 1000 }
```

SOLUTION

```
fun cappedDelay(attempt: Number, baseMs: Number, maxMs: Number): Number = if
(baseMs * attempt > maxMs) maxMs else baseMs * attempt
---
cappedDelay(payload.attempt, payload.baseMs, payload.maxMs)
```

OUTPUT

```
1000
```

Without a cap, backoff delays can grow unreasonably long after enough attempts — `cappedDelay` keeps them bounded no matter how high `attempt` climbs.

#964 Decide Whether to Retry Based on an Error Type

Decide whether an error is worth retrying, based on its type.

INPUT

```
{ "errorType": "TIMEOUT" }
```

SOLUTION

```
output application/json
---
{ shouldRetry: payload.errorType == "TIMEOUT" or payload.errorType ==
"TEMPORARY_UNAVAILABLE" }
```

OUTPUT

```
{ "shouldRetry": true }
```

Not every failure deserves a retry — a validation error will fail identically every time, but a timeout might succeed on the next attempt.

#965 Track Retry State Across Multiple Fields

Track a retry's full state — count, success, and give-up flag — across recursive attempts.

INPUT

```
{ "count": 0, "maxAttempts": 5, "succeedsOn": 3, "succeeded": false }
```

SOLUTION

```
fun attempt(state) =
  if (state.succeeded) state
  else if (state.count >= state.maxAttempts) state ++ { giveUp: true }
  else attempt(state ++ { count: state.count + 1, succeeded: state.count + 1
== state.succeedsOn })
---
attempt(payload)
```

OUTPUT

```
{ "count": 3, "maxAttempts": 5, "succeedsOn": 3, "succeeded": true }
```

Carrying the whole state Object forward through ++ on each recursive call, rather than separate parameters, keeps every field about the retry together in one place.

#966 Simulate a Circuit Breaker Opening After Too Many Failures

Decide whether a circuit breaker should be open, based on recent failure counts.

INPUT

```
{ "recentFailureCount": 6, "failureThreshold": 5 }
```

SOLUTION

```
output application/json
---
{ circuitOpen: payload.recentFailureCount >= payload.failureThreshold }
```

OUTPUT

```
{ "circuitOpen": true }
```

A circuit breaker is really just a threshold check like this one, run before every call — once too many recent failures pile up, new attempts stop until things recover.

#967 Combine try With a Retry Loop

Combine `try` and recursion to retry a simulated flaky operation until it succeeds or attempts run out.

INPUT

```
{}
```

SOLUTION

```
import * from dw::Runtime
fun flakyCall(attempt: Number, succeedsOn: Number) = if (attempt ==
succeedsOn) "OK" else fail("simulated failure on attempt " ++ (attempt as
String))
fun attemptWithRetry(attempt: Number, succeedsOn: Number, maxAttempts:
Number) =
  if ((try(() -> flakyCall(attempt, succeedsOn))).success) (try(() ->
flakyCall(attempt, succeedsOn))).result
  else if (attempt >= maxAttempts) "GAVE UP"
  else attemptWithRetry(attempt + 1, succeedsOn, maxAttempts)
---
attemptWithRetry(1, 3, 5)
```

OUTPUT

```
"OK"
```

Each recursive call wraps its own attempt in `try`, so a simulated failure on attempts 1 and 2 doesn't stop the retry loop from reaching attempt 3, where it succeeds.

#968 Log Each Retry Attempt Into an Accumulating Array

Build a log of every retry attempt, recording whether each one failed or succeeded.

INPUT

```
{}
```

SOLUTION

```
fun attemptWithLog(attempt: Number, succeedsOn: Number, maxAttempts: Number,
log: Array): Object =
  if (attempt == succeedsOn) { success: true, log: log ++ ["attempt " ++
(attempt as String) ++ ": success"] }
  else if (attempt >= maxAttempts) { success: false, log: log ++ ["attempt "
++ (attempt as String) ++ ": gave up"] }
  else attemptWithLog(attempt + 1, succeedsOn, maxAttempts, log ++ ["attempt
" ++ (attempt as String) ++ ": failed"])
---
attemptWithLog(1, 3, 5, [])
```

OUTPUT

```
{ "success": true, "log": ["attempt 1: failed", "attempt 2: failed", "attempt
3: success"] }
```

Passing the accumulating `log` Array through every recursive call, the same way Volume 8's accumulators worked, builds a complete attempt-by-attempt history for free.

#969 Decide Retry Delay Using the Attempt Number

Compute the backoff delay for one specific attempt number.

INPUT

```
{ "attempt": 3, "baseMs": 150 }
```

SOLUTION

```
fun powerOf2(n: Number): Number = if (n == 0) 1 else 2 * powerOf2(n - 1)
fun delayForAttempt(attempt: Number, baseMs: Number): Number = baseMs *
powerOf2(attempt - 1)
---
delayForAttempt(payload.attempt, payload.baseMs)
```

OUTPUT

```
600
```

This is #962's delay sequence, computed for a single attempt on demand instead of the whole sequence upfront.

#970 CargoLink Capstone: A Complete Retry-With-Backoff Simulation

Simulate a full retry sequence, logging the backoff delay used before each retry attempt.

INPUT

```
{}
```

SOLUTION

```
fun powerOf2(n: Number): Number = if (n == 0) 1 else 2 * powerOf2(n - 1)
fun attempt(n: Number, succeedsOn: Number, maxAttempts: Number, baseMs:
Number, log: Array): Object =
  if (n == succeedsOn) { success: true, totalAttempts: n, log: log }
  else if (n >= maxAttempts) { success: false, totalAttempts: n, log: log }
  else attempt(n + 1, succeedsOn, maxAttempts, baseMs, log ++ [{ attempt: n,
delayMs: baseMs * powerOf2(n - 1) }])
---
attempt(1, 3, 5, 100, [])
```

OUTPUT

```
{ "success": true, "totalAttempts": 3, "log": [{ "attempt": 1, "delayMs": 100
}, { "attempt": 2, "delayMs": 200 }] }
```

This closes out the section by combining exponential backoff timing with the attempt log from #968 — a realistic, complete picture of what a retrying integration actually did.

SECTION 8 — Pagination & Batching for Integration (Problems 971–980)

#971 Compute the Total Number of Pages Needed

Compute how many pages are needed to cover a full result set at a given page size.

INPUT

```
{ "items": [1, 2, 3, 4, 5, 6, 7], "pageSize": 3 }
```

SOLUTION

```
output application/json
---
{ totalPages: floor(sizeOf(payload.items) / payload.pageSize) + (if
(sizeOf(payload.items) mod payload.pageSize == 0) 0 else 1) }
```

OUTPUT

```
{ "totalPages": 3 }
```

`floor` handles the whole-pages part of the count, and the `mod` check adds one more page for whatever's left over — 7 items at 3 per page needs 3 pages, not 2.

#972 Extract One Page of Results

Write a reusable function that slices out one page of results by page number and page size.

INPUT

```
{ "items": ["a", "b", "c", "d", "e", "f", "g"] }
```

SOLUTION

```
fun getPage(items: Array, page: Number, pageSize: Number): Array =
  if (((page - 1) * pageSize) >= sizeOf(items)) []
  else items[((page - 1) * pageSize) to (if (page * pageSize - 1 >
sizeOf(items) - 1) sizeOf(items) - 1 else page * pageSize - 1)]
---
getPage(payload.items, 2, 3)
```

OUTPUT

```
["d", "e", "f"]
```

The end-index calculation clamps to the Array's actual last index, so the final page — even if it's shorter than a full page — doesn't try to slice past the end.

#973 Build a Pagination Metadata Object

Build a standard pagination metadata block for an API response.

INPUT

```
{ "page": 1, "pageSize": 10, "totalItems": 25 }
```

SOLUTION

```
output application/json
---
{
  page: payload.page,
  pageSize: payload.pageSize,
  totalItems: payload.totalItems,
  totalPages: floor(payload.totalItems / payload.pageSize) + (if
(payload.totalItems mod payload.pageSize == 0) 0 else 1),
  hasNext: payload.page * payload.pageSize < payload.totalItems
}
```

OUTPUT

```
{ "page": 1, "pageSize": 10, "totalItems": 25, "totalPages": 3, "hasNext":
true }
```

This is the shape most paginated APIs return alongside the actual page of data — everything a client needs to know whether, and how, to ask for the next page.

#974 Detect the Last Page

Determine whether a given page number is the final page of results.

INPUT

```
{ "page": 3, "pageSize": 10, "totalItems": 25 }
```

SOLUTION

```
output application/json
---
{ isLastPage: payload.page >= (floor(payload.totalItems / payload.pageSize) +
(if (payload.totalItems mod payload.pageSize == 0) 0 else 1)) }
```

OUTPUT

```
{ "isLastPage": true }
```

Comparing the current page against the same total-pages calculation from #973 is all it takes to know whether there's anything left to fetch.

#975 Build a Cursor for the Next Page Request

Build a cursor value for the next page request, or null when there's nothing more to fetch.

INPUT

```
{ "hasMore": true, "lastId": "ORD-1042" }
```

SOLUTION

```
output application/json
---
{ nextCursor: if (payload.hasMore) payload.lastId else null }
```

OUTPUT

```
{ "nextCursor": "ORD-1042" }
```

Cursor-based pagination hands back the last-seen ID instead of a page number, letting the next request pick up exactly where this one left off.

#976 Merge Results From Multiple Paginated Pages Into One Array

Combine several already-fetched pages of results into one flat Array.

INPUT

```
{ "pages": [{"items": ["a", "b"]}, {"items": ["c", "d"]}] }
```

SOLUTION

```
output application/json
---
payload.pages reduce ((page, acc = []) -> acc ++ page.items)
```

OUTPUT

```
["a", "b", "c", "d"]
```

This is the same `reduce`-with-a-default-accumulator pattern from Volume 4, here flattening a fetched set of pages back into one combined result.

#977 Batch an Array Into Fixed-Size Groups

Split a flat Array of order IDs into fixed-size batches for downstream processing.

INPUT

```
{ "orderIds": ["O1", "O2", "O3", "O4", "O5"] }
```

SOLUTION

```
fun chunk(arr: Array, size: Number): Array =
  if (isEmpty(arr)) []
  else if (sizeOf(arr) <= size) [arr]
  else [arr[0 to (size - 1)]] ++ chunk(arr[size to -1], size)
---
chunk(payload.orderIds, 3)
```

OUTPUT

```
[["O1", "O2", "O3"], ["O4", "O5"]]
```

This is the recursive `chunk` helper from Volume 8, back in its most common real use — splitting a large result set into batches a downstream system can actually accept.

#978 Process Each Batch Independently and Combine Summaries

Split orders into batches and compute a size and total for each batch independently.

INPUT

```
{ "orders": [{"total": 10}, {"total": 20}, {"total": 30}, {"total": 40},
{"total": 50}] }
```

SOLUTION

```
fun chunk(arr: Array, size: Number): Array =
  if (isEmpty(arr)) []
  else if (sizeOf(arr) <= size) [arr]
  else [arr[0 to (size - 1)]] ++ chunk(arr[size to -1], size)
---
chunk(payload.orders, 2) map ((batch) -> { batchSize: sizeOf(batch),
batchTotal: sum(batch map ((o) -> o.total)) })
```

OUTPUT

```
[{ "batchSize": 2, "batchTotal": 30 }, { "batchSize": 2, "batchTotal": 70 }, {
"batchSize": 1, "batchTotal": 50 }]
```

Once `chunk` has split the orders into groups, each batch can be summarized with ordinary `map` and `sum` — no different from summarizing the whole Array at once.

#979 Track Progress Across a Batch Job

Compute a percent-complete figure for an in-progress batch job.

INPUT

```
{ "processed": 42, "total": 200 }
```

SOLUTION

```
output application/json
---
{ percentComplete: floor((payload.processed / payload.total) * 100) }
```

OUTPUT

```
{ "percentComplete": 21 }
```

A progress percentage like this is a small enough calculation to run inline anywhere a batch job's status needs reporting.

#980 CargoLink Capstone: A Complete Paginated Batch Processing Summary

Batch a set of shipments and report a size and total weight for each batch.

INPUT

```
{ "shipments": [{"weight": 10}, {"weight": 20}, {"weight": 15}, {"weight": 25}], "batchSize": 2 }
```

SOLUTION

```
fun chunk(arr: Array, size: Number): Array =
  if (isEmpty(arr)) []
  else if (sizeOf(arr) <= size) [arr]
  else [arr[0 to (size - 1)]] ++ chunk(arr[size to -1], size)
---
{
  totalItems: sizeOf(payload.shipments),
  batches: sizeOf(chunk(payload.shipments, payload.batchSize)),
  batchSummaries: chunk(payload.shipments, payload.batchSize) map ((batch) ->
{ count: sizeOf(batch), totalWeight: sum(batch map ((s) -> s.weight)) })
}
```

OUTPUT

```
{ "totalItems": 4, "batches": 2, "batchSummaries": [{ "count": 2, "totalWeight": 30 }, { "count": 2, "totalWeight": 40 }] }
```

This closes out the section with the report a batch job would actually log: how much came in, how it was split, and what happened in each piece.

SECTION 9 — Schema Validation & Contract Checking (Problems 981–990)

#981 Check That All Required Fields Are Present

Write a reusable function that checks a list of required field names are all present on an Object.

INPUT

```
{ "id": "O1", "total": 50 }
```

SOLUTION

```
fun hasAllFields(o, required: Array): Boolean = isEmpty(required filter ((f)
-> o[(f)] == null))
---
hasAllFields(payload, ["id", "total", "carrier"])
```

OUTPUT

```
false
```

The parenthesized dynamic selector, `o[(f)]`, looks up a field by the variable `f` — this is what lets one function check any list of required field names.

#982 Check That No Unexpected Fields Are Present

Write a reusable function that rejects an Object containing any field outside an allowed list.

INPUT

```
{ "id": "O1", "total": 50, "extra": "x" }
```

SOLUTION

```
fun hasOnlyAllowedFields(o, allowed: Array): Boolean = isEmpty((namesOf(o)
map ((n) -> n as String)) filter ((n) -> not (allowed contains n)))
---
hasOnlyAllowedFields(payload, ["id", "total"])
```

OUTPUT

```
false
```

`namesOf` lists every field name an Object actually has, which makes it possible to check the opposite direction from #981 — that nothing extra snuck in.

#983 Check the Type of Every Field Against Expectations

Check two fields on an Object against their expected types in one combined check.

INPUT

```
{ "id": "O1", "total": "50" }
```

SOLUTION

```
fun checkTypes(o) = (o.id is String) and (o.total is Number)
---
checkTypes(payload)
```

OUTPUT

```
false
```

`total` being the numeric-looking String `"50"` instead of a real Number is exactly the kind of contract violation a type check like this exists to catch.

#984 Build a Contract Violation Report

Check three separate contract rules and collect a message for each one that's violated.

INPUT

```
{ "id": "O1", "total": "50", "items": null }
```

SOLUTION

```
fun checkContract(o) =
  (if (not (o.id is String)) ["id must be a String"] else []) ++
  (if (not (o.total is Number)) ["total must be a Number"] else []) ++
  (if (not (o.items is Array)) ["items must be an Array"] else [])
---
checkContract(payload)
```

OUTPUT

```
["total must be a Number", "items must be an Array"]
```

This is Section 2's error-collection pattern, applied specifically to type contracts instead of general validation rules.

#985 Validate That an Array of Records All Share the Same Shape

Check whether every record in an Array has exactly the same set of fields as the first one.

INPUT

```
{ "records": [{"id": "O1", "total": 50}, {"id": "O2"}] }
```

SOLUTION

```
fun sameShape(records: Array): Boolean =
  isEmpty(records) or isEmpty(records filter ((r) -> (namesOf(r) map ((n) ->
n as String)) != (namesOf(records[0]) map ((n) -> n as String))))
---
sameShape(payload.records)
```

OUTPUT

```
false
```

Comparing each record's field-name Array against the first record's is a quick way to catch a batch where one row is missing a field the others all have.

#986 Enforce a Minimum Required API Version

Check whether an incoming request's API version meets a minimum requirement.

INPUT

```
{ "apiVersion": "2", "minVersion": 3 }
```

SOLUTION

```
output application/json
---
{ supported: (payload.apiVersion as Number) >= payload.minVersion }
```

OUTPUT

```
{ "supported": false }
```

Coercing the incoming version String to a Number before comparing avoids a subtle bug where `"10" >= 3` could otherwise be compared as text instead of as numbers.

#987 Validate a Nested Object's Shape

Validate that a nested address Object has the expected field types.

INPUT

```
{ "shipTo": {"city": "Reno", "zip": 89501} }
```

SOLUTION

```
fun validateAddress(a) = (a.city is String) and (a.zip is String)
---
validateAddress(payload.shipTo)
```

OUTPUT

```
false
```

A ZIP code that arrived as a Number instead of a String is a very common real-world contract mismatch, and exactly the kind of thing nested validation is meant to catch before it breaks downstream logic.

#988 Combine Contract Checking With try for Untrusted Input

Wrap a contract-checking validator in `try` so a violation is caught, not crashed on.

INPUT

```
{ "id": "O1", "total": "50" }
```

SOLUTION

```
import * from dw::Runtime
fun validateContract(o) =
  if (not (o.id is String)) fail("id must be a String")
  else if (not (o.total is Number)) fail("total must be a Number")
  else o
---
try(() -> validateContract(payload))
```

OUTPUT

```
{ "success": false, "error": { "message": "total must be a Number" } }
```

This is Section 4's fail-fast pattern, applied specifically to schema and type contracts rather than business rules.

#989 Validate an Enum-Style Field Against a Payload-Supplied Allowed List

Validate a status field against a list of allowed values that's itself part of the payload.

INPUT

```
{ "status": "ARCHIVED", "allowedStatuses": ["OPEN", "CLOSED", "PENDING"] }
```

SOLUTION

```
output application/json
---
{ valid: payload.allowedStatuses contains payload.status }
```

OUTPUT

```
{ "valid": false }
```

Some contracts are defined by the request itself rather than hardcoded in the script — this checks against whatever list the caller provided, not a fixed one.

#990 CargoLink Capstone: Validate an Incoming Shipment Against a Full Contract

Validate a shipment against a full three-rule contract, failing with the first violated rule.

INPUT

```
{ "id": "SHIP-1", "weight": 20, "carrier": "usps" }
```

SOLUTION

```
import * from dw::Runtime
fun validateShipment(s) =
  if (not (s.id is String)) fail("id must be a String")
  else if (not (s.weight is Number) or s.weight <= 0) fail("weight must be a
positive Number")
  else if (not (["fedex", "ups", "dhl"] contains s.carrier)) fail("carrier
must be one of fedex, ups, dhl")
  else s
---
try(() -> validateShipment(payload))
```

OUTPUT

```
{ "success": false, "error": { "message": "carrier must be one of fedex, ups,
dhl" } }
```

This closes out the section with a complete contract check — type, range, and enum rules together — exactly the kind of gate a real shipment intake endpoint would run first.

SECTION 10 — Mixed Real-World Integration Capstones (Problems 991–1000)

#991 Combine Validation, Defaults, and Format Conversion in One Pipeline

Default two optional fields and convert the result into an attributed XML document.

INPUT

```
{ "id": "O1", "total": 50 }
```

SOLUTION

```
output application/xml {writeDeclaration: false}
---
{ order @(id: payload.id default "UNKNOWN"): { total: payload.total default
0, status: payload.status default "PENDING" } }
```

OUTPUT

```
<order id="O1">
  <total>50</total>
  <status>PENDING</status>
</order>
```

This final section combines ideas across every volume in the series — here, defaults from this volume feeding directly into the XML attribute and format work from Volume 9.

#992 Build a Complete Success/Error API Response Envelope

Wrap a validated transformation in a standard success/error response envelope.

INPUT

```
{ "id": "O1", "total": -5 }
```

SOLUTION

```
import * from dw::Runtime
fun processOrder(o) =
  if (not (o.total is Number) or o.total <= 0) fail("total must be a positive
Number")
  else { orderId: o.id, total: o.total, status: "PROCESSED" }
var attempt = try(() -> processOrder(payload))
---
if (attempt.success) { status: "OK", data: attempt.result } else { status:
"ERROR", message: attempt.error.message }
```

OUTPUT

```
{ "status": "ERROR", "message": "total must be a positive Number" }
```

An envelope like `{status, data}` or `{status, message}` is the standard shape almost every real API response ends up taking, success or failure.

#993 Process a Batch, Separating Successes From Failures

Process a batch of orders, cleanly separating the ones that succeeded from a count of the ones that failed.

INPUT

```
{ "orders": [{"id": "O1", "total": 50}, {"id": "O2", "total": -5}, {"id":
"O3", "total": 20}] }
```

SOLUTION

```
import * from dw::Runtime
fun processOne(o) = if (o.total > 0) { id: o.id, status: "OK" } else
fail("invalid total for " ++ o.id)
var results = payload.orders map ((o) -> try(() -> processOne(o)))
---
{
  succeeded: (results filter ((r) -> r.success)) map ((r) -> r.result),
  failed: sizeOf(results filter ((r) -> not r.success))
}
```

OUTPUT

```
{ "succeeded": [{ "id": "O1", "status": "OK" }, { "id": "O3", "status": "OK"
}], "failed": 1 }
```

Running `try` once per item, inside a `map`, is what lets one bad order fail on its own without stopping the rest of the batch from processing.

#994 Build a Complete Ingestion Pipeline: Parse, Validate, Transform

Parse an embedded JSON String, validate it, and report acceptance or rejection.

INPUT

```
{ "rawJson": "{\"id\":\"O1\",\"total\":50}" }
```

SOLUTION

```
import * from dw::Runtime
fun validate(o) = if (o.total > 0) o else fail("total must be positive")
var parsed = read(payload.rawJson, "application/json")
var attempt = try(() -> validate(parsed))
---
if (attempt.success) { status: "ACCEPTED", order: attempt.result } else {
status: "REJECTED", reason: attempt.error.message }
```

OUTPUT

```
{ "status": "ACCEPTED", "order": { "id": "O1", "total": 50 } }
```

Parse, validate, transform is the backbone of almost every real ingestion endpoint — this combines Volume 9's `read` with this volume's `try`-based validation in one small pipeline.

#995 Combine Idempotency Checking With Validation

Check for a duplicate before running validation at all, short-circuiting the rest of the pipeline.

INPUT

```
{ "seenIds": ["O1"], "order": {"id": "O1", "total": 50} }
```

SOLUTION

```
import * from dw::Runtime
fun validate(o) = if (o.total > 0) o else fail("total must be positive")
---
if (payload.seenIds contains payload.order.id) { status: "DUPLICATE" }
else (if ((try(() -> validate(payload.order))).success) { status: "ACCEPTED"
} else { status: "REJECTED" })
```

OUTPUT

```
{ "status": "DUPLICATE" }
```

Checking for a duplicate first avoids re-validating and reprocessing something that's already been handled — a real efficiency gain, not just a correctness one.

#996 Build a Retry-Aware Batch Submission Report

Run a retry simulation for each job in a batch and report each one's outcome.

INPUT

```
{ "jobs": [{"id": "J1", "succeedsOn": 2}, {"id": "J2", "succeedsOn": 8}] }
```

SOLUTION

```
fun attempt(n: Number, succeedsOn: Number, maxAttempts: Number) =
  if (n == succeedsOn) { success: true, attempts: n }
  else if (n >= maxAttempts) { success: false, attempts: n }
  else attempt(n + 1, succeedsOn, maxAttempts)
---
payload.jobs map ((j) -> { jobId: j.id } ++ attempt(1, j.succeedsOn, 5))
```

OUTPUT

```
[{ "jobId": "J1", "success": true, "attempts": 2 }, { "jobId": "J2",
"success": false, "attempts": 5 }]
```

Running the same retry helper independently per job, then merging each job's ID onto its own result with ++, builds a complete per-job submission report.

#997 Convert a Validated Batch Into a Paginated CSV Export

Filter out invalid orders, batch what's left, and export the first batch as CSV.

INPUT

```
{ "orders": [{"id": "O1", "total": 50}, {"id": "O2", "total": -5}, {"id": "O3", "total": 30}, {"id": "O4", "total": 40}] }
```

SOLUTION

```
fun chunk(arr: Array, size: Number): Array =
  if (isEmpty(arr)) []
  else if (sizeOf(arr) <= size) [arr]
  else [arr[0 to (size - 1)]] ++ chunk(arr[size to -1], size)
var validOrders = payload.orders filter ((o) -> o.total > 0)
var pages = chunk(validOrders, 2)
---
{
  pages: sizeOf(pages),
  firstPageCsv: write(pages[0], "application/csv")
}
```

OUTPUT

```
{ "pages": 2, "firstPageCsv": "id,total\nO1,50\nO3,30\n" }
```

Filtering before batching keeps the invalid row, `O2`, out of the export entirely — validation, batching, and format conversion all in one script.

#998 Build a Full Error-Handling Wrapper Around a Format Conversion

Wrap an XML parse attempt in `try`, reporting a clean status either way.

INPUT

```
{ "rawXml": "<order><id>O1</id></order>" }
```

SOLUTION

```
import * from dw::Runtime
var attempt = try(() -> read(payload.rawXml, "application/xml"))
---
if (attempt.success) { status: "OK", parsed: attempt.result } else { status:
"PARSE_ERROR" }
```

OUTPUT

```
{ "status": "OK", "parsed": { "order": { "id": "O1" } } }
```

Even parsing itself can fail on genuinely malformed input — wrapping `read` in `try` means a bad document produces a clean `PARSE_ERROR` status instead of crashing the whole flow.

#999 CargoLink Capstone: A Near-Complete Order Intake Integration

Parse a CSV manifest, validate every row, and report accepted counts alongside rejection reasons.

INPUT

```
{ "rawCsv": "id,weight\nPKG-1,10\nPKG-2,-5\nPKG-3,20\n" }
```

SOLUTION

```
import * from dw::Runtime
fun validateShipment(s) =
  if (not (s.id is String)) fail("id is required")
  else if (not (s.weight is Number) or s.weight <= 0) fail("weight must be a
positive Number")
  else s
var parsedOrders = read(payload.rawCsv, "application/csv", {header: true})
map ((row) -> row ++ { weight: row.weight as Number })
var results = parsedOrders map ((o) -> try(() -> validateShipment(o)))
---
{
  totalReceived: sizeOf(parsedOrders),
  accepted: sizeOf(results filter ((r) -> r.success)),
  rejected: sizeOf(results filter ((r) -> not r.success)),
  rejectionReasons: (results filter ((r) -> not r.success)) map ((r) ->
r.error.message)
}
```

OUTPUT

```
{ "totalReceived": 3, "accepted": 2, "rejected": 1, "rejectionReasons":
["weight must be a positive Number"] }
```

CSV parsing from Volume 9, coercion, per-row `try`-wrapped validation, and a summary report all in one script — nearly everything this volume has covered, one problem before the finale.

#1000 SERIES FINALE: The Complete CargoLink Shipment Processing Pipeline

Parse a batch of shipments, validate each one against a full contract, classify the accepted ones by weight, and report a complete summary — the 1000th and final problem in the series.

INPUT

```
{ "rawJson": "{\"shipments\":[{\"id\":\"SHIP-
1\",\"weight\":65,\"carrier\":\"fedex\"},{\"id\":\"SHIP-
2\",\"weight\":-5,\"carrier\":\"ups\"},{\"id\":\"SHIP-
3\",\"weight\":8,\"carrier\":\"dhl\"}]}" }
```

SOLUTION

```
import * from dw::Runtime
fun validateShipment(s) =
  if (not (s.id is String)) fail("id is required")
  else if (not (s.weight is Number) or s.weight <= 0) fail("weight must be a
positive Number")
  else if (not (["fedex", "ups", "dhl"] contains s.carrier))
fail("unrecognized carrier")
  else s
fun classify(weight: Number): String = weight match {
  case w if w > 50 -> "FREIGHT"
  case w if w > 10 -> "STANDARD"
  else -> "SMALL_PARCEL"
}
var parsed = read(payload.rawJson, "application/json")
var results = parsed.shipments map ((s) -> try(() -> validateShipment(s) ++ {
category: classify(s.weight) }))
---
{
  summary: {
    totalShipments: sizeOf(parsed.shipments),
    accepted: sizeOf(results filter ((r) -> r.success)),
    rejected: sizeOf(results filter ((r) -> not r.success))
  },
  accepted: (results filter ((r) -> r.success)) map ((r) -> r.result),
  rejected: (results filter ((r) -> not r.success)) map ((r) ->
r.error.message)
}
```

OUTPUT

```
{ "summary": { "totalShipments": 3, "accepted": 2, "rejected": 1 },
"accepted": [{ "id": "SHIP-1", "weight": 65, "carrier": "fedex", "category":
```

```
"FREIGHT" }, { "id": "SHIP-3", "weight": 8, "carrier": "dhl", "category":
"SMALL_PARCEL" }], "rejected": ["weight must be a positive Number"] }
```

Every idea from all ten volumes converges here: selectors and coercion, `match`-based classification, a recursive-style validation function, `try`/`fail` error handling, and JSON parsing, all in one realistic shipment-intake pipeline. Thank you for working through all 1000 problems.

END OF THE SERIES

1000 Down, 0 to Go

You've now worked through every selector, every type coercion rule, operators and conditionals, array and object transformations, string handling, dates and times, custom functions and recursion, pattern matching, reading and writing JSON, XML, and CSV, and a complete tour of validation and error handling.

That's the full **1000 DataWeave Problems With Solutions** series — ten volumes, one hundred problems each, built to take you from your very first selector to a production-grade integration pipeline. Congratulations on making it through all 1000.